Inside DeepInfer

How an LLM Inference Engine Works



Jianxin Zhao

Inside DeepInfer

How an LLM Inference Engine Works

Jianxin Zhao

Inside DeepInfer: How an LLM Inference Engine Works
© 2026 Jianxin Zhao.

Except where otherwise noted, the book's original text and illustrations, including the web and PDF editions, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International licence (CC BY-NC-SA 4.0):
https://creativecommons.org/licenses/by-nc-sa/4.0/.
You may share, translate and adapt this material for noncommercial purposes under the licence's terms, with attribution, a link to the licence and an indication of changes. Shared adaptations must use the same or a compatible licence. For a separate commercial licence, contact the author. Exceptions and limitations to copyright remain unaffected. Separately licensed code and third-party material retain their own terms.

Version 1.0, October 2026. The companion code for this version is tagged `v1.0` at
https://github.com/jzstark/inside-deepinfer-code.

This is an independent book. It is not affiliated with, sponsored by or endorsed by DeepSeek. Product and company names belong to their respective owners; this book uses them only to identify the software it describes.

The book describes DeepInfer at commit `774ce244b87cc0eb8d4b94ab9f5de2ab22049f01`, inspected in October 2026. Later versions may differ. Line references, defaults and behaviour stated here apply to that edition.

DeepInfer is distributed under the Apache License 2.0. Source excerpts are reproduced under that licence with attribution. Common leading indentation has been removed for typesetting; the code is otherwise unchanged. Appendix C records the source edition, file hashes, excerpt ranges and licence notices. The companion code is also distributed under Apache-2.0; its reuse is governed by that licence.

The experiments are teaching models written for this book. None of them runs the DeepInfer engine; lab 16 executes one unmodified DeepInfer policy file on CPU tensors. Their outputs are not performance measurements of DeepInfer. Performance figures and service descriptions attributed to DeepSeek's documentation or model report are DeepSeek's reported findings; this book has not reproduced them.

This book is provided as is, without warranty. Check a claim against the source edition before relying on it.

Preface

You type a question into ChatGPT and wait. An answer begins to appear. You ask a follow-up, referring to something in the reply, and the conversation continues. The text box gives little reason to think about the machinery serving it.

Behind that exchange, a service must keep many requests moving on shared hardware. Each answer needs repeated model calculations; each conversation brings a history that those calculations may have to read. Can one pass over the model's weights serve several requests? How much work on the history can be saved before memory runs out? Once an answer is complete, when is its memory safe to reuse?

An inference engine organises the model computation, saved state and output generation behind such a service. This book studies those decisions in DeepSeek's DeepInfer. It starts device work before the host program knows the previous result. It reserves space for guessed tokens that may be rejected. With a further optimisation enabled, it can shorten part of a prompt's computation and reconstruct the missing state approximately. Each choice raises a question about what the next operation is allowed to assume.

We will follow these choices into DeepInfer's code, using small experiments to expose what each operation needs from the last.

Why DeepInfer

DeepSeek identifies DeepInfer as the engine behind its large-scale production services. It has released an offline core alongside a deployment Q&A that describes the larger system: hardware, separate processing stages, routing and cache storage [1, 2]. The code shows how an engine advances a request; the Q&A describes how many such engines share the work and reach saved state. Together, they let us follow a request beyond the boundary of one process.

The model gives us another reason to look closely. Its attention design changes what a cache row contains. Sharing representations between layers changes which work can be omitted. Routing tokens to different experts changes which weights must be available on each device. These properties reach into memory allocation, scheduling and execution. Reading the model and engine together makes their constraints easier to understand.

DeepInfer was originally built primarily on vLLM. Its deployment Q&A credits SGLang and NanoFlow when describing asynchronous scheduling [1, 2]. If you know those systems, familiar names will help you find your bearings. We then ask what those names mean here: which design is retained, which behaviour differs, and which requirement comes from DeepSeek's model. Fixed-version comparisons and the authors' own acknowledgements give us evidence for those relationships; a familiar class name alone does not.

What you will be able to follow

By the end, you should be able to investigate a pause in generation by locating the work and waits that could explain it. You should be able to follow a cache address to its owner and determine when that storage can be reused. A throughput curve should lead you back to its counting assumptions. A passing test should lead you to the values it compares and the claim its assertion actually checks. Together, these skills give you a working understanding of inference engines and a route through DeepInfer's architecture when you want to read further or change something.

Throughout, we distinguish teaching experiments, inspected implementation, documented production architecture and proposals made in this book. We also keep different claims of correctness separate. Equal values, values within a tolerance, samples from an intended distribution and useful answers each call for a different check. These distinctions matter whenever an optimisation changes the order, amount or location of computation.

You need familiarity with functions, loops and arrays, and enough Python to read a short program. Elementary probability will help with sampling. We introduce attention, device memory and distributed execution as the questions require them. Each technical chapter begins with an experiment whose result you can predict, then opens a connected path through the source. The excerpts are printed beside the explanation, with diagrams and concrete cases to keep track of the objects crossing each boundary.

What you can try

The small Python experiments let you make mistakes cheaply. Release a simulated memory block before its queued copy finishes, and another request's data appears in the copy. Move expert work between simulated devices, and check that the answer survives the move. Enumerate a sampling rule's possible outcomes, and see exactly where its output distribution changes. You can alter a condition, predict the consequence and check it before returning to the engine code.

There are twenty-eight experiments. They require Python 3.10 or later; all but one use only the standard library. The exception runs DeepInfer's placement policy with PyTorch on a CPU. The experiments, their saved outputs and the source checks are available at `https://github.com/jzstark/inside-deepinfer-code`; Appendix C explains how the outputs were recorded. Assigned costs and simulated clocks expose dependencies and trade-offs; they do not measure accelerator performance.

The inspected engine release requires Python 3.12 and supports NVIDIA SM100 GPUs with CUDA 13, or Huawei Ascend 950 NPUs with CANN 9.2 [1]. The Q&A recommends 64 prefill GPUs plus 64 decode GPUs as the smallest *efficient service deployment*, rather than a minimum for every offline experiment [2]. Most readers will therefore study the full engine through source walkthroughs.

The route through the book

We begin with one generation loop, then ask what batching saves and what a model must remember. Once that state has a size and an owner, we can decide which requests fit, which should run next and how host and device work can overlap. Experts spread the computation across devices; speculative decoding adds a choice about how much work is worth checking. Approximate reconstruction then tests the limits of what cached state can be recovered. The final chapters examine the guarantees these choices must meet and the service that connects

engines to one another and to storage. A closing request trace brings the mechanisms back together.

The appendices support closer reading. Appendix A follows compression and indexing into attention. Appendix B maps README features to source entry points, traces a step's calls and returns, and shows how processes and devices are grouped. Use it whenever a chapter enters an unfamiliar part of the repository. Appendix C identifies the source edition and explains reproduction; a terms table locates definitions.

The detailed walkthroughs cover the main runtime mechanisms and selected model-driven state paths. Low-precision storage is included; quantisation algorithms and general kernel tuning are outside the scope. Ascend kernel internals, vision architecture, checkpoint loading and the complete distributed initialisation and recovery paths are not covered systematically. Hyper-connections and Engram receive navigation entries rather than full derivations. The repository map marks these limits and supplies places to continue reading.

For now, start with a list of integers. One of them has just been generated, but the prediction function has not processed it yet. Keeping track of that small distinction will take us a long way into DeepInfer.

Contents

Preface **ii**

1 A generation loop **1**
1.1 One decision at a time . . . 1
1.2 Keep work that will be needed again . . . 3
1.3 Rebuild a lost window . . . 5
1.4 Give an empty slot to another request . . . 7
1.5 What an engine must remember . . . 9
1.6 Follow state through DeepInfer . . . 10
1.7 Experiments to try next . . . 17

2 Where the time goes **19**
2.1 Give each cost a unit . . . 20
2.2 Find what batching shares . . . 21
2.3 Count useful output separately . . . 23
2.4 Read a request timeline . . . 23
2.5 Trace work and measurements through DeepInfer . . . 26
2.6 Experiments to try next . . . 32

3 The model shapes the engine **33**
3.1 A query chooses a mixture . . . 33
3.2 Cache the past, calculate the present . . . 35
3.3 Count rows, then count bytes . . . 37
3.4 Open DeepInfer's attention path . . . 38
3.5 Keep visible rows safe from overwrite . . . 45
3.6 Experiments to try next . . . 46

4 State needs an owner **47**
4.1 Hand out memory in blocks . . . 47
4.2 Two owners, one block . . . 48
4.3 Follow DeepInfer's ownership path . . . 49
4.4 A state slot has regions with different lifetimes . . . 56
4.5 What a cache entry means . . . 57
4.6 Experiments to try next . . . 58

5 Choose the next batch **59**
5.1 A smaller chunk is an opportunity to choose . . . 59
5.2 Follow a prompt through the scheduler . . . 61
5.3 When running requests exhaust the pool . . . 66
5.4 A position budget is not a time budget . . . 70

5.5 Experiments to try next . . . 70

6 Keep the host and device busy **71**
6.1 Overlap has a limit . . . 71
6.2 One request crosses two steps . . . 74
6.3 Read the correction path . . . 76
6.4 Last use, reuse and the cost of end-of-sequence . . . 80
6.5 Graphs constrain the input buffers . . . 81
6.6 Experiments to try next . . . 82

7 Put experts on several devices **83**
7.1 Dispatch, compute and return . . . 83
7.2 Follow DeepInfer's execution path . . . 85
7.3 Why widen the expert group? . . . 88
7.4 Use history to choose a placement . . . 89
7.5 Make the new placement take effect . . . 94
7.6 Experiments to try next . . . 97

8 Spend verification work wisely **98**
8.1 Accept or correct . . . 99
8.2 What DeepInfer verifies . . . 100
8.3 How many positions are worth the work? . . . 104
8.4 Which round supplies the confidence? . . . 108
8.5 Reported gains and confidence error . . . 112
8.6 Experiments to try next . . . 114

9 Reconstruct state approximately **115**
9.1 Where exact window recovery stops . . . 115
9.2 Where DeepInfer puts the boundary . . . 117
9.3 Which rows can a consumer see? . . . 120
9.4 Which state may enter the prefix cache? . . . 123
9.5 Count the saving; choose the error baseline . . . 125
9.6 Experiments to try next . . . 127

10 Different correctness contracts **128**
10.1 Equality needs a fixed computation order . . . 128
10.2 One flag reaches several libraries . . . 130
10.3 Golden tokens hold the trajectory fixed . . . 131
10.4 Repeatable and correctly distributed are different claims . . . 135
10.5 A grammar narrows the distribution . . . 139
10.6 Experiments to try next . . . 142

11 From an engine to a service **144**
11.1 A request crosses several owners . . . 145
11.2 Hits decide prefill demand . . . 146
11.3 State lives in three places . . . 148
11.4 A hit has two lengths . . . 151
11.5 Balancing, scaling and failure . . . 152
11.6 Check a published curve . . . 152
11.7 Experiments to try next . . . 155
11.8 One request, end to end . . . 155

A Compression and indexing **158**
A.1 A pair is complete only at its boundary 158
A.2 Temporary state must survive speculative writes 160
A.3 One compressed result feeds two caches 161
A.4 Select rows, then translate their addresses 163
A.5 Count rows, payloads and owners separately 166
A.6 Sharing has several meanings 167
A.7 Experiments to try next 168

B Repository and execution map **169**
B.1 Processes, devices and communication groups 170
B.2 Follow one step into computation 171
B.3 Follow results back to their owner 172
B.4 Start from a README feature 173
B.5 Choose a directory before choosing a file 174
B.6 Find the corresponding code in another edition 176

C Source edition and excerpts **177**

Terms and abbreviations **179**

References **181**

Chapter 1

A generation loop

Suppose a function takes a list of integers and returns a score for each possible next integer. You can use it to extend the list: call the function, choose an integer, append it and repeat. This is enough to describe the outer loop of text generation.

Running that loop for many people raises further questions. Which work can be reused? Which requests should run together? When can their memory be released? Four small programs give us the tools to follow a real request through DeepInfer. We will trace admission, scheduling, execution and result ownership, then explain why an empty scheduling queue does not end all of the engine's work.

The question

What must we add around a prediction function to serve several requests? Watch three things: which token the next call needs, which work survives a call, and which request gets the next free slot.

Figure 1.1 shows the smallest version: each prediction depends on the token just selected, until an end marker or output limit stops the loop.

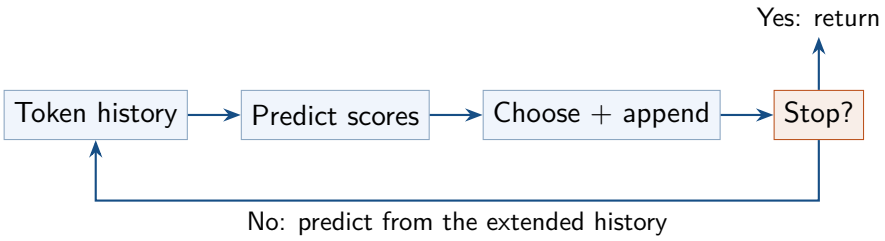


Figure 1.1: The next prediction depends on the selected token. The end marker or output limit ends the loop.

1.1 One decision at a time

Language models work with *tokens*: entries in a vocabulary, each identified by an integer. A token may represent a word, part of a word, punctuation or a special marker. The program that maps text to these integers is a *tokenizer*. The exact mapping depends on the model.

Our first experiment invents a vocabulary with five entries. Three stand for words; two mark the start and end. The integers and scores are chosen for this exercise.

Run this command from the book directory:

```
python3 labs/01_generate.py
```

Before reading the output, follow the first row of `SCORES`. The starting token has ID zero. The largest score in row zero belongs to token one, `Small`. That token selects the next row.

Listing 1.1: An invented vocabulary and its score table.
Book lab · `labs/01_generate.py` · lines 8–16

```
VOCAB = ["<start>", "Small", "steps", "help", "<end>"]
SCORES = [
    [0, 9, 1, 0, 0],  # <start> -> Small
    [0, 0, 8, 1, 0],  # Small   -> steps
    [0, 0, 0, 7, 1],  # steps   -> help
    [0, 0, 0, 0, 9],  # help    -> <end>
    [0, 0, 0, 0, 9],  # <end>   -> <end>
]
END = VOCAB.index("<end>")
```

The loop uses the last token to select a row, chooses the largest score and adds the chosen token to the history. The complete script calls this function and checks its result.

Listing 1.2: Choose one token and repeat.
Book lab · `labs/01_generate.py` · lines 19–34

```
def generate(max_new_tokens=8):
    history = [0]
    output = []
    for step in range(1, max_new_tokens + 1):
        scores = SCORES[history[-1]]
        token = scores.index(max(scores))
        print(f"step {step}: {VOCAB[history[-1]]} -> {VOCAB[token]}")
        history.append(token)
        if token == END:
            print("stop: end token")
            break
        output.append(VOCAB[token])
    else:
        print("stop: output limit")
    print("text:", " ".join(output))
    return output
```

Here is the output from running the script:

```
step 1: <start> -> Small
step 2: Small -> steps
step 3: steps -> help
step 4: help -> <end>
stop: end token
text: Small steps help
```

The table plays the role of a very small prediction function. Unlike a Transformer, it uses only the last token and has no learned weights. This restriction makes every decision visible.

Choosing the largest score is called *greedy selection*. A language model usually returns scores called *logits*. A sampler can turn them into probabilities and draw a token, or use them to make a greedy choice. We will study sampling later. Here, `max` makes the choice deterministic.

The expression `scores.index(max(scores))` finds the first position with the largest score. If two tokens tie, it therefore chooses the earlier one. The `else` attached to the `for` loop runs only when the loop reaches its limit without executing `break`.

The loop needs two ways to stop. The end marker lets the prediction function finish an answer. The output limit stops a function that never chooses that marker. Predict the result of `generate(max_new_tokens=2)`, then try it in a Python session. Does the loop reach the end marker?

1.1.1 What depends on what

Let $x_1, \ldots, x_n$ denote the input tokens. A model with fixed weights θ supplies scores:

$$z = f_\theta(x_1, \ldots, x_n).$$

The selection rule turns z into x_{n+1}. The next call then uses the extended sequence. The symbol f_θ hides the neural network for now; the dependence between calls is what matters.

In ordinary generation, the next call needs the token chosen by the previous call. Knowing that an answer will contain ten tokens does not tell us which ten tokens to feed into ten independent calls. Later, speculative decoding will give us a way to propose several tokens and verify them. It must still respect the model's generation rule.

1.2 Keep work that will be needed again

A simple implementation could pass the entire growing history to the model on every call. For a causal model, earlier positions cannot depend on tokens that have not yet been generated. Some work on those positions can therefore be saved.

To see the principle without introducing attention, use a prefix sum. Our second prediction function returns the sum of all input integers, modulo ten. The modulo operation keeps the output in the range zero to nine.

Starting with $[1, 2, 3, 5]$, the sum is eleven, so the next integer is one. Appending one makes the sum twelve; the next integer is two. Appending two makes the sum fourteen; the next integer is four.

There are two ways to do this. One sums the whole history each time. The other keeps a running total and adds only the newly generated integer before the next prediction.

```
python3 labs/02_reuse.py
```

The first function starts its sum at zero for every prediction:

Listing 1.3: Sum the whole history each time.
Book lab · `labs/02_reuse.py` · lines 9–21

```
def recompute(prompt, steps):
    history = list(prompt)
    output = []
    additions = 0
    for _ in range(steps):
        total = 0
        for token in history:
            total += token
            additions += 1
        token = total % 10
        history.append(token)
        output.append(token)
    return output, additions
```

The second sums the prompt once. Before each later prediction, it adds the previous output to the saved total:

Listing 1.4: Keep the total and add only new work.
Book lab · `labs/02_reuse.py` · lines 24–39

```
def reuse(prompt, steps):
    total = 0
    additions = 0
    processed = 0
    for token in prompt:
        total += token
        additions += 1
        processed += 1
    output = []
    for step in range(steps):
        if step > 0:
            total += output[-1]
            additions += 1
            processed += 1
        output.append(total % 10)
    return output, additions, processed
```

The full script calls both functions with the same prompt and checks their outputs and addition counts:

```
prompt: [1, 2, 3, 5]
recompute: [1, 2, 4] | additions: 15
reuse:     [1, 2, 4] | additions: 6
same output: True
history entries: 7 | entries covered by final cache: 6
```

Both versions produce $[1, 2, 4]$. The first performs fifteen additions: four before the first output, five before the second and six before the third. The second performs six: four to process the prompt, then one for each of the next two predictions.

For a prompt of length n and $m \geq 1$ output integers, the first version performs

$$n + (n+1) + \cdots + (n+m-1) = mn + \frac{m(m-1)}{2}$$

additions. The second performs $n + m - 1$.

The final output has been emitted but has not been fed back into the prediction function. The history has seven entries; the running total covers six. If we wanted another output, we would first add the last emitted integer. Figure 1.2 separates the two boundaries. DeepInfer adds another: when a device result becomes available to the host. Its asynchronous scheduler must track these boundaries separately.

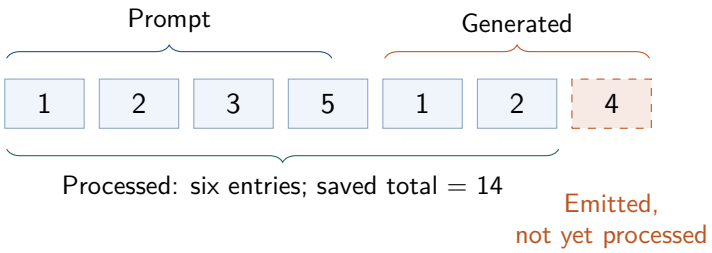


Figure 1.2: State after the third prediction in the prefix-sum experiment. The output token 4 is in the history but has not entered the saved total.

1.2.1 From a running total to a cache

The running total is a *cache*: saved information used to avoid repeating work. It is useful because it contains enough information for our next update. Appending a token preserves that property. Editing an earlier token without repairing the total would break it.

Transformer attention needs a richer cache. Earlier positions provide arrays called *keys* and *values*, which later positions use to compute their representations. Saving these arrays gives a key–value cache, usually shortened to *KV cache*. We will derive the arrays and their memory cost in later chapters. The running sum occupies constant space. A full-attention KV cache grows with the number of processed tokens, and each decode step still reads keys and values across that context. Caching avoids rebuilding them; it does not make attention a constant-cost operation. A sliding window bounds the context read by that attention layer, which changes both its memory use and what it can see.

Processing the supplied prompt is called *prefill*. Extending the sequence using saved state is called *decode*. In a conventional cached generation loop, the final prompt position supplies the scores for the first output token. The next model call processes that token to predict the second.

These names identify work, not a guarantee about how many device calls it takes. A long prompt may be split across several steps. A speculative step may verify several candidate tokens. Our simple loop gives us a starting point for understanding both changes.

1.3 Rebuild a lost window

How much history do we need to recover a cache? Change the predictor to the sum of the last three processed integers, modulo ten. A bounded `deque` holds that window: appending a fourth integer discards the oldest. This gives us the behaviour of a three-slot ring buffer without writing its index arithmetic.

```
python3 labs/04_window.py
```

Predict what happens if we lose the window after four outputs. The token history is still available, together with the number of entries already processed:

Listing 1.5: Generate with a bounded window, then rebuild from the processed suffix.
Book lab · `labs/04_window.py` · lines 11–30

```
def generate(prompt, steps, width=3):
    history = list(prompt)
    window = deque(prompt, maxlen=width)
    processed = len(prompt)
    output = []
    for step in range(steps):
        if step > 0:
            window.append(output[-1])
            processed += 1
        token = sum(window) % 10
        assert list(window) == history[max(0, processed - width):processed]
        assert token == sum(history[max(0, processed - width):processed]) % 10
        history.append(token)
        output.append(token)
    return history, output, window, processed


def rebuild(history, processed, width=3):
    return deque(history[max(0, processed - width):processed], maxlen=width)
```

The script clears the window, calls `rebuild` and checks it against the saved state. It also shows what happens if we take the wrong end of the history:

```
generated: [0, 8, 3, 1]
history entries: 8 | processed entries: 7
window before loss: [0, 8, 3]
rebuilt from processed suffix: [0, 8, 3]
same window: True
wrong suffix (includes latest output): [8, 3, 1]
next token after processing latest output: 2
```

The four outputs are $[0, 8, 3, 1]$. At the checkpoint, seven entries have been processed, and the window is $[0, 8, 3]$. The latest output, 1, has been emitted but not processed. Taking the last three entries of the entire history would include it too soon. To continue after recovery, first process that 1: the window becomes $[8, 3, 1]$, whose sum predicts 2.

Recovery is exact here because the state contains raw integers. The last three processed entries determine the whole window, regardless of the earlier history. DeepInfer's sliding-window attention (SWA) bounded replay tackles a related problem, but reconstructs an approximate attention cache: hidden states inside the replayed suffix can depend on earlier context. Chapter 9 will explain that approximation alongside cross-layer sharing. Assessing it requires model-quality checks as well as checks of buffer contents [2].

1.4 Give an empty slot to another request

So far, one request has owned the loop. Now suppose three requests are waiting. For this experiment, A needs two output tokens, B needs four and C needs one. The engine can advance at most two requests in a round.

We make strong assumptions to isolate one scheduling decision. All requests arrive at the start. Each active request gains exactly one token per round. Every round costs one abstract time unit, however many slots are occupied. There is no prefill cost or memory limit.

A fixed-batch policy admits A and B together, waits until both finish, then admits C. A second policy refills an empty slot at the start of each round. Predict when C will finish under each policy.

```
python3 labs/03_batch.py
```

The waiting queue supplies requests in arrival order. An active slot holds a request name and its remaining work; `None` means the slot is empty. The `refill` switch controls admission at the start of each round.

Listing 1.6: Two policies share the same execution model.
Book lab · `labs/03_batch.py` · lines 9–39

```
from collections import deque

JOBS = [("A", 2), ("B", 4), ("C", 1)]


def simulate(refill):
    waiting = deque(JOBS)
    active = [None, None]
    finished = {}
    rounds = 0
    unused = 0
    while waiting or any(job is not None for job in active):
        if refill or all(job is None for job in active):
            for slot, job in enumerate(active):
                if job is None and waiting:
                    name, length = waiting.popleft()
                    active[slot] = [name, length]
        rounds += 1
        names = [job[0] if job else "-" for job in active]
        print(f"round {rounds}: {' '.join(names)}")
        for slot, job in enumerate(active):
            if job is None:
                unused += 1
                continue
            job[1] -= 1
            if job[1] == 0:
                finished[job[0]] = rounds
                active[slot] = None
    print(f"rounds: {rounds} | unused slot-rounds: {unused}")
    print("finish:", ", ".join(f"{name}={finished[name]}" for name, _ in JOBS))
    return rounds, unused, finished
```

The script calls `simulate` once with `refill=False` and once with `refill=True`, and checks both results:

```
Fixed batches
round 1: A B
round 2: A B
round 3: - B
round 4: - B
round 5: C -
rounds: 5 | unused slot-rounds: 3
finish: A=2, B=4, C=5

Refill after each round
round 1: A B
round 2: A B
round 3: C B
round 4: - B
rounds: 4 | unused slot-rounds: 1
finish: A=2, B=4, C=3
```

A finishes after round two under both policies. With fixed batches, its slot stays empty while B runs in rounds three and four. C must wait until round five. Refilling the empty slot lets C run in round three alongside B. Figure 1.3 compares the two schedules.

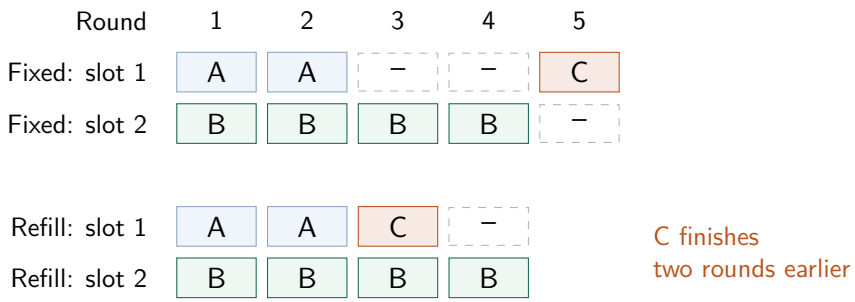


Figure 1.3: The same seven token updates occupy five fixed-batch rounds or four refilled rounds. Dashed cells are unused slots; rounds have an assigned unit cost.

Both policies do seven units of useful token work. Fixed batches provide ten slot-rounds across five rounds and leave three unused. Refilling provides eight across four rounds and leaves one unused. B still finishes after round four; the shorter request benefits from the earlier admission.

Allowing batch membership to change between generation steps is often called *continuous batching*. Admitting C in an engine also requires prompt processing and enough memory for its state.

Notice that the scheduling policy never reads the remaining length when choosing the next request. The simulator uses that length only to decide when the simulated request finishes. A real engine generally discovers completion as generation proceeds.

1.5 What an engine must remember

The programs have acquired several kinds of state: token history and an output limit, reusable computation with a processed boundary, and a waiting queue with active slots and completion records.

For each request, an engine needs to answer at least four questions:

1. Which input and output tokens belong to it?
2. How much of that sequence has the model processed?
3. Which memory holds the state needed to continue?
4. Is it waiting, running or finished?

The engine also needs shared decisions: which requests to advance and how much work to admit. A *scheduler* makes those choices. A *worker* carries out the selected model work. The separation lets us discuss a scheduling policy without rewriting the prediction function.

There is a further complication when computation runs on another device. Submitting work may return before that work finishes. A request that has left a scheduling queue may still have a device operation using its memory. Resource lifetime therefore depends on completion, as well as logical request status. We will examine this in the chapter on overlapping host and device work.

1.6 Follow state through DeepInfer

DeepInfer's README says it was originally built primarily on vLLM; this book has not identified the fork point. Its asynchronous execution makes the loop more demanding. Its zero-overhead batch scheduler (ZOS) overlaps host scheduling with device computation. The deployment Q&A credits SGLang and NanoFlow for earlier implementations of this approach [1, 2].

Follow one request through the ordinary offline interface: valid text token IDs, with no multimodal processor or structured-output grammar. We will read its admission, execution and result path, then revisit what happens when execution overlaps. This is a walkthrough of the inspected code, not a recorded engine run. Figure 1.4 gives the route. The arrows cross responsibility boundaries: the entry point prepares requests, the engine coordinates them, the scheduler selects work and the worker executes it.

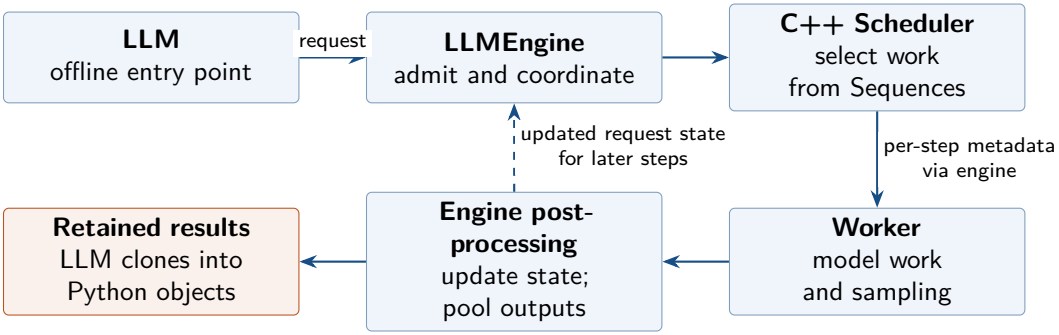


Figure 1.4: The ordinary offline request path. Arrows show data handoffs, not a single synchronous call stack. The worker calls back into the engine to process completed work; under ZOS, that work can belong to the previous step. The model, cache allocation and distributed execution remain closed boxes here [4, 5, 8, 7].

1.6.1 A request becomes persistent engine state

`LLM.generate` submits each input through `_add_request` before entering its generation loop. The helper begins by separating the request's sampling settings from the caller's object, assigning an identifier and making a token array:

Listing 1.7: Prepare one request before handing it to the engine.
DeepInfer · `deepinfer/entrypoints/llm.py` · lines 151–157

```
151 sampling_params = copy.deepcopy(sampling_params)
152 request_id = str(next(self.request_counter))
153 if prompt_token_ids is None:
154     if prompt is None:
155         raise ValueError("prompt and prompt_token_ids cannot both be None")
156     prompt_token_ids = self.llm_engine.tokenizer.encode(prompt)
157 prompt_token_ids = np.asarray(prompt_token_ids, dtype=np.int32)
```

For our token-ID input, the tokenizer branch is skipped. `deepcopy` creates independent sampling settings even if the caller supplied the same object for several requests; converting to `int32` establishes the array's element type. It does not promise another copy when the input already has that type. These are different guarantees. The helper passes the identifier, settings and token array to `LLMEngine.add_request`.

After validation and input preparation, the engine creates a C++-backed `Sequence`. The `sequence_args` dictionary carries the identifier, settings and other request metadata prepared earlier in that method:

Listing 1.8: Create the request state that survives across steps.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 594–598

```
594 seq = Sequence(
595     prompt_token_ids=prompt_token_ids,
596     compressed_prompt_token_ids=compressed_prompt_token_ids,
597     **sequence_args,
598 )
```

The two token arrays serve different input representations. Their model-specific contents are not needed to follow admission: both belong to the request whose state will survive successive scheduling rounds. For our request, no grammar construction is pending, so the next fragment takes its final `else`:

Listing 1.9: The ordinary request reaches the scheduler immediately.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 620–630

```
620 def _enqueue(s: Sequence) -> None:
621     if s.so_matcher is not None and s.so_matcher.is_aborted():
622         s.status = SequenceStatus.FINISHED_SO_FAILED
623     self.scheduler.add_sequence(s)
624     if self.multi_modal_processor is not None:
625         self.multi_modal_processor.add(request_id, multi_modal_data)
626
627 if so_pending:
628     self.so_matcher_builder.submit(seq, sampling_params.so_args, _enqueue)
629 else:
630     _enqueue(seq)
```

Here `so_pending` means that a structured-output grammar still needs to be built. Our case also has no multimodal processor, so neither extra path runs. The call to `add_sequence` crosses into C++:

Listing 1.10: Admission appends a request; it does not run the model.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 135–142

```
135 void add_sequence(std::shared_ptr<Sequence> seq) {
136     seq->request.arrival_time = get_timestamp_ms();
137     if (engine_type == EngineType::NORMAL) {
138         seq->reserve_output_tokens();
139         seq->cache.block_table.reserve(_block_table_capacity(*seq));
140     }
141     waiting.push_back(std::move(seq));
142 }
```

The scheduler records an admission timestamp and appends the sequence to `waiting`. The C++ `shared_ptr` is an ownership handle; `std::move` transfers that handle into the queue without copying the sequence's token history. The arrow in `seq->request` accesses a member through that handle. For a normal offline engine, the method also reserves capacity in vectors for output tokens and block references. Reserving a vector's capacity is not allocating the device's KV cache. That resource decision belongs to later scheduling and cache management.

This boundary explains why `generate` can submit many requests before running any model work. Admission records available work; scheduling decides what fits now. A useful consequence of this separation is that a new admission policy need not alter the model's prediction function. We infer that benefit from the division of responsibilities, rather than from a stated design motive.

1.6.2 Generated length does not measure completed work

Listing 1.11: Token history and its prompt/output boundary.
DeepInfer · csrc/engine/sequence.hpp · lines 80–88

```
80  void append_token_id(int token_id) { token_ids.push_back(token_id); }
81  int get_len() const { return static_cast<int>(token_ids.size()); }
82  int get_prompt_len() const { return prompt_len; }
83  int get_output_len() const { return get_len() - prompt_len; }
84  const std::vector<int> &get_token_ids() const { return token_ids; }
85  int get_last_token_id() const { return token_ids.back(); }
86
87  std::vector<int> token_ids;
88  int prompt_len{0};
```

The persistent `Sequence` contains input tokens, generation settings, cache bookkeeping and scheduling state. The scheduler holds this request object across steps. These methods belong to its `SequenceData` token history. Output length is total history length minus prompt length. It counts generated tokens, not cached positions or device work that has completed. Our running-total experiment shows why that distinction matters [6].

The nearby `SequenceStatus` enum includes `WAITING`, `RUNNING`, `SWAPPED` and several reasons for finishing. An enum value does not tell us which transitions the scheduler actually takes. For this request, follow admission into `RUNNING` and eventual completion. Chapter 5 examines what changes when memory pressure interrupts that route [8].

1.6.3 A scheduling decision becomes model work

Now return to the engine's `step`. Its call to the C++ scheduler returns per-request metadata and one object describing decisions for the whole batch:

Listing 1.12: Per-sequence metadata and batch-wide decisions.
DeepInfer · deepinfer/engine/llm_engine.py · lines 885–887

```
885 seq_metadata_list, scheduler_outputs = self.scheduler.schedule(
886     self.sequence_metadata_pool
887 )  # Will also free the finished sequence groups.
```

Here, *metadata* means lengths, positions and references needed to interpret arrays. `SequenceMetadata` describes selected work for a request in this step; `SchedulerOutputs` carries batch decisions, including whether the work is prompt processing and which sequences were selected. Neither is the user's completed answer. A correct token array with incorrect positions can still produce an incorrect calculation.

After preparing state transfers, the engine calls `_execute_model_async_step`. Its implementation shows exactly what crosses the worker boundary:

Listing 1.13: Pass scheduling decisions to the worker and collect output addresses.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 961–975

```
961 def _execute_model_async_step(
962     self, seq_metadata_list: list[SequenceMetadata], scheduler_outputs:
    ↪ SchedulerOutputs
963 ) -> np.ndarray[np.uint64]:
964     with cpu_timer.record("run_workers"):
965         request_outputs = self._run_workers(
966             "execute_model_step",
967             driver_kwargs=dict(
968                 seq_metadata_list=seq_metadata_list,
969                 scheduler_outputs=scheduler_outputs,
970             ),
971             is_prompt=scheduler_outputs.prompt_run,
972             is_placeholder=scheduler_outputs.is_empty(),
973         )
974     self.deferred_release.collect_request_outputs(request_outputs)
975     return request_outputs
```

`_run_workers` delegates to the driver worker's dispatch method. The per-request metadata and scheduler result travel in `driver_kwargs`; the prompt and placeholder flags are separate arguments. A placeholder means no real sequences were selected, but the worker still follows an execution path. An empty current batch therefore does not imply that all earlier work is done.

The returned array is registered with `deferred_release` before being returned to the caller. This connects the execution path to an ownership rule: these outputs are temporary, and the engine is already tracking their eventual release. We will inspect the copy that makes a result safe to keep below [5].

The comment in Listing 1.12 still says "sequence groups". vLLM v0.6.3 defines `SequenceGroup` in `vllm/sequence.py` and describes freeing finished sequence groups in its engine's `step` docstring. In the inspected DeepInfer engine, the scheduler instead handles `Sequence` objects; its Python and C++ sources define no `SequenceGroup` type [11, 12, 8]. This is consistent with inherited terminology, though it does not establish the exact fork point. Read the types and control flow before treating an old comment as a current design description.

1.6.4 One call can handle work from two steps

The opening of `Worker.execute_model_step`'s docstring gives the order:

Listing 1.14: A docstring excerpt showing current and previous work.
DeepInfer · `deepinfer/worker/worker.py` · lines 96–102

```
96 """The unified entry for model execution: every step (prompt/decode, ZOS on/off,
97 placeholder) goes through this Forward-Post_process-Sample round.
98   1. Forward current step (main model)
99   2. Post-process previous step
100  3. Sample current step
101  4. Post-process current step: in place for prompt / non-ZOS steps;
102     ZOS decode steps defer it to the next step to overlap with forward
```

The implementation makes the delayed state explicit. After current model work, it first processes the state saved from the previous call:

Listing 1.15: Collect the previous step before deciding what to retain.
DeepInfer · deepinfer/worker/worker.py · lines 143–144

```
143 # 2. post process prev step
144 request_outputs = self._post_process_step(self.prev_step_state)
```

It then samples the current work and packages its metadata, sample output and completion event in `curr_step_state`. The final branch decides whether that package is processed now or retained:

Listing 1.16: Prompt work completes here; ZOS decode leaves a pending state.
DeepInfer · deepinfer/worker/worker.py · lines 183–189

```
183 if is_prompt or not envs.DEEPINFER_ZERO_OVERHEAD_SCHEDULING:
184     request_outputs = np.concatenate([request_outputs, self._post_process_step(
    ↪ curr_step_state)])
185     self.prev_step_state = None
186 else:
187     self.prev_step_state = curr_step_state
188
189 return request_outputs
```

For prompt work or disabled ZOS, the returned outputs include current-step post-processing. For a ZOS decode step, `prev_step_state` instead becomes the current package; the returned outputs came from the earlier package. The method's return is therefore not a universal "current prediction is ready" boundary. Chapter 6 will open the completion-event and metadata-correction mechanisms behind this branch.

1.6.5 A queue can be empty while results are still pending

`LLM.generate` adds requests through `_add_request` and enters `_run_engine`. Here is the loop that collects their results:

Listing 1.17: Step while requests remain, then flush pending results.
DeepInfer · deepinfer/entrypoints/llm.py · lines 175–187

```
175 outputs: dict[str, list[RequestOutput]] = defaultdict(list)
176 while True:
177     unfinished = self.llm_engine.has_unfinished_requests()
178     step_outputs = self.llm_engine.step() if unfinished else self.llm_engine.
    ↪ flush_inflight_reqs()
179     for raw_output in step_outputs:
180         # clone step_outputs since step_outputs will be released in the next
    ↪ step
181         output = RequestOutput.clone(raw_output)
182         outputs[output.request_id].append(output)
183         if output.finished and use_tqdm:
184             pbar.update(1)
185     if not unfinished:
186         break
187 request_ids = sorted(outputs, key=int)
```

The conditional call is the important part. Once the engine reports no unfinished requests, the loop still calls `flush_inflight_reqs`, collects its outputs and only then breaks. The

docstring of `LLMEngine.flush_inflight_reqs`, line 978, says: "Flush and post-process all requests in flight under ZOS" (author's translation). Logical completion and completion of result handling are separate events [5].

The engine's flush delegates to the worker's flush method. Its body consumes exactly the state retained by the branch above:

Listing 1.18: Drain the pending step and clear its retained state.
DeepInfer · `deepinfer/worker/worker.py` · lines 281–283

```
281 request_outputs = self._post_process_step(self.prev_step_state)
282 self.prev_step_state = None
283 return request_outputs
```

`_post_process_step` returns an empty address array when its argument is `None`. Otherwise it waits for sampling completion, prepares the model results and calls the engine's post-processing callback. The engine updates sequence state through `scheduler.process_outputs`, then builds pooled request outputs. The outer loop can thus collect the same form of result from an ordinary step or the final drain. Clearing `prev_step_state` prevents that retained package from being consumed twice [5, 7].

1.6.6 An output has an owner and a lifetime

The comment immediately before `RequestOutput.clone(raw_output)` belongs to the call site above. It explains why the outer loop copies each output. The stronger ownership contract appears in `LLMEngine.step`:

Listing 1.19: The output pool's lifetime contract.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 861–871

```
861 """Run one inference step and return the generated outputs.
862
863 Returns:
864     np.ndarray[np.uint64]: entrypoint -> a list of RequestOutput objects.
865
866 Note:
867     The returned RequestOutput objects are managed by RequestOutputPool.
868     RequestOutput will be automatically released in the next step() call.
869     If you need to retain them beyond the current step, you must call
    ↪ pool.clone(list[RequestOutput]) to obtain copies,
870     and release them via pool.release(list[RequestOutput]).
871 """
```

Notice the return type: `np.ndarray[np.uint64]`. Its entries are integer addresses of C++ `RequestOutput` objects, not Python references to those objects. Keeping the array, or putting its integers in a Python list, preserves the addresses but does not keep the pointed-to objects alive. The C++ pool can destroy and reuse them at the next step [9].

The loop therefore calls `RequestOutput.clone(raw_output)` while each address is still valid. This static method is declared in `deepinfer/_C/engine.pyi`, line 182. Its C++ binding copies the object at the supplied address into a new object owned by the Python wrapper [10]. The docstring's reference to `pool.clone` does not match that exposed API: use the entry-point call and the binding to identify the operation. A Python reference protects the copied object; an integer address does not protect the pooled original.

Cloning during collection preserves the result across the next pool release. Chapter 4 returns to the binding and pool implementation in Section 4.3.9, alongside the ownership rules for cached state. Here the caller needs one rule: copy a result while its address is valid.

1.6.7 Trace the same request across the boundaries

Our ordinary request now has a complete route. After admission it is a `Sequence` in `waiting`. When selected, its per-step metadata travels through the engine to the worker. Under ZOS decode, the worker retains the current step's package and reports completed earlier work. Post-processing updates the request and creates output addresses; the offline collector copies them before the pool may reuse their storage.

For a concrete branch trace, suppose this request remains active after its last prompt chunk and through two ZOS decode calls. Name the decode packages D_1 and D_2. With no pending state on entry to that prompt call, the printed worker code gives:

Call	Package processed for output	State retained after return
Last prompt chunk	Current prompt package	`None`
Decode D_1	None	D_1
Decode D_2	D_1	D_2

The D_1 call can return no output despite doing model work. The D_2 call can return D_1's results while retaining its own. A later flush consumes whichever package remains. This predicts branch behaviour, not particular token values or the number of calls until the request finishes.

Three different objects therefore answer three different questions:

Object	What it represents
`Sequence`	A request's state across scheduling rounds.
`WorkerStepState`	One execution step's metadata, samples and completion event.
`RequestOutput`	One result record, temporary in the pool or retained as a copy.

To investigate a missing final result, first compare the worker's pending state with the outer loop's flush path. To investigate results that change after a later step, inspect collection and cloning before changing sampling. These are different failure hypotheses because the execution and ownership boundaries are different. Chapters 4–6 will open the cache, scheduling and overlap boxes in the same map.

1.7 Experiments to try next

Keep the original scripts and saved outputs as a reference. Make each change in a scratch copy, and update its assertions when you deliberately change the expected result.

1. Change a score so that the first program repeats `steps`. Does it reach the end marker, or stop at the output limit?

2. Change the prefix-sum prompt and generate five integers. Check that both versions agree. Predict their addition counts before running them.

3. In the window experiment, use width four and a two-integer prompt. Rebuild after the first output and again after several outputs. Does each recovered window predict the same next integer after processing the latest output?

4. Give A and B the same length in the batching experiment. Does refilling still reduce the number of rounds?

5. Give C a later arrival time. Which part of the simulator must change? Is an empty slot always evidence of a bad scheduling decision?

6. Follow the printed worker branch with ZOS disabled, then enabled for decode. Which step's outputs can the caller receive? Explain what final flush consumes in each case.

7. Suppose the entry point keeps raw output addresses instead of cloning. Locate the operation that invalidates that choice. Why does keeping the addresses in a Python list not fix it?

Chapter 2

Where the time goes

Can a slower generation step produce more tokens per second? In chapter 1, each simulated round cost one time unit. A real batch does not have a fixed price: more requests mean more computation and more data to read. Yet some work can serve several requests together. We will follow DeepInfer's scheduled work into its operation estimate, then trace how its benchmark turns observed step returns into a reported token rate.

The question

If we put four times as many requests in a batch, how much longer can the step take before throughput falls? Which delay would an individual reader notice?

Run the next experiment and compare batches of 16 and 64:

```
python3 labs/05_cost.py
```

The script calculates costs from hypothetical rates; it does not time Python or a device. For now, each request produces one useful output token per step. Here are selected rows from its sweep:

```
Hypothetical model; times in microseconds
R=N  transfer  compute     step   tokens/s
  1      41.0      2.0     61.0      16393
  4      44.0      8.0     64.0      62500
  8      48.0     16.0     68.0     117647
 16      56.0     32.0     76.0     210526
 32      72.0     64.0     92.0     347826
 40      80.0     80.0    100.0     400000
 64     104.0    128.0    148.0     432432
128     168.0    256.0    276.0     463768
```

A batch of 16 takes 76 microseconds; a batch of 64 takes 148. Four times as many tokens cost less than twice as much time, so the aggregate output rate rises from about 211,000 to 432,000 tokens per second. But a request that stays in the larger batch waits 148 microseconds between steps instead of 76. Higher throughput and a longer token interval can occur together. The answer to the opening question is therefore: when useful output grows fourfold, throughput does not fall provided step time grows by at most fourfold.

These are two questions about the same work. *Throughput* counts useful output across requests per unit time. *Latency* measures how long a particular event takes for a request. We need a cost model to explain the first result, then a timeline to decide which latency we mean.

2.1 Give each cost a unit

Imagine a step that reads a shared array once and also reads some saved state for each request. The shared array could contain model parameters: values reused when predicting for different users. We assign four million shared bytes, another 100,000 context bytes per request and two million arithmetic operations per evaluated position. The model gives the device separate limits for memory transfer and arithmetic, plus a fixed cost that runs before either.

The constants and the complete cost function are small enough to inspect:

Listing 2.1: A hypothetical cost model with explicit units.
Book lab · `labs/05_cost.py` · lines 13–27

```
SHARED_BYTES = 4_000_000
BYTES_PER_REQUEST = 100_000
OPS_PER_POSITION = 2_000_000
BANDWIDTH = 100_000_000_000  # bytes/s; hypothetical
COMPUTE_RATE = 1_000_000_000_000  # operations/s; hypothetical
OVERHEAD = 20e-6  # seconds; serial in this model


def cost(requests, positions, context_scale=1, compute_rate=COMPUTE_RATE):
    traffic = SHARED_BYTES + requests * BYTES_PER_REQUEST * context_scale
    operations = positions * OPS_PER_POSITION
    memory_s = traffic / BANDWIDTH
    compute_s = operations / compute_rate
    step_s = OVERHEAD + max(memory_s, compute_s)
    return memory_s, compute_s, step_s
```

Let R count active requests and N count positions evaluated in this step. The transferred bytes and operation count are

$$M(R) = 4{,}000{,}000 + 100{,}000R, \qquad F(N) = 2{,}000{,}000N.$$

Ordinary decode evaluates one position per request, so $N = R$. Verification can evaluate several positions for each request: 16 requests with six positions each give $R = 16$, $N = 96$. Those positions use the same saved context. We assume that each request's context is read once and reused across its positions, while arithmetic grows with N. This isolates the benefit of reuse; traffic for new positions and intermediate arrays, repeated reads inside kernels and changes in work with context length require a richer model.

Dividing an amount of work by a rate gives a time. With memory bandwidth $W = 10^{11}$ bytes per second and arithmetic rate $P = 10^{12}$ operations per second,

$$T_{\text{memory}}(R) = \frac{M(R)}{W}, \qquad T_{\text{compute}}(N) = \frac{F(N)}{P}.$$

For $R = N = 16$, transferring 5.6 million bytes takes 56 microseconds at the assigned rate. Performing 32 million operations takes 32 microseconds. A microsecond is 10^{-6} seconds; a millisecond is 10^{-3} seconds. Keeping seconds inside the function makes the final division into tokens per second straightforward.

If computation and memory transfer overlap perfectly, their combined duration is the larger of these two times. The script also assigns a separate serial overhead $h = 20$ microseconds, so its step model is

$$T(R, N) = h + \max\left(\frac{M(R)}{W}, \frac{F(N)}{P}\right). \tag{2.1}$$

The `max` explains the 76-microsecond result: $20 + \max(56, 32) = 76$. The smaller cost has not disappeared. Under this overlap assumption, it finishes before the larger one.

2.1.1 A bound is not a stopwatch

If W and P are upper limits on achievable rates, then $\max(M/W, F/P)$ is a lower bound for the time needed to move those bytes and perform those operations. It becomes the script's predicted duration only because we assume perfect overlap and attainment of both rates. Adding h also assumes that this overhead cannot overlap with either activity.

Actual work can violate those scheduling assumptions. Some operations depend on data that has not arrived; small device programs, called *kernels*, may not reach the assigned rates; several stages may each have a different bottleneck. Conversely, host work counted in h may overlap with device work. Do not add the maximum of two whole-step totals and expect it to describe every dependency inside the step. Chapter 6 will give those dependencies their own timeline.

The model tells us which assumption to investigate. An arithmetic optimisation can leave a transfer-limited step unchanged because arithmetic does not determine its duration.

2.2 Find what batching shares

First keep $N = R$, as in the opening sweep. At small R, the four-million-byte shared read dominates the memory cost. More requests spread that read, and the fixed overhead, across more useful outputs. At larger R, the arithmetic cost catches up. The crossover occurs where $40 + R = 2R$, with both times in microseconds: $R = N = 40$. Figure 2.1 plots both costs and the resulting output rate.

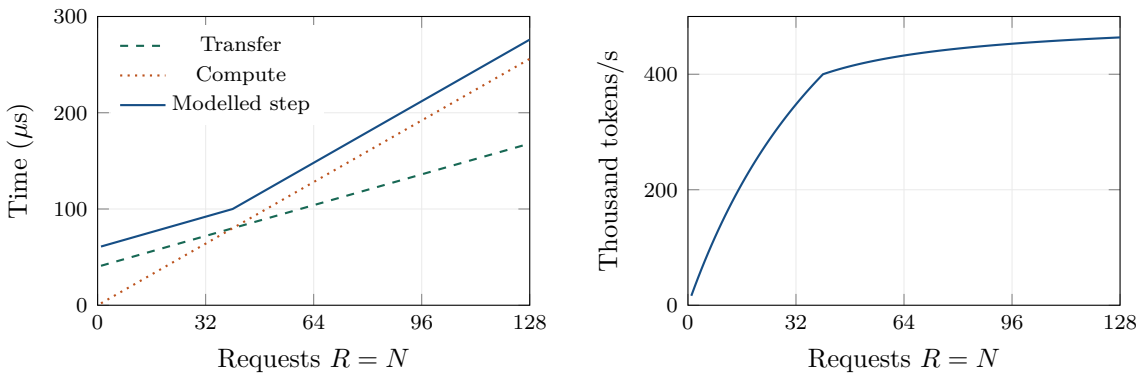


Figure 2.1: Calculated from the hypothetical model in `labs/05_cost.py`. Both panels use ordinary decode, with one evaluated position and one useful token per request per step ($N = R$). The left shows the competing costs; the right shows output throughput. Both plots read the CSV saved by the experiment.

For $R > 40$, equation 2.1 becomes $T(R, R) = 20 + 2R$ microseconds. The output rate is therefore

$$\frac{R}{(20 + 2R)\,10^{-6}} \text{ tokens per second.}$$

It approaches 500,000 tokens per second as the fixed cost becomes a smaller fraction of each step. Latency keeps growing. There is no single best batch size until we also state a latency limit, memory capacity and available work.

This reasoning is the basis of the *roofline* model: arithmetic throughput is bounded by the smaller of compute capacity P and bandwidth times *arithmetic intensity*, $I = F/M$, measured in operations per byte. The original paper calls this *operational intensity* and counts traffic between cache and main memory [18]. Here the crossover has $I = P/W = 10$ operations per byte: 80 million operations divided by eight million bytes. The roofline bound excludes our separately added serial overhead h; our plots show step time and token rate, rather than a conventional roofline plot.

This saving relies on the shared read actually being shared. A *mixture of experts* (MoE) model routes positions to selected parameter arrays called experts. A larger batch can touch more experts, so the shared-byte term need not stay fixed. Chapter 7 will follow this cost across devices in Wide-EP. Similarly, a batch of longer contexts can require more saved-state traffic per request. The next two lines change one condition at a time:

```
One change at a time (microseconds)
R=N=16, double compute rate: 76.0 -> 76.0
R=N=32, four times context traffic: 92.0 -> 188.0
```

Doubling the arithmetic rate at $R = N = 16$ reduces its compute term from 32 to 16 microseconds. Transfer still takes 56, so the step remains 76. Multiplying the per-request traffic by four at $R = N = 32$ raises the transfer term from 72 to 168 microseconds, and the step grows from 92 to 188. The argument for batching survives; the cost curve changes with the workload.

2.2.1 More positions can reuse the same context

Now hold the request count at 16 and increase only the evaluated positions. The experiment keeps transfer at 56 microseconds:

```
Same 16 requests, more positions (microseconds)
 R   N  transfer  compute     step
16  16      56.0     32.0     76.0
16  28      56.0     56.0     76.0
16  96      56.0    192.0    212.0
```

At $N = 28$, compute catches up with transfer, yet the step still takes 76 microseconds. The extra 12 positions add no step time under our assumptions. At $N = 96$, compute takes 192 microseconds and the step takes 212. Reusing context makes some verification work cheap; it does not make unlimited work free. Nor are evaluated positions necessarily useful output tokens.

2.2.2 Bytes per element are part of the model

An array's element count does not yet tell us its byte count. A conventional 32-bit value occupies four bytes; a 16-bit value occupies two. Lower-precision formats can use fewer bits for values while storing extra information needed to interpret groups of them. Allocation alignment can add further padding.

For example, an invented format holding 32 one-byte values plus one two-byte scale occupies 34 bytes, or 1.0625 bytes per value. Multiplying the element count by one would miss the scale. The example illustrates accounting, not a particular DeepInfer layout.

The deployment Q&A gives a concrete unit for later chapters: 528 bytes per token per stored layer for its MXFP8 sliding-window attention KV state [2]. That is a layout size, not a whole-model per-token budget. We will count layers and retained positions when we

derive memory requirements. For a traffic estimate, we must also establish how often those bytes are read. Bytes retained in memory and bytes transferred during a step are different quantities.

2.3 Count useful output separately

In the ordinary-decode sweep, $N = R$ evaluated positions produce R useful tokens. Verification breaks this equality. A later chapter will let a cheaper predictor propose tokens that the main model checks; some checked positions may not become output. Let $U(R, N)$ be the useful output from a step. The rate we want is

$$\text{useful throughput} = \frac{U(R, N)}{T(R, N)}. \tag{2.2}$$

For repeated steps of different sizes, measure total useful tokens divided by total elapsed time. Averaging individual step rates without weighting their durations answers a different question.

The cost script finishes with two assigned useful-output counts for the same 16 requests, evaluating either 32 or 64 positions:

```
Assigned useful-output counts; rate = U / step_s
R=16, N=32, U=24: 84 us, 285714 useful tokens/s
R=16, N=64, U=28: 148 us, 189189 useful tokens/s
```

The second choice produces more useful tokens per step, 28 instead of 24, but its time grows from 84 to 148 microseconds. Both choices read the same context; the additional arithmetic causes the increase. The useful rate falls. These assigned counts are an arithmetic exercise; chapter 8 will derive expected output from acceptance probabilities. Its policy will need both that numerator and a cost curve for the denominator.

This distinction already matters in DeepInfer. The deployment Q&A describes DSpark allocating verification work using predicted acceptance and a profiled hardware throughput curve. Its table estimates a change from fixed-5 to variable-length verification of -1.5% at 16 concurrent requests per rank, but $+11\%$ at 96 [2]. Our model suggests a reason for this trend: at low concurrency, removing positions may save little time because transfer still dominates; as concurrency grows, reducing verification arithmetic can shorten a compute-limited step. This is an interpretation, not a reproduction of those figures. Chapter 8 will combine acceptance with measured costs; the hypothetical constants cannot choose a production budget.

2.4 Read a request timeline

A step duration does not say how long a user waits for the first token. The next experiment reads `labs/data/06_request_trace.json`, a saved teaching trace with assigned timestamps. All events use the same observation clock and all times are in milliseconds. The records specify arrival, the start and end of prompt processing, and the delivery time of every output token.

```
python3 labs/06_trace.py
```

Before running it, consider request E. It arrives at 3, starts prompt processing at 30, ends that span at 42, and delivers tokens at 45, 49 and 65. How much of its first-token wait occurs before prompt work starts? Figure 2.2 marks the boundaries the script uses.

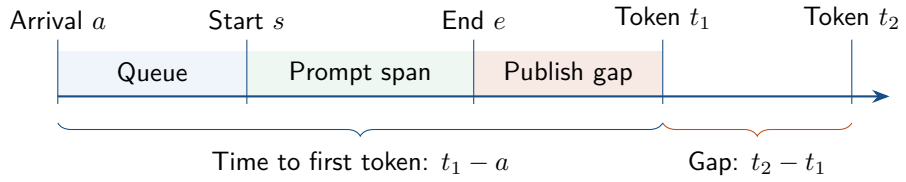


Figure 2.2: The observation boundaries used by the trace experiment. All events use one clock. Widths are schematic; the first-token wait and later token gaps measure different intervals.

For arrival a, prompt start s, prompt end e and token delivery times $t_1, t_2, \ldots$, the script computes

$$Q = s - a, \qquad S = e - s, \qquad D = t_1 - e,$$

$$\mathrm{TTFT} = t_1 - a = Q + S + D, \qquad \Delta_i = t_i - t_{i-1} \quad (i \geq 2).$$

Time to first token (TTFT) includes queueing in this definition. The *time between tokens* is a sequence of gaps Δ_i, rather than one number. We call D the *publish gap* in this book; DeepInfer does not use that name. It covers everything between the recorded end of prompt processing and the first delivery, and need not be one operation.

These boundaries are chosen for the experiment. In a service, arrival might mean receipt by a gateway, admission to an engine or transmission from a client. Token delivery might mean an engine callback or data reaching the client. Those choices include different queueing, transfer and buffering costs. Write them down before comparing two measurements. Real records from several clocks also need clock alignment before we can subtract their timestamps.

Here is the complete calculation for one request:

Listing 2.2: Separate first-token waiting from later token gaps.
Book lab · `labs/06_trace.py` · lines 14–23

```
14 def analyse(request):
15     arrival, start, end = (request[key] for key in ("arrival", "prompt_start", "
       ↪ prompt_end"))
16     tokens = request["tokens"]
17     assert tokens and arrival <= start <= end <= tokens[0]
18     assert tokens == sorted(tokens)
19     queue, prompt, publish = start - arrival, end - start, tokens[0] - end
20     ttft = tokens[0] - arrival
21     gaps = [later - earlier for earlier, later in zip(tokens, tokens[1:])]
22     assert ttft == queue + prompt + publish
23     return queue, prompt, publish, ttft, gaps
```

The saved output includes all five requests:

```
Synthetic teaching trace: assigned timestamps, not engine measurements
All times in ms; gaps exclude the wait for the first token
id  queue  prompt  publish  TTFT  gaps
A       2       6        2    10  4,4,4
B       2       6        2    10  -
C       8       8        3    19  4,4
D       8       4        3    15  4
E      27      12        3    42  4,16
TTFT: n=5, mean=19.2, p50=15, p95=42
Token gaps: n=8, mean=5.5, p95=16
Whole trace: 13 tokens / 65 ms = 200.0 tokens/s
```

E waits 27 milliseconds before prompt processing begins, then spends 12 in the prompt span and three in the publish gap. Its TTFT is 42 milliseconds. Its later gaps are four and 16 milliseconds. Shortening the first-token queue would not, by itself, remove that last pause. Conversely, reporting only its later token gaps would hide the long initial wait.

A faster kernel can leave first-token latency almost unchanged when the request spends most of its time waiting to run. Suppose an illustrative request waits 30 milliseconds, then takes 10 milliseconds to reach its first token. Halving all of that latter work reduces TTFT from 40 to 35 milliseconds, not to 20. The timeline tells us which part an optimisation can shorten.

The prompt span measures elapsed time between two markers, not pure arithmetic. If a real request's prompt work is chunked and interleaved with other requests, that span can include intervening waits. Likewise, device execution, host processing and delivery can overlap across requests. Adding every recorded operation duration is not generally a way to recover wall-clock time.

2.4.1 Look beyond an average

The five TTFTs have a mean of 19.2 milliseconds, but the slowest request waits 42. The script also reports sample percentiles. It sorts the values and chooses the element at rank $\lceil pN \rceil$, counting ranks from one:

Listing 2.3: The nearest-rank convention used for sample percentiles.
Book lab · `labs/06_trace.py` · lines 26–28

```
26 def percentile(values, fraction):
27     """Nearest-rank percentile of a non-empty sample, for 0 < fraction <= 1."""
28     return sorted(values)[ceil(fraction * len(values)) - 1]
```

The sample median, or p50, is 15 milliseconds. With only five values, p95 is the maximum, 42. This tiny example shows the calculation; it cannot estimate a production tail reliably. Real comparisons need enough requests, a workload description and a stated percentile convention.

There are eight later-token gaps, with mean 5.5 milliseconds and p95 of 16. Request B has only one output token, so it contributes no gap. Substituting zero would invent an observation. Pooling all gaps also weights requests with more output tokens more heavily than requests with fewer; averaging one mean per request is another convention.

Finally, the script counts 13 output tokens over the 65 milliseconds from the first arrival to the last delivery: 200 tokens per second. This whole-trace rate includes the initial waits

and final completion. It is not a steady-state decode rate. An observation window chosen to include only busy steps would have a different denominator.

2.5 Trace work and measurements through DeepInfer

Chapter 1 followed requests through admission, scheduling and execution. We now follow the quantities attached to that work: selected positions, estimated operations, observed intervals and reported throughput. Figure 2.3 adds two paths around the same engine. Keeping them separate prevents a common mistake: treating a count of operations as if it were a measurement of the time taken to execute them.

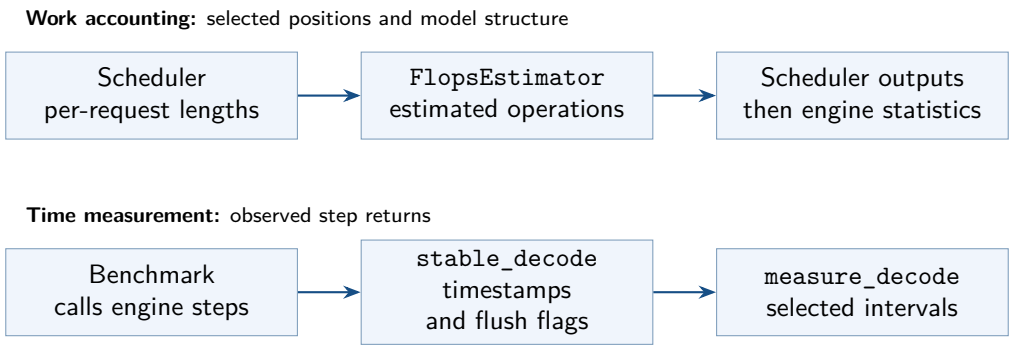


Figure 2.3: Two accounting paths around the engine in Figure 1.4. The upper path estimates scheduled work; the lower observes elapsed time. The benchmark's token-rate formula uses the lower path plus request and draft counts, not the FLOPs estimate [8, 14, 5, 15].

2.5.1 From scheduled positions to an operation count

The C++ scheduler constructs per-request metadata after selecting a batch. Inside its loop over selected sequences, it obtains two lengths:

Listing 2.4: Scheduled work combines cached, pending and new positions.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 696–697

```
696 int seqlen_q = scheduler_outputs.seqlens_q[i];
697 int seqlen_k = seq->cache.num_tokens + seq->schedule.total_inflight_tokens +
      ↪ seqlen_q;
```

Here `seqlen_q` counts the new positions selected for this request. `seqlen_k` extends the cached prefix by pending positions and the new work. For an assigned example with 128 cached positions, two in flight and three selected, the metadata has $q = 3$, $k = 133$. It does not say that all 133 positions have completed. The pending term connects directly to the delayed worker state from chapter 1.

Across requests, the sum of selected q values plays the role of N in our cost model; the number of selected requests plays the role of R. Two batches with the same N can have different request counts and context lengths. The teaching model represents that difference with context bytes per request. DeepInfer retains individual lengths and model structure rather than reducing every step to one batch-size scalar.

When statistics are enabled, the same metadata-construction loop calls an estimator and adds its two results to the batch totals:

Listing 2.5: Estimate work from the lengths already chosen by scheduling.
DeepInfer · csrc/engine/scheduler.hpp · lines 712–719

```
712 if (log_stats) {
713     const bool is_prompt = scheduler_outputs.prompt_run;
714     const int full_len = seq->input.tokens.get_len();
715     const auto [attn, total] =
716         flops_estimator->calc_flops(seqlen_q, seqlen_k, is_prompt, full_len,
    ↪ is_prompt ? seq->cache.swa_valid_start : 0);
717     attn_flops += attn;
718     total_flops += total;
719 }
```

The call's extra arguments describe prompt extent and which sliding-window state is valid. Chapters 3 and 9 will explain why these boundaries affect the model's work. For now, follow the returned pair into the estimator itself:

Listing 2.6: Separate attention work while including it in the total.
DeepInfer · csrc/engine/flops_estimator.hpp · lines 37–45

```
37 // Return (attention, total); total already includes attention. Scheduler
38 // calls this directly from its existing per-request metadata construction.
39 std::pair<int64_t, int64_t> calc_flops(int q, int k, bool is_prompt, int
    ↪ full_len, int swa_valid_start = 0) const {
40     const auto [full, consumer] = query_ranges(q, k, is_prompt, full_len,
    ↪ swa_valid_start);
41     const int64_t attn = calc_attn_flops(full, consumer, is_prompt);
42     const int64_t non_attn = calc_non_attn_flops(full.rows(), consumer.rows());
43     const int64_t logits = calc_logits_flops(q, is_prompt);
44     return {attn, attn + non_attn + logits};
45 }
```

The method derives row ranges, counts attention, counts other model operations and counts the projection that produces token scores, called logits. Its return value is $(A, A + O + L)$, not three independent counters. Adding its two returned numbers would count attention twice. The two row ranges accommodate differences between layers; chapter 9 explains the full and consumer row ranges when it counts truncated prefill.

Back in the scheduler, lines 730–731 divide the accumulated counts by 10^{12} and store them in `attn_tflops` and `total_tflops`. Despite their names, these fields hold tera-operation *amounts* for the scheduled batch, not operations per second. The engine copies them into `StepStats` during post-processing. No elapsed time appears in the estimator call or that unit conversion [8, 14, 5].

Notice the direction of the dependency: selected lengths feed this logging branch. This call is not choosing the verification budget. DSpark's documented use of a profiled cost curve is a different path, which chapter 8 will trace. The distinction matters when locating code to investigate a scheduling choice.

2.5.2 An operation estimate has a scope

A floating-point operation is often abbreviated to *FLOP*. Counting a multiply and an addition as two operations is a common convention. DeepInfer's estimator goes further: it assigns format-dependent weights to express work in FP8-equivalent FLOPs.

Listing 2.7: Precision weights and the estimator's stated scope.
DeepInfer · csrc/engine/flops_estimator.hpp · lines 13–27

```
13 // GEMM coefficients include multiply-add (2 FLOPs) and the FP8-equivalent
   ↪ weight.
14 namespace flops_coef {
15 constexpr int64_t FP4 = 1;
16 constexpr int64_t FP8 = 2;
17 constexpr int64_t BF16 = 4;
18 constexpr int64_t TF32 = 8;
19 constexpr int64_t FP32 = TF32; // Accounting reference, not actual kernel
   ↪ precision.
20 } // namespace flops_coef
21
22 // Estimate main-model work in FP8-equivalent FLOPs; FP32/TF32 use weight 4.
23 // Count major contractions regardless of bottleneck; exclude draft/padding/
   ↪ replicas.
24 // MFU measures target-model FLOPs throughput; DFlash is extra compute used
25 // to accelerate that model, so its FLOPs are excluded.
26 // Omit small arithmetic (norms/activations). TODO: vision.
27 // Scheduled q/k ignore acceptance/ZOS corrections.
```

The comments use GEMM for matrix multiplication and MFU for model FLOPs utilisation, which compares the model's arithmetic rate with a compute-capacity reference.

FP8 denotes an eight-bit floating-point format family; FP4 uses four-bit values, and BF16 is a sixteen-bit format. The coefficients here describe normalised arithmetic accounting, not bytes per element. The factor for a multiply-add is already included: the FP8 coefficient is two and the BF16 coefficient is four. The comment's FP32/TF32 precision weight of four likewise becomes the code's coefficient of eight after multiplication by two. Dividing this estimate by an arbitrary device's advertised operation rate would mix conventions unless the rate uses compatible weights and scope [14].

The comments also exclude draft-model work, padding and replicas, and say that scheduled lengths have not incorporated acceptance or ZOS corrections. The q and k values we traced are therefore scheduling-time inputs to an accounting estimate, not a complete record of what every kernel finally ran. A higher main-model FLOPs rate does not prove that a request completed sooner, especially if extra work outside that count was used to obtain it.

2.5.3 A benchmark chooses its observation window

Now follow the lower path in Figure 2.3. The benchmark drives the same `LLMEngine.step` that chapter 1 inspected, but it controls when step returns enter a measurement window. A bounded queue stores timestamps and flags:

Listing 2.8: Reserve one more endpoint than the number of desired intervals.
DeepInfer · `examples/benchmark.py` · lines 380–381

```
380 stable_prefill: deque[tuple[int, float, bool]] = deque(maxlen=
    ↪ prefill_measure_steps + 1)
381 stable_decode: deque[tuple[float, bool]] = deque(maxlen=decode_measure_steps +
    ↪ 1)
```

`decode_measure_steps` is 60 in this edition, so the decode queue holds 61 endpoints. Adjacent endpoints define 60 intervals. This is a different use of a queue from the scheduler's `waiting`: it stores observations, not requests available to run.

The surrounding control flow separates unmeasured work from collection:

Listing 2.9: Run two unmeasured segments before filling the decode window.
DeepInfer · `examples/benchmark.py` · lines 499–510

```
499 for _ in range(decode_warmup_steps):
500     if not llm.llm_engine.has_unfinished_requests():
501         break
502     run_decode_step(measure=False)
503 set_engine_log_interval(20)
504 for _ in range(decode_measure_steps):
505     if not llm.llm_engine.has_unfinished_requests():
506         break
507     run_decode_step(measure=False)
508 set_engine_log_interval(0)
509 while len(stable_decode) < stable_decode.maxlen and llm.llm_engine.
    ↪ has_unfinished_requests():
510     run_decode_step(measure=True)
```

The first loop performs up to 50 warm-up steps. The next performs up to 60 unmeasured steps with a different logging interval. Only the final loop calls `run_decode_step(measure=True)`. Filling 61 entries can require more than 61 calls because the inner function also checks the running-request count. Every loop can stop early when no unfinished requests remain. A separate profiler segment follows this window; its calls use `measure=False`.

The benchmark records a host timestamp after `step()` returns, and only when the running-request count matches its requested decode concurrency:

Listing 2.10: Select step endpoints for the decode window.
DeepInfer · `examples/benchmark.py` · lines 435–440

```
435 def run_decode_step(*, measure: bool) -> None:
436     countdown_before = lb_flush_countdown() if measure else None
437     llm.llm_engine.step()
438     if measure and len(llm.llm_engine.scheduler.running) == args.
    ↪ decode_num_requests:
439         flushed = countdown_before is not None and lb_flush_countdown() >
    ↪ countdown_before
440         stable_decode.append((time.perf_counter(), flushed))
```

`flushed` here tracks expert-load-balancing statistics flushes. It does not mean the final in-flight-request flush from chapter 1. The interval calculation filters out steps marked by those statistics flushes:

Listing 2.11: Steps per second over selected intervals.
DeepInfer · `examples/benchmark.py` · lines 399–402

```
399 clean = [ts - prev[0] for prev, (ts, flushed) in pairwise(stable_decode) if not
        ↪ flushed]
400 if not clean:
401     return {}
402 avg_sps = len(clean) / sum(clean)
```

The numerator is the number of selected intervals; the denominator is their summed duration. The timestamps are qualifying step-return boundaries, not per-request token arrivals. If an intervening call fails the concurrency check, it creates no entry, but its elapsed time can still lie between stored endpoints. There is no separate counter of those omitted calls in `stable_decode`. The selected-window rate is most straightforward when concurrency stays stable, which is what the benchmark arranges its workload to study [15].

2.5.4 Follow one interval through the filter

Use four assigned endpoint times to read the filter. These milliseconds are illustrative inputs, not benchmark results; the implementation stores seconds.

Endpoint (ms)	Flush flag	Interval (ms)	Treatment
0	False	—	First endpoint only
4	False	4	Keep
10	True	6	Exclude
15	False	5	Keep

`pairwise` forms adjacent pairs before the comprehension applies its filter. The flag at 10 excludes the interval from 4 to 10; the next pair still starts at 10. The result is $2/(0.004 + 0.005)$, about 222.2 selected intervals per second. Removing the flagged endpoint first would incorrectly join 4 to 15, giving $2/0.015$, about 133.3. The order of pairing and filtering is part of the metric's definition.

The source detects a statistics flush when its countdown increases across the step. This protects the chosen comparison from that maintenance cost; it does not make the cost disappear from service execution. If no eligible intervals remain, `measure_decode` returns an empty dictionary. A caller must not interpret that as a measured zero-throughput run.

2.5.5 From a local step rate to a reported token rate

The output dictionary converts that step rate into a token rate with this line:

Listing 2.12: The reported decode-token formula; enclosing dictionary omitted.
DeepInfer · `examples/benchmark.py` · line 408

```
408 "decode_tokens_per_s_per_gpu": avg_sps * args.decode_num_requests * (envs.
        ↪ DEEPINFER_USE_NEXTN_DECODE + 1) / cp_size,
```

The source itself explains how to interpret and convert this metric. Here are the complete comments on lines 405 and 407 (author's translations; `nextn` is the configured draft length):

> "With VARLEN=0, each step always submits 1 + `nextn` tokens, so it is independent of acceptance rate." (Line 405, attached to `decode_avg_sps`.)
>
> "Full-acceptance accounting: count 1 + `nextn` tokens per request per step; to use a different τ, recalculate from `decode_avg_sps`." (Line 407.)

Here τ is the average useful output per request per step. The fixed submission size is the premise for holding step rate fixed while changing that output count. It is not a guarantee that every other runtime cost is invariant.

For an illustrative `cp_size=1`, 16 requests, 200 steps per second and five draft tokens, it reports $200 \times 16 \times 6 = 19{,}200$ tokens per second. Using $\tau = 3$ at that same step rate gives useful throughput of 9,600. This is the conversion the comment describes. With `VARLEN=1`, acceptance affects the number of submitted positions and hence can change step rate, as the comment on line 539 also warns. We must measure that policy's time and useful output together, rather than reuse the fixed-length timing.

The denominator `cp_size` also needs a scope. The launcher runs a separate request batch for each data-parallel (DP) rank, and only rank 0 writes the metrics. Each rank uses `cp_size` devices for context parallelism (CP), which shares its context work across those devices. Thus this "per GPU" metric divides rank 0's local request throughput by its CP device count; it is not a sum over all ranks divided by all devices. Here `ep_size` is the total device count and `dp_size` the number of request-serving ranks, with `ep_size` = `dp_size` × `cp_size`. Equal rates across ranks would make the two averages agree, but this reporting code does not check that condition [15, 16, 17]. Preserve useful output counts and elapsed times for every rank needed in a system-wide comparison.

`measure_decode` returns its dictionary to the rank-0 reporting block. That block formats the console result and can save configuration together with the measured fields:

Listing 2.13: Persist the selected metrics with the configuration that defines them.
DeepInfer · `examples/benchmark.py` · lines 554–559

```
554 if args.result_jsonl is not None:
555     result = benchmark_config | prefill_metrics | decode_metrics
556     args.result_jsonl.parent.mkdir(parents=True, exist_ok=True)
557     with args.result_jsonl.open("a", encoding="utf-8") as f:
558         f.write(json.dumps(result, default=str) + "\n")
559     print(f"Benchmark result appended to {args.result_jsonl.resolve()}")
```

The saved decode fields include the selected interval count, step rate and maximum selected interval, plus the full-acceptance token rate. They do not contain the raw endpoint queue or observed useful token counts. Recomputing a rate under another assumed τ is possible; auditing a different interval filter or the realised useful output requires additional records. This follows from the dictionary being written, not from the file's JSON format.

We can now locate three different changes. To change which calls count as measurement, inspect the window loops and `run_decode_step`. To change the exclusion rule, inspect the `pairwise` comprehension. To replace full-acceptance accounting with useful throughput, obtain useful counts over the same time window before changing the reporting formula. Editing the FLOPs estimator would address none of those three questions.

A cost curve belongs to a hardware setup

The deployment Q&A puts support for multiple accelerator vendors among its system goals. The inspected release supports NVIDIA SM100 and Huawei Ascend 950; its README limits batch-invariant inference to NVIDIA [1, 2].
Record the platform, numerical formats, kernels, parallel layout and measurement boundary with a curve. A result on one setup is not a rate constant for another. The model above can organise either investigation, but its parameters must come from the setup being studied.

The next chapter opens the context-byte term in this model. It explains what the cached rows contain, which reads remain after caching, and how DeepInfer's attention layout determines their size.

2.6 Experiments to try next

1. Impose a 150-microsecond step limit in the hypothetical model. Which integer request count, with $N = R$, gives the highest output rate within it? What changes if per-request traffic is multiplied by four?

2. Halve the fixed overhead, then halve the arithmetic cost. Compare the effects at $R = N = 4$ and $R = N = 64$. Which assumption makes each change useful? Then hold $R = 16$ and find the largest N that adds no time over ordinary decode.

3. In a scratch copy of the trace, delay all of E's token deliveries by ten milliseconds without moving its prompt markers. Which of TTFT, publish gap, later-token gaps and whole-trace throughput change?

4. For a benchmark report, write down the numerator, the time endpoints and any excluded intervals. Which additional raw count would let you turn a full-acceptance rate into an observed useful-output rate?

5. In the assigned endpoint table, mark the endpoint at 15 as a flush too. Which interval survives? Then mark the endpoint at 4 as a flush: which branch of `measure_decode` runs, and what should a report say?

6. Locate the logging branch that calls `calc_flops`. Explain why reducing its estimate alone cannot change the number of positions selected for verification or the benchmark's reported token rate.

Chapter 3

The model shapes the engine

Chapter 2 counted context bytes as an input to its cost model. What do those bytes contain, and why must the engine keep reading them? Chapter 1 saved a running sum whose state fitted in one number, however long the input became. Attention keeps rows of numbers, and a new query can read many of them. What does caching save if the engine still has to read the past?

The question

Which work disappears when we cache keys and values? Which work remains? What changes if each query can see only a fixed window?

Run a small attention calculation:

```
python3 labs/07_attention.py
```

The inputs are five vectors, each containing two numbers. The script first rebuilds the calculation for every growing prefix, then repeats it while keeping keys and values. Its first comparison reports:

```
full: cached outputs match every recomputed prefix
projected rows: recompute=15, cache=5
query/key pairs: recompute=35, cache=15
retained KV rows: 5
last output: 2.790791, 0.930264
```

The cached calculation projects five input rows instead of fifteen. It also avoids recalculating the outputs for old positions. But it still evaluates fifteen query/key pairs: one for the first position, two for the second, and so on up to five. Caching removes repeated work on the past; the current query still consults that past.

We will make those rows visible before opening DeepInfer's model code. The source walk then follows one layer with sliding-window attention (SWA) from its input to a cache write and an attention read. Along the way, three different quantities will emerge: the number of visible positions, the number of allocated positions and the bytes needed to represent one position.

3.1 A query chooses a mixture

A vector is an ordered list of numbers. Its *dot product* with another vector of the same length multiplies corresponding entries and adds the products: $[1, 2] \cdot [3, 4] = 1 \cdot 3 + 2 \cdot 4 = 11$. Attention uses such a score to decide how much one position should contribute to another.

Each input row produces a *query*, a *key* and a *value*. The query describes what the current position seeks. Each key supplies a score against that query. The values supply the numbers that will be mixed. Our lab uses simple fixed projections:

Listing 3.1: Three roles for one input row.
Book lab · `labs/07_attention.py` · lines 12–14

```
12 def project(x):
13     a, b = x
14     return (a, b), (a - b, a + b), (a + 2 * b, 2 * a - b)
```

In a trained model, learned arrays determine these transformations. Here the small formulas let us inspect every entry:

```
position | input = query | key     | value
       0 | (1, 0)        | (1, 1)  | (1, 2)
       1 | (0, 1)        | (-1, 1) | (2, -1)
       2 | (1, 1)        | (0, 2)  | (3, 1)
       3 | (2, 0)        | (2, 2)  | (2, 4)
       4 | (0, 2)        | (-2, 2) | (4, -2)
```

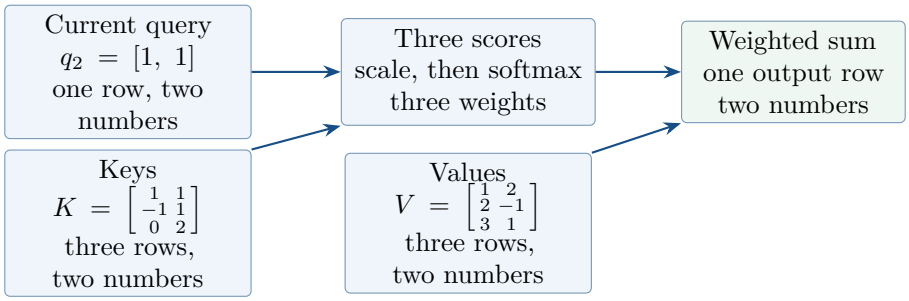


Figure 3.1: The lab at position 2. Each key earns one weight; that weight scales the corresponding value row. The output width comes from the values.

At position 2, the query is $[1, 1]$. It can see keys at positions 0, 1 and 2. Their dot products are $2, 0, 2$. Divide each by $\sqrt{2}$, where two is the query width. This scaling is part of scaled dot-product attention [19]. We then turn the scores into positive weights that sum to one. The operation is called *softmax*: exponentiate each score and divide by the sum of the exponentials. A larger score gets a larger share.

For scores s_j, the weight of position j is

$$a_j = \frac{\exp(s_j - m)}{\sum_r \exp(s_r - m)}, \qquad m = \max_r s_r.$$

Subtracting the same maximum leaves the mathematical ratios unchanged and keeps the largest exponent at zero. Each output entry is a weighted sum of the corresponding entries in the value rows:

Listing 3.2: Scores become weights; weights mix value rows.
Book lab · `labs/07_attention.py` · lines 17–26

```
def attend(query, keys, values):
    scores = [sum(a * b for a, b in zip(query, key)) / sqrt(len(query))
              for key in keys]
    peak = max(scores)
    masses = [exp(score - peak) for score in scores]
    total = sum(masses)
    weights = [mass / total for mass in masses]
    return tuple(
        sum(weight * value[d] for weight, value in zip(weights, values))
        for d in range(len(values[0])))
```

In Figure 3.1, one query row meets three key rows. The resulting three weights multiply three value rows, producing one output row. A matrix is simply a rectangular array of such rows. For N queries of width d, L keys of width d, and values of width d_v, the shapes are

$$Q:\ N \times d, \quad K:\ L \times d, \quad V:\ L \times d_v, \qquad \text{output}:\ N \times d_v.$$

The familiar expression $\text{softmax}(QK^{\mathsf{T}}/\sqrt{d})V$ describes the same calculation for several query rows. The transpose turns each key row into a column so that matrix multiplication computes all the dot products.

3.2 Cache the past, calculate the present

Generation uses *causal* attention: position i can see positions 0 through i, including itself, but no later position. A later token therefore cannot change an earlier output in this layer. With fixed model parameters and unchanged earlier inputs, the earlier keys and values remain valid when another input arrives.

The incremental loop projects one new row, appends its key and value, and calculates only the new output:

Listing 3.3: Retain keys and values across queries.
Book lab · `labs/07_attention.py` · lines 41–52

```
def incremental(xs, window=None):
    keys, values, outputs = [], [], []
    pairs = 0
    for x in xs:
        query, key, value = project(x)
        keys.append(key)
        values.append(value)
        if window is not None:
            keys, values = keys[-window:], values[-window:]
        outputs.append(attend(query, keys, values))
        pairs += len(keys)
    return outputs, pairs, len(keys)
```

With `window=None`, the lists grow. Old queries are not needed to calculate the new output, but old keys and values are. In a causal stack of layers, the same argument applies layer by layer: each layer keeps its own state derived from that layer's inputs.

The lab's recomputation baseline projects every row and calculates every causal output in each prefix. Across prefixes of lengths 1 through 5, that is $1+2+3+4+5 = 15$ projected rows and $1+3+6+10+15 = 35$ query/key pairs. The cached version projects five rows and calculates fifteen pairs. These counts compare two specified algorithms; they are not instruction counts for an optimised kernel.

The script checks every output in every prefix with absolute and relative tolerances of 10^{-12}. The mathematical calculation is unchanged. The test permits floating-point rounding differences; it does not establish bitwise agreement between different device kernels.

For a long context of length L, one full-attention decode query still uses L keys and L values. Its score and mixture work grow with L, as does the amount of logical KV data it consumes. During prefill, many queries are available together, allowing kernels to reuse data across them. This is the distinction between requests R and evaluated positions N in chapter 2: several queries for one request can share its context. The number of logical references alone does not tell us how many bytes cross a particular memory interface.

3.2.1 Give each query a window

A sliding window of width W changes the visible range to

$$\max(0, i - W + 1), \ldots, i.$$

The current position occupies one of the W places. Figure 3.2 shows the resulting band of visible positions.

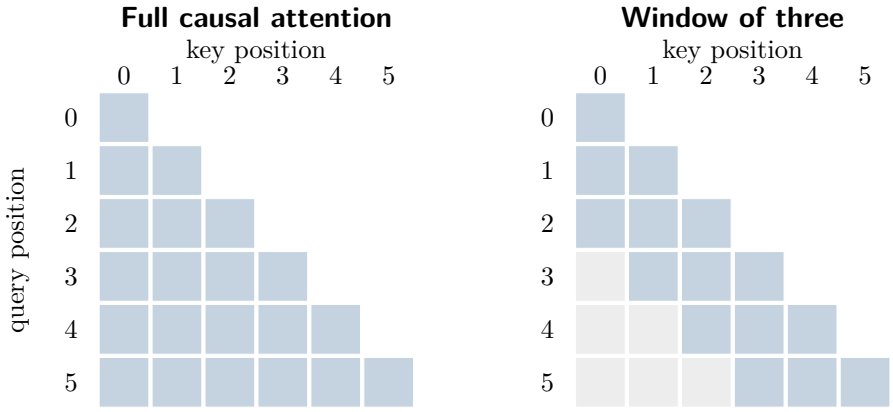


Figure 3.2: Blue cells contribute to the output. White cells are in the future; grey cells have left the window. A window of three includes the current position and at most two earlier positions.

The same incremental loop implements a window by retaining only its last W keys and values. For $W = 3$, the experiment reports:

```
window=3: cached outputs match every recomputed prefix
projected rows: recompute=15, cache=5
query/key pairs: recompute=31, cache=12
retained KV rows: 3
last output: 3.000000, 1.000000
```

After the window fills, each new query reads three rows. Across five queries, the pair count is $1+2+3+3+3 = 12$. The last output is $[3, 1]$, whereas full attention returned approximately $[2.790791, 0.930264]$. We can check the windowed result by hand. The final query is $[0, 2]$; the last three keys all have second component 2. Their dot products are therefore all 4, and their scaled scores are all $4/\sqrt{2}$. Softmax gives each row weight $1/3$. The corresponding values are $[3, 1]$, $[2, 4]$ and $[4, -2]$, whose average is exactly $[3, 1]$.

These are two separate comparisons. Caching agrees with recomputation *under the same visibility rule*. Restricting full attention to a window changes the rule and can change the output. Whether a model performs its task well with that restriction is a model-quality question; agreement between two implementations cannot answer it.

Chapter 1's last-three-numbers exercise recovered its state exactly from those raw inputs. This lab also projects each row directly from a fixed input. At a deeper model layer, the inputs themselves can depend on earlier attention. Replaying a short suffix need not reconstruct those inputs exactly. Chapter 9 will examine that different contract.

3.3 Count rows, then count bytes

A model usually calculates several attention mixtures, called *heads*, at each layer. Several query heads may share the same stored keys and values. For memory accounting, count stored KV heads rather than assuming that each query head owns a separate copy.

For an illustrative model with separate K and V arrays of equal width, let n_ℓ be the number of layers, h_{KV} the stored KV heads, d their width, L the context length and b the bytes per element. One request's KV payload is

$$C_{\mathrm{full}} = 2n_\ell h_{\mathrm{KV}} dLb.$$

The factor two counts keys and values. A model whose every layer uses a window W needs at most

$$C_{\mathrm{window}} = 2n_\ell h_{\mathrm{KV}} d \min(L, W) b$$

for those visible rows. These formulas exclude allocation padding, other state and temporary buffers.

```
python3 labs/08_kv_layout.py
```

With 32 layers, eight KV heads, width 128, a context of 4,096 and two-byte elements, the lab gives:

```
generic full KV: 536,870,912 B = 512 MiB
generic window KV: 16,777,216 B = 16 MiB
```

One mebibyte (MiB) is 2^{20} bytes. A 128-position window reduces this example's payload from 512 MiB to 16 MiB. Independent requests multiply the capacity requirement; shared prefixes and shared layers need their own ownership accounting.

The element size must describe the actual representation. BF16 uses two bytes per value. An eight-bit quantised value uses one byte, but its scale factors also need storage. Quantisation represents numbers with a smaller set of values, generally introducing error. A capacity calculation can count those bytes without claiming that the resulting model is numerically identical.

3.4 Open DeepInfer's attention path

Before following one attention call, locate it inside the prediction function from chapter 1. A token ID first selects a learned vector, its *embedding*. Successive layers transform these vectors into *hidden states*: the current representations of the input positions. Attention mixes information across visible positions. A feed-forward sublayer transforms each position's representation using learned weights; in a mixture of experts (MoE), a gate chooses which expert networks perform that transformation. Chapter 7 follows those experts across devices.

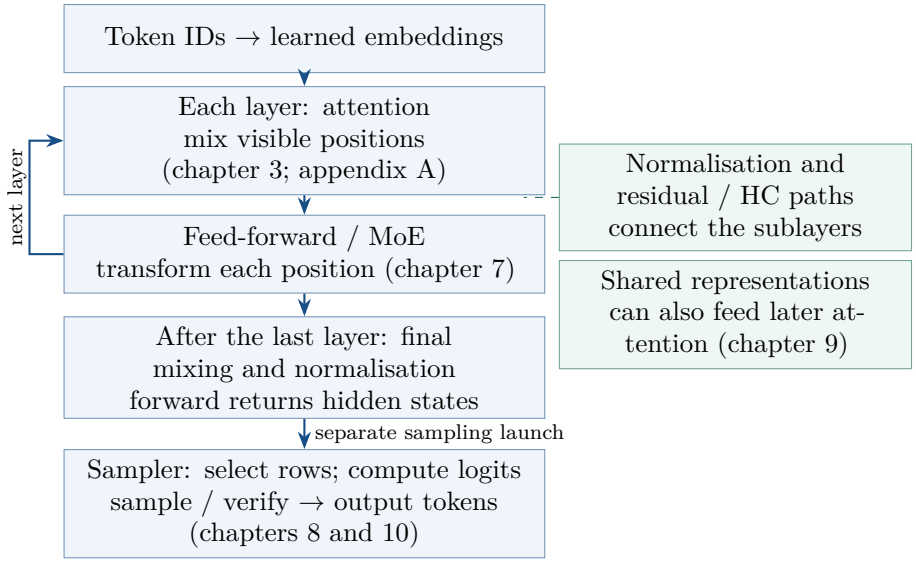


Figure 3.3: A computation map for the text path. The worker launches the layer stack and sampling separately; residual and HC details are grouped by responsibility.

DeepInfer's text path makes these stages concrete. In `deepinfer/model_executor/models/deepseek.py`, `DeepseekModel.forward` looks up input embeddings at lines 160–162 and calls the layers at lines 210–218. The layer's `forward` in the adjacent `decoder_layer.py` calls `self.self_attn` at lines 155–161, then `self.mlp` at lines 169–173 [20, 21].

Normalisation and residual paths connect these transformations. A simple residual adds a sublayer's input to its output, preserving a route for the earlier representation. DeepInfer also supports hyper-connections (HC), which mix several residual streams; the figure marks that surrounding machinery without deriving it. Appendix B locates HC and the additional Engram embedding path. Chapter 9 opens a different connection: a shared hidden representation that supplies later attention layers.

The main forward returns hidden states, not token IDs or vocabulary scores. `DeepseekForCausalLM.sample` passes those states and `lm_head.weight` to `Sampler`; its `get_logits` call produces the vocabulary scores for selected rows. Sampling then chooses output tokens or verifies drafts [20, 58]. Thus chapter 6's separate forward and sampling launches have a real data boundary. Chapters 8 and 10 open the sampling stage. Here we follow the persistent state read by attention inside the layer stack.

The generic two-array formula is useful, but it is not DeepInfer's cache layout. The V4 path stores a shared KV representation, supplies explicit indices to its attention backend, and may combine SWA with another cache. We will follow the SWA part of an ordinary decode step. Compression, sparse selection and cross-layer sharing remain named inputs here; Appendix A follows them into the attention read. Figure 3.4 marks the path we take through the layer.

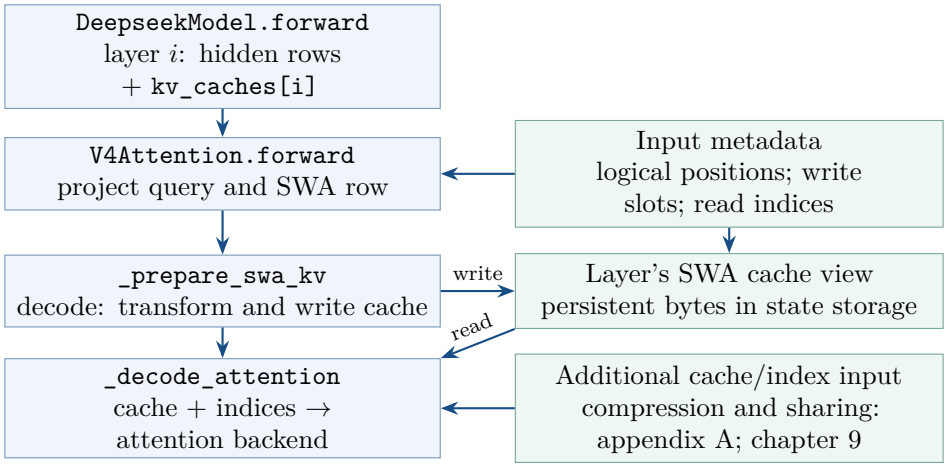


Figure 3.4: Opening the model-execution box from chapter 1. The selected path is one SWA-enabled V4 layer during decode. Metadata chooses addresses; the cache view supplies storage. Additional attention inputs remain visible but unopened.

3.4.1 Project the current rows

`DeepseekModel.forward` passes each layer its hidden states and its cache view; the decoder layer forwards these to `V4Attention`, along with position and address metadata [20, 21]. The cache survives the call. Of the model loop's optional inputs, `cross_layer_runtime` carries cross-layer state, `embed_hook` supplies an n-gram embedding lookup, and `dflash_input` holds input for the draft model.

Inside `V4Attention.forward`, projection and cache preparation are adjacent:

Listing 3.4: Project new rows, then prepare the SWA cache.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 787–800

```
787 q_a, kv_swa = self._project_q_and_swa(
788     hidden_states,
789     shared_hidden_bf16,
790 )
791 del hidden_states, hidden_states_bf16, shared_hidden_bf16
792 kv_swa, wait_kv_swa = self._prepare_swa_kv(
793     kv_swa,
794     kv_cache.swa_cache,
795     input_metadata,
796     use_cp,
797 )
798
799 q_b, index_q_b = self._project_q_b(q_a, input_metadata.positions)
800 del q_a
```

For a layer without shared-hidden input, `_project_q_and_swa` applies one combined projection and splits its output into `q_a` and `kv_swa`. The former is an intermediate query representation; `_project_q_b` expands it into query heads. With N evaluated positions, H query heads and width d, the resulting query layout is $N \times H \times d$. The new SWA projection has $N \times d$ elements, with one shared row per position rather than a separate stored row for each query head [22].

This is already an architectural constraint on the engine. It must keep the projected representation expected by this model and backend, rather than allocate separate K and V

arrays for every stored head. The projection and normalisation details need not be mastered to follow the lifetime of that storage.

3.4.2 A decode projection writes persistent state

In the decode branch of `_prepare_swa_kv`, the output destination is a cache plus a write-slot mapping:

Listing 3.5: Select a format and a persistent output destination.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 422–432

```
422 assert not use_cp
423 cast_args = CastArgs(
424     "e4m3",
425     self.swa_cache_num_per_channels,
426     round_sf=True,
427     sf_format="ue8m0",
428 )
429 out_args = KVCacheOutArgs(
430     swa_cache,
431     input_metadata.slot_mapping_store_swa,
432 )
```

`cast_args` selects the smaller numerical format. `out_args` pairs the cache with the assigned write slots. The fused operation receives both alongside the parameters for transforming the row:

Listing 3.6: Transform each new row and write its assigned slot.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 433–448

```
433 result = fused_norm_rope_hadamard_cast_store_kvcache(
434     kv_swa,
435     norm_args=NormArgs(self.kv_swa_norm.weight, self.kv_swa_norm.
    ↪ variance_epsilon),
436     rope_args=RopeArgs(
437         input_metadata.positions,
438         self.rotary_emb.cos_sin_cache,
439         self.qk_rope_head_dim,
440         self.rotary_emb.is_neox_style,
441         offset=self.qk_head_dim - self.qk_rope_head_dim,
442     ),
443     cast_args=cast_args,
444     out_args=out_args,
445 )
446 if not input_metadata.is_prompt:
447     assert result is None
448     return None, None
```

Read the arguments before the long function name. `norm_args` supplies normalisation parameters. `rope_args` supplies position-dependent rotations, commonly called rotary positional embeddings or RoPE. The output arguments direct the result into `swa_cache` at `slot_mapping_store_swa`.

The assertion and return at the end matter: decode expects no returned tensor from this operation. Its useful result is the change to persistent cache storage. A reader who follows only Python return values would miss that state transition.

3.4.3 The 528-byte row is defined in code

The SWA allocation calls `_kv_bytes_per_token` with `use_fp4=False`. That helper selects `V41_FORMAT` and checks that the model dimensions and scale grouping match it [23]. Its definition is short:

Listing 3.7: A physical format for one shared KV row.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/cache_common.py` · lines 109–118

```
109 V41_FORMAT = KVCacheFormat(
110     quantized_dtype="e4m3",
111     quantized_elements=512,
112     elements_per_sf=32,
113     bf16_elements=0,
114     do_cast=True,
115     sf_format="ue8m0",
116     note="V41, main/swa kv.",
117     token_interleaved=True,
118 )
```

There are 512 eight-bit values, with one scale per 32 values. `ue8m0` is the encoded scale format; this implementation stores each scale in a one-byte unsigned integer. The format's properties therefore give

$$512 \text{ data bytes} + (512/32) \text{ scale bytes} = 528 \text{ bytes}.$$

This agrees with the deployment Q&A's MXFP8 SWA figure [24, 2]. It includes scale overhead. There is no additional factor two: keys and values use the same stored row.

This is the cache-saving idea behind *Multi-head Latent Attention* (MLA), introduced in DeepSeek-V2. Instead of retaining expanded keys and values for every head, MLA retains a smaller learned representation, a *latent vector*, from which both are obtained. The query heads share that latent state. The V2 paper also caches a separate position-encoding component; its complete cache is not just the latent vector [28].

The inspected V4 implementation has its own layout. The cache helper asserts `kv_lora_rank + rope_head_dim == layout.num_elements`; `V41_FORMAT` fixes that total at 512. This combines the latent and position-related dimensions, rather than counting 512 ordinary key elements per query head. The model's `Wkv` projects into this shared space. The backend, `decode_mla_mqa_sparse`, uses the row for key scoring and value accumulation: its call receives `head_dim_v=self.qk_head_dim`, with no separate value cache. The accumulated context is transformed by `_wv_and_o_proj` afterwards [23, 29, 22].

Compare one layer and one token in the two layouts:

$$C_{\text{separate}} = 2 \times 8 \times 128 \times 2 = 4{,}096 \text{ B}, \qquad C_{\text{V4 SWA}} = 512 + 16 = 528 \text{ B}.$$

The first uses about 7.76 times as many bytes. This compares the illustrative separate-K/V layout with the inspected SWA row; it includes differences in head sharing, dimensions and precision. It is not a measured compression ratio for the same model. Both quantities are per layer per token, before multiplying by window length or counting other model state.

The cache engine exposes part of its allocated state storage as a tensor view:

Listing 3.8: View the SWA region without allocating another cache.
DeepInfer · `deepinfer/worker/cache_engine.py` · lines 418–422

```
418 assert start % 32 == 0
419 if blocks is not None:
420     swa_cache = blocks[:, start : start + swa_size].as_strided(
421         (blocks.shape[0], swa_block_size, 1, swa_size_per_token), (blocks.stride
    ↪ (0), swa_size_per_token, swa_size_per_token, 1)
422     )
```

The shape is

$$(\text{state slots},\ \text{ring positions},\ 1,\ \text{bytes per row}).$$

`as_strided` supplies both shape and distances between entries in the underlying allocation. The leading stride is `blocks.stride(0)`: moving to another request's state slot can skip more than this layer's SWA region. The one in the third dimension records the shared KV layout. The final dimension counts bytes, including scales, rather than 528 independent model features.

The second lab separates visible rows from allocated rows:

```
SWA row: 512 data B + 16 scale B = 528 B
128 visible rows, one layer: 67,584 B
134 allocated rows, one layer: 70,752 B
```

The 128-position window specifies what attention may use. We can derive the 134-position ring from the code. In `cache_engine.py`, lines 88–93, `get_inference_swa_block_size` adds `DEEPINFER_MAX_BACKTRACKING_TOKENS` to the window. In `deepinfer/environ.py`, lines 129–132, the draft limit `DEEPINFER_USE_NEXTN_DECODE` defaults to 5; the backtracking allowance is the larger of an explicit override (zero when unset) and that limit plus one:

$$S_{\text{inference}} = 128 + \max(0, 5 + 1) = 134.$$

The cache helper's docstring explains the purpose: "nextn requires backtracking by n tokens" (author's translation) [23, 30]. The expression chooses six extra positions, so this is a configured allowance rather than simply the five possible drafts.

This separates the model's requirement from the engine's execution choice. The window determines which positions a query may see; the draft configuration adds room for backtracking. The default 134 is their combination, not a new attention window or a constant prescribed by MLA. As with the shared latent row, the useful comparison is between what the model needs to read and what the engine must retain to execute it safely.

Decode metadata uses `get_kvstore_swa_block_size`. With `DEEPINFER_PREFILL_STORE_EXTRA_STATE` disabled, as it is by default, that helper returns the same 134. Enabling it adds `DEEPINFER_PREFILL_STORE_EXTRA_SIZE`, whose default is 128. Within that enabled branch, YOCO truncation (chapter 9) also adds the non-negative configured `DEEPINFER_YOCO_PREFILL_KEEP_TOKENS` when truncation is on. Thus the lab's $S = 134$ follows the default environment settings for a 128-token window; overrides can enlarge it. Chapter 4 follows the resulting allocations. The separately configured replay-retention length, such as 140, is not the attention window or the default ring size.

3.4.4 Trace a window boundary and a ring boundary

For a query at position p, let S be `swa_block_size` and b be `state_id`. The logical length is $\min(p+1, W)$. A position p occupies flattened slot

$$bS + (p \bmod S).$$

The modulo wraps within one request's ring. Multiplying the state ID by S selects that request's block. These flattened slots count token rows; they are not byte offsets or Python object addresses.

Use the default $W = 128$, $S = 134$, and choose state ID $b = 2$. This request owns flattened slots 268 through 401. The scalar transcription in the lab produces:

```
position | logical range | first/last read slot | write slot
     127 |   0..127      | 268 / 395            | 395
     128 |   1..128      | 269 / 396            | 396
     133 |   6..133      | 274 / 401            | 401
     134 |   7..134      | 275 / 268            | 268
```

At position 127, attention can see logical positions 0 through 127. At position 128, position 0 leaves the window and position 128 enters. The write goes to slot 396; the ring has not wrapped.

At position 134, the logical window is 7 through 134. Its physical read indices are 275 through 401, followed by 268. The write to 268 replaces logical position 0, which left the visible window six steps earlier. It does not replace the oldest visible position, 7. Window eviction and physical overwrite are different events when $S > W$.

The lab checks partial windows, repeated wraps and separation between state IDs. It implements the inspected address formulas using Python integers. It does not run the metadata kernel. The source path is what connects the calculation to DeepInfer's arrays.

3.4.5 Expand the read indices

The CUDA decode-metadata kernel constructs those bounds and write slots:

Listing 3.9: Derive read bounds and a write address from the position.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/cuda/prepare_decode_metadata.py` · lines 138–144

```
138  swa_len = T.min(pos + 1, window_size)
139  window_start_pos = pos - swa_len + 1
140  swa_off = state_id * swa_block_size + window_start_pos % swa_block_size
141  kv_swa_index_offset_len[q_off, 0] = swa_off
142  kv_swa_index_offset_len[q_off, 1] = swa_len
143
144  slot_mapping_store_swa[q_off] = state_id * swa_block_size + pos % swa_block_size
```

`V4Attention.forward` calls `compute_attn_info` before projecting the new rows. On decode, that method calls `get_kv_swa_index_decode` if the index array has not yet been prepared. It obtains the physical ring length from `get_kvstore_swa_block_size` [25]. The index helper receives two sizes: `window_size` determines the number of output columns; `swa_block_size` determines physical wraparound. The helper selects the index kernel's ring branch [25, 26]:

Listing 3.10: Wrap read addresses within the selected state block.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/cuda/get_kv_index.py` · lines 59–64

```
else:
    for j in T.Parallel(B):
        index[j] = offset // block_size * block_size + (offset + i * B + j) %
    ↪ block_size
for j in T.Parallel(B):
    index[j] = T.if_then_else(i * B + j < length, index[j], -1)
T.copy(index, kv_index[bx, i * B : (i + 1) * B])
```

Here $iB + j$ is an output column processed by the kernel, not a model position or a batch size. The division recovers the block base from the first read offset. The modulo wraps the successive columns inside that block. Columns beyond the valid length receive -1, so a short request does not read unfilled positions.

3.4.6 The attention read consumes cache and indices together

After the new SWA rows have been stored, the decode path supplies the cache and its read indices to the backend:

Listing 3.11: Pair the SWA storage with its visible indices.
DeepInfer · deepinfer/model_executor/models/v4_modules/attn.py · lines 640–647

```
if self.use_swa:
    attn_kv_args = dict(
        key_cache=kv_cache.swa_cache,
        indices_in_kvcache=kv_swa_index,
        key_cache_extra=kv_cache.ca_cache,
        indices_in_kvcache_extra=kv_ca_index,
        index_length_extra=dense_index_len,
    )
```

The additional cache fields keep another source of context visible in the interface. They prevent a misleading conclusion: a 128-token SWA component does not mean that the whole V4 model can use only 128 tokens. This chapter's capacity and visibility bounds apply to the SWA component.

Those arguments reach the attention operation here:

Listing 3.12: Read the selected rows to produce context vectors.
DeepInfer · deepinfer/model_executor/models/v4_modules/attn.py · lines 660–670

```
    context, _ = decode_mla_mqa_sparse(
        q_b,
        head_dim_v=self.qk_head_dim,
        sched_meta=input_metadata.flash_mla_sched_meta_dict[self.sched_meta_key
    ↪ ],
        softmax_scale=self.softmax_scale,
        attn_sink=self.attn_sink_params,
        batch_invariant=envs.DEEPINFER_BATCH_INVARIANT,
        fused_norm_rope_cast_kwargs=fused_kwargs,
        **attn_kv_args,
    )
return context
```

`q_b` supplies the current queries. The cache arguments supply stored rows, and `softmax_scale` is initialised to $d^{-1/2}$, the scaling used in the lab. The returned `context` is then passed to `_wv_and_o_proj` to produce the attention sublayer's output [22].

The backend also explains a potentially surprising detail. Its non-fused FlashMLA call sets `causal=False`. The comment at `attention_backend.py:133` says, in the author's translation, "sparse attention supports only non-causal mode" [27]. This call therefore asks the kernel to apply no causal mask of its own. The index rows must exclude future positions; otherwise the kernel could read them. Causality is enforced by the address construction we just traced.

3.5 Keep visible rows safe from overwrite

During decode, the current row is stored before attention reads the window. Could prefill do the same for a whole chunk? A chunk may write slots whose old contents are still needed by its early queries.

The prompt branch therefore constructs a temporary attention input. It places current SWA rows after the cached SWA rows, with any compressed rows after those. It loads the old window before storing the new one:

Listing 3.13: Preserve the old window before writing overlapping slots.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 845–857

```
845 if self.use_swa:
846     if input_metadata.is_prompt:
847         load_paged_kv_cache(
848             kv_cache.swa_cache,
849             input_metadata.load_segments_swa,
850             total_k_swa_cached,
851             out=kv[:total_k_swa_cached],
852         )  # [num_seqs * window_size, 1, qk_head_dim]
853         kv_swa_full_index = input_metadata.kv_swa_full_index  # [total_q,
    ↪ window_size]
854         # The SWA store slots may overlap the ranges loaded above, so prefill
    ↪ must store only after the cache load.
855         store_kv_cache(kv_swa, kv_cache.swa_cache, input_metadata.
    ↪ slot_mapping_store_swa)
856     else:
857         kv_swa_index = input_metadata.kv_swa_index  # [total_q, window_size]
```

The prefill indices refer to the assembled input, whereas decode indices refer directly to cache slots. The two branches implement compatible visibility through different physical layouts.

For a small example, take a window of three and a ring of five. Suppose the cached prefix ends at position 3 and the new chunk contains positions 4, 5 and 6. Query 4 needs old positions 2 and 3 plus new position 4. Query 6 needs only new positions 4, 5 and 6. Writing positions 5 and 6 wraps into slots 0 and 1, which are harmless in this particular chunk. Extend the chunk through position 7, however, and its write to slot 2 would destroy position 2 before query 4 had read it. Loading the old rows first makes all these queries safe to compute together.

3.5.1 Several decode queries can also overwrite old rows

The same hazard applies to a verification step. With five available draft tokens and fixed-length verification, one request supplies six query positions, $p, \ldots, p+5$. This is a decode call: the scheduler marks its output as non-prompt, and the model metadata preserves that

choice. Its SWA path writes the new rows before the attention read. Chapter 8 follows the draft and verification machinery; here we use only its query count and write order [31].

For a full window, the earliest query, at p, needs old rows beginning at $p-127$. In a ring of length S, writes at $p,\dots,p+5$ replace the previous occupants at $p-S,\dots,p+5-S$. Avoiding damage to that earliest query requires

$$p+5-S<p-127, \qquad S\geq 133.$$

This is a derived lower bound for these six writes, not a proof that 133 satisfies every engine requirement. With $S=128$, the last five writes replace rows still needed by query p. The default $S=134$ satisfies the bound and preserves the configured backtracking allowance. The lab checks all six queries after the writes for $S=128,133,134$:

```
S=128: overwritten visible rows per query = [5, 4, 3, 2, 1, 0]
S=133: overwritten visible rows per query = [0, 0, 0, 0, 0, 0]
S=134: overwritten visible rows per query = [0, 0, 0, 0, 0, 0]
```

Each entry counts visible rows whose previous contents were overwritten for that query. The two mechanisms now have a common explanation: prefill copies old rows before overwriting them; decode reserves space beyond the visible window.

The model therefore shapes both the cache representation and its access rules. The next chapter follows the storage behind these views: who allocates a state slot, which requests may share state, and when the engine can reuse it.

3.6 Experiments to try next

1. Change the attention lab's window to one, then to a value larger than the input length. With one, each output should equal its own value row. With the larger window, outputs should agree with full attention. Keep the prefix-agreement checks.

2. In the layout lab, change the generic KV-head count from eight to one. Explain why query-head count does not occur in its capacity function. Then halve the element size and identify which quantisation overheads the generic formula still omits.

3. Trace state ID 3 at position 134 with a 134-position ring. Derive the first read slot, last read slot and write slot. They should be 409, 402 and 402. Explain why none belongs to state ID 2.

4. In the prefill example, extend the chunk through position 8. Which other old row would an early store destroy, and which queries still need it? Use the checklist below to review the ordering.

A cache-reading checklist

Identify the query positions, the rows each query may read, the physical slots that new rows overwrite, and the point at which old rows have been copied. List every query that still needs an overwritten row; there may be more than one. In DeepInfer, find the cache load, the store mapping and the different prefill/decode index arrays before changing their order.

Chapter 4

State needs an owner

Chapter 3 ended with an address: a state ID multiplied by a ring length, plus a position modulo that length. Something must hand out those IDs. It must also decide when an ID can be given to another request. A request's memory need grows one token at a time, and requests finish in an order nobody chose in advance. Two requests may even begin with the same text.

The question

How can requests of different and changing lengths share one finite memory pool? When two requests use the same stored prefix, when is it safe to reuse that memory?

4.1 Hand out memory in blocks

Suppose the engine gave each request one contiguous region sized for its longest possible answer. Most requests stop early, so much of that memory would sit unused. Reserving exactly the current length fails the other way: the next token needs room that may already belong to someone else.

A common answer is to divide memory into equal *blocks*, each holding a fixed number of token positions. A request owns a list of block IDs, called its *block table*. Position p lives in block $table[\lfloor p/B \rfloor]$ at offset $p \bmod B$, where B is the block size. The blocks need not be adjacent. Any free block can serve any request. This paged design was popularised for language-model serving by vLLM [32].

Run the first experiment:

```
python3 labs/09_blocks.py
```

It uses a pool of eight blocks with four positions each. Block 0 is never handed out; the number 0 can therefore mean "no block". Before reading the output, predict when request A, which starts with six tokens, first needs a third block.

```
admit A(6), B(9)              A=[1, 2] B=[3, 4, 5]         free=[6, 7]
A reaches 8 tokens            A=[1, 2] B=[3, 4, 5]         free=[6, 7]
A reaches 9 tokens            A=[1, 2, 6] B=[3, 4, 5]      free=[7]
A at 11, two drafts reserved  A=[1, 2, 6, 7] B=[3, 4, 5]   free=[]
drafts rejected; A at 12      A=[1, 2, 6] B=[3, 4, 5]      free=[7]
admission check for 4 blocks: LATER
B finishes                    A=[1, 2, 6]                  free=[7, 5, 4, 3]
admission check for 4 blocks: OK
admission check for 8 blocks: NEVER
```

Six tokens need $\lceil 6/4 \rceil = 2$ blocks and nine need three, so A receives block 6 at its ninth token, not at its seventh. The last block in a table is usually only partly full. With block size B, at most $B - 1$ positions are idle per request, however long the request becomes.

The fourth line reserves room for two tokens that do not yet exist. Chapter 8 will introduce *draft tokens*: cheap guesses that the model verifies in the next step. Positions for a draft must exist before the step writes it. Here 11 tokens plus two drafts need $\lceil 13/4 \rceil = 4$ blocks, so A takes the last free block. Both drafts are then rejected and the verified step adds one token. Twelve tokens need only three blocks, and block 7 returns to the pool. A table can shrink as well as grow.

The admission checks distinguish two kinds of refusal. Four blocks are not free while B runs, so the answer is `LATER`: the request can wait. Eight blocks exceed the seven that this pool can ever issue, so the answer is `NEVER`. Waiting would not help.

Each block carries an owner count. The pool allocates from the front of its free list and returns a block to the back when the count reaches zero. When B finishes, its blocks are released in reverse table order. The free list therefore ends as $[7, 5, 4, 3]$: B's last block will be reused before its first. The check in `show` confirms after every step that each usable block is either owned or free, never both and never neither.

4.2 Two owners, one block

Owner counts matter once two requests share a block. Suppose a parent request has processed ten prompt tokens and a second request, the child, should continue from the same prefix. Recomputing the prefix would repeat work; copying all its blocks would repeat memory. The child can instead point to the parent's blocks and add one to each block's count. We call that *pinning* a block.

Sharing is safe only for rows that neither owner will write again. Positions 0–7 fill the first two blocks and are complete. Positions 8 and 9 sit in a partly filled third block. The next token of either request would be written there. So the child keeps the two full blocks and receives a private copy of the partial one. Copying only when a write would otherwise affect another owner is called *copy-on-write*.

```
python3 labs/10_share.py
```

The experiment stores token IDs in place of key and value rows, so a wrong block shows up as a wrong number:

Listing 4.1: Pin the parent's table; keep full blocks and copy a partial tail.
Book lab · `labs/10_share.py` · lines 38–48

```
38 def fork(pool, parent, processed, copies):
39     child = list(parent)
40     pool.pin(child)
41     keep = processed // BLOCK_SIZE
42     unpin = child[keep:]
43     child[keep:] = []
44     if processed % BLOCK_SIZE:
45         tail = pool.allocate()
46         copies.append((unpin[0], tail))
47         child.append(tail)
48     return child, unpin
```

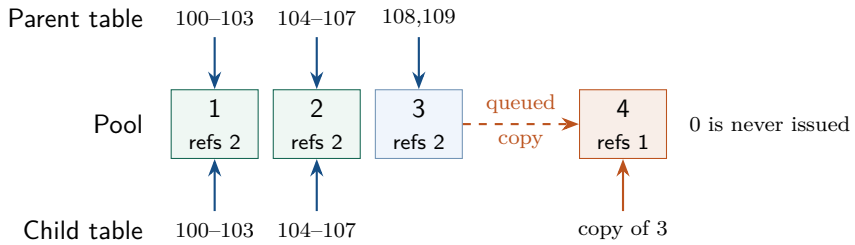


Figure 4.1: The fork in `labs/10_share.py`, before the queued copy runs. Both tables point to the two full blocks, so each has two owners. The parent's partial tail is copied into block 4; the child's pin on block 3 is released only after the copy. Token IDs stand in for KV rows.

In both runs, the fork gives blocks 1, 2 and 3 two owners, as Figure 4.1 shows. The parent then finishes. Its release lowers every count by one, but blocks 1 and 2 still belong to the child and do not enter the free list. The script asserts this. Ending one owner's lifetime has not ended the data's lifetime.

```
Release the tail pin before the queued copy
  fork: parent=[1, 2, 3] child=[1, 2, 4] refs 1:2 2:2 3:2 4:1 copy=[(3, 4)]
  parent finishes: free=[]; shared [1, 2] kept
  writer receives block 3 before the copy runs
  child reads [100, 101]...[900, 901]: correct=False

Release the tail pin after the queued copy
  fork: parent=[1, 2, 3] child=[1, 2, 4] refs 1:2 2:2 3:2 4:1 copy=[(3, 4)]
  parent finishes: free=[]; shared [1, 2] kept
  writer waits; receives block 3 after the copy
  child reads [100, 101]...[108, 109]: correct=True
```

The difference lies in the tail. The copy from block 3 to block 4 is queued, standing for work submitted to a device that runs later. In the first run, the host releases the child's pin on block 3 as soon as it has queued the copy. The parent has already finished, so the block's count reaches zero, so a new writer receives it and stores its own rows. When the queued copy finally runs, it copies the writer's data. The child reads 900 and 901 where it expected 108 and 109.

In the second run, the host keeps the pin until the copy has run. The writer must wait for a block, then receives block 3 safely. The safe policy costs capacity for a short time: a block that no request will use again stays unavailable until queued work has finished reading it.

The writer in this lab does not wait for the copy queue. If every later write were queued behind the copy in the same order, the early release would not corrupt this example. A host-side allocator usually cannot assume that about every future writer. The rule "release a source only after work that reads it has completed" avoids depending on that assumption.

4.3 Follow DeepInfer's ownership path

DeepInfer implements both experiments' ideas, with one addition: there are two pools. Figure 4.2 places them on the request path from chapter 1. We will follow one request from admission to release, then a fork of its prefix.

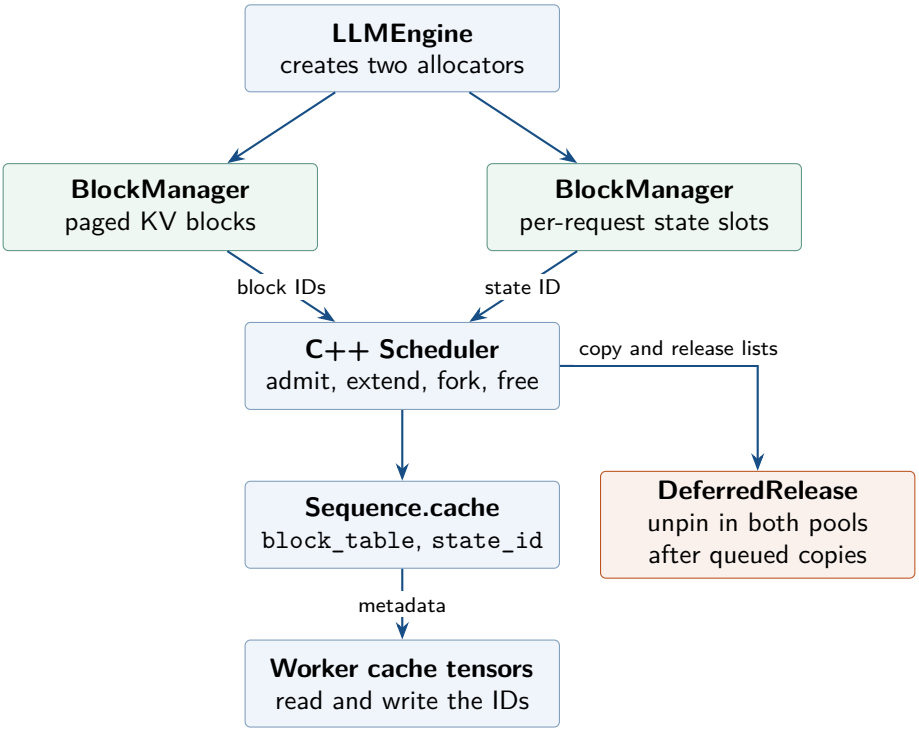


Figure 4.2: The ownership path in this chapter. One allocator class manages two pools. The scheduler stores the IDs in each request's cache state and records copies; the engine releases copied sources only after the copy event completes [5, 34, 33].

4.3.1 Two pools, one allocator class

The engine creates both pools when it starts:

Listing 4.2: One allocator class for paged blocks and state slots.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 265–266

```
265 self.block_manager = BlockManager(cache_config.num_gpu_blocks)
266 self.state_manager = BlockManager(cache_config.num_state_seqs) if cache_config.
      ↪ bytes_of_state_per_seqs > 0 else None
```

The first manager issues paged KV blocks. For the V4 model, the block size is 128 token positions (`model_metadata_utils.py`, lines 134–135). The cache engine's field descriptions place the compressed-attention cache and the index cache in these blocks, each block holding 128/ratio rows for its compression ratio (`cache_engine.py`, lines 206–219). The compression and indexing appendix (Appendix A) explains those rows. Here they are simply rows assigned to token ranges.

The second manager issues *state slots*, one per running request. A slot holds the SWA ring from chapter 3 and the compression state. The pool exists only if the model has such state. Its size is $\lfloor 1.2 \times \texttt{max_num_seqs} \rfloor + 1$ by default (`cache_engine.py`, lines 201–202; `environ.py`, line 162). Because slot 0 is never issued, this provides $\lfloor 1.2 \times \texttt{max_num_seqs} \rfloor$ usable slots [23, 30].

The order of sizing is revealing. After a profiling run, the worker subtracts the fixed state reservation from available device memory, doubled for what its comment calls the state store and a copy buffer. Only the remainder becomes paged blocks (`deepinfer/worker/base.py`,

lines 237–246). The number of state slots follows from the request limit; the number of blocks follows from whatever memory is left [36].

The block table retains the paged-addressing idea introduced at the start of this chapter. The additional allocation boundary follows the model: compressed KV and index rows belong to token blocks, while the SWA ring and rolling compression state belong to a request. DeepInfer gives both regions the same allocator machinery but different capacities and copy lifetimes. This is a comparison between a general paging mechanism and its use for this model, not a claim that every other engine has only one kind of state.

4.3.2 An owner count decides when a block is free

Both managers use the same small structure, `Blocks`. Its `allocate` method, lines 82–86, takes the head of an intrusive free list, a linked list threaded through the block records themselves, and sets the block's owner count to one. Its comment says, in the author's translation, that removal from the list has already checked for a zero count. The other two operations change an existing count:

Listing 4.3: Share a live block; release one owner.
DeepInfer · `csrc/engine/block_manager.hpp` · lines 88–103

```
88  void pin_block(int block_id) {
89      TORCH_CHECK(block_id > 0 && block_id < static_cast<int>(_blocks.size()));
90      auto &block = _blocks[block_id];
91      TORCH_CHECK(block.ref_count > 0, "try to pin free block");
92      block.ref_count += 1;
93  }
94
95  void free(int block_id) {
96      TORCH_CHECK(block_id > 0 && block_id < static_cast<int>(_blocks.size()));
97      auto &block = _blocks[block_id];
98      block.ref_count -= 1;
99      TORCH_CHECK(block.ref_count >= 0, "ref_count corrupted: ref_count < 0");
100     if (block.ref_count == 0) {
101         _push_back(block_id);
102     }
103 }
```

`pin_block` adds an owner but refuses a free block: sharing must start from a live one. `free` removes one owner and returns the block to the back of the list only at zero. The Python binding exposes `free_blocks` under the name `unpin_blocks` (`pybind_scheduler.cpp`, line 68). Releasing a request and releasing a pin are the same operation [34, 35].

The structure's constructor explains the lab's reserved block. Its comment at line 27 says, in the author's translation, "GPU block 0 is unused, so 0 here means empty". Block 0 terminates the free list; later we will see it mark a copy with no destination.

`can_allocate_nolock`, lines 128–136, returns the lab's three answers. A request for at least `num_blocks` blocks is `NEVER`, because block 0 can never be issued. The scheduler combines this answer with the state pool: if blocks are available but no state slot is, the result becomes `LATER` (`scheduler.hpp`, lines 194–201). A request needs both kinds of storage to run [33].

4.3.3 Reserve positions that the history does not show

How many blocks should a request own? DeepInfer's answer includes positions that are not yet tokens:

Listing 4.4: Count history, in-flight and draft positions.
DeepInfer · csrc/engine/sequence.hpp · lines 291–293

```
int get_num_blocks() const {
    return (get_len() + schedule.total_inflight_tokens + get_nextn_len() + cache
   ↪ .block_size - 1) / cache.block_size;
}
```

The first term is the token history from chapter 1. The second counts positions of steps that have been scheduled but whose results have not yet reached the host. Under ZOS, the scheduler plans step $N + 1$ while step N is still running, so the history is behind the work already submitted. The third term counts draft tokens awaiting verification. Memory must cover all three, or a step could write beyond its table.

When a request is first admitted for prompt work, `allocate_seq` takes this many blocks and one state slot (`scheduler.hpp`, lines 518–519; `block_manager.hpp`, lines 207–214). The count uses the whole current prompt. A long prompt may be processed in several chunks, but its blocks are reserved at the first one. Chunking divides the computation, not the memory claim [33, 34].

4.3.4 Each decode step adjusts the table

Before each decode step, the scheduler brings every running table back to the required length:

Listing 4.5: Grow by one block or release the surplus.
DeepInfer · csrc/engine/scheduler.hpp · lines 817–828

```
void _append_slot(Sequence &seq) {
    int num_blocks = seq.get_num_blocks();
    if (static_cast<decltype(num_blocks)>(seq.cache.block_table.size()) <
   ↪ num_blocks) {
        seq.cache.block_table.emplace_back(block_manager->allocate_single_nolock
   ↪ ());
    } else if (static_cast<decltype(num_blocks)>(seq.cache.block_table.size()) >
   ↪  num_blocks) {
        std::vector<int> redundant_blocks(seq.cache.block_table.begin() +
   ↪ num_blocks, seq.cache.block_table.end());
        block_manager->free_blocks_nolock(redundant_blocks);
        seq.cache.block_table.erase(seq.cache.block_table.begin() + num_blocks,
   ↪ seq.cache.block_table.end());
    }
    TORCH_CHECK(static_cast<decltype(num_blocks)>(seq.cache.block_table.size())
   ↪ == num_blocks,
        "len(block_table) = ", seq.cache.block_table.size(), ", but should be ",
   ↪  num_blocks, ". seq_len = ", seq.get_len());
}
```

The growth branch takes one block; the lab's general `resize` can take more. With 128-position blocks and, by default, at most six new positions per request in a step, one block is enough; the final check would catch a shortfall. The shrink branch is the lab's

rejected-draft case: when fewer positions are reserved, surplus blocks go back immediately. The decode loop calls this function only after checking that a block is free. If none is, it preempts another request (lines 629–640). Chapter 5 examines that choice [33].

4.3.5 A written row is not yet a valid row

A table says which blocks a request owns. It does not say which rows inside them are meaningful. That is the job of the request's processed-token count:

Listing 4.6: Advance the processed count, then correct it.
DeepInfer · csrc/engine/sequence.hpp · lines 216–221

```
216 void advance_optimistically(int token_count) {
217     previous_num_tokens = num_tokens;
218     num_tokens += token_count;
219 }
220
221 void commit_processed_tokens(int token_count) { num_tokens = previous_num_tokens
    ↪  + token_count; }
```

When a step's output is processed, the scheduler first advances `num_tokens` by every position the step evaluated (`scheduler.hpp`, line 927). For decode, it then commits the true count. The comment at line 998 says, in the author's translation: "decode: correct `cache.num_tokens` from the optimistically advanced value to the number actually accepted (rejected drafts do not occupy KV)" [33, 6].

"Do not occupy" describes meaning, not bytes. The verification step wrote rows for its drafts into the request's blocks. After rejection those bytes remain until a later token overwrites them, but they lie beyond `num_tokens` and no reader should use them. Validity is decided by the count, not by whether bytes are present. Chapter 3 drew the same distinction between leaving the window and being overwritten. Here it applies to rows that were never accepted.

4.3.6 Fork a prefix by pinning it

Prefix sharing enters through `Scheduler::fork_seq` (lines 1061–1122). The released benchmark uses it to give a batch of decode requests one cached prompt, in `fork_request_batch` (`examples/benchmark.py`, lines 331–353). Production prefix reuse also involves the external KV cache system, which the release does not include [1, 15]. The fork itself is a few lines:

Listing 4.7: The child pins the parent's blocks and state slot.
DeepInfer · csrc/engine/scheduler.hpp · lines 1110–1114

```
1110 block_manager->pin_blocks(fork_seq->cache.block_table);
1111 if (state_manager) {
1112     fork_seq->cache.state_id = seq->cache.state_id;
1113     state_manager->pin_block(seq->cache.state_id);
1114 }
```

At this point the child shares everything, including the parent's state slot. That cannot last. A state slot holds a ring that every decode step writes, as chapter 3 showed. Two requests cannot advance one ring.

The benchmark's forks share a fully processed prompt, so they take the scheduler's path for forks entering decode. It separates what must be private:

Listing 4.8: Copy the mutable state and the last block; release the rest.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 602–618

```
602 if (forked) {
603     if ((state_manager && !state_manager->can_allocate_single_nolock()) || !
    ↪ block_manager->can_allocate_single_nolock()) {
604         break;
605     }
606     _clone_state(*seq);
607     int new_block = block_manager->allocate_single_nolock();
608     int num_blocks = (seq->cache.num_tokens + seq->cache.block_size - 1) / seq->
    ↪ cache.block_size;
609     TORCH_CHECK_LE(num_blocks, static_cast<int>(seq->cache.block_table.size()));
610     clone_blocks.emplace_back(seq->cache.block_table[num_blocks - 1]);
611     clone_blocks.emplace_back(new_block);
612     for (size_t i = num_blocks; i < seq->cache.block_table.size(); ++i) {
613         clone_blocks.emplace_back(seq->cache.block_table[i]);
614         clone_blocks.emplace_back(0);
615     }
616     seq->cache.block_table[num_blocks - 1] = new_block;
617     seq->cache.block_table.erase(seq->cache.block_table.begin() + num_blocks,
    ↪ seq->cache.block_table.end());
618 }
```

The fork waits unless one state slot and one block are free. `_clone_state` allocates a new state slot and records an (old, new) copy (lines 203–212). `num_blocks` counts the blocks that hold processed rows. The last of them is copied into the new block; this path copies it even when it happens to be full. Every later parent block is recorded with destination 0: release without copying. The table then points to the shared complete blocks followed by the private copy.

Forks whose prompt continues beyond the shared prefix are admitted through the prompt path instead (lines 412–437). There the shared part is the `num_reserve_blocks` complete blocks, and the tail is copied only when the processed count ends partway through a block. The comment at line 427 says "has a tail, needs copy" (author's translation). That is the rule in our lab [33].

This is the lab's rule with one more kind of state. Complete prefix blocks are read but not written, so they are shared. The partial tail and the state ring will be written, so each owner gets its own copy.

4.3.7 Release a source only after its copy

The copy lists reach the device during `LLMEngine.step` (`llm_engine.py`, lines 889–905). The C++ helper queues a device copy for each pair with a nonzero destination and adds every source to a release list, whether or not it was copied (`scheduler.hpp`, lines 1288–1300). The engine does not release those sources at once:

Listing 4.9: Record an event after the queued copies.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 95–98

```
95 def collect_clone_unpins(self, states_to_unpin: np.ndarray[np.int32],
   ↪ blocks_to_unpin: np.ndarray[np.int32]) -> None:
96     assert self.clone_unpins_to_release is None
97     self.clone_done_evt.record()
98     self.clone_unpins_to_release = (states_to_unpin, blocks_to_unpin)
```

An *event* marks a point in the device's work queue; the host can later wait until the device has passed it. The release happens in `DeferredRelease.release`. The engine calls it from `pre_sampler_sync` later in the same step, as the comment at line 904 says. By then the worker has launched the forward pass and sampling (`worker.py`, lines 113–164):

Listing 4.10: Wait for the copies, then drop the pins.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 109–116

```
109 if self.clone_unpins_to_release is not None:
110     states_to_unpin, blocks_to_unpin = self.clone_unpins_to_release
111     self.clone_unpins_to_release = None
112     self.clone_done_evt.synchronize()
113     if states_to_unpin.size > 0:
114         self.state_manager.unpin_blocks(states_to_unpin)
115     if blocks_to_unpin.size > 0:
116         self.block_manager.unpin_blocks(blocks_to_unpin)
```

This is the second run of the sharing lab. The host waits for the copy event before the sources can return to either free list. The copies were queued before the forward pass, so the host's wait does not hold back the forward and sampling work already queued behind them [7]. The class docstring states the general purpose: collect resources to release, then release them at the next `pre_sampler_sync`, overlapping with device computation (lines 68–71). The same object delays the output-pool release from chapter 1 and the release of per-step metadata [5].

4.3.8 Finishing returns every reference

A finished or preempted request gives back what it owns:

Listing 4.11: Release the draft slot, state slot and blocks.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 738–745

```
738 void free_seq_nolock(Sequence &seq) {
739     release_mtp_info_slot(seq);
740     if (seq.cache.state_id != -1) {
741         state_manager->free_blocks_nolock({seq.cache.state_id});
742     }
743     block_manager->free_blocks_nolock(seq.cache.block_table);
744     seq.cache.block_table.clear();
745 }
```

`release_mtp_info_slot` returns a third resource, a slot for draft information that chapter 8 will use. Then the state slot and every block lose one owner. A block shared with a fork survives; an unshared block returns to its free list. `free_blocks_nolock` walks the table in reverse (`block_manager.hpp`, lines 147–149), which is why the lab's free list ended with B's blocks in reverse order. The source does not explain that choice.

4.3.9 Output storage has a different owner

Chapter 1's result records belong to `RequestOutputPool`. The caller must clone any record it keeps before the pool releases it. That ownership change differs from pinning a KV block: cloning creates a new object, while pinning adds an owner of the same block. The binding's `clone` constructs a new object from the one at the supplied address:

Listing 4.12: Copy the addressed output into a separately owned object.
DeepInfer · `csrc/engine/pybind_output.cpp` · lines 16–18

```
16 py::class_<RequestOutput>(engine, "RequestOutput")
17     .def_static(
18         "clone", [](uint64_t ptr) { return std::make_unique<RequestOutput>(*
   ↪ reinterpret_cast<RequestOutput *>(ptr)); }, py::arg("ptr"))
```

Here `reinterpret_cast` treats the integer as an object address; `make_unique` allocates a separate object managed by the binding. In contrast, the pool's release loop destroys the original and makes its slot available again:

Listing 4.13: Recycle storage only after the caller has had its chance to copy.
DeepInfer · `csrc/engine/outputs.hpp` · lines 72–80

```
72 void release(py::array_t<uint64_t> ptrs_py) {
73     const uint64_t *ptrs = ptrs_py.data();
74     int size = ptrs_py.size();
75     for (int i = 0; i < size; ++i) {
76         auto p = reinterpret_cast<RequestOutput *>(ptrs[i]);
77         std::destroy_at(p);
78         free_.push_back(p);
79     }
80 }
```

The deferred-release helper invokes this operation at a later `pre_sampler_sync`. Merely appending `raw_output` to a Python list would leave an integer referring to destroyed or reused storage. Cloning during collection preserves the result across that boundary. The pointer-based interface allows explicit reuse of pool slots; the cost is that callers must follow a lifetime contract that ordinary Python references do not express [9, 10, 5].

4.4 A state slot has regions with different lifetimes

Ownership is not only about who may write. It also determines how long data must survive and where it may go. The `StateBlock` docstring in `cache_engine.py`, lines 221–241, describes one request's slot. In the author's translation of its central rule: state that must be stored goes in a front *stored region*; state that need not be stored, but is still needed while running, goes in a rear *dropped region*. Taking the first `state_size_to_disk` bytes of the *inference* layout therefore excludes the dropped region by truncation. The next sentences of the docstring add a condition (lines 226–227). If extra prefill state is enabled, each slot holds the longer kvstore ring from chapter 3, which must first be reordered

into the inference window; only when it is disabled can the original buffer be cut directly. `GPUCacheInterface.select_state_slice_to_store` implements both branches: a direct slice when `DEEPINFER_PREFILL_STORE_EXTRA_STATE` is off, and a per-view reordering otherwise (lines 763–817) [23].

The stored region holds the SWA rings of the layers whose window must survive across requests. The dropped region holds the SWA layers that the deployment Q&A says are recovered by prefill recomputation, and the compression state. Chapter 9 explains why those layers can be recomputed [2].

The code sets `state_size_to_disk` to the padded total for the stored SWA layers, using the inference ring length (line 306). Chapter 3 derived that ring as 134 positions of 528 bytes:

$$134 \times 528 = 70{,}752 \text{ bytes per stored layer.}$$

That is already a multiple of 32, so the 32-byte padding adds nothing. The Q&A states that V4.1 stores its first 20 layers, giving

$$20 \times 70{,}752 = 1{,}415{,}040 \text{ bytes per sequence,}$$

the figure it reports [2]. The layer count comes from the model configuration, which this book does not inspect; the arithmetic shows that the Q&A's number follows from the inspected layout.

The released engine prepares this layout but leaves its use to the missing storage system. `LLMEngine.should_copy_state` returns `False`, and `do_save_kvcache` and related methods are empty hooks; their docstrings call them virtual functions (`llm_engine.py`, lines 811–822). The Q&A describes the production behaviour: copying the window at 128-token boundaries, keeping SWA state in the producing machine's host memory and fetching it across machines on a hit. That is documented service design, not released engine code [5, 2].

4.5 What a cache entry means

We can now answer chapter 1's third question, which memory holds the state needed to continue, with some precision. Two different questions hide in it.

The first asks which rows form a committed prefix: rows that later steps, a fork or a store may reuse. Such a row must satisfy four conditions together:

1. The request owns the block or state slot, alone or through a pin.
2. The row's position is below the request's processed-token count, `num_tokens`.
3. The row has not been replaced by a later write to the same physical slot, as in chapter 3's ring.
4. Any queued copy that fills it has completed before it is read.

The second asks which rows the current step's computation may read. That range is larger. When the scheduler builds a step's metadata, it sets the key length to `num_tokens` plus the positions of steps still in flight plus this step's query positions (`scheduler.hpp`, line 697). Attention in this step reads rows that the step has just written for its own queries and drafts, before the host has advanced `num_tokens`. Those rows are visible because the step's metadata and indices say so. They join the committed prefix only if the later correction accepts them [33].

Ownership is recorded in owner counts, the committed prefix in `num_tokens`, the current step's visibility in its metadata and indices, physical replacement in address arithmetic, and

copy completion in device events. No single data structure answers the whole question. The next chapter asks what happens when the free lists run short and the scheduler must decide who waits.

4.6 Experiments to try next

1. In `labs/09_blocks.py`, reserve five drafts instead of two at length 11. How many blocks must `resize` add at once? DeepInfer's `_append_slot` adds at most one. State a condition on block size and positions added per step under which one block is always enough.

2. In `labs/10_share.py`, set `PROCESSED` to 8 instead of 10. Predict the copy list, the owner counts and whether releasing the pin early still corrupts the child. The script's final check expects corruption only when a tail copy was queued; confirm that it passes.

3. Change `release` in either lab to walk blocks in forward order. Which block does the next allocation receive? Suggest a reason an engine might prefer one order, and describe an experiment that could test it. The source gives no reason, so label your answer as a hypothesis.

4. The state pool provides $\lfloor 1.2 \times$ `max_num_seqs` $\rfloor$ usable slots. List situations in which more than `max_num_seqs` slots can be held at once. Check each against the fork and release code before relying on it.

Chapter 5

Choose the next batch

A free block answers whether a request can own more memory. It does not answer whether that request should run next. One user has submitted a long document; another wants a short answer; a third is already receiving tokens. They compete for computation as well as storage.

The question

If we split a long prompt into smaller pieces, will short requests get their first token sooner? What should happen when the next step needs memory that another request owns?

5.1 A smaller chunk is an opportunity to choose

Chapter 1's batching experiment gave each active request one token per round. Now give the scheduler prompt work and arrivals. Request A arrives at time 0 with 12 prompt positions. B and C arrive at time 3 with two each. A step processes one request's prompt, or part of it. Give a step with q positions the assigned duration $1 + q$: one unit of overhead plus one per position. The first output arrives when the prompt finishes. This experiment isolates prefill; it does not schedule subsequent decode tokens.

```
python3 labs/11_chunks.py
```

Compare three rules. First, finish each prompt in arrival order, also called *first in, first out* (FIFO). Second, allow at most four positions per step but put an unfinished request back at the front. Third, use the same four-position limit and put it at the back, behind arrivals seen during that step. Predict which rule helps B before reading the output.

```
Whole prompts, FIFO (budget 12)
   0->13: A q=12 waiting=['B', 'C']
  13->16: B q= 2 waiting=['C']
  16->19: C q= 2 waiting=[]
  first-token delays: A=13, B=13, C=16
  work=16 positions; steps=3; finish=19
```

With whole prompts, A occupies time 0–13. B finishes at 16, so its first-token delay is $16 - 3 = 13$. Now cut A into chunks, keeping its continuation at the front:

```
Chunks, continuation at front (budget 4)
   0-> 5: A q= 4 waiting=['A', 'B', 'C']
   5->10: A q= 4 waiting=['A', 'B', 'C']
  10->15: A q= 4 waiting=['B', 'C']
  15->18: B q= 2 waiting=['C']
  18->21: C q= 2 waiting=[]
  first-token delays: A=15, B=15, C=18
  work=16 positions; steps=5; finish=21
```

The chunks create boundaries at 5 and 10, but the scheduler chooses A again at both. B's delay rises to 15 because A now pays the overhead three times. Moving the continuation to the back changes who runs at those boundaries:

```
Chunks, continuation at back (budget 4)
   0-> 5: A q= 4 waiting=['B', 'C', 'A']
   5-> 8: B q= 2 waiting=['C', 'A']
   8->11: C q= 2 waiting=['A']
  11->16: A q= 4 waiting=['A']
  16->21: A q= 4 waiting=[]
  first-token delays: A=21, B=5, C=8
  work=16 positions; steps=5; finish=21
```

The third rule uses those boundaries to change requests. At time 5 the queue is B, C, A. B finishes at 8, five units after its arrival. C finishes at 11, eight units after its arrival. A pays for their earlier answers: its own first token moves from time 13 to 21. The queue operation responsible for the difference is small:

Listing 5.1: Choose where an unfinished prompt returns.
Book lab · `labs/11_chunks.py` · lines 38–45

```
38          admit()  # Arrivals during this chunk precede a rotated continuation.
39          if remaining[name]:
40              if rotate:
41                  waiting.append(name)
42              else:
43                  waiting.appendleft(name)
44          else:
45              first[name] = now
```

All three runs process 16 positions. The chunked runs take five steps instead of three, so their total duration is 21 rather than 19. Smaller chunks have changed latency and overhead without reducing the useful work. Chapter 2's distinction between work and elapsed time applies here too. Figure 5.1 draws the three runs.

Chunk size and queue order are separate decisions. A short step creates a point at which other work *could* run. Whether it does depends on what the scheduler chooses there. The rotation rule is a teaching alternative; we will now inspect where DeepInfer actually puts an unfinished prompt.

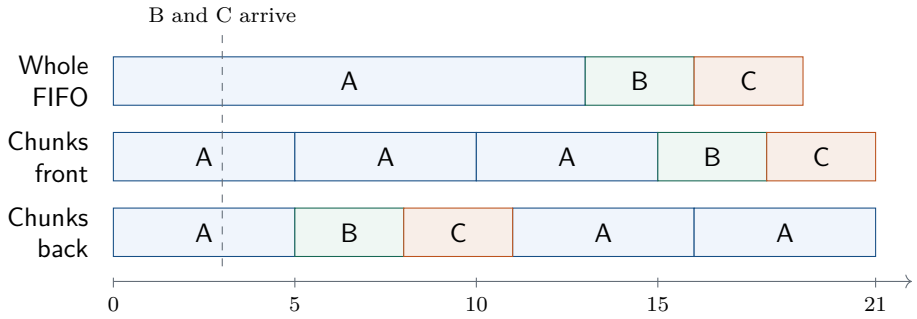


Figure 5.1: The three runs of `labs/11_chunks.py`. Widths show assigned time, including one overhead unit per step. Only the bottom policy uses a chunk boundary to let B and C overtake A.

5.2 Follow a prompt through the scheduler

In chapter 1, `LLMEngine.step` called the C++ scheduler and passed its metadata to the worker. Open that scheduler box. It makes three decisions in sequence: choose a kind of step, choose requests that fit, then describe the positions that the worker will evaluate. The block and state managers from chapter 4 participate in the second decision.

The queue names have a useful historical comparison. vLLM v0.6.3 keeps `waiting`, `running` and `swapped` queues of sequence groups (`vllm/core/scheduler.py`, lines 340–348). Its preemption path can either discard cached progress for recomputation or swap blocks out for later recovery [13]. DeepInfer's C++ scheduler keeps `waiting`, `running` and `prompted`. In this version, only `fork_seq` adds requests to `prompted` (`scheduler.hpp`, line 1118); lines 581–618 handle their admission to decode. Although the `SWAPPED` status name remains, this scheduler has no corresponding swapped queue or swap branch in `_preempt` [37, 6].

We use that fixed vLLM release as a comparison, not an identified fork point. The familiar waiting/running distinction is only the starting point. Follow DeepInfer's rank-wide step-type votes and its separate in-flight counts below: they determine what a request may do before the preceding device result is known. Figure 5.2 places these decisions on the request path.

5.2.1 Choose prefill or decode

DeepInfer's released scheduler distinguishes three engine types. A `PROMPT` engine runs the prefill path; a `COMPLETION` engine runs the decode path. A `NORMAL` engine can do either (`scheduler.hpp`, lines 288–295). A normal engine chooses one kind for each step. It does not build a batch mixing prompt chunks and decode rows in these two paths [37].

That choice also involves other participants. A *data-parallel rank* is one participant with its own request batch in a cooperating group. The ranks must agree on the kind of step. Detailed distribution belongs to chapter 7; here the consequence is that a non-empty local queue cannot decide alone.

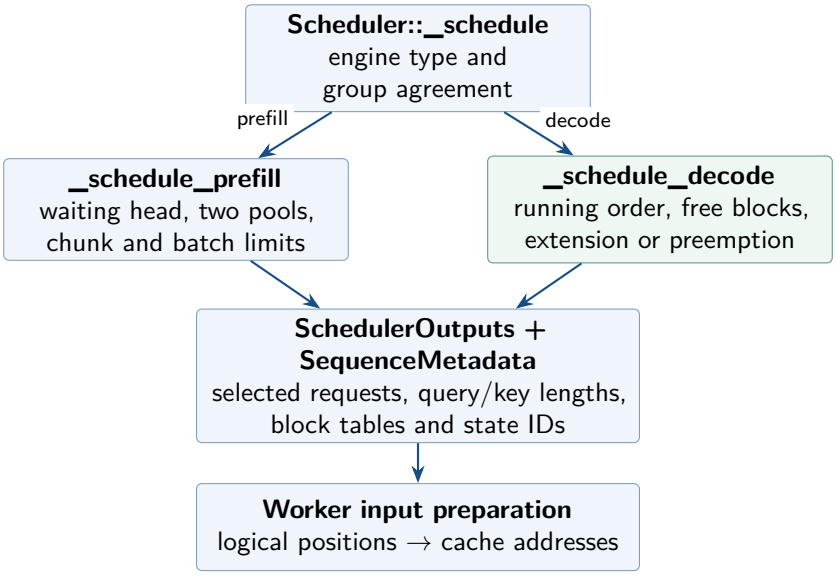


Figure 5.2: Open the scheduler box from chapter 1. Admission and preemption use chapter 4's ownership state; input preparation turns the selected lengths into chapter 3's addresses. Output processing later commits progress or requeues work.

Listing 5.2: Some decode counts permit a prefill attempt.
DeepInfer · csrc/engine/scheduler.hpp · lines 308–313

```
308 bool is_any_point = (decode_step != 0) && (decode_step %
    ↪ prefill_any_attempt_interval == 0);
309 bool is_all_point = (decode_step % prefill_attempt_interval == 0);
310 if (!is_any_point && !is_all_point) {
311     return _schedule_decode(acquire);
312 }
313 bool want = (get_gpu_cache_usage() <= GPU_CACHE_PREFILL_WATERMARK) &&
    ↪ _has_prefill_waiting();
```

At an ALL attempt, every rank must want prefill; at an ANY attempt, one is enough. Lines 315–319 implement the votes. A rank wants prefill only if its cache usage is at or below the watermark and the head of its waiting queue is schedulable. The default intervals are 70 and 100 decode steps respectively, from `environ.py`, lines 174–178. An ANY point takes precedence when the two coincide. A prefill step resets `decode_step` to zero, which is another ALL attempt point [30, 37].

Agreement to try is not admission. After trying, the ranks vote again to check whether any produced a prefill result. If all attempts returned no work, they fall back to decode (lines 328–340). The source comment explains why: an empty prompt step would reset the counter, invite another unsuccessful attempt and prevent running requests from making progress. The ANY interval therefore offers an opportunity for isolated prefill work; it is not a wall-clock latency guarantee or a promise that memory will be available.

5.2.2 The head of the queue must fit

`_schedule_prefill` examines `waiting.front()`. For a fresh request, it checks the prompt length and asks whether both pools can provide the storage required by `get_num_blocks`. Chapter 4 established that this reserves blocks for the whole prompt, even when computation will be chunked.

Listing 5.3: Wait for a possible allocation; reject an impossible one.
DeepInfer · csrc/engine/scheduler.hpp · lines 386–397

```
auto can_allocate = _can_allocate_with_state(seq->get_num_blocks());
if (can_allocate == AllocStatus::LATER) {
    break;
} else if (can_allocate == AllocStatus::NEVER) {
    LOG_WARN("Input prompt ({} tokens) is too long and "
             "exceeds the capacity of block_manager",
         num_prompt_tokens);
    seq->schedule.status = SequenceStatus::FINISHED_IGNORED;
    ignored_seqs.push_back(seq);
    waiting.pop_front();
    continue;
}
```

The difference between `break` and `continue` is a scheduling policy. `LATER` leaves the request at the head and stops this pass. A smaller request behind it is not tried. This is *head-of-line blocking*: work that could fit waits behind work that cannot yet fit. `NEVER` marks the request ignored, removes it and tries the next one. It cannot become runnable by waiting.

For example, suppose three blocks are free. The head needs four and the next request needs one. If the pool can eventually provide four, the head receives `LATER` and neither request is admitted in this pass. Choosing to skip the head would use the free memory now, but repeated skips could keep a large request waiting indefinitely. The released path chooses to stop; the code establishes that choice without establishing which policy is best for a service.

Two other limits follow (lines 440–449). The scheduler limits the number of requests alongside those already running, and subtracts each selected request's *whole history length* from `prefill_sum_seqlen_k_remain`. Despite the name, this second check does not charge only the current chunk's key length. It is separate from the query-position budget below. A request can pass the storage check and still fail one of these limits.

5.2.3 Choose how many positions to evaluate

Chunking is off by default: `EngineArgs.chunked_prefill` is `False` in `deepinfer/engine/arg_utils.py`, line 42. The shared example setup enables it in `examples/_config.py`, line 120, unless the caller overrides that setting. The chunked paths below assume it has been enabled [39, 17].

The unsatisfied prompt work is `get_len() - cache.num_tokens`. The scheduler first caps that work by the remaining query budget:

Listing 5.4: Fit a prompt chunk into the remaining position budget.
DeepInfer · csrc/engine/scheduler.hpp · lines 470–475

```
if (seq_workload > (max_num_batched_tokens - num_batched_tokens) / cp_align *
    ↪ cp_align) {
    new_workload = std::min(seq_workload, (max_num_batched_tokens -
    ↪ num_batched_tokens) / cp_align * cp_align);
    seqlen_q = new_workload;
    TORCH_CHECK(
        seqlen_q < seq_workload, "_schedule: is_chunked must be true:, seqlen_q:
    ↪ ", seqlen_q, " seq_workload: ", seq_workload);
}
```

Let T be `max_num_batched_tokens`, U the positions already charged to this batch and a the alignment. The available allowance here is

$$a \left\lfloor \frac{T - U}{a} \right\rfloor .$$

For example, with $T = 259$, $U = 252$ and $a = 4$, only four of the seven remaining positions are available to the next chunk. The constructor sets `cp_align` to 1 for one context-parallel participant, or twice the group size otherwise (line 90). Context parallelism divides context computation across devices; Appendix B shows how its ranks are grouped and how each sequence is split between them. For this chapter, alignment means that useful positions and charged positions need not match.

The cap is not the final chunk size. Lines 477–510 respect explicit checkpoints, move boundaries away from the inside of an image, check whether chunking is enabled and enforce a minimum chunk length. Once the chunk passes those checks, the scheduler commits the decision:

Listing 5.5: Charge padded positions and allocate before dispatch.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 515–526

```
515 num_batched_tokens += (seqlen_q + cp_align - 1) / cp_align * cp_align;
516
517 TORCH_CHECK(seq->schedule.status == SequenceStatus::WAITING, "status ", int(seq
    ↪ ->schedule.status));
518 if (seq->cache.block_table.empty()) {
519     acquire.allocate_seq(*seq);
520 }
521
522 if (!is_chunked) {
523     num_curr_seqs += 1;
524     seq->schedule.status = SequenceStatus::RUNNING;
525     allocate_mtp_info_slot(*seq);
526 }
```

The first line charges $a\lceil q/a \rceil$ positions for q useful queries. Five queries at alignment four cost eight budget positions. This is an accounting rule, not a claim that every padded position produces an output token.

The fresh request now owns blocks and a state slot. It enters the selected list and leaves `waiting`. If this is only a partial prompt, its status stays `WAITING`; if it is the final chunk, its status becomes `RUNNING`. Status is not queue membership: a partially processed request can temporarily be outside the waiting queue while its chunk is in flight.

When `process_outputs` handles an unfinished chunk, it puts the request back at the *front*:

Listing 5.6: An unfinished prompt returns ahead of waiting arrivals.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 1016–1019

```
1016 TORCH_CHECK(seq_ptr->schedule.status == SequenceStatus::WAITING,
1017     SPRINT_VARS(
1018         "In process_outputs, should be waiting", seq.cache.num_tokens, seq.input
     ↪ .tokens.get_len(), seq.get_nextn_len()));
1019 waiting.push_front(seq_ptr);
```

That is the front-return rule from the experiment, not its rotation rule. Real execution adds overlap, group agreement and the possibility of selecting several prompts in one step,

with a limit: a `NORMAL` engine stops selecting requests as soon as it selects an unfinished chunk (lines 543–545). That chunk is therefore the last request selected in the step. Returned to the front after processing, it again precedes later arrivals. The experiment's times do not describe this engine, but these two queue operations explain why chunking alone does not let a later short prompt overtake a long one.

5.2.4 Prefill can keep decode waiting

Combine that return to the front with the step-type rule. The worker processes a prompt step's outputs before returning, even with ZOS enabled (`worker.py`, lines 101–102 and 182–187). An unfinished A is therefore back at the head before the next scheduling call. Since prefill reset `decode_step` to zero, that call is another ALL attempt. With one rank, prefill can continue while the head remains schedulable, cache usage is at most the 0.9 watermark and the admission checks succeed. There need not be a decode step between A's chunks [7, 37].

With several ranks, consecutive ALL attempts require every rank to want prefill. At an ANY attempt, however, some ranks may have no local work. At either kind of attempt, a rank that wanted prefill may still admit nothing, for example because its head request needs blocks that are not free. If another rank has scheduled prefill, such ranks execute an empty prompt step (`scheduler.hpp`, lines 332–340); their running decode requests do not advance either. During these prompt steps the engine schedules no new decode work. It may still deliver results from an earlier decode step, which the worker post-processes before sampling the current step. Once those results are drained, waiting for the next decode step can lengthen chapter 2's inter-token intervals. This consequence follows from the inspected control flow; it is not a measured engine result.

The scope matters. `NORMAL` serves offline or combined prefill/decode execution. The production Q&A describes separate prefill and decode instances, the separate roles represented by `PROMPT` and `COMPLETION` in the released scheduler. The repeated choice between prefill and decode described here belongs to the normal engine, not that separated production architecture [2, 37].

5.2.5 A chunk becomes positions and addresses

Trace a text-only request with 300 prompt tokens. Use a normal engine with chunking enabled, a query budget of 128, alignment one, no prefix hit and no draft positions. Assume all other limits permit admission, enough memory is free and each previous chunk has been processed before the next is scheduled.

At its first admission, `get_num_blocks` asks for $\lceil 300/128 \rceil = 3$ blocks. The allocator also gives it a state slot. All three chunks keep those allocations:

Chunk	Cached before	Query length	Key length	Query positions
1	0	128	128	0–127
2	128	128	256	128–255
3	256	44	300	256–299

`Scheduler::schedule` builds a metadata record for each selected request. Its key length is the processed count plus in-flight positions plus this step's query length (lines 696–707). Our trace has no in-flight positions from another step. The `SequenceMetadata` constructor takes the block-table reference and state ID from the sequence (`sequence.hpp`, lines 374–381).

One downstream consumer makes the connection to chapter 3 explicit. The prefill input builder converts those lengths and that state ID into SWA write addresses:

Listing 5.7: Turn a scheduled prompt chunk into ring destinations.
DeepInfer · csrc/engine/input_metadata/prepare_tensor_input_v4.cpp · lines 98–109

```
98  auto slot_mapping_store_swa = get_or_create_tensor(result, has_out, "
       ↪ slot_mapping_store_swa", {total_q}, torch::kInt32);
99  auto p_slot_mapping_store_swa = slot_mapping_store_swa.mutable_data_ptr<int32_t
       ↪ >();
100 int32_t i_q = 0;
101 for (auto p_seq : seq_metadata_list) {
102     for (int32_t j = 0; j < p_seq->seqlen_q; ++j, ++i_q) {
103         if (p_seq->seqlen_q - j <= swa_block_size)
104             p_slot_mapping_store_swa[i_q] = p_seq->state_id * swa_block_size + (
       ↪ p_seq->seqlen_k - p_seq->seqlen_q + j) % swa_block_size;
105         else
106             p_slot_mapping_store_swa[i_q] = -1;
107     }
108 }
109 TORCH_CHECK_EQ(i_q, total_q);
```

Within a chunk, query j is at logical position $p = \texttt{seqlen_k} - \texttt{seqlen_q} + j$. For the second chunk, $j = 0$ gives $p = 128$. In state slot 2 with the default ring length 134, its destination is $2 \times 134 + 128 = 396$; query $j = 6$ wraps to $2 \times 134 + 0 = 268$. All our chunks fit within the ring. For a chunk longer than the ring, the condition keeps only its last ring-length positions; earlier destinations receive -1. Chapter 3 explained why prompt attention has a temporary input and reads the old cache before writing these final rows [38].

The scheduler has therefore chosen more than request names. It has chosen a range of logical positions, a budget charge and the storage through which the worker will address those positions.

5.3 When running requests exhaust the pool

Admission is not the last memory decision. A request already producing tokens can need another block. *Preemption* means taking resources away from such a request so another can proceed. It may suspend work for later recovery or terminate the request. DeepInfer has both outcomes.

Run a second experiment:

```
python3 labs/12_preempt.py
```

The pool has 19 usable blocks, each with four positions. A, B and C own all of them. B and C have just generated a token beyond the end of their last block; that token is in their history but has no cache row yet.

Request	History length	Output length	Processed	Blocks owned
A	20	8	19	5
B	45	2	44	11
C	13	4	12	3

Which request should give up its blocks? B owns the most, but it has generated the least output. These are different possible priorities. The experiment uses the inspected decode ordering, then applies each of the three branch conditions in `_preempt`:

```
stop=False, A.so_status=0
  output order: [('A', 8), ('C', 4), ('B', 2)]
  candidate=B; victim=A; blocks A=5, B=11, C=3
  A: WAITING, processed=0, blocks=[], state=-1
  waiting=['A', 'D'], ignored=[], free=[3, 2, 1]
  history retained=True; scheduled=['B', 'C']

stop=True, A.so_status=0
  output order: [('A', 8), ('C', 4), ('B', 2)]
  candidate=B; victim=A; blocks A=5, B=11, C=3
  A: FINISHED_PREEMPTED, processed=19, blocks=[], state=-1
  waiting=['D'], ignored=['A'], free=[3, 2, 1]
  history retained=True; scheduled=['B', 'C']

stop=False, A.so_status=1
  output order: [('A', 8), ('C', 4), ('B', 2)]
  candidate=B; victim=A; blocks A=5, B=11, C=3
  A: FINISHED_PREEMPTED, processed=19, blocks=[], state=-1
  waiting=['D'], ignored=['A'], free=[3, 2, 1]
  history retained=True; scheduled=['B', 'C']
```

A loses five blocks. B and C take one each, leaving three free. In the first run A goes ahead of D in the waiting queue and loses its processed cache count. Its 20-token history survives. In the second run a stop flag makes A finish; in the third an active structured-output status does so even though the stop flag is false. All three return A's blocks and state slot. The nonzero processed count printed for a finished A is no permission to read its freed storage.

5.3.1 Read the comparator and both ends of the queue

After admitting ready forks from `prompted`, the decode scheduler sorts `running` and selects requests:

Listing 5.8: Select from the short-output end; preempt from the other end.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 624–640

```
624 std::sort(running.begin(), running.end(),
625     [](const SequencePtr &a, const SequencePtr &b) { return a->get_output_len() > b->
    ↪ get_output_len(); });
626 scheduled.reserve(max_seqs);
627 size_t l = 0, r = running.size();
628 while (l < r && scheduled.size() < max_seqs) {
629     auto &seq = running[r - 1];
630     while (!block_manager->can_allocate_single_nolock() && r - l > 1) {
631         auto victim_seq = running[l];
632         _preempt(victim_seq, ignored_seqs);
633         l += 1;
634     }
635     if (block_manager->can_allocate_single_nolock()) {
636         _append_slot(*seq);
637         scheduled.push_back(std::move(seq));
638     } else {
639         _preempt(seq, ignored_seqs);
640     }
```

The comparator orders output lengths from greatest to least: A, C, B in the experiment. The candidate is `running[r - 1]`, so B goes first. If no block is free, the victim is `running[l]`, so A is preempted. Output length is already generated work, not remaining answer length or allocated memory. Nothing here can tell that B will finish soon.

Read two nearby comments with that trace in mind. The decode scheduling comment says it preempts the shortest output (line 557, author's translation), while the body takes a victim from the longest-output end. In `sequence.hpp`, line 56, the comment on the preemption finish reason says that the request occupying the most KV cache is forcibly terminated (author's translation). The example separates those quantities: A is selected although B owns more blocks. The control flow also permits recomputation. These comments do not fully describe the inspected policy.

There is another constraint in the body: even a candidate whose existing tail has space is considered only after the free-block check succeeds. `_append_slot` might then allocate nothing. Thus "my table is large enough" does not by itself guarantee that this loop will schedule the request. If no other victim remains and no block becomes free, it preempts the candidate itself. Pins from chapter 4 matter here: freeing one request does not necessarily free any shared block.

After choosing survivors, the scheduler sorts them again, this time by total history length, before building per-request query lengths (lines 648–657). The first sort determines who survives memory pressure; the second determines the batch order passed onwards. Ties in these comparators do not establish an arrival-order guarantee [37].

5.3.2 Termination and recomputation are different contracts

The default depends on the entry point. Unless the caller supplies a value, offline `LLM` sets `stop_preempted_request` to `False` (`llm.py`, line 68), giving the recovery policy used in the lab's first run. Construction paths that retain the `EngineArgs` default instead use `True` (`arg_utils.py`, line 40), which requests termination. The structured-output guard still applies in either case [4, 39].

The preemption branch first asks whether it is allowed to recover this request:

Listing 5.9: A stop flag or nonzero structured-output status prevents recomputation.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 770–782

```
770 const bool so_blocks_recompute = seq->structured_output.status != 0;
771 if (stop_preempted_request || so_blocks_recompute) {
772     if (so_blocks_recompute && !stop_preempted_request) {
773         LOG_WARN("Request {} is preempted and stops: structured output (
    ↪ so_status {}) can not be recomputed.",
774                  seq->request.request_id, seq->structured_output.status);
775     } else {
776         LOG_WARN("Request {} is preempted and stops.", seq->request.request_id);
777     }
778     seq->schedule.status = SequenceStatus::FINISHED_PREEMPTED;
779     free_seq_nolock(*seq);
780     ignored_seqs.push_back(seq);
781 } else {
782     LOG_WARN("Request {} is preempted and will be recomputed.", seq->request.
    ↪ request_id);
```

Structured output constrains generation to an allowed form, such as a grammar, and carries state beyond token IDs. The actual guard is `structured_output.status != 0`; it is not a test of whether the original request merely supplied a grammar. The log message says

this state cannot be recomputed by the present path. `Sequence::reset_for_recompute` helps us see the boundary:

Listing 5.10: Reset cached progress, scheduling progress and draft state.
DeepInfer · `csrc/engine/sequence.hpp` · lines 307–311

```
307 void reset_for_recompute() {
308     cache.reset_for_recompute();
309     schedule.reset_for_recompute();
310     mtp.reset_for_recompute();
311 }
```

The helpers reset the current and previous processed counts and the SWA valid start; they clear both in-flight counters and selected draft state. They do not erase the token history, sampling configuration or structured-output matcher (`sequence.hpp`, lines 134–142, 223–227 and 258–261). A recovery path would have to establish a compatible matcher state too. That is an inference from the state separation, not a claim that replaying a grammar is impossible [6].

In the recovery branch, line 783 calls this reset. Its Chinese comment points to the reset helpers as the inventory of rolling state (author's translation). The remaining operations put the request back and return its resources:

Listing 5.11: Put recovery work first, then release its allocations.
DeepInfer · `csrc/engine/scheduler.hpp` · lines 785–788

```
785     waiting.push_front(seq);
786     seq->schedule.status = SequenceStatus::WAITING;
787     free_seq_nolock(*seq);
788 }
```

When A returns to prefill in our case, its history still has 20 tokens but its processed count is zero. The prompt path therefore sees 20 positions of work, including tokens already generated. It rebuilds state for that history; it does not erase the eight output tokens and start a new answer. This recovery adds computation and delay. It does not by itself prove bitwise identity after replay; chapter 10 separates the relevant correctness contracts.

In the vLLM comparison, an unspecified preemption mode selects recomputation when `get_max_num_running_seqs()==1` and swapping otherwise; a configured mode can override this. Its recomputation path sets the sequence to `WAITING`, frees its blocks and resets computed progress (`vllm/core/scheduler.py`, lines 1472–1510) [13]. DeepInfer also has a rebuild-from-history path, but here the choice is between recomputation and termination, with engine-type and structured-output guards. Those are different recovery contracts despite the shared term `preempt`.

Recomputation also requires a route back to prefill. The scheduler constructor rejects a `COMPLETION` engine with `stop_preempted_request=false` (lines 102–104): that engine has no prefill scheduling path. Termination is reported as `insufficient_system_resource` by `get_finished_reason`. A larger service could retry elsewhere, but that is not an action performed by this branch.

Preemption may happen while a previous step is still in flight. In `process_outputs`, the comment and check at lines 917–924 describe skipping results for a decode request already reset to `WAITING`. Otherwise stale output could advance the newly reset count or return the request to the wrong queue. This is where scheduling meets chapter 6's subject: work can finish after the host has decided that its result is no longer usable.

5.4 A position budget is not a time budget

The released prefill path caps query positions, request counts, full lengths and allocations. These are useful constraints, but they are not equivalent to a maximum step duration. Chapter 2 showed that compute and memory costs depend on different quantities. Equal query counts can still read different amounts of context or route work to different experts.

The deployment Q&A describes an additional production policy: a fitted quadratic $\text{cost}(q, c)$, where q is the number of new positions and c the cached prefix length. A nanosecond budget named `compute_quota_limit` and a 32,768-token cap constrain the selected chunk. The policy also reduces the budget across a data-parallel group [2]. The inspected `_schedule_prefill` does not implement that quadratic solver. Its printed budget branch uses remaining token capacity and alignment. The timed model in the Q&A belongs to the documented service; we should not read it into these source lines. The runtime model used for draft verification is a separate path, which chapter 8 will examine.

We can now follow a batch decision from a waiting queue to a worker address, and follow memory pressure back to a reset or a reported failure. The remaining question is when those decisions can safely be made while an earlier batch is still executing.

5.5 Experiments to try next

1. In `labs/11_chunks.py`, remove the per-step overhead. Predict which first-token delays become equal and which still depend on queue order. Update the expected results before running the altered experiment.

2. In the memory experiment, give A and B equal output lengths. What additional rule would make the victim deterministic? Python's sort is stable; the C++ call shown here is `std::sort`. Do not infer the engine's tie behaviour from the lab.

3. Trace a candidate whose existing block table is large enough when the free list is empty. Locate the check that still causes a preemption. Propose a change, then list the ownership and in-flight cases it would need to preserve. Label the change as a proposal, not an existing DeepInfer policy.

4. In `_schedule_prefill`, compare a first admission, an unfinished chunk and a forked prefix. Which can own a state slot while its status is `WAITING`? Use this to revisit chapter 4's extra state-slot exercise.

Chapter 6

Keep the host and device busy

Chapter 1 found two oddities in DeepInfer's loop. One call to the worker can handle work from two steps, and the outer loop must flush a pending step after the scheduler reports no unfinished requests. Chapter 5 found a third: a result can arrive for a request that the scheduler has already reset. All three come from one design choice. The host plans the next step before it has read the result of the current one.

The question

The host must choose step $N + 1$ before it has read step N's result. What can it decide without that result? What must the device supply instead? When may storage or metadata from step N be reused?

6.1 Overlap has a limit

Two parties take part. The *host* is the CPU process that schedules requests, prepares metadata and submits work. The *device* executes submitted work in order on a queue, called a *stream*. Submitting work and executing it are different events. The host can queue a forward pass and move on before the device starts it. Chapter 4 introduced *events*: markers in the queue that the host can wait for.

A decode step has two device operations here: a *forward* pass that evaluates the model, and *sampling* that chooses tokens from its output. The host has four kinds of work: scheduling, submission, post-processing of sampled tokens, and engine callbacks. The next forward needs the tokens chosen by the previous sample. The question is whether the host must read them first.

```
python3 labs/13_overlap.py
```

The experiment assigns durations to these operations and simulates both orders. In the synchronous order, each call submits forward and sampling, waits for sampling to finish, then post-processes its own result. The ZOS order follows `Worker.execute_model_step` (`worker.py`, lines 113–189). During decode call $N + 1$, the host:

1. prepares inputs and queues forward $N + 1$;
2. waits for `sample_end_evt` of step N, then post-processes step N;
3. queues sampling for step $N + 1$ and runs the engine callbacks in `pre_sampler_sync`;
4. records `sample_end_evt` for step $N + 1$ and keeps the step's state for the next call.

The lab puts forward and sampling on one device queue, so forward $N + 1$ cannot start before sample N finishes even though the host queued it earlier. The worker's comment at line 231 states that ordering for the correction work queued before forward; this chapter's release table returns to the cases where a single queue is assumed rather than shown [7].

```
Assigned units. interval = repeating step; idle = device idle per step;
gap = device wait between forward and sample; wait = host wait for sample 3;
first = first sample end; finish = end of final post-processing.
case                       mode  interval idle gap wait first finish
baseline                   sync     20      8      0     10     16    120
baseline                   ZOS      12      0      0      2     16     80
slow scheduling            sync     29     17      0     10     25    174
slow scheduling            ZOS      19      7      0      0     25    120
long post-processing       sync     30     18      0     10     16    180
long post-processing       ZOS      20      8      5      0     16    127
correction before forward  ZOS      13      0      0      3     16     85

Timer view of the baseline: a host timer around 'launch' records 1 unit; the
   ↪ device forward it queued lasts 10.
```

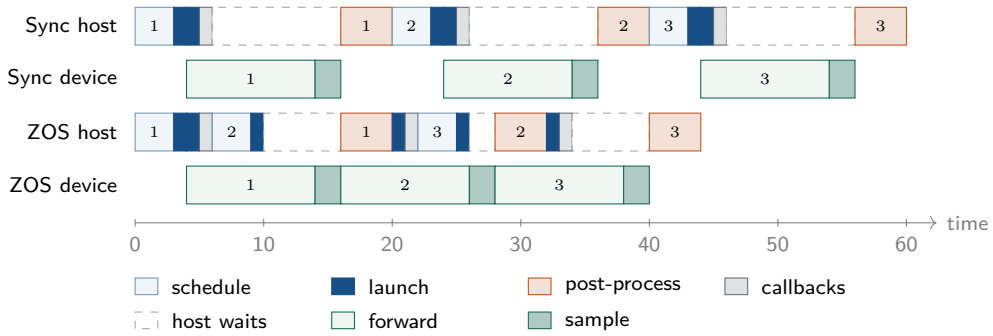


Figure 6.1: Three baseline decode steps from `labs/13_overlap.py`, drawn from data the script writes. Numbers name the step. In the synchronous run the host waits for each sample before post-processing it, so the device idles during that work and the next scheduling pass. Under ZOS, forward $N + 1$ is queued before the host waits for sample N. Durations are assigned units, not measurements.

In the baseline, the device does 12 units of work per step: forward 10 and sampling 2. The synchronous run takes 20. Its device is idle for 8 units while the host post-processes (4), schedules the next step (3) and submits the next forward (1). The host also waits 10 units for each sample. The ZOS run takes 12, the device's own work, and the host waits only 2. Its ten units of host work fit behind the next forward. Figure 6.1 draws the first three steps of both runs.

The other two cases show the limit. When scheduling takes 12 units, the host needs 19 units per step, more than the device's 12. ZOS still helps, from 29 to 19 units, but the device now idles for 7 units every step. Overlap can hide host work only behind device work that is already queued. In the third case, post-processing takes 14 units. The host cannot queue sample $N+1$ until it has post-processed step N, so the device finishes forward $N + 1$ and then waits. The gap is $14 + 1 - 10 = 5$ units: post-processing plus the sampling submission, less the forward. Overlap moved the waiting; it did not remove the dependency.

The first sample ends at time 16 in both baseline runs. Overlap does not shorten the first step, because nothing is queued before it. At the end, the ZOS run must still wait for

and post-process its pending step, the flush from chapter 1. The step interval describes repeating throughput; the fill and the drain are separate costs.

One more detail is visible in the code order. The worker records `sample_end_evt` after the callbacks, not immediately after queuing sampling (lines 161–168). The lab models that event as complete at the later of the sampling end and its own submission. Waiting for the event can therefore take longer than waiting for sampling itself would.

The loop below produces the timeline. `work` advances the host clock; `enqueue` starts device work at the later of that clock and the end of previously queued work. `wait` advances the host clock only if the event is still pending. The cost keys name scheduling (S), submission (L), post-processing (P), callbacks (C), forward (F), sampling (M) and optional correction (X):

Listing 6.1: Queue the next forward before collecting the previous result.
Book lab · `labs/13_overlap.py` · lines 38–56

```
38      for n in range(1, steps + 1):
39          work("schedule", n, costs["S"])
40          work("launch", n, costs["L"])
41          d2h_done = enqueue("correct", n, costs["X"]) if zos and costs["X"] and n
    ↪  > 1 else 0
42          enqueue("forward", n, costs["F"])
43          if zos and n > 1:
44              wait(sample_end[n - 1], n - 1)  # sample_end_evt(N).synchronize()
45              work("post", n - 1, costs["P"])
46              wait(d2h_done, n)  # d2h_evt.synchronize() before sampler metadata
47          work("launch", n, costs["L"])
48          enqueue("sample", n, costs["M"])
49          work("callbacks", n, costs["C"])
50          sample_end[n] = max(free, t)  # the event is recorded after callbacks
51          if not zos:
52              wait(sample_end[n], n)
53              work("post", n, costs["P"])
54      if zos:
55          wait(sample_end[steps], steps)  # final flush of the pending step
56          work("post", steps, costs["P"])
```

6.1.1 Which interval does a timer see?

The last line of the output repeats chapter 2's warning in a new form. A host timer around a submission measures the submission: one unit, while the forward it queued lasts ten. `CPUTimer.record` reads the host clock before and after the caller's block (`timer.py`, lines 50–63). The worker uses it around the wait in post-processing:

Listing 6.2: Time the wait, then read only completed graph timers.
DeepInfer · `deepinfer/worker/worker.py` · lines 262–266

```
262 with cpu_timer.record("worker.post_process.sync"):
263     step_state.sample_end_evt.synchronize()
264 with cpu_timer.record("worker.post_process.graph_timer_flush"):
265     # flush the time of completed graph, no effect on the running graph.
266     cuda_graph_timer.flush_out()
```

The first block measures how long the host waited for the device. In the lab's terms, that is the `wait` column: 2 units per ZOS baseline step and 0 when scheduling is slow. A

large wait means the host was ahead; a small wait with idle device time means the host was behind. Device timing comes from recorded events instead. `CUDAGraphTimer.flush_out` waits for each recorded pair before reading its elapsed time (lines 196–212), and the comment notes that only completed graphs are flushed. Neither kind of timer includes a request's queueing or delivery; chapter 2's boundaries still apply [46].

6.2 One request crosses two steps

Chapter 1 identified the asynchronous-scheduling precedents acknowledged by the deployment Q&A. The idea carried forward is to overlap host preparation with device execution. The question here is how DeepInfer makes that overlap work when the host's information is a step behind. Its implementation matches rows by request identity, supplies tokens and length corrections on the device, and retains results and metadata until their readers are finished. The next sections trace those obligations. They explain this implementation's choices without claiming that row remapping or deferred release originated here.

Overlap has a price in information. When the host schedules step $N + 1$, step N is still running. The host does not yet know which token step N will sample. If step N verified draft tokens, it does not know how many were accepted. The device knows both soon after sampling, and the next forward runs on the device. DeepInfer therefore lets the device complete the host's one-step-old metadata.

```
python3 labs/14_lagging_metadata.py
```

The experiment translates the inspected matching code and correction formula into lists. It builds a batch of four requests and checks the corrected inputs against a synchronous reference that already knows step N's result. We add one missing fact at a time.

6.2.1 The missing token

Begin without draft tokens (`nextn=0`). Each request evaluates one position per step, so the scheduler knows the length exactly. It lacks only the input token, which step N is still sampling.

```
A. missing token, same order: nextn=0, total_q=4, rows with padding=4, same-order
   ↪ path
  id remap q k_cpu base corrected true  inputs
  11     0 1    33   33        33   33  [501]
  12     1 1    34   34        34   34  [502]
  13     2 1    35   35        35   35  [503]
  14     3 1    36   36        36   36  [504]

B. reordered survivors and a new request: nextn=0, total_q=4, rows with padding=4,
   ↪  seq_id map
  id remap q k_cpu base corrected true  inputs
  13     2 1    35   35        35   35  [503]
  11     0 1    33   33        33   33  [501]
  15    -1 1    41   41        41   41  [515]
  12     1 1    34   34        34   34  [502]
```

Rows are matched by `seq_id`, a stable request identifier, not by position in the batch. The `remap` column gives the matching row of the previous batch, or -1 when there is none. A matched row reads its input token from the previous sample's output on the device.

Request 15 was not in step N; the host already knows its token, 515, and supplies it. In the second batch the survivors have been reordered, as chapter 5's length sort can do, and request 14 has left. The order changed, but every matched row still finds its token. The corrected and synchronous key lengths agree because nothing about length was missing.

6.2.2 The missing accepted length

Now let each request verify two draft tokens per step (`nextn=2`), so each step evaluates three positions. After verification, a request keeps its main token plus the accepted drafts. The host cannot know that number while scheduling step $N + 1$.

The scheduler reserves the maximum. When it schedules a request, it records the step's query length as in flight (`scheduler.hpp`, line 726). The next step's key length is the committed count plus in-flight positions plus the new query length (line 697). That is an upper bound [37].

```
C. missing accepted length, EOS and graph padding: nextn=2, total_q=12, rows with
   ↪ padding=8, same-order path
  id remap q k_cpu base corrected true  inputs
  11     0 3    37   34        35   35  [601, 611, 612]
  12     1 3    38   35        37   37  [2, 621, 622]
  13     2 3    39   36        39   39  [603, 631, 632]
  14     3 3    40   37        40   40  [604, 641, 642]
  evaluated although step N sampled EOS: [12]; their step N+1 output is not
   ↪ published
```

Follow request 11. It had 31 committed positions before step N. Step N reserved 3 and step $N + 1$ asks for 3, so the host's key length is 37. With drafts enabled, the C++ preparation subtracts the current query length, giving a base of 34 (`zos.cpp`, line 188). The device then applies the reference formula. For real row i with remap r_i and query length q_i,

$$k'_i = k_{\text{base},i} + q_i - \begin{cases} \texttt{nextn} - a_{r_i}, & r_i \geq 0, \\ 0, & r_i = -1, \end{cases}$$

where a_{r_i} counts the drafts accepted in the matched previous row, excluding the main token. Request 11 accepted none: $34 + 3 - (2 - 0) = 35$. The synchronous reference agrees: $31 + 1 + 0 = 32$ committed positions after step N, plus three new queries. Request 13 accepted both drafts, so its reserved length was already right: $36 + 3 - 0 = 39$. The base exists so that the formula does not add q twice [40, 43].

The input tokens change too. A matched row's first input is the token sampled in step N; the remaining inputs are the new drafts that step N proposed. Request 12 shows a further consequence. Its step N sample is the end-of-sequence token, number 2 here. The host learns this only when it post-processes step N, after forward $N + 1$ has been queued. Step $N + 1$ therefore evaluates request 12 anyway. Evaluation and publication are separate: the row can be computed, but its result will not be returned to the user. Section 6.4 follows that row to the end of its life.

The batch has 12 query positions, so the worker selects a captured graph for 16 positions and adds four padding rows. There are now eight metadata rows: four real requests with three positions each, and four padding rows with one position each. The script checks that the padding rows keep $q = k = 1$ and remap -1: the correction applies only to the first `n_seq` real rows. Section 6.5 explains the padding.

The lab computes the inputs and lengths with this list calculation. It visits only the real `rows`, leaving padding unchanged, and checks the results against a separate reference based on the known accepted counts:

Listing 6.3: Translate token selection and length correction into lists.
Book lab · `labs/14_lagging_metadata.py` · lines 63–71

```
63 def correct(rows, prev, remap, q, k, nextn):
64     inputs = []
65     for i, row in enumerate(rows):
66         old = prev[remap[i]] if remap[i] >= 0 else None
67         inputs.append([row.fresh[j] if old is None else
68                        old.sampled if j == 0 else old.drafts[j - 1] for j in
   ↪ range(q[i])])
69         if nextn:  # worker.py:220-227; padded rows are left unchanged
70             k[i] += q[i] - (nextn - old.accepted if old else 0)
71     return inputs
```

6.2.3 A query length chosen on the device

Chapter 8 adds a third unknown. With variable-length verification, the number of drafts each request verifies depends on confidence estimates from the previous sample. The host fixes only the step's total query length; the device divides it among requests. Chapter 8 describes that allocation. Here only its dependency matters. The sampler's host metadata needs the per-request lengths, which now exist only on the device. A non-blocking copy returns them to pinned host memory, and the host must wait for the copy's event, `d2h_evt`, before updating that metadata (`worker.py`, lines 146–152).

The last row of lab 13 adds this case as a one-unit correction queued before forward. It runs after sample N and before forward $N + 1$ on the same stream. Its event is therefore complete soon after the host's existing wait for sample N; the step lengthens by the correction's device time, from 12 to 13 units. That result depends on the assigned durations. The code establishes only the order.

6.3 Read the correction path

The correction happens in `Worker._correct_seqlens_qk`, called before forward when the worker has a previous decode step. It first builds the host-side inputs:

Listing 6.4: Match the batches, then copy the host arrays to the device.
DeepInfer · `deepinfer/worker/worker.py` · lines 209–216

```
209 remap_cpu, seqlens_q_cpu, seqlens_k_cpu, has_overlap = prepare_zos_inputs(
210     seq_metadata_list, prev.seq_metadata_list, n_seq_with_padding, self.nextn
211 )
212 if not has_overlap:
213     return None
214 remap = remap_cpu.cuda(non_blocking=True)
215 seqlens_q = seqlens_q_cpu.cuda(non_blocking=True)
216 seqlens_k = seqlens_k_cpu.cuda(non_blocking=True)
```

`prepare_zos_inputs` first tries the cheap case. If both batches have the same length and the same `seq_id` at every position, row i matches row i (`zos.cpp`, lines 148–157). Otherwise it builds a map:

Listing 6.5: Match by request identity when the order has changed.
DeepInfer · `csrc/engine/zos.cpp` · lines 159–176

```
159 bool has_overlap = same_order && !seq_metadata_list.empty();
160 if (!same_order) {
161     std::unordered_map<int64_t, int32_t> prev_indices;
162     prev_indices.reserve(prev_seq_metadata_list.size());
163     for (int32_t i = 0; i < static_cast<int32_t>(prev_seq_metadata_list.size());
    ↪  ++i) {
164         if (!prev_seq_metadata_list[i]->is_placeholder) {
165             prev_indices.insert_or_assign(prev_seq_metadata_list[i]->seq_id, i);
166         }
167     }
168     for (int32_t i = 0; i < static_cast<int32_t>(seq_metadata_list.size()); ++i)
    ↪  {
169         auto prev = prev_indices.find(seq_metadata_list[i]->seq_id);
170         remap_indices[i] = prev == prev_indices.end() ? -1 : prev->second;
171         has_overlap |= remap_indices[i] >= 0;
172     }
173 }
174 if (!has_overlap) {
175     return {std::nullopt, std::nullopt, std::nullopt, false};
176 }
```

Placeholder rows from the previous step are excluded; they carry no request. If no row matches, the function returns nothing and the worker skips correction. The host then has complete information for every row in the batch. The function next writes the arrays that the device will correct:

Listing 6.6: Write base lengths for real rows and fixed values for padding.
DeepInfer · `csrc/engine/zos.cpp` · lines 184–194

```
184 for (int32_t i = 0; i < static_cast<int32_t>(seq_metadata_list.size()); ++i) {
185     const auto *seq = seq_metadata_list[i];
186     remap_ptr[i] = remap_indices[i];
187     seqlens_q_ptr[i] = seq->seqlen_q;
188     seqlens_k_ptr[i] = seq->seqlen_k - (nextn > 0 ? seq->seqlen_q : 0);
189 }
190 for (int64_t i = seq_metadata_list.size(); i < n_seq_with_padding; ++i) {
191     remap_ptr[i] = -1;
192     seqlens_q_ptr[i] = 1;
193     seqlens_k_ptr[i] = 1;
194 }
```

Line 188 is the subtraction that defines the base. Lines 190–194 fill the padding rows that the lab checked. Back in `_correct_seqlens_qk`, the worker queues length correction before calling `execute_main_model`:

Listing 6.7: Correct lengths on the device; copy query lengths back if they vary.
DeepInfer · `deepinfer/worker/worker.py` · lines 218–234

```
218 if envs.DEEPINFER_ENABLE_MTP_VARLEN:
219     compute_seqlens_q_by_confidence_rates(total_q, remap[:n_seq], prev.
    ↪ sampler_output.confidence_rates_gpu, seqlens_q[:n_seq])
220 if self.nextn > 0:
221     compute_seqlens_k_zos(
222         seqlens_k[:n_seq],
223         seqlens_q[:n_seq],
224         remap[:n_seq],
225         prev.sampler_output.num_accepted_tokens_gpu,
226         nextn=self.nextn,
227     )
228
229 d2h_evt = None
230 if envs.DEEPINFER_ENABLE_MTP_VARLEN:
231     # H2D, correction, and D2H are ordered on the current stream; only varlen
    ↪ rewrites this pinned source.
232     seqlens_q_cpu[:n_seq].copy_(seqlens_q[:n_seq], non_blocking=True)
233     d2h_evt = torch.cuda.Event()
234     d2h_evt.record()
```

The first branch is chapter 8's allocation; the second is the key-length formula. Both slice to `n_seq`, leaving padding untouched. The comment states the ordering that lab 13 assumed: the host-to-device copy, the correction and the return copy are all queued on the current stream, ahead of forward. The kernel's arithmetic has a reference implementation in the same module:

Listing 6.8: The reference arithmetic behind the fused kernel.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/compute_seqlens_k_zos.py` · lines 44–52

```
44 seqlens_k += seqlens_q
45 mask = remap != -1  # [n_seq] bool
46 # Clamp remap to valid index (use 0 for fresh seqs, will be masked out)
47 remap_clamped = remap.clamp(min=0)  # [n_seq]
48 # Gather prev_num_accepted_tokens for all seqs (invalid ones get tokens[0],
    ↪ doesn't matter)
49 accepted = prev_num_accepted_tokens[remap_clamped.long()]  # [n_seq]
50 # Compute adjustment: only apply where mask is True
51 adjustment = torch.where(mask, nextn - accepted, torch.zeros_like(seqlens_k))
52 seqlens_k -= adjustment
```

Lab 14 translates these lines. Scenarios A and B check matching and fresh inputs without drafts; scenario C checks accepted-length correction for matched requests and leaves padding unchanged. Fresh requests with drafts are not exercised. These checks do not run the kernel or test its handling of real tensor layouts [43].

Model input preparation then uses the corrected query lengths to decide which token positions to gather. Fresh rows get their tokens from the host (`zos.cpp`, lines 99–113); matched rows read the previous outputs on the device. The gather's docstring states both branches:

Listing 6.9: Matched rows read the previous outputs; fresh rows read host tokens.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/gather_input_ids_zos.py` · lines 21–30

```
"""Gather input_ids from fresh_tokens or prev sampled/predict tokens.

For each (seq i, token j) with j < seqlens_q[i], let q_base = cu_seqlens_q[i-1]
(0 for i == 0):
    if remap[i] >= 0 (old seq):
        j == 0: out[q_base+j] = prev_sampled_token_ids[remap[i]]
        j >  0: out[q_base+j] = prev_predict_token_ids[remap[i], j-1]
    else (fresh seq):
        out[q_base+j] = fresh_tokens[i, j]
"""
```

`prepare_token_ids_zos` calls this gather when a remap exists, passing the corrected `seqlens_q` (`prepare_tensor_input_zos.py`, lines 235–241). For matched rows, the previous step's sampled token and new drafts reach the next forward without a round trip through the host [40, 42].

6.3.1 Two sets of metadata describe one step

The corrected lengths feed forward $N + 1$. Sampling for step $N + 1$ needs host-side metadata too, and it is prepared later. By the time the worker reaches it, it has post-processed step N, so each sequence's committed count now includes step N's accepted tokens. `update_metadata_for_sampling_zos` uses that count:

Listing 6.10: Rebuild sampling lengths from the newly committed count.
DeepInfer · `csrc/engine/zos.cpp` · lines 299–303

```
metadata->seqlen_k = metadata->seqlen_q + seq.cache.num_tokens;
if (remap_ptr[i] >= 0) {
    metadata->refresh_for_sampling(seq, nextn);
}
scheduler_outputs.seqlens_q[i] = metadata->seqlen_q;
```

The comment above, line 298, explains the choice in the author's translation: an extra forward after end-of-sequence may not have appended its token to the sequence, so sampling uses the cached length. Request 12 in lab 14 is such a case. Matched rows also refresh their draft and sampling inputs from the sequence. Forward metadata and sampling metadata both describe step $N+1$, but they become valid at different times and from different sources: the device correction, then the host's committed count [40].

6.3.2 When the correction does not run

ZOS is on by default only for some engine types: `NORMAL` and `COMPLETION`, not `PROMPT` (`engine_type.py`, lines 16–17). `EngineArgs` applies that default with `os.environ.setdefault`, so an existing `DEEPINFER_ZERO_OVERHEAD_SCHEDULING` setting wins (`arg_utils.py`, lines 51–55). Prompt steps always post-process their own result in the same call; chapter 1 printed that branch. A prompt step can still post-process a preceding decode step, because that

happens earlier in the call. Correction returns early when there is no previous state, when the previous step was a placeholder, or when no row matches (`worker.py`, lines 198–213) [44, 39, 7].

6.4 Last use, reuse and the cost of end-of-sequence

Request 12 sampled its end-of-sequence token in step N. Follow it through three consecutive decode calls:

1. Call N queues the forward and the sample that choose the end-of-sequence token. The host has not read it.

2. Call $N+1$ schedules request 12 again and queues forward $N+1$ with its row. It then post-processes step N, which marks the request finished and publishes its last token. Sampling for step $N+1$ still includes the row.

3. Call $N+2$ begins with scheduling. `free_finished_seqs` removes the finished request from the running list and hands it on as finished (`scheduler.hpp`, lines 562 and 752–763). Forward $N+2$ no longer includes it. Post-processing step $N+1$ waits for its sampling event and skips the finished row (lines 905–913). `pre_sampler_sync` then releases its storage:

Listing 6.11: Finished requests return their storage after the wait.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 844–846

```
844 if self.engine_type in [EngineType.COMPLETION, EngineType.NORMAL]:
845     for seq in scheduler_outputs.finished_seqs:
846         self.finalize_seq(seq)
```

`finalize_seq` calls the scheduler's `free_seq` (lines 804–806), which chapter 4 followed into the block allocator. The last device access to request 12's storage belongs to step $N+1$. The worker waited for that step's sampling event earlier in call $N+2$, so the release follows the last use [5, 33].

The trace assumes consecutive decode calls. A prefill step in between changes when the decode cleanup pass runs, so the release point needs a separate trace.

The extra row is the price of not waiting. Each finished request is evaluated once more after its last published token. As an illustration, suppose there are no drafts and each decode step produces one useful token. If answers averaged 200 tokens, one extra evaluation per answer would add about 0.5 per cent to decode rows, ignoring the first token produced by prefill. That figure is arithmetic, not a measurement. With drafts, both the number of normal decode steps and the number of positions in the extra step change. Chapter 5's stale-result guard is the same idea applied to a request preempted while its step was in flight, and chapter 1's final flush drains the one step that no later call will collect.

6.4.1 What makes a release safe

Chapter 4's second lab released a block while a queued copy still needed it, and a new writer corrupted the copy. That lab did not prove that every release after submission is unsafe. The governing condition is narrower: a conflicting use by a new owner must follow the last access by the old owner. A host can meet it by waiting for an event. It can also rely on an order that is guaranteed some other way, such as two operations on the same stream. Allocator reuse, the lifetime of host objects and the publication of a result are different

questions, and DeepInfer answers them differently. Table 6.1 lists the release points met so far.

Resource or release	Condition that permits reuse
Preempted KV blocks	`_preempt` returns allocations at once, while an earlier step may still be in flight. Same-stream order is a plausible explanation for safe reuse, not a complete proof here; it needs the audit below.
Finished request	In consecutive decode calls, the worker has waited for the extra step's sampling event before `finalize_seq` releases storage.
Fork copy source	`clone_done_evt` completes before the source's pin is dropped (chapter 4).
Output pool	Returned addresses stay valid for the caller until the next release phase; the caller must clone what it keeps (chapter 1). This is a host-facing contract, not a device event condition.
Sequence metadata	`WorkerStepState` keeps it until post-processing. `LLMEngine.post_process` then collects it for deferred release (line 1013); `DeferredRelease.release` returns it to the pool in `pre_sampler_sync` (lines 100–103).
Borrowed scheduler lists	The copy and zero lists must be consumed before the next scheduling call clears and reuses their storage (`scheduler.hpp`, lines 279–284).

Table 6.1: Release points and the condition that permits reuse. Only the first row lacks a complete argument in the inspected source.

The first row remains open. A complete argument must account for every access to the reused cache: eager forward, graph replay, draft execution, copies and zeroing, and any storage hooks. Graph capture uses its own stream with explicit waits (`cuda_graph_runner.py`, lines 100–137); that fact alone neither proves nor disproves safe runtime reuse. The release's storage hooks are empty, so they say nothing about the unreleased service. Until that audit is done, the first row is a scoped inference from the inspected control flow [5, 41, 45].

6.5 Graphs constrain the input buffers

A *CUDA graph* records a fixed sequence of device operations, with fixed shapes and buffer addresses, so that the host can later replay it with one submission. Fewer submissions mean less host work per step, which lab 13 showed to be the limit on overlap. The cost is rigidity.

DeepInfer captures graphs for a list of batch sizes: powers of two up to an interval, then multiples of that interval, with a default interval of 8 (`cuda_graph_runner.py`, lines 41–51; `environ.py`, line 155). The largest size is `max_num_seqs` times 1 + `nextn`, and `PROMPT` engines do not use decode graphs (`model_runner.py`, lines 72–79). Selection uses `total_q`, the number of query positions, not the number of requests (`cuda_graph_runner.py`, lines 143–148). Chapter 2's distinction between requests and positions decides which graph runs. The worker adds padding rows to reach the selected size (`worker.py`, lines 202–207). Lab 14's 12 positions became 16.

The graph's inputs must also be where it expects them. Under ZOS, the runner is built with device-side input storage (`model_runner.py`, line 78), and the graph data's docstring says that source tensors share device storage in that mode (`cuda_graph_runner.py`, lines 26–31). The corrections in this chapter therefore write into buffers that replay reads. Correcting a host copy instead would require the host to read the previous result first, which is the wait ZOS was built to avoid [45].

We can now label each piece of state by the step that produced it, the event or stream order that allows the next read, and the condition that allows reuse. The next chapter spreads this work across devices, where a step must also wait for other devices.

6.6 Experiments to try next

1. In `labs/13_overlap.py`, find the smallest post-processing cost that creates a gap between forward and sample in the ZOS run. Derive it from the other durations before running the script.

2. Set the forward duration to 2 and keep the baseline host costs. Which side limits the ZOS interval? What would graph replay need to reduce to help?

3. In `labs/14_lagging_metadata.py`, give request 14 an accepted count of 0 in scenario C. Predict its base, corrected and true key lengths.

4. Mark one previous row as a placeholder in scenario B. Translate the placeholder check from `zos.cpp`, lines 151 and 164, into the lab, then predict which rows match.

5. For one row of the release table, list every access you would need to find to turn its condition into a proof. Separate device accesses from host object lifetimes.

Chapter 7

Put experts on several devices

Chapter 3 placed attention and a feed-forward transformation inside each model layer. We now open that second part. DeepSeek's V4 models use many networks called *experts* in their MoE layers. For every token, a *gate* scores the experts and chooses a few. Each token therefore uses only part of the layer's weights, but different tokens use different parts. When the experts are spread over many devices, tokens must travel to the experts they chose, and the results must travel back.

The question

How does DeepInfer send each token's expert work to the devices that hold those experts and bring the results back? Why can one busy device limit the benefit, and how does the engine move experts to reduce that imbalance?

7.1 Dispatch, compute and return

A layer built this way is a *mixture of experts* (MoE). The gate gives each token a short list of chosen experts and a weight for each. The layer's output is the weighted sum of the chosen experts' outputs. DeepSeek's design also has a *shared expert* that every token uses, without being chosen [28].

```
python3 labs/15_dispatch.py
```

The experiment gives four experts, A to D, simple integer formulas, and the shared expert the formula $x + 10$. Two devices, called *ranks*, each own three tokens. Each expert occupies a *slot* on one rank. The routing weights are fractions, so the distributed result can be compared exactly with direct evaluation.

```
1. one expert per token: slots=['A', 'B', 'C', 'D'], top-1
  assignments per rank=[5, 1], busiest=5, sent to another rank=4
  per slot: 0:A=4, 1:B=1, 3:D=1
2. two weighted experts: slots=['A', 'B', 'C', 'D'], top-2
  assignments per rank=[8, 4], busiest=8, sent to another rank=6
  per slot: 0:A=5, 1:B=3, 2:C=2, 3:D=2
3. same experts, other placement: slots=['A', 'C', 'B', 'D'], top-2
  assignments per rank=[7, 5], busiest=7, sent to another rank=5
  per slot: 0:A=5, 1:C=2, 2:B=3, 3:D=2
4. replicas of A and B: slots=['A', 'B', 'C', 'D', 'A', 'B'], top-2
  assignments per rank=[6, 6], busiest=6, sent to another rank=4
  per slot: 0:A=2, 1:B=2, 2:C=2, 3:D=2, 4:A=3, 5:B=1
token results: (0, 0)=41/2, (0, 1)=21, (0, 2)=65/3, (1, 0)=107/3, (1, 1)=11, (1,
   ↪ 2)=124/5
```

The first run gives each token one expert. *Dispatch* sends each token's value to the rank holding its expert. The receiving rank computes the result, and *combine* returns it to the token's own rank. A token must carry its identity on the journey, or its result could not be put back in the right place. Four of six tokens chose A, so rank 0, which holds A and B, receives five of the six assignments.

The second run gives each token two experts with weights. A token now creates two assignments, possibly on different ranks. Each result is multiplied by its weight before it returns; the origin adds the shared expert's output, computed locally, and the returned contributions. The script checks every token's result against direct evaluation. Rank 0 receives 8 assignments and rank 1 only 4.

The third run moves B and C between ranks. Every token result is unchanged; only the work per rank and the number of assignments crossing ranks change. That is the freedom a placement policy uses.

The fourth run adds two slots, holding a second copy of A and of B. We now distinguish a *logical expert*, the function the gate chooses, from a *physical expert*, a copy of its weights in one slot. A copy is called a *replica*. A replica adds another place to compute A; it does not add an expert to any token's choice. The lab picks a replica with DeepInfer's rule, shown in the next section. Rank 0 and rank 1 now receive six assignments each, and the results are still unchanged.

The dispatch loop keeps the origin rank and token index with each assignment. `replicas` maps an expert to its slots; integer division finds a slot's owner:

Listing 7.1: Choose a replica and keep the token's return address.
Book lab · `labs/15_dispatch.py` · lines 42–53

```
    per_rank = len(slots) // 2
    replicas = {e: [s for s, name in enumerate(slots) if name == e] for e in
    ↪ EXPERTS}
    inbox, sent = {0: [], 1: []}, 0
    for token in TOKENS:  # dispatch: each selection goes to one physical slot
        rank, index, x, picks = token
        norm = sum(w for _, w in picks[:top_k])
        for expert, weight in picks[:top_k]:
            copies = replicas[expert]
            slot = copies[(rank + index * 23333) % len(copies)]
            owner = slot // per_rank
            inbox[owner].append(((rank, index), slot, x, weight / norm))
            sent += owner != rank
```

Each owner computes its contributions. The same token key groups them on return, and `direct` supplies a reference without dispatch:

Listing 7.2: Recombine weighted results by token identity.
Book lab · `labs/15_dispatch.py` · lines 54–59

```
    partial = {}  # compute on the owner; the weight is applied before return
    for owner, items in inbox.items():
        for key, slot, x, weight in items:
            partial.setdefault(key, []).append(weight * EXPERTS[slots[slot]](x))
    combined = {(r, i): shared(x) + sum(partial[(r, i)]) for r, i, x, _ in
    ↪ TOKENS}
    expected = {(r, i): direct((r, i, x, p), top_k) for r, i, x, p in TOKENS}
```

The exact check proves that the teaching model routes and recombines correctly. It does not establish that DeepInfer's floating-point kernels give bitwise-equal results for different placements; a different summation order can change the last bits.

7.2 Follow DeepInfer's execution path

`DeepseekMoE` in `v4_modules/mlp.py` executes the layer; the neighbouring `moe.py` defines the parameter containers it receives. Its constructor fixes the slot arithmetic (lines 196–202): the number of physical experts is the number of routed logical experts plus the extra replica slots, and each rank of the *expert-parallel* (EP) group holds an equal share. An assertion requires the total to divide evenly [47].

The execution crosses several repositories, all within DeepSeek's library stack. DeepGEMM, DeepEP and TileKernels supply kernels and communication; DeepInfer connects them to the model and the request loop. The boundary is concrete:

Boundary	Division of responsibility
Replica selection	DeepInfer supplies logical-to-physical maps and replica counts; the DeepGEMM or TileKernels gate chooses a physical copy.
Dispatch and combine	DeepInfer calls DeepEP's `EPBuffer` with rows, weights, events and a load counter, and uses the returned handle.
Expert computation	DeepInfer selects the explicit or fused path and connects the shared contribution; DeepGEMM and TileKernels perform the kernel work.
Load balancing	The statistics window, policy call, weight shuffle and mapping updates are implemented in this repository.

The inspected calls and pinned dependencies establish these boundaries [47, 48, 55]. The model determines the routed expert functions and their selections. Extra physical copies are a deployment choice for serving uneven routed work, rather than additional logical experts required by the model.

7.2.1 From a token's choice to a slot

The forward method has two paths. Begin with the explicit one:

Listing 7.3: Take the fused path, or gate and dispatch explicitly.
DeepInfer · deepinfer/model_executor/models/v4_modules/mlp.py · lines 571–585

```
571 if self.use_mega_moe:
572     return self.mega_moe_forward(hidden_states, hidden_states_bf16,
    ↪ input_metadata)
573
574 topk_idx, topk_weights = self.gate_forward(hidden_states_bf16, input_metadata)
575
576 try:
577     # Dispatch
578     event_overlap = self.dispatch(
579         hidden_states,
580         topk_idx,
581         topk_weights,
582         input_metadata,
583         None,
584     )
585     moe_in_hidden, _dispatched_topk_idx, dispatched_topk_weights, handle =
    ↪ event_overlap.current_stream_wait()
```

The gate returns, for each token, the chosen experts and their weights. Those indices are already physical slots. The gate receives the load balancer's maps:

Listing 7.4: The gate maps logical choices to physical slots.
DeepInfer · deepinfer/model_executor/models/v4_modules/mlp.py · lines 392–400

```
392     to_physical_map=eplb_layer.logical_to_physical_map,
393     logical_count=eplb_layer.logical_count,
394     scoring_func=self.scoring_func,
395     out=out,
396 )
397 if self.use_mega_gate:
398     topk_idx, topk_weights = deep_gemm.bf16_mega_gate(hidden_states, self.gate.
    ↪ weight, **gate_kwargs)
399 else:
400     topk_idx, topk_weights = moe_topk_gate(self.gate(hidden_states), **
    ↪ gate_kwargs)
```

`to_physical_map` lists the slots holding each logical expert, and `logical_count` says how many there are. On GPUs the gate runs in DeepGEMM's `bf16_mega_gate`; otherwise it uses the TileKernels gate. Both are DeepSeek libraries maintained in separate repositories. At DeepInfer's pinned submodule commit, DeepGEMM chooses a replica by

$$(\texttt{rank} + 23333 \times \texttt{token}) \bmod \texttt{logical_count}[e],$$

where `token` is the token's index in its own rank's batch. The source comment says the purpose: spread one logical expert's tokens over its replicas, differently on every rank. The TileKernels version required by DeepInfer uses the same rule [52, 53].

The rule does not look at current load. It spreads tokens by their index, so replicas share work only approximately. In lab 15, A's five assignments split two and three. The placement policy later in this chapter assumes an equal split; that assumption is an estimate.

Follow token $(0, 1)$ in the lab's fourth run. Its value is 5 and its logical choices are A and C, each with weight $1/2$. The layout is ABC/DAB, so A's slots are $[0, 4]$ and C's only slot is 2. For A, the replica rule gives $23333 \bmod 2 = 1$: the second copy, slot 4 on rank 1. For C it gives zero: slot 2 on rank 0. The equivalent row of `topk_idx` is therefore $[4, 2]$, paired with `topk_weights` $[1/2, 1/2]$.

7.2.2 Through the experts and back

`dispatch` hands the hidden states, slot indices and weights to DeepEP's communication buffer [47]. Like the asynchronous work in chapter 6, the transfer runs on its own schedule. `current_stream_wait` makes the compute stream wait for its event before using what arrived. The results are the rows received by this rank, their weights and a *handle*. The handle carries the bookkeeping needed later: how many received rows belong to each local expert, used by the expert computation (line 596), and the routing information needed to send results home.

For our token, dispatch sends one row to rank 1's local slot 1 (global slot 4) and one to rank 0's local slot 2. The received weights follow those rows into `routed_experts_forward`; the handle's `psum_num_recv_tokens_per_expert` describes their grouping by local expert. DeepInfer's rows are hidden-state vectors; the lab uses the scalar 5 so that we can calculate the contributions by hand.

Where do the weights take effect? Not in combine. Inside the expert computation, the activation step receives them:

Listing 7.5: Routing weights scale each row inside expert computation.
DeepInfer · `deepinfer/model_executor/models/v4_modules/mlp.py` · lines 436–442

```
436 down_in_hidden = swiglu_forward_and_per_token_cast(
437     gate_up_proj_out,
438     **_get_swiglu_kwargs(self.use_fp8_act),
439     psum_num_tokens_per_expert=psum_num_recv_tokens_per_expert,
440     topk_weights=topk_weights,
441     clamp_value=self.swiglu_clamp_value,
442 )
```

This matches the lab, which multiplies each result by its weight before return. The name `combine` could suggest that the weighted sum happens there; the code shows that the rows are already weighted when they leave the experts.

Return and the shared expert follow:

Listing 7.6: Compute the shared expert locally; combine adds it on return.
DeepInfer · deepinfer/model_executor/models/v4_modules/mlp.py · lines 600–610

```
600 # Combine
601 combine_pre_evt = self.ep_buffer.capture()
602 with limit_num_sms(envs.DEEPINFER_USERBUFFER_COMM_SMS, enabled=PLATFORM.is_gpu):
603     shared_out = self.shared_forward(hidden_states)
604 event_overlap = self.combine(
605     moe_out_hidden,
606     shared_out,
607     handle,
608     combine_pre_evt,
609 )
610 moe_combined_hidden, _combined_topk_weights = event_overlap.current_stream_wait
    ↪ ()
```

The shared expert runs on the token's own rank. `limit_num_sms` lowers the number of streaming multiprocessors that its kernels may use by the number set aside for communication, so the transfers keep their share of the device (`parallel_utils/userbuffer.py`, lines 40–51). `combine` passes its output as `bias` (line 505): the shared contribution is added when the routed results arrive. A token's final row is therefore the shared output plus its weighted routed results, as in the lab [47].

In the scalar trace, A returns $(2 \times 5 + 1)/2 = 11/2$ from rank 1 and C contributes $(5 - 4)/2 = 1/2$ on rank 0. The dispatch handle lets combine return the remote contribution to the original row on rank 0. The shared contribution is $5 + 10 = 15$, supplied as `bias` in the explicit engine path. The final value is $11/2 + 1/2 + 15 = 21$, the lab's result for $(0, 1)$.

7.2.3 The fused path

The explicit stages are a readable account of the data movement, but they are not the only path. Lines 184–188 require the whole EP group to lie within one NVLink domain, a set of devices joined by a fast direct interconnect. They also require either a GPU platform or FP8 activations with FP4 weights, and a supported matrix-multiplication format. When these conditions hold, `forward` returns early through `mega_moe_forward` (lines 528–563).

That method still calls `gate_forward`, writing the choices into a symmetric buffer shared across the group. A single DeepGEMM call then receives the buffer, both expert weight matrices, the shared-expert weights when they are fused, and the load counter discussed below (line 555). The routing, expert work, shared work and load counting are all present; they are no longer separate Python stages. The README reports its decode benchmarks on 64 GPUs within one scale-up domain, the setting in which this condition can hold, but the book has not established which path those runs took [47, 1].

7.3 Why widen the expert group?

Figure 7.1 shows the arrangement the deployment Q&A describes for V4.1 decode. Attention runs with *data parallelism* (DP): each rank processes its own requests. The MoE stage runs with expert parallelism over the same ranks, with the DP size equal to the EP size. The Q&A reports decode groups of EP64 to EP128, calls this Wide-EP, and says that it substantially reduces per-step memory traffic in the expert part [2]. The released code supports other group sizes; these numbers are a reported production setting.

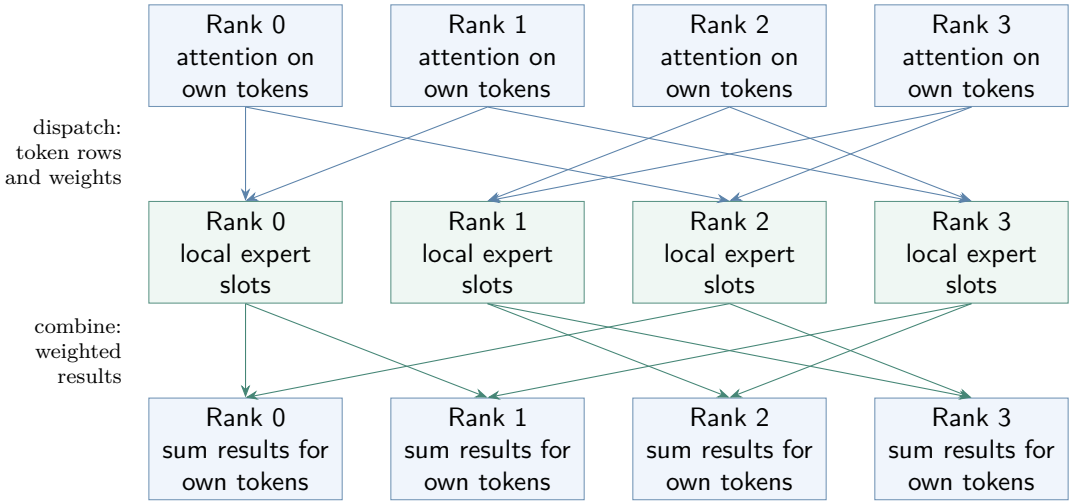


Figure 7.1: The same four devices in two stages of one layer. Attention runs data parallel: each rank processes its own tokens. The MoE stage sends each selected expert assignment to the rank holding that expert slot and returns the results to the token's origin, where they are summed. Arrows are illustrative.

Chapter 2's cost model explains the attraction. Take an illustrative layer with $E = 256$ routed experts, each token choosing $k = 8$, and $B = 32$ tokens per rank. These are teaching numbers, not V4.1's configuration. With P ranks and no replicas:

Ranks P	Experts per rank E/P	Mean assignments per expert PBk/E
8	32	8
64	4	64

Holding the per-rank batch fixed, the wider group gives each rank fewer expert weight matrices to read per layer, and each matrix serves more tokens on average. In chapter 2's terms, the expert computation moves towards higher arithmetic intensity. The comparison assumes every expert receives some tokens, the same model and the same hardware.

Two costs limit the gain. Each rank still sends Bk assignments and receives their results, whatever the group size, and in a wider group a larger share of them leave the rank. The step must also wait for the slowest rank. Lab 15's second run had 8 assignments on one rank and 4 on the other. Those counts identify the busiest rank in the lab; it has no duration model. Kernel efficiency and communication also affect the time, so the counts do not establish step latency.

7.4 Use history to choose a placement

The engine reduces imbalance with *expert-parallel load balancing* (EPLB): it observes which experts were busy and periodically changes which slots hold which experts. Figure 7.2 shows the cycle.

7.4.1 Counting work

The counter appears in the MoE calls themselves: explicit dispatch passes the layer's `expert_count` as `cumulative_local_expert_recv_stats` (line 474), and the fused call passes the same tensor (line 555). It has one entry per local physical slot and layer.

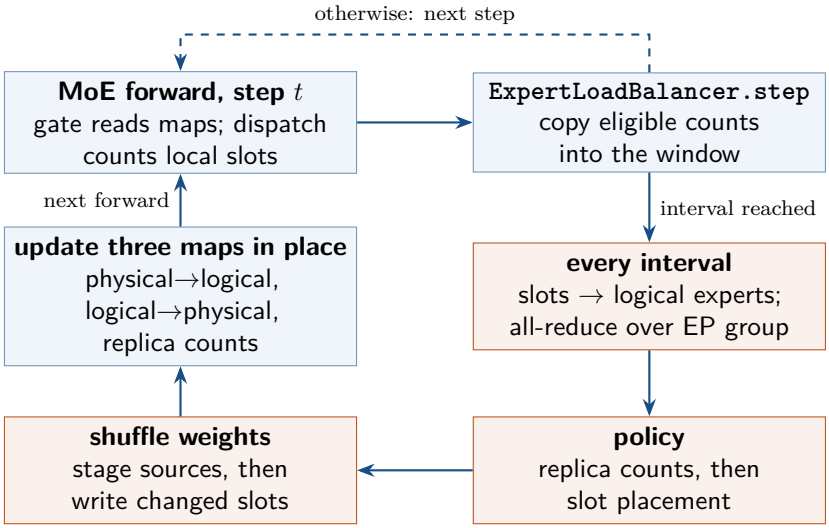


Figure 7.2: The EPLB cycle with rearrangement enabled. Eligible steps record counts; at an interval, the engine chooses a new placement, moves weights and updates the maps. After either branch, `step` records load statistics and clears the counter before the next forward pass [47, 48, 49, 50].

After sampling, `DeepseekForCausalLM` calls `ExpertLoadBalancer.step` (`deepseek.py`, lines 402–404). That method updates the rearrangement history when enabled, records load statistics and finally zeroes the counter. The rearrangement history is a ring of recent steps, the *window*. Every *interval* of eligible steps, it triggers `_rearrange_experts` [48].

The policy needs counts per logical expert across the whole group, not per local slot. First, `_rearrange_experts` selects this rank's mapping and allocates space for logical counts:

Listing 7.7: Select the local slots' logical identities.
DeepInfer · `deepinfer/model_executor/eplb/__init__.py` · lines 235–248

```
235 window_size, num_moe_layers, num_local_physical_experts = self.
        ↪ _expert_count_window.shape
236 num_physical_experts = self.physical_to_logical_map.shape[1]
237 num_logical_experts = self.logical_count.shape[1]
238 local_physical_to_logical_map = self.physical_to_logical_map[
239     :,
240     ep_group.rank() * num_local_physical_experts : (ep_group.rank() + 1) *
        ↪ num_local_physical_experts,
241 ]
242 logical_expert_token_counts = torch.zeros(
243     window_size,
244     num_moe_layers,
245     num_logical_experts,
246     dtype=torch.int,
247     device=local_physical_to_logical_map.device,
248 )
```

Next, `scatter_add_` adds each local slot's count to the logical expert that slot held. The all-reduce sums over ranks, so replicas on different ranks contribute to one logical total:

Listing 7.8: Aggregate counts across the group, then call the policy.
DeepInfer · `deepinfer/model_executor/eplb/__init__.py` · lines 249–260

```
249 logical_expert_token_counts.scatter_add_(
250     -1,
251     local_physical_to_logical_map.to(torch.int64).expand_as(self.
    ↪ _expert_count_window),
252     self._expert_count_window,
253 )
254 dist.all_reduce(logical_expert_token_counts, group=ep_group)
255
256 next_physical_to_logical_map, next_logical_to_physical_map, next_logical_count =
    ↪  rebalance_experts_global(
257     logical_expert_token_counts,
258     num_physical_experts,
259     num_local_physical_experts,
260 )
```

The window keeps one row per step; it does not add the steps together. The policy uses that structure.

7.4.2 Why keep the steps separate?

```
python3 labs/16_placement.py
```

This experiment requires PyTorch, but no GPU. It loads an unmodified copy of DeepInfer's `policy.py`, checked against the source manifest, and calls `rebalance_experts_global` directly. It imports nothing else from the engine. The saved output records the PyTorch version in `results/provenance.json`.

```
History by step: [[10, 10, 0, 0], [0, 0, 10, 10]]
  candidate AB/CD: score=40
  candidate AC/BD: score=20
  policy AC/BD: score=20

Skewed history by step: [[30, 6, 4, 0], [20, 10, 0, 6], [40, 2, 6, 2]]
  initial ABC/DAB: score=70.00
  policy AAD/ABC: score=68.00
  physical->logical: ['A', 'A', 'D', 'A', 'B', 'C']
  logical->physical: {'A': [0, 3, 1], 'B': [4], 'C': [5], 'D': [2]}
  replicas: {'A': 3, 'B': 1, 'C': 1, 'D': 1}
```

The first history has four experts, two ranks and two slots per rank. In step 1 only A and B are busy; in step 2 only C and D. Over both steps every expert has 10 tokens, so both placements give each rank a total of 20. The steps tell a different story. With AB/CD, rank 0 does all the work in step 1 and rank 1 all of it in step 2: the busiest rank carries 20 in each step, 40 in total. With AC/BD, each rank carries 10 in each step, and the policy chooses AC/BD. A policy that saw only the totals could not tell the two placements apart. The policy's score is the sum over steps of the busiest rank's estimated load:

$$\sum_t \max_r L_{t,r}.$$

The lab loads the checked file directly, bypassing the package initialisers:

Listing 7.9: Load only the vendored policy module.
Book lab · `labs/16_placement.py` · lines 19–24

```
19 def load_policy():
20     spec = importlib.util.spec_from_file_location("eplb_policy", POLICY)
21     assert spec is not None and spec.loader is not None
22     module = importlib.util.module_from_spec(spec)
23     spec.loader.exec_module(module)
24     return module
```

Its `place` function adds a one-layer dimension to the history, calls the real policy and checks the returned maps and replica counts:

Listing 7.10: Call DeepInfer's policy and check its maps.
Book lab · `labs/16_placement.py` · lines 44–53

```
44 def place(policy, history, num_physical, per_rank):
45     tokens = torch.tensor([[step] for step in history], dtype=torch.int)  # [
   ↪ steps, layers=1, experts]
46     p2l, l2p, count = policy.rebalance_experts_global(tokens, num_physical,
   ↪ per_rank)
47     slots, l2p, count = p2l[0].tolist(), l2p[0].tolist(), count[0].tolist()
48     for expert, physical in enumerate(l2p):  # check both maps and the replica
   ↪ counts
49         valid = [p for p in physical if p >= 0]
50         assert len(valid) == count[expert] == slots.count(expert)
51         assert all(slots[p] == expert for p in valid)
52     assert len(slots) == num_physical and sum(count) == num_physical
53     return slots, l2p, count
```

7.4.3 Two decisions: replicas, then places

`rebalance_experts_global` first decides how many copies each logical expert gets. One extra slot at a time, it asks which expert's copy would most reduce the sum over steps of the largest per-copy load:

Listing 7.11: Score each candidate for the next replica.
DeepInfer · `deepinfer/model_executor/eplb/policy.py` · lines 111–116

```
111 score = tokens_per_expert_all_diff / logical_count  # NOTE: Values in score must
    ↪  be different from each other
112 score_added = tokens_per_expert / (logical_count + 1)
113 values, indices = score.max(-1, keepdim=True)
114 values = values.expand_as(score).contiguous()
115 score.scatter_(-1, indices, score_added.gather(-1, indices))
116 values.scatter_(-1, indices, score.max(-1, keepdim=True).values)
```

`score` is each expert's load per copy in each step. `score_added` is what it would be with one more copy. Giving a copy to the current busiest expert lowers the maximum; giving it to anyone else leaves the maximum unchanged. After these lines, `values` holds, for each candidate, the step's largest per-copy load if that candidate received the copy. Summing over steps and taking the smallest picks the replica (line 117). Line 109 adds a tiny amount per expert so that equal loads still have a single maximum [49].

In the lab's skewed history, A carries 30, 20 and 40 tokens in the three steps, far more than any other expert. Both extra slots go to A, giving it three copies.

The second decision places the six copies on two ranks of three slots each. The helper that does this, `_batch_greedy_indices`, takes the copies in descending order of their total per-copy load over the window (lines 31–35). For each copy it computes, for every rank, the score that would result from placing it there (lines 58–60): each step's busiest-rank load after the addition, summed over steps. It then commits the best rank:

Listing 7.12: Place the copy on the best rank that still has a free slot.
DeepInfer · `deepinfer/model_executor/eplb/policy.py` · lines 63–69

```
63 rank = scores.argmin(dim=-1)
64 occupied = occupancy[layers, rank]
65 indices[layers, rank * capacity + occupied] = physical_order[:, job_index]
66 occupancy[layers, rank] = occupied + 1
67 updated = loads[layers, rank] + vector
68 loads[layers, rank] = updated
69 torch.maximum(peaks, updated, out=peaks)
```

Full ranks were excluded on line 62, and `argmin` breaks ties towards the lower rank. The result in the lab is AAD/ABC, with an estimated score of 68. The starting layout, ABC/DAB, which gives the two extra slots to A and B, scores 70. The lab checks that both maps agree: every slot listed for an expert holds that expert, and each expert's replica count equals its number of slots.

These scores are estimates in the policy's own model. The policy divides an expert's load equally among its copies, whereas the gate's index rule splits them approximately. The algorithm is greedy, so it does not guarantee the best placement. The score leaves out communication, kernel efficiency and the cost of moving weights. A difference between 70 and 68, or between 40 and 20, is not a measured speed-up.

7.4.4 Compare two DeepSeek placement policies

DeepSeek also published a standalone EPLB implementation. At the fixed commit `d52c72d`, it already separates logical experts from physical replicas, allocates redundant copies and packs them into equal-capacity device groups. Its entry point returns the same three kinds of map/count tensors. Its README leaves load prediction to the caller and suggests a moving average of historical statistics [54]. The useful comparison is therefore how the two policies represent and use load, not whether replication first appears in DeepInfer.

Decision	Standalone EPLB at `d52c72d`	DeepInfer at `774ce244`
Load input	One scalar per layer and logical expert.	One count per step, layer and logical expert.
Next replica	Copy the expert with the largest scalar load per existing replica.	Score each candidate by the sum of per-step maximum per-copy loads after adding it.
Placement	Sort scalar copy loads, then choose the least-loaded pack with space.	Sort total copy loads, then score candidate ranks using per-step maximum rank loads.
Topology	Entry point selects hierarchical group/node packing or a global fallback.	The inspected global policy takes physical and per-rank slot counts, without group/node parameters.

In the standalone code, `replicate_experts` chooses copies and `balanced_packing` places them. The entry point is `rebalance_experts`. Compare these with the two DeepInfer functions just read [54, 49].

The input difference can change a decision. Consider two experts with loads $(10, 0)$ in one step and $(10, 18)$ in the next, and one spare slot. If the caller supplies totals $(20, 18)$ to the scalar policy, it copies A. Keeping both steps, a copy of A gives per-step maxima 5 and 18, summing to 23; a copy of B gives 10 and 10, summing to 20. DeepInfer's replica score therefore chooses B. The third case in lab 16 calls DeepInfer's policy with this history, three physical slots and one slot per rank. It checks the returned maps and asserts the replica counts:

```
Replica-choice history by step: [[10, 0], [10, 18]]
  replicas: A=1, B=2
```

These are calculated loads, not timings. The standalone policy is compared by source inspection; this lab runs only DeepInfer's policy. The chapter's AB/CD example shows the same information loss at the placement stage.

This fixed-version comparison establishes implementation differences. It does not identify a direct code lineage between the repositories or show that one policy is always better. DeepInfer's runtime contribution, which we follow next, is to collect the history and make the chosen placement take effect between model steps.

7.5 Make the new placement take effect

7.5.1 The slot budget is fixed

Extra slots cost memory: each holds a full copy of one expert's weights in every MoE layer. The number is fixed when the model is built. The loader reads a requested minimum from `DEEPINFER_OVERRIDE_NUM_EXTRA_EXPERTS`, which defaults to zero, and `resolve_num_extra_experts` rounds it up so that the total divides evenly among EP ranks (`model_loader.py`, lines 83–107; `eplb/__init__.py`, lines 21–28). The example setup requests at least 32 (`examples/_config.py`, lines 77–79). As an illustration, 256 routed experts plus 32 requested extras make 288 slots; at EP64 the engine adds 32 more so that $320 = 64 \times 5$. The Q&A reports, for decode, one extra slot per GPU before rounding [51, 48, 17, 2].

A later policy call does not add slots. It only chooses which logical experts occupy them. Comparing configurations with different slot budgets must therefore charge the extra weight memory: device memory that chapter 4's block pool could otherwise use.

7.5.2 Move weights, then switch the maps

After the policy returns, `_rearrange_experts` moves weights and installs the new maps:

Listing 7.13: Shuffle weights into their new slots, then update the maps.
DeepInfer · `deepinfer/model_executor/eplb/__init__.py` · lines 261–278

```
261 if self._intranode_shuffle:
262     expert_model_parallel_shuffle_inplace_intranode(
263         self._ipc_weight_buffers,
264         self.physical_to_logical_map,
265         self._expert_weights,
266         next_physical_to_logical_map,
267         ep_group,
268     )
269 else:
270     expert_model_parallel_shuffle_inplace(
271         self.physical_to_logical_map,
272         self._expert_weights,
273         next_physical_to_logical_map,
274         ep_group,
275     )
276 self.physical_to_logical_map.copy_(next_physical_to_logical_map)
277 self.logical_to_physical_map.copy_(next_logical_to_physical_map)
278 self.logical_count.copy_(next_logical_count)
```

The three tensors are updated in place. The per-layer state that the gate receives is a view of those same tensors (lines 346–376), so the next forward pass reads the new mapping without any other hand-over. The weights must already be in their new slots when that happens, which is why the shuffle comes first. The rank-0 metrics path synchronises the device around the operation and logs its elapsed time (lines 226–228 and 280–288) [48].

Moving weights has its own hazard. Suppose slot 2 should receive the expert that slot 5 holds, while slot 5 should receive slot 2's expert. Copying one directly into the other would destroy a source that is still needed. The general point-to-point shuffle avoids that by staging. Copies within a rank and transfers from other ranks both write into separate receive buffers first (`shuffle.py`, lines 107–138). Only after every transfer has completed does it write the changed slots:

Listing 7.14: Write changed slots from the staging buffers.
DeepInfer · `deepinfer/model_executor/eplb/shuffle.py` · lines 141–147

```
141 recv_slot = dict(experts_to_recv)
142 for i in range(num_local):
143     if origin_local[i] == current_local[i]:
144         continue
145     src = i if received[i] else recv_slot[current_local[i]]
146     for weight, buf in zip(weights, recv_buffers, strict=True):
147         weight[i].copy_(buf[src])
```

Unchanged slots are skipped. A changed slot takes its new weights either from its own staged copy or from the buffer that received that expert from another rank. When the whole group lies within one NVLink domain, the intranode path instead copies each rank's weights into a shared mirror, one layer at a time. Two barriers bracket the mirror refresh; their comments say, in the author's translation, "wait for every rank to finish reading the

previous layer's mirror" and "wait for every rank's mirror to be ready" (lines 192–195). Reads then come from mirrors, or from a slot that has already received its new weights [50].

Apply the general shuffle to the lab's actual transition, ABC/DAB to AAD/ABC. Global slot 2 on rank 0 must change from C to D. D currently lives in global slot 3, rank 1's local slot 0. Rank 1 sends those weights into rank 0's receive buffer at local index 2. Once the transfer completes, the final loop copies that buffer entry into rank 0's weight slot 2. Meanwhile C must move from old slot 2 to rank 1's slot 5. Staging both transfers preserves C before slot 2 is overwritten. Figure 7.3 shows these two transfers; the other changed slots use the same shuffle.

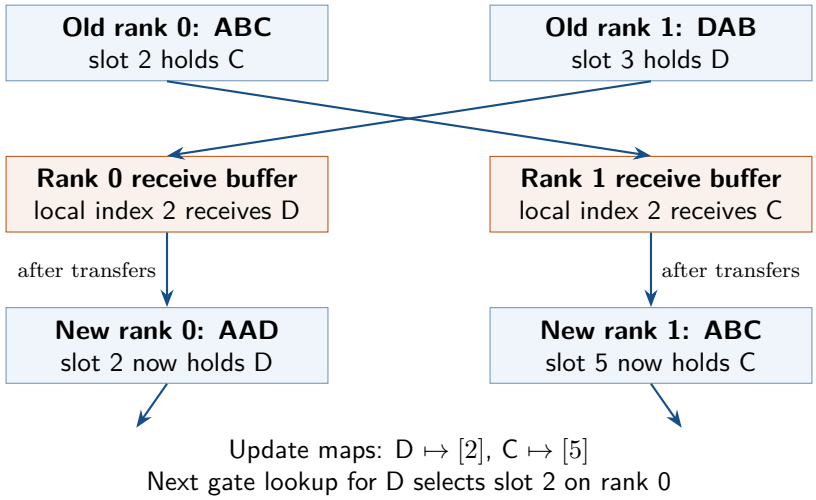


Figure 7.3: Two transfers in the lab's ABC/DAB to AAD/ABC transition, traced through DeepInfer's general P2P shuffle. Slot numbers are global; each rank's receive-buffer indices start at zero. Staging protects C before slot 2 receives D. Other slot changes are omitted.

Only after the shuffle is queued does the engine queue the three tensor updates. For this layer, `physical_to_logical_map` becomes $[A, A, D, A, B, C]$. The valid entries in `logical_to_physical_map` are A: $[0, 3, 1]$, B: $[4]$, C: $[5]$, D: $[2]$, with `logical_count` $[3, 1, 1, 1]$. A's list is in the order returned by the policy, not sorted slot order; the replica formula indexes that list. These are the lab's actual outputs.

On the next forward pass, consider token $(0, 2)$, which selects D in lab 15. D still has one replica, so its replica index is zero. The gate now reads `logical_to_physical_map`[D,0] as 2 rather than 3. Dispatch therefore sends that assignment to rank 0 rather than rank 1, and the destination slot now holds D's weights. The logical computation is unchanged; its location has changed.

7.5.3 When rearrangement runs

Rearrangement is off by default. `DEEPINFER_EXPERT_REARRANGEMENT_STEP_INTERVAL` defaults to zero, and `_update_rearrangement` returns early in that case (`environ.py`, lines 115–117; `eplb/__init__.py`, lines 147–148). For an EP group with more than one rank, `step` still calls `_update_load_balance_stats` and clears `expert_count` (lines 131–138). Load statistics, including their periodic collective, remain active when rearrangement is off; this is the statistics flush discussed in chapter 2. When it is enabled, the window must lie between one step and the interval (lines 106–111), and the first rearrangement comes after a quarter of the interval, to correct start-up imbalance sooner (lines 121–123). A normal engine leaves prefill steps out of the window unless a separate setting asks for a

rearrangement straight after prefill (lines 83–84 and 149–159). The example setup uses a three-step window and, when EPLB is enabled, a 30-step interval (`examples/_config.py`, lines 80–83) [30, 48, 17].

The deployment Q&A reports very different production settings: a 100-step window and a 1,000-step interval for prefill, and 1,000 and 10,000 for decode. It reports a ratio of slowest to mean rank load of 1.074 and 1.045 for two prefill configurations and 1.090 for decode [2]. Those are DeepSeek's reported measurements; this book has not reproduced them.

Both settings reflect a trade. A placement is chosen from the past, so it lags a workload that changes. Moving weights takes time on every rank. As a teaching model only: if a rearrangement costs R seconds and saves s seconds per step for the next M steps, it pays when $sM > R$. Long intervals bet that expert popularity changes slowly. The rank-0 log already records rearrangement time, which supplies an observation of R. This book has not run the engine to obtain that value or measured the subsequent saving s and how long it persists. Choosing M needs those measurements.

We can now follow a token's expert work from the gate to another rank and back, and follow the counts it leaves behind into a new placement. These choices affect the cost of a forward pass. The next chapter asks which draft positions are worth evaluating at that cost, allocating a verification budget across requests using predicted useful output and measured step times.

7.6 Experiments to try next

1. In `labs/15_dispatch.py`, give token $(1, 1)$ the choices A and D. Predict the per-rank assignments in runs 2 and 4 before running the script.

2. Change the replica rule in the lab to always pick the first copy. Which rank becomes busiest in run 4? What does the index rule buy, and what can it not guarantee?

3. In `labs/16_placement.py`, add a third step in which only A and C are busy. Does the policy still choose AC/BD? Compute both scores by hand first.

4. Give the skewed history four slots per rank. How many replicas does each expert receive, and does the score improve? Relate the extra slots to weight memory.

5. Using the teaching model $sM > R$, list the quantities you would need to measure to choose a rearrangement interval, and where in DeepInfer you would record each one.

Chapter 8

Spend verification work wisely

Chapter 4 reserved room for *draft tokens*: cheap guesses that the model checks in its next decode step. Chapter 3 counted the six query positions that such a step evaluates, and chapter 6 corrected the host's lengths once the device knew how many guesses survived. Two questions remain. What makes keeping a guess legitimate when we sample rather than always choose the most likely token? And how many guesses are worth the work of checking? The first is a question about probability, the second about cost. DeepInfer answers both in code that runs at every decode step. The README calls this feature *DSpark and confidence-scheduled verification*; the deployment Q&A explains how DSpark uses predicted acceptance to allocate a variable verification budget [1, 2].

The question

A cheap predictor prefers B; the model we want to sample prefers A. Can we keep some of the predictor's guesses and still obtain the model's distribution? Then, if we can verify several guesses together, how many should we check?

```
python3 labs/17_verification.py
```

The experiment enumerates three possible tokens with exact fractions. The *target* distribution is the one we want; the *draft* distribution belongs to the cheaper predictor. The standard column uses an exact correction rule. The penalised column changes that correction, using an exaggerated penalty so that the difference is visible:

```
Exact probabilities; illustrative alpha=9/10
token  target  draft  standard  penalised  difference
  A     1/2    1/3       1/2      19/39      -1/78
  B     1/3    1/2       1/3        1/3          0
  C     1/6    1/6       1/6       7/39       1/78
Standard output equals target; penalised output obeys both bounds.
```

The standard rule recovers the target exactly. The modified rule moves 1/78 of probability from A to C. Both columns sum to one, and both accept proposed tokens using the same test. The difference comes from what happens after a rejection. DeepInfer uses this second kind of correction, with a much smaller penalty. Understanding that choice comes before deciding how much verification work to schedule.

Figure 8.1 locates the two decisions in DeepInfer's loop. DFlash, DeepInfer's draft model, proposes tokens; a pool retains them with their proposal probabilities and confidence estimates. The next verification step reads that proposal, but may check only a prefix. We

first explain acceptance and correction, then the budget, and finally which round supplies each piece of information.

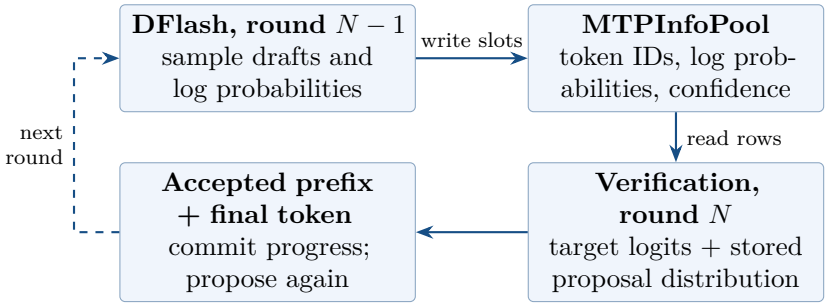


Figure 8.1: The proposal and verification path. Scheduling chooses how much of the stored proposal to verify. Round labels identify the data's generation, not when the host receives it.

8.1 Accept or correct

Write $P(x)$ for a target probability and $Q(x)$ for the probability with which the draft actually proposes token x. Draw from Q, then accept the proposal with probability

$$a(x) = \min\left(1, \frac{P(x)}{Q(x)}\right).$$

We only need this ratio for tokens that can be drawn: $Q(x) > 0$. The accepted probability mass at token x is therefore $\min(P(x), Q(x))$. Where the draft offers too much mass, the test removes the excess. Where it offers too little, acceptance alone cannot fill the gap.

Set $s(x) = [P(x) - Q(x)]_+$, where $[z]_+ = \max(z, 0)$, and let $\Delta = \sum_x s(x)$. Since both distributions sum to one, Δ is also the total rejection probability. On rejection, draw a replacement from s/Δ. The resulting probability of token x is

$$A(x) = \min(P(x), Q(x)) + \Delta \frac{s(x)}{\Delta} = P(x).$$

If $\Delta = 0$, every proposal is accepted and no replacement is needed. This is the probability argument behind the standard speculative sampling algorithms of Leviathan, Kalman and Matias and of Chen and colleagues [56, 57].

In our example, the accepted masses of A, B and C are $1/3, 1/3, 1/6$. Only B can be rejected: it is proposed with probability $1/2$ and accepted with probability $2/3$, leaving total rejection mass $1/6$. Only A has a positive residual. Sending every rejection to A gives $1/2, 1/3, 1/6$, the target distribution. Notice that the replacement need not be another draw from the target itself; it must fill precisely the missing mass.

The lab enumerates every branch. Here `residual` has already been formed and `mass` is its sum:

Listing 8.1: Count accepted and replacement mass for every proposal.
Book lab · `labs/17_verification.py` · lines 22–33

```
    for token, q in enumerate(draft):
        if not q:
            continue
        accept = min(F(1), target[token] / q)
        output[token] += q * accept
        rejected = q * (1 - accept)
        if rejected:
            assert mass > 0
            for replacement, weight in enumerate(residual):
                output[replacement] += rejected * weight / mass
    assert sum(output) == 1
    return output
```

The full lab checks equality on a grid of three-token distributions with probabilities in sixths. The grid includes identical distributions, disjoint supports and zero probabilities. These are exact calculations, not a histogram from a finite sample. The derivation explains why equality holds generally under the stated assumptions; the grid checks the implementation on those cases.

For a draft prefix, apply the argument at each position conditional on the prefix already accepted. A target forward pass can compute the distributions for several positions together. If a draft is rejected, emit its replacement and discard later candidates: their target distributions were evaluated after a prefix we did not keep. If all checked drafts are accepted, emit one further target sample. Thus checking k drafts can yield up to $k+1$ tokens. Stopping conditions can shorten the published result. Exact one-position correction does not, by itself, prove every adaptive choice of which positions to check.

8.2 What DeepInfer verifies

Chapter 6 followed the worker from forward to sampling. Inside that sampling stage, `Sampler._verification` maps flat logit rows to requests and local positions. It marks the first $q-1$ rows of each request for draft verification; the final row is the extra target sample if the prefix survives. The CUDA metadata helper expresses this as `step + 1 < length` [58].

The verifier needs the distribution that produced the previous draft, not a new prediction of that distribution. Its call to `process_sampling` receives stable pool storage and a row mapping:

Listing 8.2: Supply stored proposal probabilities and IDs to verification.
DeepInfer · `deepinfer/model_executor/layers/sampler.py` · lines 439–457

```
(
    logprobs,
    token_index_cur,
    candidate_token_ids,
    accept_result_verify,
    draft_selected_logprobs,
    accept_probs,
) = process_sampling(
    logits,
    driver_inputs.kernel,
    self.force_topk,
    row_to_req=row_to_req,
    local_step=local_step,
    is_verify=is_verify,
    last_nextn_token_logprobs=mtp_info_pool.logprobs if speculative_inputs is
    ↪ not None else None,
    last_nextn_token_logprobs_index=mtp_info_pool.logprobs_index if
    ↪ speculative_inputs is not None else None,
    draft_token_ids_pool=mtp_info_pool.token_ids if mtp_info_pool is not None
    ↪ else None,
    mtp_info_read_rows=speculative_inputs.mtp_info_read_rows if
    ↪ speculative_inputs is not None else None,
)
```

The read-row mapping selects the stored proposal for each verification row. The helper `process_sampling`, in `layers/sampler_utils.py`, transforms the target logits and invokes the sampling kernel. In `force_topk` mode it first selects a smaller vocabulary and remaps the stored draft probabilities into the target's selected vocabulary. The optional index pool preserves the connection to original token IDs. A missing draft token cannot be treated as if it had an arbitrary index in that smaller space [59].

Here and below, P means the target distribution *after* the applicable sampling transforms, including temperature and top-k/top-p filtering. It is not necessarily the raw model softmax. Likewise Q must be the distribution used to sample that draft. Structured-output masks are applied before this call, and golden overrides can later replace results. We analyse stochastic verification without such overrides; chapter 10 treats their separate contracts.

8.2.1 The acceptance test is standard

The CUDA kernel reads the target and stored draft log probabilities. After accounting for target filtering, it performs this test:

Listing 8.3: Accept using the target-to-draft probability ratio.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/cuda/fused_sample.py` · lines 391–394

```
accept_lp = T.min(target_lp - draft_lp, T.cast(0, T.float))
verify_seq = sample_positions[row] * 3 + 2
T.rng_init(seed_val, verify_seq)
accepted_s[0] = T.rng_rand_float(32, "uniform") < T.exp(accept_lp)
```

Exponentiating the minimum gives $\min(1, P/Q)$. The random stream uses the request's seed and a value derived from the sampling position, rather than the decode step number. That choice matters when batching changes, but it is only one part of a deterministic execution contract.

8.2.2 The correction deliberately leaves a small bias

Further down the same kernel, replacement weights are formed differently from the standard residual:

Listing 8.4: Subtract a slightly reduced draft probability.
DeepInfer · deepinfer/model_executor/layers/tilelang_ops/cuda/fused_sample.py · lines 438–445

```
    for j in T.Parallel(SAMPLE_BD):
        pf[j] = T.Select(
            i0 * SAMPLE_BD + j < vocab_size,
            T.max(pf[j] - T.exp(draft_lf[j] - DRAFT_PROB_PENALTY), 0.0),
            0.0,
        )
if do_sample:
    accumulate(sample_cfg, pf, sample_acc_frag)
```

`DRAFT_PROB_PENALTY` is 5×10^{-5}. Its comment says that the penalty prevents rejection sampling from finding nothing to sample (author's translation of `tilelang_ops/common.py`, line 53). Writing $\alpha = \exp(-5 \times 10^{-5})$, these weights are

$$r(x) = [P(x) - \alpha Q(x)]_+.$$

Reducing the subtracted draft probability adds residual mass. It can give a token a positive replacement weight even when its standard residual is zero. In the lab, $\alpha = 9/10$: A's residual becomes $1/5$, C's becomes $1/60$, and B's remains zero. Rejection still occurs with probability $1/6$. Normalising those new weights gives A $12/13$ and C $1/13$ of the replacements, explaining the transfer of $1/78$.

The source docstring gives a bound for this modified rule. It names the draft p and the target q; our notation reverses those roles, using Q and P. Its argument, rendered here in the book's notation (author's translation), starts by writing

$$\varepsilon = 1 - \alpha, \qquad r(x) = s(x) + d(x), \qquad D = \sum_x d(x).$$

The added mass satisfies $0 \le d(x) \le \varepsilon Q(x)$, so $D \le \varepsilon$. With the unchanged acceptance test,

$$A(x) - P(x) = \frac{\Delta d(x) - D s(x)}{\Delta + D}.$$

For the upper bound, $d(x) \le \varepsilon P(x)$ also holds: check separately $P(x) \ge Q(x)$ and $P(x) < Q(x)$. Hence $A(x) - P(x) \le d(x) \le \varepsilon P(x)$. For the lower bound, use $s(x) \le \Delta$, $D \le \varepsilon$ and $\Delta \le 1$ to obtain $P(x) - A(x) \le D\Delta/(\Delta + D) \le \varepsilon/(1 + \varepsilon)$. The no-rejection case is immediate. Together these give the docstring's per-token bound [60]:

$$\boxed{P(x) - \frac{\varepsilon}{1+\varepsilon} \le A(x) \le (1+\varepsilon)P(x).}$$

Lab 17 checks both inequalities for four rational values of α on the same distribution grid:

```
Exact grid: 3136 bound checks (includes equal and disjoint supports).
Configured penalty=5e-5: epsilon approximately 0.0000499988
```

The rational 9/10 makes the difference visible and exactly computable. The configured exponential is not a rational number; its displayed ε is a decimal approximation. These calculations verify the idealised rule. The general bound comes from the argument above, not from the number of examples tried.

There is a further implementation boundary. `accumulate` converts the weights to fixed-point integers before summing them. If the accumulated replacement mass is zero, lines 506–518 of the kernel fall back to the first token attaining the maximum logit. Floating-point probability transforms, fixed-point accumulation and that fallback are not covered by the docstring's ideal-arithmetic bound. For example, in exact arithmetic with $\alpha < 1$, $\sum_x r(x) \geq 1 - \alpha > 0$; the zero-mass fallback is not a branch that our rational model should invent. We have inspected these implementation choices, not established a combined error bound for the GPU kernel [58, 61].

This is the chapter's comparison boundary: DeepInfer retains the standard acceptance rule and modifies the rejection residual. The inspected code and its comment support that difference. They do not support a claim that the released sampler preserves the target distribution exactly.

8.2.3 Only a surviving prefix is published

Some sampled candidates follow a rejected draft and cannot be published. The helper `get_accept_result` finds which remain usable. After identifying the request and its first row, the kernel scans earlier positions:

Listing 8.5: A candidate is usable only while all earlier verification rows survive.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/cuda/sampler.py` · lines 261–276

```
261 req = T.alloc_var(T.int32, row_to_req[row])
262 start = T.alloc_var(T.int32, ftp[req])
263 alive = T.alloc_var(T.bool, True)
264 prev = T.alloc_var(T.int32, start)
265 while alive and prev < row:
266     prev_accept = T.alloc_var(T.bool, raw_accept[prev])
267     if has_golden:
268         prev_golden = T.alloc_var(T.int32, golden[prev])
269         if prev_golden != -1:
270             prev_draft_row = T.alloc_var(T.int32, read_rows[prev])
271             prev_accept = prev_golden == draft_token_ids_pool[prev_draft_row]
272     alive = is_verify[prev] and prev_accept
273     prev += 1
274 token_ids_out[row] = token
275 accept_out[row] = alive and is_verify[row] and accept
276 is_final_out[row] = alive and (not is_verify[row] or not accept)
```

Ignoring the explicit golden overrides, `alive` remains true only through an accepted draft prefix. The first rejection is a final row; later rows are neither accepted nor final. If no draft rejects, the extra, non-verification row is final. `finalize_verification` collects that final token and the accepted draft IDs. Suppose four positions checked three drafts and the second draft failed. The first draft and the second row's replacement survive; the third

draft and extra target sample do not. Chapter 4's processed-token count is later corrected to this actual progress, even though the forward already evaluated and wrote state for more positions.

8.3 How many positions are worth the work?

```
python3 labs/18_verification_budget.py
```

Lab 18 translates DeepInfer's CPU budget search and checks it against exhaustive prefix choices on a small input. There are two requests, each with three available drafts. Their older cumulative confidence values are $(0.90, 0.72, 0.36)$ and $(0.80, 0.40, 0.10)$. The first request's raw estimates were $(0.90, 0.80, 0.50)$: reaching draft 3 requires accepting drafts 1 and 2, so the estimated chance of keeping the prefix is their product, 0.36. These are predictions, not the acceptance probabilities computed by verification.

The lab forms the prefix products with `itertools.accumulate`, using multiplication:

Listing 8.6: Turn conditional confidence estimates into prefix estimates.
Book lab · `labs/18_verification_budget.py` · lines 20–22

```
20 def cumulative(raw):
21     assert all(0 <= r <= 1 for row in raw for r in row)
22     return [list(accumulate(row, mul)) for row in raw]
```

The script prints every candidate under four assigned cost curves. Here is the staircase case:

```
stairs: synthetic time units; sampler=2, slope=0
  positions graph expected time  output/time
          2     2     2.00   10       0.2000
          3     4     2.90   12       0.2417
          4     4     3.70   12       0.3083
          5     6     4.42   14       0.3157
          6     6     4.82   14       0.3443
          7     8     5.18   30       0.1727
          8     8     5.28   30       0.1760
  chosen total=6, q(N-1)=[2, 4]
  allocation using N-2 instead: [3, 3]
```

There is one main input position per request before adding any drafts. Start with two expected output tokens and two positions. Sorting all cumulative values gives $0.90, 0.80, 0.72, 0.40, 0.36, 0.10$. Taking the first four extras raises the score to $2 + 0.90 + 0.80 + 0.72 + 0.40 = 4.82$, with six positions of work. The assigned cost is 14, so the score is $4.82/14 \approx 0.3443$. A seventh position crosses a graph boundary and costs 30; its extra expected output cannot pay for that jump. Figure 8.2 shows the three assigned cost curves.

Changing only the cost curve changes the winning total: flat costs choose all eight positions, the staircase chooses six, and the steep curve chooses four. In the fourth case, moving the steep curve's second graph size from four to five makes five positions best: the fifth now fits in the cheaper bucket. Within a bucket, another position with positive expected value improves the score at no extra *modelled* time. This is chapter 2's distinction between request count and evaluated positions expressed through graph sizes.

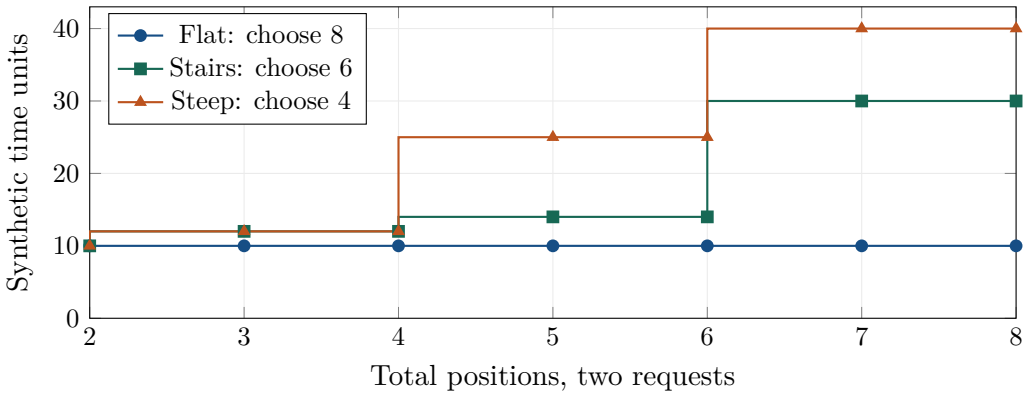


Figure 8.2: Lab 18's assigned costs. Positions within one graph bucket share a cost; crossing a boundary can change which budget wins. The choices maximise estimated output per time, using the same older confidence values.

The script also changes the confidence history used for allocation. In the staircase output, the total is six but the lengths are $[2, 4]$; allocating with the older values would give $[3, 3]$. The total and its distribution are separate decisions. We will trace their rounds after inspecting the search.

8.3.1 A search over totals, then a second allocation

The decode scheduler calls `MTPVarlen::schedule` after choosing requests. It sums the returned query lengths into `num_batched_tokens`. Here `bsz` counts *positions*, not requests. The class builds two sorted queues of cumulative confidence values, one to choose this total and one to divide it among requests. Its comparator sorts by descending value, breaking ties by request index (lines 210–214) [62].

For estimates in $[0, 1]$, cumulative values along a request never increase. A later position cannot strictly outrank an earlier one. Equal values need not retain position order in the C++ sort: the result stores a *count* per request and uses that many positions from the beginning. Thus counts still specify prefixes. This is why cumulative estimates, rather than unmultiplied per-position confidence, fit this allocation rule.

For each total, the search scores the smallest graph large enough:

Listing 8.7: Round positions to a graph bucket and score expected output per time.
DeepInfer · csrc/engine/mtp_varlen.hpp · lines 234–249

```
int cur_bsz_idx = 0;
auto update_best = [&]() {
    while (cur_bsz_idx < num_bsz_selections && cur_bsz > _decode_perf_curve[
    ↪ cur_bsz_idx].first) {
        ++cur_bsz_idx;
    }
    if (cur_bsz_idx == num_bsz_selections) {
        return;
    }
    const auto &[graph_size, baseline_ms] = _decode_perf_curve[cur_bsz_idx];
    const double time_ms = baseline_ms + slope_ms * graph_size + sampler_ms;
    const double cur_throughput = cur_expected / time_ms;
    if (cur_throughput > best_throughput) {
        best_throughput = cur_throughput;
        best_bsz = cur_bsz;
    }
};
```

There are three terms in `time_ms`: a startup forward measurement for the graph, an online slope times the graph size, and an estimated sampler time. Before this excerpt the slope is clamped to zero or above. Candidates beyond the largest available graph are ignored. The strict `>` keeps the earlier, smaller total when two scores tie.

The loop adds one candidate's estimated output and one position at a time:

Listing 8.8: Consider the main positions first, then every larger candidate total.
DeepInfer · csrc/engine/mtp_varlen.hpp · lines 252–258

```
update_best(); // All the golden tokens

for (auto &acc : acc_queue_for_bsz) {
    cur_expected += acc.first;
    cur_bsz += 1;
    update_best();
}
```

The final allocation deliberately uses the other queue:

Listing 8.9: Keep the chosen total while distributing its extra positions.
DeepInfer · csrc/engine/mtp_varlen.hpp · lines 263–266

```
seqlens_q.assign(num_seqs, 1);
for (int i = 0; i < best_bsz - num_seqs; ++i) {
    seqlens_q[acc_queue_for_seqs_q[i].second] += 1;
}
```

Lab 18 preserves these decisions, including the tie rule, graph limit and non-negative slope. Its independent check enumerates every possible pair of prefix lengths under the older confidence values and confirms the winning total. It also tests nonzero slopes, absent drafts and equal scores. Fractions remove rounding ambiguity in the teaching input; the production search uses floating-point values. The check does not say that the newer allocation maximises

the older score, or that either predicts measured throughput exactly. EOS, confidence errors and changes in workload can all separate the estimate from useful published output.

8.3.2 The engine supplies the time model

`ModelRunner.profile_decode_perf` measures the startup curve, if a cached curve is unavailable. It replays each captured graph once to warm up and five times with CUDA timing events. Expert routing is forced random during profiling. With `NUM_TRIM=1`, the sorted measurements lose one value at each end:

Listing 8.10: Average the middle three measurements, then average across EP ranks.
DeepInfer · `deepinfer/worker/model_runner.py` · lines 617–623

```
617 times_ms.sort()
618 trimmed = times_ms[NUM_TRIM : NUM_MEASURE - NUM_TRIM]
619 avg_time_ms = sum(trimmed) / len(trimmed)
620 sum_time = torch.full((1,), avg_time_ms, dtype=torch.float32, device="cuda")
621 dist.all_reduce(sum_time, op=dist.ReduceOp.SUM, group=group)
622 avg_time_ms = sum_time.item() / group.size()
623 results.append((batch_size, avg_time_ms))
```

For five assigned times $9, 10, 11, 12, 40$, this rule gives 11. It does not average the best four, nor does it describe the maximum rank time. The cached startup curve is a baseline; current context lengths and routing can change forward cost [63].

Chapter 6 already located the device events and the wait before post-processing. The feedback connection is now visible in `LLMEngine.post_process`:

Listing 8.11: Feed completed device timings back into the budget estimator.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 995–1002

```
995 if step_state.forward_events is not None and step_state.sample_events is not
    ↪ None:
996     # Worker has synchronized this step's sample_end_evt before calling
    ↪ post_process.
997     self.scheduler.update_mtp_runtime(
998         len(seq_metadata_list),
999         step_state.graph_size,
1000        step_state.forward_events[0].elapsed_time(step_state.forward_events[1]),
1001        step_state.sample_events[0].elapsed_time(step_state.sample_events[1]),
1002    )
```

`update_runtime` compares forward time with the baseline at graph size g, storing the normalised residual $(T_{\text{forward}} - T_{\text{startup}}(g))/g$. `ForwardRuntimeEstimator` keeps 5,000 steps. It takes a median within each graph size having at least ten samples, then a median across those graph-size medians. A frequently used graph does not gain extra votes in that second median. This is a robust residual-slope estimate, not a least-squares fit. If no graph meets the threshold, the slope is zero. The schedule clamps a negative estimate to zero, making the startup curve the lower bound on estimated forward cost [62].

Sampler time has no startup curve. Its estimator keeps nine observations and requires five before returning a median. The search prefers history for the same request count, then a global sampler history, then zero. For example, a forward baseline of 10 at $g = 6$, an estimated slope of 0.5, and a sampler estimate of 2 give $10 + 0.5 \times 6 + 2 = 15$ in consistent time units. The lab assigns these inputs; it does not reproduce startup profiling or measure a device.

8.3.3 Fixed length is the default

Variable-length verification is off by default: `DEEPINFER_ENABLE_MTP_VARLEN` is false. The draft limit is five, set by `DEEPINFER_USE_NEXTN_DECODE`. The actual fixed-length branch in `MTPVarlen::schedule` is an empty performance curve: each request receives one plus its available draft count (lines 152–157). `WorkerBase.warm_up_model` obtains a curve only under the graph and variable-length conditions in `worker/base.py`, lines 275–281. Turning on the flag alone does not guarantee a usable curve [30, 63].

With five available drafts, fixed verification evaluates six positions per request. Chapters 3 and 6 have already traced those positions through decode SWA writes and lagging metadata. Variable verification changes which prefix is evaluated; it still needs the same attention and lifetime rules.

8.4 Which round supplies the confidence?

Call the current verification step N. Its draft was produced in round $N-1$. That draft has three different kinds of associated information: its token IDs, the log probabilities of the distribution used to sample it, and a confidence head's estimates of acceptance. The first two drive sampling correctness. The third predicts where verification work will be useful. Return to Figure 8.1: we can now follow the writes and reads behind its pool boundary, then distinguish the histories used by scheduling.

8.4.1 Produce once, store the distribution, verify later

DeepInfer's draft model is the `DFlash` module in `model_executor/models/mtp/`, named after multi-token prediction (MTP); `nextn` is its configured draft count. DFlash starts from $[\text{golden}, \text{noise}, \ldots]$, where the first token is the just-selected target output. Its draft layers process these positions together, and `sample_drafts` computes base logits for them. It then samples position by position. An optional Markov correction changes the logits using the previous draft token (lines 308–314). This permits a parallel base computation while retaining that dependency during sampling [64].

The confidence head runs before sampling the token at that position. The sampling call also writes the log probabilities actually used:

Listing 8.12: Predict confidence and sample the draft with the request's sampling parameters.
DeepInfer · deepinfer/model_executor/models/mtp/dflash.py · lines 315–330

```
315 if confidence_rates is not None:
316     self.confidence_head(
317         hidden_states_before_norm[step_i :: self.max_MTP_value],
318         w1,
319         out=confidence_rates[:, step_i],
320         step_i=step_i,
321     )
322 _, token_index_next, draft_token_ids, _, _, _ = process_sampling(
323     logits,
324     SamplingKernelInputs(
325         sampling_input.kernel_inputs.logits, sampling_input.kernel_inputs.
    ↪ seq_seeds, sample_positions_table[step_i]
326     ),
327     self.force_topk,
328     logprobs_out=next_n_token_logprobs[:, step_i],
329     tag="mtp",
330 )
```

The same request sampling parameters feed this call, but the logits come from the draft model. Equal parameters do not mean equal distributions. With `force_topk`, sampled indices are mapped back to token IDs and the index mapping is retained. After the position loop, all proposal data is stored:

Listing 8.13: Keep the proposal and its distribution in the assigned pool slots.
DeepInfer · deepinfer/model_executor/models/mtp/dflash.py · lines 339–345

```
339 self.mtp_info_pool.store(
340     sampling_input.mtp_info_write_slots,
341     prediction,
342     next_n_token_logprobs,
343     next_n_token_logprobs_index,
344 )
345 return prediction
```

`MTPInfoPool` owns GPU arrays for IDs, log probabilities, optional confidence and optional top-k indices. Each slot has `nextn` rows; slot zero is reserved for padding. The scheduler assigns slots, and verification's read-row mapping selects their contents. This closes the path back to the pool inputs printed earlier. Recomputing a draft distribution later with different filtering would not justify the probability argument [65].

Host post-processing separately updates the confidence history in `process_MTPInfo`. It moves `accept_rates` into `prev_accept_rates`, multiplies the new raw confidence values along each prefix, and stores those products as the new `accept_rates` (`scheduler.hpp`, lines 1239–1250). Despite their names, these fields contain confidence-head estimates. They are not the verifier's measured `accept_probs` values.

8.4.2 Choose a total with older information

Two counters locate the host's history: `schedule_cnt` advances on scheduling; `process_cnt` advances when processing MTP information. The budget selector checks that the host is either caught up or one step behind. Its comment specifies this division of rounds (author's translation of `mtp_varlen.hpp`, lines 159–167):

Decision	Host caught up	Host one step behind (ZOS)
Counter relation	`schedule_cnt` `= process_cnt`	`schedule_cnt` `= process_cnt + 1`
Total budget	$N-2$: `prev_accept_rates`	$N-2$: `accept_rates`
Per-request lengths	$N-1$: `accept_rates`	CPU provisionally uses $N-2$; GPU reallocates using $N-1$

In code, `post_process_paired` is the equality of the two counters. Line 204 chooses `prev_accept_rates` when they match and `accept_rates` when they do not, supplying $N-2$ to the budget queue in either case. Line 207 always supplies the current host `accept_rates` to the allocation queue. That second queue is current enough without ZOS, but one round stale with it [62, 6].

The comment begins with a stronger constraint than a timing observation: confidence at later positions must not affect whether an earlier draft token is verified (author's translation of line 159). Later confidence may depend on earlier sampled tokens, so using it to decide whether to keep those tokens can introduce selection bias. The code documents this concern and its choice of rounds; it does not provide a complete proof for the adaptive policy. Keeping the total-budget rule consistent with and without ZOS is a plausible additional reason for using $N-2$, but that is an inference, not a stated explanation that replaces the constraint.

The inspected construction appears to respect that constraint. We state the argument as an inference, not a proof. Take draft position i of one request. First, the DFlash loop runs the confidence head before sampling the token at that position, so c_i can depend on earlier draft tokens through the Markov input but not on x_i itself. Second, cumulative values never increase along a request, and the allocation records counts, so the request's later positions cannot take a slot ahead of i. Third, competing positions from other requests depend on their own drafts, not on this request's x_i. Finally, the total comes from round $N-2$, before these drafts existed. Whether x_i is verified may therefore depend on the prefix before it, but not on x_i or later tokens of the same request.

Given an accepted prefix, x_i is then a draw from Q that the test either accepts or replaces. If verification stops before x_i, the final row is a target sample instead. In ideal arithmetic with the standard residual, both routes give the target's conditional distribution at that position. A complete argument for the engine would also need to cover structured-output masks, golden overrides, the penalised residual and numerical effects.

8.4.3 Reallocate on the device without changing the total

In ZOS, `Worker._correct_seqlens_qk` receives the CPU's fixed `total_q`. If the current batch overlaps the previous one, it passes the remap and the previous sampler's device confidence to the GPU length helper. Those device values belong to $N - 1$, even though host post-processing has not yet published them. The kernel first makes cumulative rates:

Listing 8.14: Form prefix confidence from the previous device batch.
DeepInfer · deepinfer/model_executor/layers/tilelang_ops/cuda/schedule_mtp_varlen.py · lines 26–37

```
# Phase 1: compute cumprod, reading confidence rates via remap (0.0 for fresh
    ↪ seqs)
with T.Kernel(num_seqs, threads=128) as pid:
    acc = T.alloc_var(T.float32)
    acc = 1.0
    T.pdl_sync()
    prev_idx = remap[pid]
    if prev_idx == -1:
        acc = 0.0
    for j in T.serial(nextn):
        if prev_idx != -1:
            acc = acc * prev_confidence_rates[prev_idx, j]
        cum_rates[pid, j] = acc
```

Each position then counts how many other positions outrank it. Equal values are ordered by flat index, combining request index and position:

Listing 8.15: Rank each draft position, including ties.
DeepInfer · deepinfer/model_executor/layers/tilelang_ops/cuda/schedule_mtp_varlen.py · lines 58–68

```
# Higher rate wins; tie-break by flat index (lower index wins)
for j in T.serial(nextn):
    my_flat_idx = T.alloc_var(T.int32)
    my_flat_idx = pid * nextn + j
    beats = T.alloc_var(T.int32)
    beats = 0
    if other_rate > my_rates[j]:
        beats = 1
    if other_rate == my_rates[j] and other_idx < my_flat_idx:
        beats = 1
    rank_reducer[j] += beats
```

A position wins if this rank is smaller than `total_q - num_seqs`. The request's query length becomes one plus its winning positions. The kernel does not rerun the time-model search. In lab 18's staircase case, the total remains six while the newer values change the allocation from $[3, 3]$ to $[2, 4]$. The six evaluated positions still fit the same graph bucket.

The worker next corrects key lengths using the chosen query lengths and the previous accepted counts. It copies query lengths into pinned host memory and records `d2h_evt`. Forward uses the corrected device inputs. Before launching sampling, the worker waits for that copy and calls `update_metadata_for_sampling_zos`. This is chapter 6's third metadata correction: the host's initial lengths described a provisional division of work, and sampling must see the division that actually ran [66, 7].

Fresh rows need care. On the CPU, an empty rate vector is filled with ones if `has_draft` is true, and zeros otherwise. The predicate includes a request whose host processing is one

step behind. On the GPU, an unmatched row (`remap=-1`) has zero cumulative confidence. But zero confidence does not force a query length of one: after all positive positions win, zero-valued positions can take the remaining slots. The scalar rank-count reference shows this with two surviving requests and one new row:

```
GPU rank-count reference; remap=[0,-1,1]
  total=7: q=[3, 1, 3]
  total=8: q=[3, 2, 3]
  CPU missing-rate fill=1, same total=7: q=[3, 3, 1]
  Both conserve the budget; zero confidence can receive surplus slots.
```

The CPU comparison deliberately supplies ones for that new row to expose the difference between the two local rules. Both allocations conserve the total. It is not a trace proving that this particular combination of host history, placeholder drafts and pool-slot validity reaches the engine. Establishing that requires following admission and proposal ownership together. In particular, `_correct_seqlens_qk` skips correction if there is no overlap at all. We leave the mixed-batch reachability question open rather than calling the different fill values a budget bug.

The README's deterministic-sampling recipe requires variable-length verification to be disabled, alongside batch invariance and a fixed sequence seed. The acceptance RNG's position-based indexing does not remove the runtime timings that influence the budget. Different measured timings could select different verification work; that is an inference about the restriction, developed with the other contracts in chapter 10 [1].

8.5 Reported gains and confidence error

8.5.1 Less verification can help or hurt

The deployment Q&A reports estimated throughput from EP64/DP64 benchmark data, comparing fixed-five verification with variable length. Lab 18 stores the six rows as sourced CSV data and recomputes the gains:

```
DSpark: DeepSeek's reported throughput estimates, not lab timings
  concurrency fixed dynamic gain
           16  4891    4816  -1.5%
           32  8964    9111  +1.6%
           48 12032   12467  +3.6%
           60 13730   14466  +5.4%
           72 14962   15961  +6.7%
           96 16817   18660 +11.0%
```

These are DeepSeek's reported estimates, not timings from our experiment. At concurrency 16, average verified draft length falls from 5 to 4.44, while accepted length falls from 3.32 to 3.24; estimated throughput falls by about 1.5%. At 96, verified length falls to 3.24 and accepted length from 3.30 to 3.15, yet estimated throughput rises by about 11%. At the reported operating point of roughly 60 requests per rank, the table gives $14466/13730 - 1 \approx 5.4\%$ [2].

The lower accepted length at 96 is compatible with greater useful throughput if the step cost falls enough. Conversely, dropping positions inside a cheap graph bucket may save little time while reducing output. Our synthetic curves explain those possibilities; they neither reproduce the production workload nor isolate the cause of its gains. The decision depends

on both sides of chapter 2's ratio: useful output and elapsed time. A higher acceptance rate by itself settles neither.

8.5.2 What the confidence metric compares

The engine records `avg_confidence_error`. Verification adds |`confidence`−`accept_probs`| for every row marked for verification, and the logger divides the accumulated error by the verification count. Rows after an earlier rejection can still contribute because the mask marks evaluated verification positions, not only the emitted prefix. This compares confidence with a probability-like value, rather than a realised zero-or-one result [67].

There is a subtlety in that value. Even when a token was accepted, the CUDA kernel computes the full residual mass for the metric:

Listing 8.16: Derive the logged probability proxy from accumulated residual mass.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/cuda/fused_sample.py` · lines 471–479

```
471 if accepted_s[0]:
472     # accept_probs still requires the full fixed-point residual mass.
473     residual_mass = T.alloc_reducer((1,), sample_cfg.scale_dtype, replication="
    ↪ all")
474     T.fill(residual_mass, 0)
475     for j in T.Parallel(SAMPLE_BD):
476         residual_mass[0] += sample_acc_frag[j]
477     T.finalize_reducer(residual_mass)
478     if T.get_thread_binding() == 0:
479         accept_probs[row] = T.max(1.0 - T.cast(residual_mass[0], T.float) /
    ↪ sample_cfg.scale, 0.0)
```

This is the *penalised, fixed-point* residual, not the standard residual. Ignoring numerical rounding, the standard test's acceptance probability, averaged over proposals at a fixed prefix, is $\beta = \sum_x \min(P(x), Q(x)) = 1 - \Delta$. The logged proxy instead uses $1 - \sum_x r(x) = 1 - \Delta - D$, with a clamp at zero. Thus it can be smaller by up to ε in ideal arithmetic. For the exaggerated lab penalty, the difference is visible:

```
Ideal arithmetic, before fixed-point rounding:
  mean acceptance of standard test: 5/6
  1 - penalised residual mass:      47/60
  difference:                       1/20
```

Neither quantity is the acceptance chance $a(x)$ of a particular token already drawn, and neither is that token's realised acceptance outcome. Fixed-point rounding adds another distinction in the implementation. Calling the metric an exact conditional acceptance probability would lose these boundaries. Calling it the confidence error reported by this engine preserves the useful question: how well does the head predict the quantity the engine actually logs?

Verification spends work on possible future tokens. The next chapter reduces prefill work by reconstructing only a suffix of some model state. Its guarantee is different: the sampling residual's probability bound does not bound errors in reconstructed hidden states. We must first ask which dependencies the shortened computation preserves.

8.6 Experiments to try next

1. In lab 17, use $\alpha = 99/100$. Predict the direction of each probability change before running. Check the bound and explain why the zero-rejection case needs no normalisation of a residual.

2. For one token in the opening example, calculate its acceptance chance after it is proposed, the mean acceptance over all proposals, and the logged ideal-arithmetic proxy. Which of these could equal zero or one in a single run?

3. In lab 18, raise the forward slope while keeping the startup curve fixed. Predict the winning total. Explain why the slope multiplies the graph size rather than the unpadded position count.

4. Replace the newer history with the older one. Which values in the saved output change? Explain why the total-budget search does not need to change.

5. Give the scalar GPU allocator more budget than there are positive-rate positions. Predict which zero-valued position wins first. Then identify the engine conditions that would need checking before treating that example as a reachable request trace.

6. Trace one DFlash proposal from its pool write through verification to final output. At each boundary identify the token IDs, stored distribution, confidence round and completion event. Separate a demonstrated numerical guarantee from an inference about adaptive selection or determinism.

Chapter 9

Reconstruct state approximately

Chapter 1 rebuilt a three-entry window by replaying three inputs. Chapter 4 then encountered state that DeepInfer deliberately leaves out of its persistent cache. Rebuilding that state sounds like the same operation. The difficulty is what an entry contains: a later layer caches representations produced by earlier layers, and those representations may depend on positions outside the window. DeepInfer shortens prefill by accepting an approximation in this local state, while preserving the full source for another part of the cache.

The question

If each layer reads only three positions, can replaying the last three inputs rebuild two layers' caches exactly? What changes if both layers take their cached entries from the same shared input?

```
python3 labs/19_replay.py
```

The lab starts with the integers 1 to 8, two layers and a window of three. Each layer adds the mean of its visible entries to its current input. One version caches each layer's own input; the other always caches the shared original input. It compares a full prefill with a replay of inputs 6, 7 and 8, then uses each resulting cache to process the next input, 9:

```
KV source    last row (full / suffix)  next row (full / suffix)
shared          22 / 22                   25 / 25
per-layer       28 / 57/2                 32 / 193/6
```

Shared inputs preserve both results. Per-layer inputs change the last prefill result from 28 to 57/2, and the next decode result from 32 to 193/6. The error survives the handover from prefill to decode. We will find the same dependency distinction in DeepInfer's full block-KV projections and truncated sliding-window attention (SWA).

9.1 Where exact window recovery stops

The lab uses identity KV projections: the cached row is the input row itself, and the attention weights are uniform. For a layer input h_i and cached values v_j, its rule is

$$h'_i = h_i + \frac{1}{|J_i|} \sum_{j \in J_i} v_j, \qquad J_i = \{\max(s, i - W + 1), \ldots, i\}.$$

Here W is the window and s is the first available position. The full calculation starts at zero; suffix replay starts later. The addition of h_i is a *residual*: it carries that row's input

into its output. This deliberately small rule isolates the dependencies without introducing softmax or floating-point round-off.

The two versions differ in one line of the layer loop. `boundary` is the unchanged input, while `hidden` is the preceding layer's output:

Listing 9.1: Choose the source of cached rows before computing a layer.
Book lab · `labs/19_replay.py` · lines 16–25

```
    caches = []
    for _ in range(depth):
        kv = boundary if shared else hidden
        caches.append(kv[-window:])
        output = []
        for i, value in enumerate(hidden):
            visible = kv[max(0, i - window + 1):i + 1]
            output.append(value + sum(visible) / len(visible))
        hidden = output
    return hidden, caches
```

Consider the per-layer version. The first layer's full output at the position holding 6 is $6 + (4 + 5 + 6)/3 = 11$. Replaying from 6 loses 4 and 5, giving $6 + 6 = 12$. At the positions holding 7 and 8, the replay gives $27/2$ and 15, instead of 13 and 15. These three outputs become the second layer's KV rows. Its last query therefore reads different values even though all three positions are present:

$$\text{full: } 15 + \frac{11 + 13 + 15}{3} = 28, \qquad \text{replay: } 15 + \frac{12 + 27/2 + 15}{3} = \frac{57}{2}.$$

The first changed cache entry is in layer 2, at zero-based position 5. Recovering the raw entries recovered layer 1's KV, but did not recover the representations needed by layer 2.

In the shared version, both layers read the original values 6, 7 and 8 for the last position. That position's residual is also unchanged: it had its whole window in the preceding layer. Its final result is $8 + 7 + 7 = 22$. The retained shared KV is exact even though some earlier retained hidden rows are not. More generally, with shared KV, row-local operations and a replay that contains the last row's window, the last row stays exact by induction over layers: its own previous representation and its visible shared KV are unchanged. This reasoning also permits queries that depend on that row's previous representation. It does not permit cached KV that depends on other changed hidden rows.

9.1.1 A window grows backwards across layers

To recover three KV inputs at layer 2, first recover the three layer-1 outputs that produce them. Their combined input span contains five positions. Replaying inputs 4 through 8 therefore restores the retained KV and final output of our two-layer model exactly. The lab checks this equality with fractions.

For a window of W entries at layer D, each preceding layer adds $W - 1$ positions to the left. The current position is already included, so it must not be counted twice. The span is

$$R = W + (D - 1)(W - 1). \tag{9.1}$$

For 20 layers with a 128-position window, $R = 2{,}541$. The deployment Q&A describes the conservative product $20 \times 128 = 2{,}560$ [2]. Both express the growth of SWA dependencies across layers; 2,541 is the tighter span under the inclusive causal-window convention.

This counts input positions required by the local dependency chain, conditional on shared inputs and global KV already being available. It is not the sum of all layer-row computations. Those intervals can shrink as the computation moves upwards. Nor does it account for the work of producing the shared input. The lab checks these claims for windows 2 to 4 and one to three layers, on every 0/1 input two positions longer than the span: 5,456 cases. A shared-input replay of one window and a per-layer replay of R positions each reproduce the final row, the retained KV and one subsequent decode step. For every window and depth, replaying $R - 1$ positions changes the final row or the decode result for at least one input, so no shorter replay is exact for this rule. These are exact statements about the teaching rule, not about DeepInfer's kernels.

9.2 Where DeepInfer puts the boundary

DeepInfer's V4.1 path combines two sources of KV. Consumer layers project block KV from a shared representation produced at a boundary in the model. Their SWA KV instead comes from each layer's own input. Thus the engine can preserve the source of block KV while shortening the consumer computation that builds local SWA state.

The design draws on YOCO (*You Only Cache Once*). The terminology comes from three places:

YOCO paper	Model report and Q&A	DeepInfer code
Self-decoder	Causal encoder	Producer work before `n_prefill_layer`
Cross-decoder	Decoder	Consumer layers, starting at `n_prefill_layer`
Shared global KV	Shared encoder representation	Boundary normalisation; `shared_hidden` and its full-row counterpart

CED means *Causal Encoder-Decoder*. The model report explicitly credits YOCO as an inspiration. The YOCO paper's cross-decoder reads a shared global KV cache and permits prefill to exit early without changing the final output. CED uses layer-dependent global-KV projections from shared hidden states and retains a separate per-layer SWA path. That local path requires reconstruction when decoder prefill is shortened [68, 69]. The table maps roles; it does not make these architectures identical.

Our concrete case is a long prompt with no prefix-cache hit, processed in one chunk. We explicitly enable `compact` mode. The producer layers process all prompt rows. At the boundary, two row layouts emerge (Figure 9.1). We first follow those layouts, then examine how DeepInfer selects the retained positions.

9.2.1 One layer performs two jobs

The Hugging Face adapter's `_n_prefill_layer` returns the index of the first layer whose compression ratio is 1. Its docstring describes that layer as part of the prefill side. The runtime predicate includes the same index among consumers [70, 71]:

Listing 9.2: The boundary index is also the first consumer index.
DeepInfer · `deepinfer/model_executor/models/v4_modules/yoco.py` · lines 19–21

```
19 def is_yoco_consumer_layer(cfg: AttrDict, layer_idx: int) -> bool:
20     """YOCO consumer layer: [n_prefill_layer, n_layer) in the main model."""
21     return yoco_enabled(cfg) and cfg.n_prefill_layer <= layer_idx < cfg.
       ↪ num_hidden_layers
```

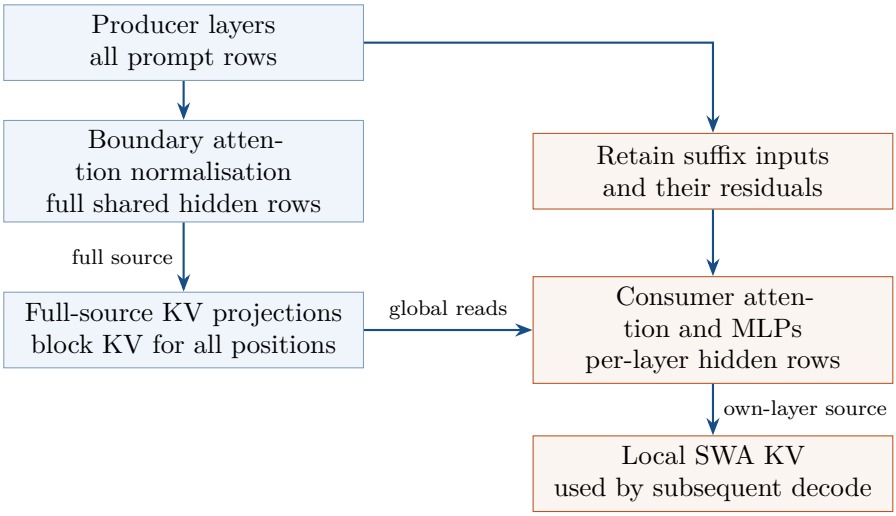


Figure 9.1: DeepInfer's compact prefill has two row layouts. Full shared inputs supply block KV; retained inputs continue through consumer layers. The boundary layer belongs to the consumer side even though it also normalises the full inputs. This diagram shows dependencies, not kernel timing.

The executed operations resolve the apparent disagreement. In `DeepseekModel.forward`, the boundary selects a consumer metadata view and retains the original metadata for full-source work:

Listing 9.3: Keep full metadata while switching the consumer's view.
DeepInfer · `deepinfer/model_executor/models/deepseek.py` · lines 196–198

```
196 consumer_metadata = input_metadata.get_consumer_view()
197 cross_layer_runtime.consumer_full_metadata = input_metadata
198 input_metadata = consumer_metadata
```

At this point the hidden tensor still contains the original rows. The boundary layer makes compact copies of the hidden inputs and residuals, but normalises the full inputs as well:

Listing 9.4: Save a full normalised source, then continue with compact inputs.
DeepInfer · `deepinfer/model_executor/models/decoder_layer.py` · lines 137–143

```
137     compact_inputs, full_inputs = compact_yoco_boundary(
138         hidden_states, residual, input_metadata, cross_layer_runtime, hc_enabled=self.
    ↪ hc_enabled
139     )
140     cross_layer_runtime.consumer_full_hidden = self.attn_norm_forward(*full_inputs)[1]
141     hidden_states, residual = compact_inputs
142     del compact_inputs, full_inputs
143 hidden_states, hidden_states_bf16, residual = self.attn_norm_forward(hidden_states,
    ↪ residual, hc_post_intervention)
```

`consumer_full_hidden` holds the full attention-normalised input. The normalisation on the last line supplies the compact input for this layer's attention. A few lines later, the boundary saves the compact normalised copy as `shared_hidden`; the comment explicitly distinguishes it from the full copy (lines 147–152, author's translation) [21].

The boundary therefore supplies full shared rows and runs consumer attention on compact rows. It is not a full producer attention layer followed by a separate first consumer layer.

Counting it twice would misstate both the architecture and the saved computation. Its MLP also runs on the compact rows passed through attention.

9.2.2 Block KV keeps its full source

Consumer layers that own block KV call `_store_yoco_full_kv`. This method reads `consumer_full_hidden` into a local variable named `full_hidden`. It checks the row count against `full_metadata`, then projects and stores the KV:

Listing 9.5: Project and store block KV from full shared rows.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 704–709

```
704 assert self.shared_hidden_kv_proj is not None
705 projected_kv = self.shared_hidden_kv_proj(full_hidden)
706 with userbuffer.cp_session(enabled=use_cp):
707     wait_projected_kv = _make_cp_gather_getter(projected_kv, use_cp)
708     pre_rope_kv = self._store_main_kv(wait_projected_kv, kv_cache, full_metadata)
709 cross_layer_runtime.pre_rope_kv = pre_rope_kv
```

Both operations cover source positions excluded from consumer attention and MLP computation. Layers that reuse another layer's block KV need not repeat that projection. In our no-hit case, truncation has not removed the inputs required to construct block KV. This is exactness of the dependency, not a claim that every kernel layout yields bitwise-identical arithmetic [22].

There is a second kind of cross-layer sharing in the repository. `SharedLayerTopology` maps layers to sources for KV, index keys, selected indices and candidate blocks. These segmented maps are distinct from sharing the boundary representation. Its validation requires a KV source at the YOCO boundary: a share-KV segment cannot cross that boundary (`cross_layer.py`, lines 114–116). These maps govern reuse of computed entries between layers; the shared hidden path determines the input from which those entries are projected [72].

9.2.3 SWA keeps a per-layer dependency

In the HF adapter, `yoco_swa_share_hidden` is false. The local KV therefore comes from the current hidden input. The method `_project_q_and_swa` selects that source alongside the query projection:

Listing 9.6: The SWA source is selected separately from the block-KV source.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 394–400

```
394 q_a_kv_swa = self.q_a_kv_swa_proj(hidden_states)
395 if self.yoco_swa_share_hidden:
396     assert shared_hidden_bf16 is not None, f"layer {self.layer_idx}:
    ↪ yoco_swa_share_hidden requires shared_hidden_bf16"
397     return q_a_kv_swa, self.kv_swa_proj(shared_hidden_bf16)
398 if self.use_swa:
399     return q_a_kv_swa.split([self.q_lora_rank, self.qk_head_dim], dim=-1)
400 return q_a_kv_swa, None
```

With the flag false, `split` separates query and local KV components produced from `hidden_states`. With it true, a separate projection takes `shared_hidden_bf16`. These correspond to the two dependency choices in the opening experiment. In the inspected V4.1 configuration, a consumer row near the retained region's left edge loses SWA neighbours.

Its changed output can become a later consumer's KV input. Preserving full block KV does not repair that local dependency.

The toy's shared-input case explains why moving KV to a shared source can remove this particular dependency. It does not establish that enabling the flag would make DeepInfer's whole computation exact: the rest of the model, indexing and numerical kernels would need their own checks. The engine we are reading uses the per-layer choice.

9.3 Which rows can a consumer see?

Let the prompt length be L, the retained count K, and the window W. The retained region starts at $s_{\text{keep}} = L - K$, clipped by the actual prompt extent. For our small diagram, $L = 12$, $K = 4$, $W = 3$. Full-source block KV still covers all 12 positions; consumer queries retain positions 8 through 11 (Figure 9.2).

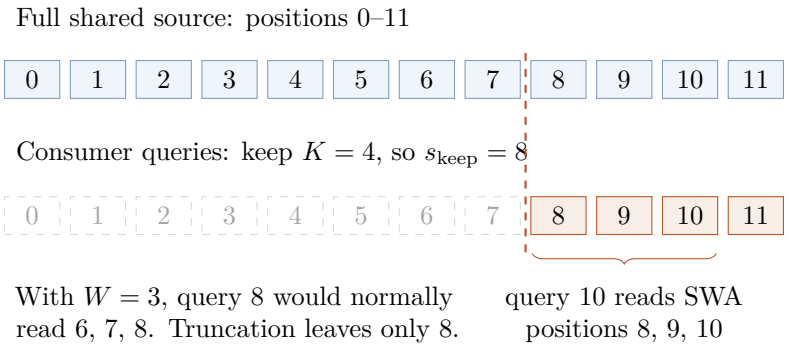


Figure 9.2: A small row-layout example with $L = 12$, $K = 4$ and $W = 3$. Block KV still has its full source. The first retained SWA queries lose neighbours; their changed outputs can become later layers' local KV.

9.3.1 Compaction changes indices, not positions

The C++ function `fill_yoco_truncation_metadata` prepares per-request clipping and compact query indices; `update_yoco_prompt_v4` checks the keep count before calling it (lines 203 and 349–359). In a chunk starting at `q_start` with `q_len` rows, `data_len` is the full sequence length used to locate the suffix:

Listing 9.7: Remove a chunk's prefix, retaining at least one query row.
DeepInfer · `csrc/engine/input_metadata/prepare_tensor_input_v4.cpp` · lines 225–230

```
225 int32_t clip_prefix = compact ? std::clamp(data_len - keep_tokens - q_start, 0,
      ↪ q_len - 1) : 0;
226 clip_prefixes[i_seq] = clip_prefix;
227 full_bases[i_seq] = full_base;
228 int32_t keep_len = q_len - clip_prefix;
229 p_cu_c[i_seq + 1] = p_cu_c[i_seq] + keep_len;
230 full_base += q_len;
```

For a single 12-row chunk, this removes eight rows and leaves four. For a four-row chunk covering positions 0–3 of that same prompt, it removes only three: the upper clamp is $q - 1$. The remaining position 3 is a placeholder, outside the final retained region. It keeps the chunk's consumer computation nonempty; it is not another useful suffix token.

The metadata view preserves original positions while replacing query-aligned arrays and counts:

Listing 9.8: Construct a compact view without changing original token positions.
DeepInfer · `deepinfer/model_executor/input_metadata/yoco.py` · lines 101–110

```
assert self.yoco_compaction_enabled
view = copy.copy(self)
qi = self.yoco_consumer_query_indices
view.total_q = self.yoco_consumer_total_q
view.cu_seqlens_q = self.yoco_consumer_cu_seqlens_q
view.positions = self.yoco_consumer_positions
view.slot_mapping_store_swa = self.yoco_consumer_slot_mapping_store_swa
view.ks = self.yoco_consumer_ks
view.ke = self.yoco_consumer_ke
view.kv_ca_index_offset_len = self.yoco_consumer_kv_ca_index_offset_len
```

In the one-chunk example, compact row 0 represents original position 8. RoPE positions and ring slots must follow the latter. The view also replaces SWA pivots and clears derived attention fields so they can be regenerated for the new layout. It is a shallow copy: fields that were not replaced can still share tensors with the original metadata, and the docstring forbids modifying those shared tensors in place (author's translation) [73]. With context parallelism, further code redistributes the retained rows across ranks; the source and consumer layouts still remain distinct.

9.3.2 Three lower bounds

Keeping a row does not grant it access to every old cache entry. The decisive line in `yoco_swa_visibility` is:

Listing 9.9: Combine the window, retained-region and valid-state bounds.
DeepInfer · `csrc/engine/input_metadata/prepare_tensor_input_v4.cpp` · line 155

```
int32_t abs_lo = std::clamp(std::max({pos - window_size + 1, rect_lo,
    ↪ valid_start}), 0, pos + 1);
```

`rect_lo` is the retained-region start; `valid_start` is the earliest valid state position. For a query at position i, the lower bound is

$$\ell_i = \mathrm{clamp}\big(\max(i - W + 1, s_{\mathrm{keep}}, s_{\mathrm{valid}}), 0, i + 1\big).$$

Visible positions run from ℓ_i through i. If $\ell_i = i + 1$, that interval is empty. The comment says that the valid-state bound prevents attention from reading zeroed slots (author's translation, lines 151–154) [38].

For query 8 in the diagram, the usual window is 6–8 but the kept-region bound raises the start to 8. Query 10 sees 8–10, a full three-position window. Its current layer has all three positions, yet their representations may already carry earlier truncation error. This is the distinction the lab exposed. The Q&A's $\max(s, i - 127)$ expresses the replay case with a 128-position window; the code also carries the valid-state bound and the empty-interval case.

A placeholder must not overwrite live ring state. The slot assignment checks whether the position actually belongs to the retained region:

Listing 9.10: A placeholder has no SWA write destination.
DeepInfer · csrc/engine/input_metadata/prepare_tensor_input_v4.cpp · lines 267–271

```
267 bool in_rect = pos >= data_len - keep_tokens;
268 if (in_rect && q_len - j <= swa_block_size)
269     p_slot_c[i_qc] = p_seq->state_id * swa_block_size + pos % swa_block_size;
270 else
271     p_slot_c[i_qc] = -1;
```

The extra length condition limits writes to the trailing ring capacity, as in chapter 5. The placeholder at position 3 receives -1; an ordinary retained row uses its original position modulo the ring size. Physical bytes left from older work do not override these validity rules.

9.3.3 Modes, defaults and limits

`off` keeps the ordinary prefill. `mask` retains all query rows but narrows consumer SWA visibility. `compact` uses that narrowed visibility and removes query rows, apart from placeholders. Mask therefore supplies a comparison for the compact layout. It removes no consumer rows; only its count of visible SWA pairs falls. The benchmark's help text says that mask computes every consumer token and saves no work [15]. Both enabled modes truncate the local dependency.

The numbers governing this path have different jobs:

Number	Meaning and evidence
128	SWA window for the V4.1 setting in the Q&A; supplied by model configuration
134	Inference ring: window plus maximum backtracking, $128 + (5 + 1)$ under the chapter 3 defaults
140	README's suggested explicit keep count for V4.1
$K \geq 134$	Runner assertion under those window/backtracking settings
$L - 134$	Production prefix-hit cap in the Q&A; not a released hit-policy implementation
2,541 / 2,560	Derived SWA dependency span / Q&A's conservative product

The Q&A describes replaying 128 tokens; the README tells users to retain 140. These are not interchangeable defaults. `environ.py` defaults truncation to `off` and the keep count to zero. To enable it, users must supply a valid count. `ModelRunner.load_model` asserts that the count covers the window plus maximum backtracking. That assertion protects capacity after backtracking; it does not prove exact reconstruction [1, 30, 45].

Both enabled modes disable prompt log probabilities. The engine enforces this in `LLMEngine.add_request` by clearing `prompt_logprobs`: compact omits prompt rows, and mask changes their local context. Compact imposes two further restrictions when the YOCO path is enabled:

Listing 9.11: Compact consumer layouts constrain auxiliary model features.
DeepInfer · `deepinfer/model_executor/models/deepseek.py` · lines 129–138

```
if dflash_target_layer_ids:
    in_consumer = [t >= cfg.n_prefill_layer for t in dflash_target_layer_ids]
    assert all(in_consumer) or not any(in_consumer), (
        "yoco consumer compaction requires DFlash target layers on the same side of "
        f"n_prefill_layer ({cfg.n_prefill_layer}), got {dflash_target_layer_ids}"
    )
    self.dflash_targets_in_consumer = all(in_consumer)
assert all(layer_id < cfg.n_prefill_layer for layer_id in engram_layer_ids), (
    "yoco consumer compaction does not support Engram in consumer layers yet."
)
```

All DFlash target layers must fall on the same side of the boundary, so its collected features have a consistent row layout. Engram layers must remain before the consumer boundary. These two checks belong to compact mode; they are not blanket restrictions on mask [20, 5].

9.4 Which state may enter the prefix cache?

Chapter 4 divided a `StateBlock` into stored and dropped regions. We can now explain the consumer SWA part of that division. In `worker/cache_engine.py`, `_is_dropped_swa_layer` returns true for layers at or beyond `n_prefill_layer`, provided the model has a consumer region. It does not inspect the truncation mode. The counts use that predicate:

Listing 9.12: Partition SWA state according to the model boundary.
DeepInfer · `deepinfer/worker/cache_engine.py` · lines 265–273

```
self.swa_num_layer = sum(int(GPUCacheInterface._has_SWA(attn_module)) for attn_module in
    ↪ runtime_attn_modules)
self.swa_store_num_layer = sum(
    int(GPUCacheInterface._has_SWA(attn_module))
    for layer_id, _, attn_module in iter_runtime_attn_modules(attn_module_list, cfg.
    ↪ num_hidden_layers)
    if not _is_dropped_swa_layer(cfg, layer_id)
)
self.swa_drop_num_layer = self.swa_num_layer - self.swa_store_num_layer
self.swa_block_size_kvstore = get_kvstore_swa_block_size(window_size)
self.swa_block_size_inference = get_inference_swa_block_size(window_size)
```

Later, `state_size_to_disk` is assigned only the stored SWA region's inference-layout byte count. Consumer SWA remains in the dropped region alongside compression state. The transfer from the prefill engine to the decode engine (PE to CE in the docstring and Q&A) includes that region, because decode needs it. "Dropped" thus means excluded from prefix persistence, not released at the end of prefill [23].

This policy makes the next prefill rebuild consumer SWA instead of retrieving it. With truncation enabled, that reconstruction is approximate. With it off, consumer computation is full only when nothing is restored. `load_state_from_store_to_prefill_cache` copies stored state into a prefill slot after a hit. Its first statement asserts that truncation is not

`off` whenever the model has dropped SWA layers (lines 836–837). A hit without truncation would need consumer SWA for positions before the hit, and nothing stored supplies it. The release therefore ties restored prefix state to truncated consumer replay, although the cache partition itself is the same in either mode [23].

The Q&A identifies the first 20 layers as the stored V4.1 layers, and the model report gives 20 encoder and 20 decoder layers. These counts are documented model settings, not numbers implied by the generic predicate [2, 69].

9.4.1 The other bounded replay

The deployment Q&A uses *bounded replay* for two operations. Our source path so far is the decoder operation:

	Decoder replay	Encoder replay
Trigger	Prefill with truncation enabled	Block-KV cache hit without usable SWA
Work	Rebuild consumer SWA over a bounded suffix	Replay the hit prefix's last block, then process the uncached suffix
Evidence	Released prefill and consumer code	Q&A's production design; released hooks without the driving hit policy

For encoder recovery, the Q&A describes reusing the replayed prefix's block KV without recomputing or overwriting it. Only its SWA is rebuilt; the uncached suffix produces both kinds of KV. The production cache manager chooses a block-aligned hit and backs off one block when qualifying SWA state is missing. That cache-management implementation is outside the release [2].

The released hooks support parts of such a path, but do not drive it. `swa_valid_start` is initialised or reset to zero in `KvCacheState`; it has a binding, but the inspected Python code contains no writer. The scheduler reserves and clears `zero_states` without filling it. It stores `truncate_len` without reading it. Following those names alone would therefore not reconstruct the production hit-recovery algorithm [6, 8].

The documented hit cap also leaves an integration question. Capping a hit at $L - 134$ guarantees at least 134 uncached rows. Those rows could supply the shared hidden inputs needed by decoder replay; that connection is an inference about the unreleased driver. An aligned hit exactly at the cap leaves 134 rows, not the README's 140. The released metadata code makes the gap concrete. With $q_{\text{start}} = L - 134$, $s_{\text{keep}} = L - 140$ and $W = 128$, the first query's lower bound is $L - 140$ unless s_{valid} is raised. Its visibility therefore includes six consumer ring entries from before the hit (`start_slot` is 122). Restoring a hit writes only the stored region, so nothing in the release fills those entries. This is our reading of the metadata code; we have not executed it. Block alignment and an SWA miss can lengthen the suffix, but neither supplies a universal answer for the exact-cap case. The release does not establish how that service policy and the suggested keep setting are combined. Chapter 11 will discuss the documented policy with this limit intact.

9.5 Count the saving; choose the error baseline

```
python3 labs/20_replay_cost.py
```

The second lab translates DeepInfer's query-range and SWA pair-count formulas into standard-library Python. It checks them against a reference that enumerates every visible query-key pair. The first examples use $W = 3$ and $K = 4$:

```
window=3, keep=4; entries are consumer rows / SWA pairs
case           off       mask     compact
short         2/ 3    2/ 3    2/ 3
long         12/33   12/ 9    4/ 9
clipped       4/ 9    4/ 0    1/ 0
crossing      4/12    4/ 3    2/ 3
valid-start   2/ 3    2/ 3    2/ 3
```

The long case covers all 12 prompt positions. Full SWA has $1 + 2 + 10 \times 3 = 33$ pairs. Truncating visibility leaves $1+2+3+3 = 9$ pairs in either enabled mode. Mask still executes 12 query rows; compact executes four. In the clipped case, the chunk covers positions 0–3. Compact executes its one placeholder with no visible SWA keys. The crossing case covers positions 6–9: only 8 and 9 remain useful, with one and two visible keys respectively. The valid-start case begins at 10 and excludes all earlier state; its two queries have one and two keys even with truncation off. These cases separate row counts from attention-pair counts.

9.5.1 The estimator knows about placeholders

`FlopsEstimator::query_ranges` starts with the full query interval $[k - q, k)$. Its consumer adjustment follows the same chunk rule as the metadata builder:

Listing 9.13: Count consumer queries and visible SWA pairs after truncation.
DeepInfer · csrc/engine/flops_estimator.hpp · lines 267–277

```
267 QueryRange consumer = full;
268 if (is_prompt && (compact || has_truncated_swa)) {
269     const int cutoff = full_len - keep_tokens;
270     // A fully clipped compact chunk still executes its last query row.
271     if (compact)
272         consumer.begin = std::clamp(cutoff, start, k - 1);
273     const int swa_start = std::max(swa_valid_start, cutoff);
274     consumer.swa_pairs =
275         window_size > 0 ? pairs(std::max(0, consumer.begin - swa_start), std::max(0, k -
    ↪  swa_start), 1, window_size) : 0;
276 }
277 return {full, consumer};
```

`pairs` uses a prefix sum. For a window W, the cumulative pair count through n positions is

$$S(n) = \frac{m(m+1)}{2} + (n - m)W, \qquad m = \min(n, W).$$

Subtracting two prefix counts gives an interval's work. Subtracting `swa_start` first expresses the interval relative to its valid local source; clamping to zero handles queries before that source. The lab compares these formulas with direct key enumeration on 8,910 cases, including short sequences, chunk boundaries and nonzero valid starts.

A 1,024-position prompt processed in four 256-position chunks illustrates why keep count alone is insufficient. With $K = 140$, the first three chunks each execute one placeholder consumer row and the last executes 140. The total is 143, whereas one full chunk executes 140 consumer rows. The source estimator counts the extra rows even though they have no visible SWA keys.

9.5.2 Why the saving can approach one half

Ignore differences in layer cost for a moment. For P producer layers, D consumer layers and one prefill chunk, the ratio of retained layer-row work to full layer-row work is

$$\frac{PL + D\min(L, K)}{(P + D)L}.$$

V4.1 has equal halves: the model report gives 20 encoder and 20 decoder layers [69]. With $P = D$ and L much larger than K, the ratio approaches one half. The lab's illustrative equal-cost calculation gives:

```
Equal layer costs, 20 producer + 20 consumer layers, keep=140
length    full row-work    compact row-work    ratio
    64             2560                2560   1.0000
  1024            40960               23280   0.5684
 16384           655360              330480   0.5043
```

At length 64 there is nothing to remove. At 1,024, the ratio is about 0.5684; at 16,384 it is about 0.5043. This explains the scale of the Q&A's "nearly halving" claim, rather than reproducing a model-specific FLOP count.

DeepInfer's actual estimator retains work that the row proxy omits. `calc_kproj_flops` takes the full query range for block-KV projections. `calc_attn_flops` combines full and consumer row terms, SWA pair terms, and global-attention pair terms. The boundary normalisation-related contractions also retain full-row work. `calc_non_attn_flops` weights producer and consumer rows using their own coefficients. Chapter 2's FP8-equivalent accounting still applies; these counts do not turn directly into execution time [14].

The repository's `tests/engine/test_flops_estimator.py` compares the native estimator with per-layer, per-query accounting, including off, mask, compact and chunk cases. It requires model metadata and the compiled extension. Our lab checks the translated range and pair formulas, not that native test or the full model's cost [74].

9.5.3 Equivalence needs the right comparison

The opening experiment compares shortened computation with full computation. The released YOCO checks ask a different question: does compact match mask? Their L1 check compares metadata, visible keys and slots. L2 compares model log probabilities and top-token results along a fixed teacher-forced trajectory. Both modes use truncated SWA, so agreement between them cannot establish that truncation preserves the full model [75].

The script also supports an untruncated baseline by setting `keep_tokens=0`; it then selects `off`. The capability to produce a baseline is not a published error measurement. No full-versus-truncated error measurement was found in the inspected release. The model report states that the reconstructed decoder SWA KV is not mathematically equivalent to a full decoder pass. It reports a negligible effect on response quality, and simulates the same replay during post-training [69]. These are the authors' findings, not an error

bound reproduced here. The last point means the approximation is partly a model property: post-training adapted the model to it, so it is not only an engine shortcut.

An evaluation of the approximation would hold inputs and the continuation trajectory fixed, then compare full and truncated intermediate state and log probabilities. A separate task evaluation would ask whether those differences matter to answers. Exact state equality, numerical agreement, and acceptable model quality are three different claims. Chapter 10 returns to the contracts checked by the released equivalence tests.

9.6 Experiments to try next

1. Use equation (9.1) for four layers and a five-position window. List the intervals required at each layer to recover the final window. Which number counts input positions, and which counts layer-row computations?

2. In lab 19, retain four inputs instead of three or five. Find the first differing second-layer cache entry. Is the final prefill result exact? Check one subsequent decode step as well.

3. A prompt has 12 positions, keep count four and window three. For a chunk covering positions 4–7, find the compact query, its SWA visibility and its write slot. Explain why a query can execute without updating the ring.

4. Switch truncation from compact to off. Does `_is_dropped_swa_layer` move consumer SWA into the stored region? Trace where that state is still needed before it can be released. Then follow a prefix hit into `load_state_from_store_to_prefill_cache`. Why must it refuse to run?

5. A block-aligned hit leaves 134 uncached rows. Which part of a 140-row retained suffix lies before the hit? Identify the released evidence and the missing service behaviour needed to say how those rows are supplied.

6. Mask and compact produce identical log probabilities. State what this establishes and propose a different comparison for the approximation error.

Chapter 10

Different correctness contracts

The previous chapter separated two questions: whether mask and compact compute the same truncated model, and whether that model is close enough to a full prefill. A passing test answers the question encoded in its assertion. Changing the question can change the verdict without changing the engine.

The question

A row contains $10^{16}, -10^{16}, 1$. Why might its sum change when an unrelated row joins the batch? If two computations pass a check whose threshold is zero, have they necessarily produced equal values?

```
python3 labs/21_equality.py
```

The experiment makes the reduction order depend on the number of rows. It then keeps that order fixed, and finally tries two other checks:

```
row-count-dependent: single=1, batched=0
row-count-dependent: masked=0, compact=1
fixed order: A=1 in every layout; 6 subset/order checks
adjacent float32 inputs: equal=False, abs diff=1.16415321827e-10
exp(d)-1-d metric=0; zero does not establish equality
logprob lists differ; both perplexities=7.389056099
```

The unrelated row changes the first result from 1 to 0. Fixing the order removes that difference. Yet a numerical difference metric can still return zero for unequal inputs, and two different logprob lists can have equal perplexity. These are three different observations, not three ways to prove the same claim.

Scope and order. We first explain the reduction, then follow DeepInfer's batch-invariance flag into its libraries. Golden tokens let us compare computations along a fixed history. A second experiment separates repeatable random draws from a correct output distribution. Finally, we trace how grammar masks and matcher state constrain speculative output. The released tests supply the questions and assertions throughout; this chapter inspects them but does not run their model workloads.

10.1 Equality needs a fixed computation order

In exact arithmetic both ways of grouping the row give 1. In Python's usual binary64 arithmetic, adding 1 to 10^{16} loses the 1. The lab's single-row path adds from left to right:

$$(10^{16} + (-10^{16})) + 1 = 1.$$

Its multi-row path gives alternating entries to two lanes. The first lane adds $10^{16} + 1$, the second contains -10^{16}, and the lane totals add to 0. A *reduction* combines several values into one; its grouping is part of the numerical computation.

Here is the entire layout-dependent rule. Masking keeps the batch shape but skips unwanted outputs; compacting removes those rows before the call:

Listing 10.1: Choose the lane count from the batch shape.
Book lab · `labs/21_equality.py` · lines 19–27

```
def reduce_rows(rows, active, fixed=False):
    lanes = 1 if fixed or len(rows) == 1 else 2
    result = {}
    for name, values in rows:
        if name in active:
            partials = [left_sum(values[lane::lanes]) for lane in range(lanes)]
            result[name] = left_sum(partials)
    return result
```

Neither operation changes the retained row's values. Nevertheless, two physical rows select two lanes even if only one output is wanted. One compact row selects one lane. With `fixed=True`, both calls use one lane and agree. The lab checks all six combinations of nonempty retained subsets and row orders for its two-row input, comparing masked and compact calls against a direct sum.

This isolates one cause of a batch-dependent result. An engine also has to align dtypes, fused operations, kernel choices and the visible attention keys. Removing *query rows* can preserve the remaining computation when rows are independent in those respects. Removing keys from a retained query changes its mathematical input, as chapter 9 showed. No choice of addition order can undo that change. Figure 10.1 sets this comparison beside the other two execution changes this chapter tests.

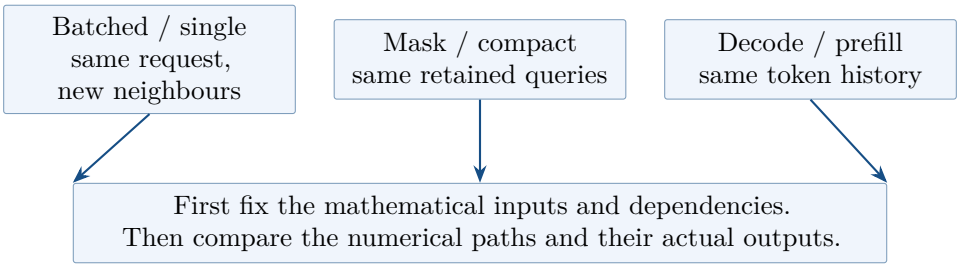


Figure 10.1: Three changes of execution shape. They need compatible numerical paths as well as compatible inputs.

We will distinguish four kinds of claim. An *exact comparison* requires equal observed values. A *tolerance check* admits a specified difference under a specified metric. A *distributional check* asks whether sampled outputs follow the intended probabilities. A *task or format check* asks whether an answer satisfies a requirement, such as a grammar. A valid format does not establish a factually correct answer.

Several released comments call direct comparisons "bitwise". We will state the comparator: Python `==` and `torch.equal` compare numerical values, not raw byte strings. For example, numerical equality does not distinguish positive and negative zero. More importantly for this chapter, a zero *threshold on a computed metric* need not even establish value equality. We return to the lab's counterexample when we open `test_kl.py`.

10.2 One flag reaches several libraries

DeepInfer's README offers a batch-invariance setting: a request's result should not depend on its batch neighbours. The worker translates that setting into library configuration before running model computations [1, 76]. At lines 67–69 of `deepinfer/worker/runtime_config.py`, the condition is "batch invariance enabled, or NPU". Its body is:

Listing 10.2: Enable deterministic algorithms in two kernel libraries.
DeepInfer · `deepinfer/worker/runtime_config.py` · lines 68–69

```
68 deep_gemm.use_deterministic_algorithms(True)
69 tile_kernels.use_deterministic_algorithms(True)
```

The NPU condition does not mean the complete feature is available there. Lines 61–63 separately assert that `DEEPINFER_BATCH_INVARIANT` is off on NPU. The adjacent comment says to remove this restriction after full support; the comment on line 67 says that the NPU setting prevents unstable CI numerics (*author's translations*). These are distinct scopes: deterministic algorithms in particular libraries, and an engine-wide batch-invariance promise. The README describes the latter as NVIDIA-only.

The flag also reaches attention. The two sparse MLA decode calls in `deepinfer/model_executor/models/v4_modules/attn.py`, lines 666 and 1002, pass `batch_invariant=envs.DEEPINFER_BATCH_INVARIANT`. Earlier in the same file, it changes the choice of fused normalisation and rotary-position operations. Read the entire condition:

Listing 10.3: The flag removes one route into the fused path.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 87–92

```
87 self.use_fused_norm_rope_attn_rope_cast = (
88     # disabled in dense packed-KV decoding
89     ((not envs.DEEPINFER_BATCH_INVARIANT and cfg.engine_type == EngineType.
   ↪ PROMPT) or self.is_sparse or not self.has_packed_kv)
90     and self.num_heads in (64, 128)
91     and PLATFORM.is_gpu
92 )
```

When the flag is on, `EngineType.PROMPT` alone no longer enables this fusion. But `is_sparse` or `not has_packed_kv` can still do so, subject to the head-count and GPU conditions. Saying that the flag disables all fused prefill paths would miss those alternatives. Aligning prefill and decode numerics is a plausible reason for controlling fusion; the predicate alone does not prove that explanation or the resulting equality.

Chapter 7 introduced a second source of layout changes: expert replicas. DeepInfer's DeepEP buffer has this option:

Listing 10.4: Restrict reduction choices when invariance and expert rearrangement are both enabled.
DeepInfer · `deepinfer/model_executor/parallel_utils/userbuffer.py` · line 79

```
79 allow_multiple_reduction=not (envs.DEEPINFER_BATCH_INVARIANT and envs.
   ↪ DEEPINFER_EXPERT_REARRANGEMENT_STEP_INTERVAL > 0),
```

The restriction requires *both* batch invariance and a positive expert rearrangement interval. Recall that replica selection includes a token's index in the batch. Changing neighbours can therefore change its destination rank. A changed route may change how partial results are summed. That is a control-flow inference linking the replica rule to this option; these callers do not reveal or prove the libraries' internal reduction order.

Owner	Responsibility visible at this boundary
DeepInfer	Choose modes, construct metadata, pass flags and compare end-to-end results
DeepGEMM, TileKernels	Implement the requested deterministic kernel algorithms
FlashMLA	Provide sparse MLA and fused norm/RoPE kernels; receive the sparse decode invariance option
DeepEP	Perform expert communication/combine with the selected reduction option

This chapter's comparison is a division of responsibility, not a claim that DeepInfer invented deterministic reductions. DeepGEMM, TileKernels, FlashMLA and DeepEP are independent DeepSeek repositories. Chapter 7 records the pinned evidence for the replica rule and the caller's communication paths. The attention calls lead to FlashMLA, named in chapter 3: DeepInfer's attention backend imports its sparse attention and fused norm/RoPE operations, while the V4 attention module also imports the fused operations directly [27, 22]. The manifest records FlashMLA commit `2e5429fc`, pinned by DeepInfer's `vendor/FlashMLA` submodule. This identifies the dependency; it does not audit its kernel internals. A flag at any one boundary cannot establish the whole engine contract; the next section examines what the released tests actually compare.

10.3 Golden tokens hold the trajectory fixed

Suppose two runs disagree at token 7. By token 8, their histories differ, so their model scores need not agree even in exact arithmetic. To locate the numerical difference, a test can force the same token history while continuing to compute the model's probabilities. This is *teacher forcing*. DeepInfer calls the forced sequence `golden_token_ids`.

The batch-invariance test first generates a batch of 64 requests. It then replays each request alone with the batched output as its golden sequence:

Listing 10.5: Hold the trajectory fixed while changing the batch.
DeepInfer · `tests/models/test_batch_invariant.py` · lines 99–105

```
99  for token_ids, output in zip(prompt_token_ids, outputs, strict=True):
100     sp = copy.copy(sampling_params)
101     sp.golden_token_ids = [x.token_id for x in output]
102     output_single = llm.generate(None, sp, [token_ids], use_tqdm=False)[0]
103     result_batched = _get_result(output)
104     result_single = _get_result(output_single)
105     assert result_batched == result_single, f"{dp_rank=}, {result_batched=}, {
      ↪ result_single=}"
```

The helper `_get_result` collects token IDs, their logprobs, and the top-5 IDs and logprobs. The assertion compares the whole tuple. However, equality of the output token IDs is imposed by the override. The substantial numerical check is whether the model assigns the same logprobs and top-5 results along that history. It is not a comparison of two independent stochastic draws [77].

10.3.1 What the override changes

The verification path in `Sampler._verification` first computes candidates, then calls `get_accept_result`. Its reference helper makes the override visible:

Listing 10.6: Replace candidates and acceptance decisions for golden rows.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/sampler.py` · lines 237–246

```
237 accept = raw_accept
238 tokens = candidate_token_ids
239 if golden is not None:
240     use_golden = golden != -1
241     accept = torch.where(use_golden, golden == draft_token_ids, accept)
242     tokens = torch.where(use_golden, golden, tokens)
243 blocked = ~(is_verify & accept)
244 prefix = torch.cat([blocked.new_zeros(1, dtype=torch.int32), blocked.cumsum(0,
    ↪ dtype=torch.int32)])
245 alive = prefix[:-1] == prefix[ftp[row_to_req.long()].long()]
246 effective_accept = alive & is_verify & accept
```

A golden value of -1 means no override. Otherwise, the candidate becomes the golden token and acceptance becomes the test "draft equals golden". The prefix calculation then prevents a request from accepting drafts beyond its first blocked row. The CUDA implementation repeats the same override before scanning preceding rows in `deepinfer/model_executor/layers/tilelang_ops/cuda/sampler.py`, lines 252–276 [78].

For a golden continuation $[a, c]$ and drafts $[a, b]$, the first draft matches, the second does not, and the output still follows $[a, c]$. The model's stochastic acceptance probabilities did not decide that trajectory. This is why a golden-token test is useful for numerical comparison and unsuitable as evidence that rejection sampling preserves a distribution. Also keep these testing tokens separate from DFlash's leading "golden" input: chapter 8 used that name for a committed token anchoring proposal generation.

10.3.2 The released inventory

The following checks vary different things. Test filenames beginning with `test_` are under `tests/models/`; the references identify each fixed-version file [77, 80, 82, 75].

Check	Change / observable	Assertion and boundary
`test_batch_invariant.py`	64 requests / one; token and top-5 logprobs	Tuple `==`; single replay uses golden tokens
`test_ve_batch_invariant.py`	Image batch composition; encoder embeddings	Same shapes and `torch.equal`; vision path
`test_kl.py`	Incremental decode / full prefill; logprobs	Prompt lists equal; continuation metric ≤ 0, not direct equality
YOCO L2	Mask / compact; logprobs, top-k, optional confidence	Direct value comparisons on a golden trajectory
`test_offline_inference.py`	Current run / stored baseline; decode perplexity	Exact scalar equality, platform and EP/CP specific
`test_sampler.py`	Four sampling settings; transformed samples	Aggregate Kolmogorov–Smirnov (KS) p-value above 10^{-5}, enough valid trajectories
`examples/_structured_output.py`	Conflicting prompts / required format	Syntax, normal stop and EOS at temperature zero

The vision test deserves its own row: it compares image embeddings before language-model sampling. It checks one image against repeated and mixed-shape batches using

`torch.equal`. Passing a language-output comparison would not by itself identify whether that earlier representation was unchanged.

10.3.3 Decode versus prefill: inspect the metric

`test_kl.py` uses MTP 5, context parallelism 2 and expert parallelism 8, with image input and batch invariance enabled. Its text tokens and golden continuation are drawn uniformly from non-special tokens, not from the model distribution. Sampling uses temperature 0.5 and top-p 0.95. The first pass decodes that continuation. The second appends it to the prompt and obtains its logprobs through prefill. This holds the history fixed while changing how it is computed.

The compared values are *processed* logprobs. Temperature changes the scores, top-p keeps a nucleus of high-probability tokens, and the remaining tokens receive zero probability. The sampling reference shows the order:

Listing 10.7: Transform the logits before reporting log probabilities.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/fused_sample.py` · lines 209–211

```
209 logits.mul_((1.0 / transform_inputs.temperatures).unsqueeze(1))
210 apply_top_p_top_k_ref(logits, transform_inputs.top_ps, transform_inputs.top_ks)
211 logprobs = torch.log_softmax(logits, dim=-1)
```

The top-p helper replaces excluded logits with $-\infty$, so their logprobs stay $-\infty$. Prompt logprobs use the same request settings: `csrc/engine/prepare_sample.cpp`, lines 186–188, copies temperature, top-p and top-k into the prompt arrays. The prompt-logprob helper passes those arrays to `fused_sample` before looking up each prompt token [77, 59].

The original prompt's logprob lists are compared directly. The continuation's logprobs are converted to float32 tensors and passed to `cal_diff`; the test accepts its fourth field at a threshold of zero. Before either comparison, it replaces $-\infty$ with -9999. Those transformations are part of what the test checks. If a token lies outside both nuclei, both values become the same sentinel, irrespective of their excluded logits. Such a position records agreement on exclusion, not agreement between finite logprobs. If nucleus membership differs between the runs, that normally produces a sentinel-versus-finite discrepancy.

Uniformly chosen tokens could make these exclusions common when the model's nucleus contains few vocabulary entries. The fraction depends on the model and histories; it has not been measured here, and the test does not report it. To assess numerical coverage, record positions excluded on both sides, positions retained on both sides, and membership disagreements before replacing infinities. Finite-position differences and exclusion agreement answer different questions.

Two comparisons help locate this limitation. YOCO L2 uses temperature 0.5 and top-p 1.0, leaves top-k disabled and asserts that `DEEPINFER_SAMPLER_FORCE_TOPK` is zero: there is no nucleus or top-k truncation in that setup. The batch-invariance test instead obtains its golden tokens by sampling the batched run, rather than uniformly choosing vocabulary entries. Its chosen tokens therefore come from that run's retained distribution; the single-request replay must match their logprobs and top-5 results. Neither contrast makes the selected probabilities an independent oracle for the whole model [75, 77].

The continuation comparison has a second limitation, in the metric itself:

Listing 10.8: A field named kl is computed from chosen-token logprob differences.
DeepInfer · `deepinfer/testing.py` · lines 4–12

```
def cal_diff(x: torch.Tensor, y: torch.Tensor) -> tuple[float, float, float,
    ↪ float]:
    if torch.equal(x, y):
        return 0.0, 0.0, 0.0, 0.0
    x, y = x.double(), y.double()
    rmse = ((x - y) * (x - y)).mean().sqrt().item()
    cos_diff = 1 - 2 * (x * y).sum().item() / max((x * x + y * y).sum().item(),
    ↪ 1e-12)
    chebyshev_dis = (x - y).abs().max().item()
    kl = ((torch.exp(x - y) - 1) - (x - y)).mean().item()
    return cos_diff, rmse, chebyshev_dis, kl
```

Write $d = x - y$. The last field averages $e^d - 1 - d$ over the selected tokens. A KL divergence between two categorical distributions would require summing over their token probabilities with the appropriate weights. This expression is a discrepancy measure on the supplied logprobs; its name does not make it that distributional divergence.

In real arithmetic, $e^d - 1 - d$ is non-negative and zero only at $d = 0$. Floating-point subtraction changes the conclusion. Lab 21 uses adjacent float32 values, both exactly representable in Python's binary64 floats:

$$x = -0.0010000000474974513, \quad y = -0.0009999999310821295, \quad |x - y| = 2^{-33}.$$

The lab evaluates the same scalar expression and gets zero. The original PyTorch helper also returns zero for these inputs on CPU despite `torch.equal` being false. The first return branch is sufficient for zero, but it is not necessary. To assert exact value equality, compare the values directly; to allow a tolerance, choose and state a metric that measures the intended difference.

This comparison completes part of chapter 5's recomputation story. Rebuilding decoded history through prefill should reconstruct compatible representations and probabilities. The test probes that boundary for its selected inputs and settings. It does not actually preempt and resume a request, and its zero metric does not prove identical continuation. Sampling state, auxiliary state and the cache must also be compatible. Nor does this test enable chapter 9's truncated-prefill mode.

10.3.4 Two other baselines

YOCO L2 compares mask with compact under batch invariance, fixing the golden trajectory and checking the other relevant settings. It directly compares chosen logprobs, top-k IDs and logprobs, and optional DFlash confidence values. The context-parallel derivation comment in `deepinfer/model_executor/input_metadata/yoco.py`, lines 136–139, explicitly points to an end-to-end equality check. Neither a local derivation nor agreement between these two truncated modes measures their error against full prefill. That remains chapter 9's separate quality question.

The offline-inference test uses a different baseline: stored decode perplexities in `tests/models/offline_decode_ppl.json`. It checks the current run against those scalars, rather than comparing two fresh runs. The baseline is tied to the platform, workload and parallel configuration: NVIDIA uses EP 4 / CP 2; Ascend uses EP 16 / CP 8. Both have two data-parallel ranks. Changing that configuration requires a new baseline.

For chosen-token logprobs $\ell_1, \ldots, \ell_n$, perplexity is $\exp(-\sum_i \ell_i/n)$. Lab 21 compares $[-1, -3]$ with $[-2, -2]$. Both give e^2, although neither token's logprob agrees. Exact equality of an aggregate is an aggregate regression check, not tokenwise equality.

10.4 Repeatable and correctly distributed are different claims

Numerically equal probabilities are only part of repeatable sampling. A draw also consumes randomness. A shared stream hands its next value to whichever request happens to run next. Moving a request changes the value it receives.

```
python3 labs/22_sampling_contracts.py
```

```
shared stream AB: A=0, B=1
shared stream BA: A=1, B=0
keyed draws: all 6 batch orders and single requests agree
q=1, position 7: fresh target, token=1
q=3, position 7: accepted draft, token=0
ideal marginal in both budgets: [1/2, 1/2]
```

The shared stream changes both requests' tokens when their order is reversed. The keyed version survives all six orders of three requests and each request run alone. But changing a verification budget still changes one token, even though the toy target and draft have identical distributions.

10.4.1 Keys belong to requests, positions and operations

The teaching generator hashes an explicit request seed and a position/role key. It uses SHA-256, not Python's process-randomised `hash`, and returns an exact rational in $[0, 1)$:

Listing 10.9: A stable teaching draw, independent of the batch slot.
Book lab · `labs/22_sampling_contracts.py` · lines 19–23

```
19 def keyed_draw(seed, position, role, draft=False):
20     effective_seed = seed + (DRAFT_OFFSET if draft else 0)
21     message = f'{effective_seed}:{3 * position + role}'.encode('ascii')
22     bits = int.from_bytes(sha256(message).digest()[:8], 'big')
23     return F(bits, 2**64)
```

DeepInfer uses its device RNG instead. The following line numbers refer to `deepinfer/model_executor/layers/tilelang_ops/cuda/fused_sample.py`. In that file, the effective seed is the request's `seq_seed` plus a namespace offset. The wrapper sets that offset to zero for the default path and to 1,000,000,007 for MTP. The CUDA kernel uses three position-derived sequences [79]:

Operation	RNG sequence	`cuda/fused_sample.py` line
Fresh target sample	$3p$	450
Rejected-draft replacement	$3p + 1$	481
Accept/reject draw	$3p + 2$	392

Here p is the kernel's `sample_positions` field. The seed is selected at line 227. Batch slot is not part of these RNG keys.

There is an important intermediate transformation. For packed verification rows, `process_sampling` expands the per-request parameters using `row_to_req` and `local_step`. Its reference expansion is:

Listing 10.10: Preserve the request seed and mix in its local verification offset.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/sampler.py` · lines 61–66

```
req = row_to_req.long()
return SamplingKernelInputs(
    LogitsTransformInputs(params.logits.temperatures[req], params.logits.top_ps[
    ↪ req], params.logits.top_ks[req]),
    params.seq_seeds.view(torch.int64)[req].view(torch.uint64),
    params.sample_positions[req] + local_step * DRAFT_POS_HASH_MUL,
)
```

`DRAFT_POS_HASH_MUL` is 10,000,019, not 1. Thus p is not always a literal absolute token index. Reordering requests while preserving their local work preserves this construction; changing local verification positions need not. DFlash separately forms proposal positions from the base, the last valid offset and the draft index, then samples in the MTP seed domain. The lab deliberately uses simple absolute positions to isolate the role/domain change. It reproduces neither this position mixing nor the device generator.

10.4.2 Why the budget matters even with a fixed seed

Chapter 8's timing curve influences the total verification budget and each request's query count. The sampling metadata determines the role of each row:

Listing 10.11: The final query draws afresh; preceding queries verify drafts.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/sampler.py` · lines 18–25

```
row_to_req = torch.repeat_interleave(
    torch.arange(ftp.shape[0], dtype=ftp.dtype, device=ftp.device),
    seqlen_q,
    output_size=total_q,
)
local_step = torch.arange(total_q, dtype=ftp.dtype, device=ftp.device) - ftp[
    ↪ row_to_req.long()]
is_verify = local_step != seqlen_q[row_to_req.long()] - 1
return row_to_req, local_step, is_verify
```

For a query count $q = 1$, local row 0 is the final row and takes a fresh target sample. For $q = 3$, rows 0 and 1 verify drafts, while row 2 draws afresh. At the same output position, the first run may use a target draw and the second an accepted proposal drawn earlier in the MTP namespace. Equal seeds do not make those two operations consume the same randomness.

Lab 22 sets target and draft probabilities to $(1/2, 1/2)$, making every draft acceptable. Its budget loop follows that final-row rule:

Listing 10.12: Changing q changes which operation produces a position.
Book lab · `labs/22_sampling_contracts.py` · lines 39–52

```python
def budget_output(seed, start, q):
    # q-1 verified drafts, then one fresh target draw, as in the metadata rule.
    output, roles = {}, {}
    for local in range(q):
        position = start + local
        verify = local != q - 1
        if verify:
            draft = token(keyed_draw(seed, position, 0, draft=True))
            assert keyed_draw(seed, position, 2) < 1  # target/draft ratio = 1
            output[position], roles[position] = draft, 'accepted draft'
        else:
            output[position] = token(keyed_draw(seed, position, 0))
            roles[position] = 'fresh target'
    return output, roles
```

With seed 11 at position 7, a fresh target draw produces token 1; the accepted draft produces token 0. Under ideal uniform draws each rule still has exactly the distribution $(1/2, 1/2)$. Equality of marginal distributions does not require equality for a particular seed. The SHA-256 example supplies a concrete repeatability counterexample; the marginal statement follows from the two sampling rules, not a claim that a finite list of hash outputs proves uniformity.

This gives a source-based explanation for the README recipe: enable `DEEPINFER_BATCH_INVARIANT`, disable `DEEPINFER_ENABLE_MTP_VARLEN`, and provide a fixed `SamplingParams.seq_seed` on NVIDIA. Measured timings can change the budget; the budget can change the sampling operation and its keys. This does not mean every timing perturbation changes a token.

No end-to-end test of that complete recipe was found in the inspected `tests/models/` inventory: none sets `seq_seed`. Embedded sampler and RNG kernel tests are different checks. The evaluation driver supplies per-sample seeds but initially sets varlen to 1; its later `-env` loop can override that value. Its default setup is therefore not the README's recipe. Seeds alone, or the helper's repeatability docstring, do not establish that those evaluation runs are repeatable [79].

10.4.3 Test the probabilities, not just the seed

`test_sampler.py` asks whether the sampled tokens agree statistically with their reported probabilities. It uses MTP 5 and four settings $(T, \text{top-p}) = (0, 1), (1, 0.95), (8, 0.95), (1, 1)$, with 128 trajectories per setting and up to 2,048 output tokens per trajectory [80]. It does not force a golden continuation.

A categorical sample is not a uniform number. A *randomised probability integral transform* (PIT) turns it into one under the target distribution. Order the tokens, give token x an interval of width p_x, and let $F(x)$ be the interval's upper end. Given the sampled token, draw an independent uniform U and form

$$Z = F(x) - Up_x.$$

If token x is chosen with probability p_x, that probability is spread uniformly over an interval of the same width. The intervals together fill $[0, 1]$ uniformly. The construction works conditionally on each token's history; the probabilities can change between positions.

DeepInfer calls the upper endpoint `sum_smaller_probs`. It sorts by ascending probability, breaking ties by column index, and *includes* the sampled token. Its reference calculation makes the tie rule explicit:

Listing 10.13: Build an inclusive CDF in probability order.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/sum_smaller_probs.py` · lines 38–44

```
pivot = logprobs.gather(1, sample_results.unsqueeze(1))
mask_less = logprobs < pivot
col_idx = torch.arange(logprobs.shape[1], device=logprobs.device).unsqueeze(0)
mask_equal_before = (logprobs == pivot) & (col_idx <= sample_results.unsqueeze
    ↪ (1))
out = (logprobs.exp() * (mask_less | mask_equal_before)).sum(dim=1)
out.masked_fill_(invalid, 0.0)
return out
```

Lab 22 uses target probabilities $(1/2, 1/4, 1/4)$. The token order is 1, 2, 0, with intervals $[0, 1/4]$, $[1/4, 1/2]$, and $[1/2, 1]$. A biased sampler instead chooses token IDs 0, 1 and 2 with probabilities $(3/4, 1/8, 1/8)$. Exact interval calculations give:

```
PIT quarter masses, correct: 1/4, 1/4, 1/4, 1/4
PIT quarter masses, biased:  1/8, 1/8, 3/8, 3/8
```

The correct sampler places a quarter of its PIT mass in each quarter of the unit interval. The biased sampler places too much in the upper half. These are exact rational mass calculations, not a histogram or a run of DeepInfer's statistical test.

The released test filters the arrays before constructing PIT values and calling a Kolmogorov–Smirnov (KS) check:

Listing 10.14: Transform a trajectory before checking uniformity.
DeepInfer · `tests/models/test_sampler.py` · lines 83–94

```
ssp_arr = np.array(ssp, dtype=np.float64)
p_arr = np.exp(np.array(log_probs, dtype=np.float64))
if not (np.all(np.isfinite(ssp_arr)) and np.all(np.isfinite(p_arr)) and np.all(
    ↪ p_arr > 0)):
    return None
if (ssp_arr - p_arr < -0.01).sum() > 0:
    return None
rng = np.random.RandomState(seed)
u = np.clip(ssp_arr - p_arr * rng.uniform(0, 1, size=n), 0, 1)
ks_stat, ks_quantile = ks_test_uniform(u.tolist())
if ks_stat is None:
    return None
return ks_stat, ks_quantile, n
```

KS measures the largest gap between the sample's cumulative distribution and the uniform CDF. The helper turns that gap into an approximate p-value using an asymptotic formula with a finite-sample correction. A p-value describes how unusual such a discrepancy would be under the null model; it is not the probability that the sampler is correct. The helper returns a p-value under the name `ks_quantile`. A second KS check compares those trajectory-level values with a uniform distribution.

The actual failure gate is short:

Listing 10.15: Only the all-trajectory aggregate decides statistical failure.
DeepInfer · tests/models/test_sampler.py · lines 242–247

```
242 if len(all_quantiles) == 0:
243     errors.append(f"DP {dp_rank} {config_name}: no valid trajectories for all,
    ↪ skip={skip_counts}")
244 elif all_pv is None:
245     errors.append(f"DP {dp_rank} {config_name}: computing the all p_value failed
    ↪ ")
246 elif all_pv <= 1e-5:
247     errors.append(f"DP {dp_rank} {config_name}: all p_value={all_pv} is too
    ↪ small; the sampler may have a problem")
```

`all_pv` uses the whole trajectories. The test also prints `next1_pv`, based on tokens after steps marked by `draft_tok`, but does not reject on that value. The filtering is consequential: if *any* position has $\texttt{ssp} - p < -0.01$, the helper returns `None` and that whole trajectory is omitted, not failed. An inclusive CDF should satisfy $\texttt{ssp} \geq p$; the tolerance filter can therefore discard evidence of the very probability inconsistency being investigated. The helper also omits missing or non-finite data and non-positive probabilities, then clips the surviving PIT values to $[0, 1]$. If fewer than two valid trajectory-level values remain, the aggregate is unavailable and the test fails. Otherwise, the aggregate may pass despite discarded trajectories.

Passing means this finite test did not detect a discrepancy at its threshold. It cannot prove exact distributional equality or exclude chapter 8's small, deliberate residual penalty. There is another uncovered claim: PIT compares samples with the distribution the engine reports after its transformations. If temperature or top-p is applied wrongly, both sampling and the reported probabilities could use the same wrong transformed logits and still pass. An independent check of the transformations would be needed to detect that error. This test also does not enable structured output, which adds another transformation of the distribution.

10.5 A grammar narrows the distribution

A grammar allows only certain continuations of the text already accepted. DeepInfer represents the allowed token IDs as a bitmask. Its sampler applies that mask to the logits before sampling and draft verification:

Listing 10.16: Apply each grammar mask to its request and speculative position.
DeepInfer · deepinfer/model_executor/layers/sampler.py · lines 428–437

```
428 ftp = decode_inputs.mtp_first_token_positions
429 seqlen_q = decode_inputs.mtp_seqlens_q
430 row_to_req, local_step, is_verify = prepare_verification_metadata(ftp, seqlen_q,
    ↪  total_q, self.max_MTP_value)
431 speculative_inputs = driver_inputs.speculative
432
433 # Apply structured-output masks before sampling and draft verification.
434 if so_indices is not None:
435     for step, indices in enumerate(so_indices):
436         if len(indices):
437             apply_token_bitmask_by_step_inplace(logits, so_bitmasks[step],
    ↪ indices, ftp, step)
```

For an allowed set S, simple masking and renormalisation would turn a distribution p into $p(x)/\sum_{y\in S} p(y)$ for $x \in S$, and zero elsewhere. That describes a probability contract only after defining the order of masking, temperature, top-k/top-p and speculative correction. A format test alone cannot establish this distribution. DeepInfer places the grammar mask before `process_sampling`, whose transformations and verification we traced in chapter 8.

10.5.1 The mask belongs to a tentative history

The grammar compiler and matcher are provided by the external XGrammar library, pinned as `xgrammar==0.2.2`. DeepInfer's wrapper in `deepinfer/structure_output/matcher.py` owns trigger progress, bound-token state, EOS bookkeeping and rollback. Unlike the independent DeepSeek kernel repositories, XGrammar is an external project supplying grammar machinery. DeepInfer supplies the integration with speculative requests [81].

On the host, `Sampler._get_so_mask` first advances the matcher with the previous step's accepted tokens and final output. Status 1 marks initial setup; status 2 enables subsequent updates:

Listing 10.17: Commit actual output before constructing the next masks.
DeepInfer · `deepinfer/model_executor/layers/sampler.py` · lines 351–359

```
351 try:
352     if so_status[idx] == 2:
353         for token in seq_metadata.last_accepted_token + [seq_metadata.
    ↪ last_output_token]:
354             if not matcher.accept_token(token):
355                 raise RuntimeError(f"matcher rejected step token {token},
    ↪ upstream invariant broken")
356     elif so_status[idx] == 1:
357         so_status[idx] = 2
358     else:
359         raise RuntimeError(f"unexpected so_status {so_status[idx]}")
```

Now suppose a teaching grammar permits `ab!`, `ad!`, `cb!` and `cd!`, with one character per token. Before any output, the mask allows $\{a, c\}$. Proposed drafts $[a, b]$ require two later masks: after a, allow $\{b, d\}$; after ab, allow $\{!\}$. Those masks assume a prefix that has not yet passed target verification (Figure 10.2).

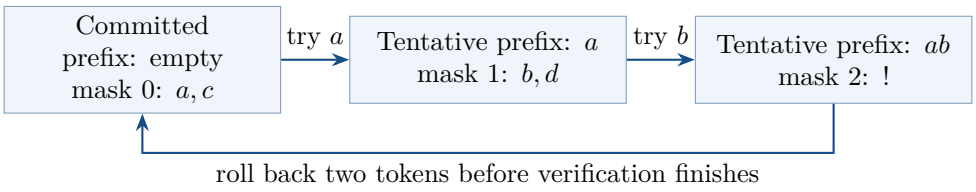


Figure 10.2: A teaching grammar with character tokens. Saved masks describe tentative prefixes; the persistent matcher returns to committed state.

DeepInfer fills the mask for position 0, then tentatively advances through the drafts. This is the start of that loop:

Listing 10.18: Construct later masks by accepting a speculative prefix.
DeepInfer · deepinfer/model_executor/layers/sampler.py · lines 366–370

```
366 num_accepted_steps = 0
367 for k, draft_token in enumerate(seq_metadata.last_next_n_token):
368     if matcher.accept_token(draft_token):
369         if matcher.constrained():
370             mask_filled = matcher.fill_vocab_mask(self.bitmasks_cpu[k + 1], idx)
```

The omitted lines 371–372 assert that filling succeeded and register the request's mask index. The loop then counts successful tentative accepts and restores the matcher:

Listing 10.19: Undo tentative advances, including when a later draft fails.
DeepInfer · deepinfer/model_executor/layers/sampler.py · lines 373–376

```
373         num_accepted_steps += 1
374     else:
375         break
376 matcher.rollback(num_accepted_steps)
```

If verification rejects a and selects c, the next host update commits c, not ab. If it accepts a but rejects b, it commits a and the replacement token. The saved mask for a later row is usable only along the speculative prefix for which it was built; accepted-prefix logic prevents later candidates from becoming output after an earlier rejection.

The wrapper's rollback capacity is `nextn+2`. Its `rollback` method restores the inner matcher's state and the wrapper's trigger/EOS/bound bookkeeping. The extra two positions are an implementation allowance; the constant alone does not establish why precisely two are required. What matters for this path is that tentative advancement and restoration stay paired.

10.5.2 Failure ends the request

A compilation future can time out or fail. Its wrapper returns an aborted matcher. When the engine enqueues a request with such a matcher, it marks `FINISHED_SO_FAILED` in `deepinfer/engine/llm_engine.py`, lines 621–622. During sampling, a missing or aborted matcher sets `so_status` to -1; runtime exceptions also mark it aborted and set that status. The scheduler closes the path by checking the stored status:

Listing 10.20: A recorded grammar failure becomes a terminal sequence status.
DeepInfer · csrc/engine/scheduler.hpp · lines 976–980

```
976 if (seq.structured_output.status == -1) {
977     seq.schedule.status = SequenceStatus::FINISHED_SO_FAILED;
978     seq.append_token_id(0);
979     seq.append_compressed_token_id(0);
980     token_cnt++;
```

The two appended zero IDs are the scheduler's failure bookkeeping. The branch marks the sequence failed instead of continuing unrestricted generation. The preceding comment explicitly says to consult the stored status, not only the report for this step: a step with no active structured-output work may report no status (*author's translation*). A missing report must not erase an earlier failure.

This is also the boundary of chapter 5's preemption discussion. The scheduler sets `so_blocks_recompute` whenever structured-output status is nonzero. Such a request terminates on preemption even if ordinary requests would be recomputed. Restoring KV would not, by itself, restore the matcher and its trigger history. Reconstructing that state would be a requirement for a new recovery implementation, not a feature present in this release. The speculative rollback above is implemented; preemption recovery of the matcher is not.

10.5.3 Where the constraint begins, and what the tests cover

A trigger can leave an initial part of the output unconstrained. Once its token sequence matches, the grammar takes over. A bound token has a more specific meaning: if it fires while the trigger is pending, the wrapper forces the remaining trigger tokens and then enters the grammar. It is not an ending delimiter that releases the constraint. These rules live in `accept_token` and `fill_vocab_mask`, with their state included in rollback.

The released structured-output example supplies three prompts asking for incompatible formats, then requests a regex, a JSON schema and an EBNF grammar. All use temperature zero. It requires nonempty output, a normal `stop` finish reason and a final EOS token before decoding the text.

The format checks are explicit:

Listing 10.21: Check the returned text against the requested format.
DeepInfer · `examples/_structured_output.py` · lines 54–63

```
54 if schema_type == "regex":
55     assert re.fullmatch(so_args["schema"], text), f"regex: {text!r}"
56 elif schema_type == "json":
57     value = json.loads(text)
58     assert isinstance(value, dict) and set(value) == {"color", "count"}, f"json:
   ↪  {text!r}"
59     assert value["color"] in {"red", "green", "blue"}, f"json: {text!r}"
60     assert type(value["count"]) is int and value["count"] in {1, 2, 3}, f"json:
   ↪ {text!r}"
61 else:
62     assert schema_type == "ebnf"
63     assert re.fullmatch(r"answer=(yes|no);level=[1-3]", text), f"ebnf: {text!r}"
```

The JSON case checks keys, allowed values and a strict integer type. The EBNF case checks the resulting text with a full-match regex. The offline-inference test explicitly invokes these examples. They establish these syntax and termination checks, not stochastic masked-distribution correctness, factual accuracy, or coverage of every trigger, timeout and rollback path. The sampling KS test does not use structured output, so its result cannot fill that gap.

10.6 Experiments to try next

1. Change lab 21 to keep two reduction lanes for every batch size. Does batch equality return? Does the result have to equal the one-lane result? Explain why numerical consistency and an exact-arithmetic answer differ.

2. Replace the zero-metric check with an assertion that matches an exact value-equality contract. Then propose a separate check for a stated absolute tolerance. What should either check do with non-finite values?

3. Construct two different three-token logprob lists with equal perplexity. Which test in the inventory could distinguish them directly?

4. For one request with four candidate rows, mark the verified rows and fresh row. Reduce its budget to two. Identify which operation changes at the second row and which parts of the RNG key are relevant.

5. In the grammar diagram, accept a, reject b, and choose d. Trace the matcher's tentative state, rollback state and next committed state. What additional state would a preemption/recompute implementation need?

6. Complete this template for a released test and for a proposed end-to-end deterministic-sampling test. Include the model and hardware configuration; do not treat a recipe as an executed test.

Field	Question to answer
Change	Which execution choice varies between the runs?
Fixed inputs	Tokens, history, seed, grammar, model, dtype, hardware and flags?
Observable	Token IDs, logprobs, embeddings, an aggregate, distribution or syntax?
Preprocessing	Are values filtered, cast, forced or discarded before comparison?
Assertion	Direct equality, a named metric and tolerance, or a statistical rule?
Uncovered claim	What plausible failure could still pass?

You can now follow a DeepInfer correctness claim from its configuration through the relevant library or state machine to the assertion that checks it, distinguishing fixed history from sampled output, stable random keys from a stable sampling path, and grammar validity from the probability or quality of the answer. Those boundaries tell you which test to run when changing the engine—and when a new test is needed.

Chapter 11

From an engine to a service

Chapter 10 examined correctness contracts within an engine. A service also passes requests and state between engines and storage. Those boundaries decide where work can run and what it must wait for.

The question

Four requests share a long prefix. Two prefill workers are available, but only one holds the prefix. Should all four requests wait for that worker? If both workers could reuse it, which queue would become shorter?

Run the service experiment:

```
python3 labs/23_service.py
```

All four requests arrive at time zero. Each prompt has 1,024 tokens, of which 768 can be reused. Matching takes one time unit; prefill processes 128 new tokens per unit. Two decode workers each need four units per request. A handoff adds one unit, with no limit on concurrent transfers. These are assigned costs. The workers serve one request at a time, without batching or eviction. Prefix fetching adds no delay in either case; only the P/D handoff has an assigned transfer cost.

The first case sends every request to the worker holding the local prefix. The second lets either worker fetch that same prefix, at the same assigned cost:

```
Assigned time units; four requests, two P and two D workers
local affinity: P ends [3, 5, 7, 9]; D ends [8, 10, 12, 14]
shared cache: P ends [3, 3, 5, 5]; D ends [8, 8, 12, 12]
shared, no hits: P ends [9, 9, 17, 17]; D ends [14, 14, 22, 22]
shared, slower handoff: P ends [3, 3, 5, 5]; D ends [10, 10, 14, 14]
affinity, slower handoff: P ends [3, 5, 7, 9]; D ends [10, 12, 14, 16]
```

With local affinity, prefill finishes at 3, 5, 7 and 9. With a shared cache it finishes at 3, 3, 5 and 5. The last answer arrives two units earlier, not four: decode now makes the last pair wait. With equal handoff delays, sharing saves two units in both comparisons: 14 versus 12 with a one-unit handoff, and 16 versus 14 with a three-unit handoff. A shared cache with the slower handoff finishes no earlier than local affinity with the faster one. That comparison changes two variables: the extra transfer cost offsets the saving from sharing.

The no-hit case supplies a baseline for reuse: both prefill workers remain available, but processing all 1,024 positions per request moves the last completion from 12 to 22, with the handoff delay held at one unit.

Scope and order. We follow the deployment Q&A from prefix matching to routing, state movement and token output. Released DeepInfer code fixes the engine roles and the state layout at those boundaries. The Q&A describes the service that connects them. We then audit a published performance curve, using chapter 2's accounting and chapter 8's acceptance model. Our experiments calculate queues and rates; they do not run a serving cluster.

11.1 A request crosses several owners

The engine of chapter 1 was responsible for advancing requests. A service must first decide which engine should own each request and how its reusable state will reach that engine. Figure 11.1 separates the request's route from the storage it can use. A released component does not imply that its production orchestration is also released.

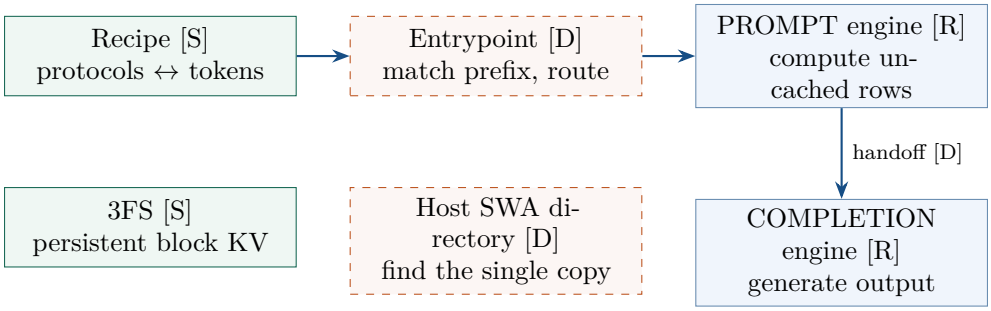


Figure 11.1: The documented text-request path. [R]: code released in DeepInfer; [S]: separately released project; [D]: service logic described in the Q&A. The two storage boxes show the state fetched by the PE through the described transfer driver; matching itself consults local metadata. Output tokens return through Recipe's decoding path.

DeepInfer names the roles directly:

Listing 11.1: Three engine roles.
DeepInfer · deepinfer/engine/engine_type.py · lines 11–13

```
11 NORMAL = 0  # process prompts & completions
12 PROMPT = enum.auto()  # only process prompts
13 COMPLETION = enum.auto()  # only process completions
```

A prefill engine, abbreviated PE in the Q&A, uses the PROMPT role. A decode or completion engine, CE, uses COMPLETION. NORMAL combines the two. After scheduling, the engine checks that a nonempty PROMPT step is actually a prompt step:

Listing 11.2: The prompt engine must not decode.
DeepInfer · deepinfer/engine/llm_engine.py · lines 947–949

```
947 if not scheduler_outputs.is_empty() and self.engine_type == EngineType.PROMPT:
948     assert scheduler_outputs.prompt_run, "Prompt engines should not perform
    ↪ decoding"
949     assert seq_metadata_list[0].seqlen_q > 0, "seqlen_q should be > 0"
```

These are executable role boundaries. They do not locate the machine-selection policy, the network transfer, or the service's retry logic. The distinction also matters when reading benchmarks: `examples/benchmark.py` constructs its LLM through `create_llm` without overriding the engine role. It therefore uses `EngineArgs.engine_type = EngineType.NORMAL`,

the default at `deepinfer/engine/arg_utils.py`:41. The examples README calls the separate prefill and decode measurements "Stage decoupling". Those windows are inside one combined engine deployment; they are not a disaggregated P/D service [17, 15, 83].

Released in DeepInfer	Separately released	Described service logic
Engine roles, scheduling, state-layout helpers, EPLB internals, benchmarks	DeepSeek Recipe for protocol/-token conversion; 3FS for distributed storage	Entrypoint matching, kvcache-store, GC-worker policy, stage load balancers, autoscaling and P/D transfer orchestration

Table 11.1: Three access boundaries. Public projects in the middle column do not supply a pinned version of the production deployment.

The Q&A identifies one property shared with the DeepSeek-V3/R1 inference system: attention uses data parallelism, with equal DP and EP sizes. It also points to DualPath's intra-group scheduling for the decode balancer, but gives no link or bibliographic reference for DualPath. These establish a stated continuity in that parallelism choice and an acknowledged scheduling precedent. The separate 3FS repository is a dependency boundary. None establishes a line-by-line derivation of the service code, which is absent here [2, 86].

Image requests add an encoder before prefill: the Q&A describes E/P/D separation. Its vision-encoder balancer first chooses a machine randomly, then chooses the GPU with the fewest assigned patches, counting width and height in patch units. Recipe handles the API-to-model representation. This adds a workload and a queue to the map, without changing why prefill and decode need different capacity signals [2].

11.2 Hits decide prefill demand

A prefix hit is useful before any bytes move. The Entrypoint matches a hash chain against metadata in the local kvcache-store's CHECK role. A hash chain identifies a block together with its preceding prefix, so reuse concerns a matching history, not just an equal block of tokens. The resulting hit length determines how much prompt work remains. Routing needs that answer; the Q&A explicitly says matching cannot be hidden [2].

For prompt length L and computation resuming at H, let $q = L - H$. Chapter 5's documented quota model estimates work from the new positions q and their existing context c. The service can then choose a prefill tier, an engine with quota left, and a rank within it:

$$\text{match} \longrightarrow H \longrightarrow (q, c) \longrightarrow \text{cost}(q, c) \longrightarrow \text{routing and capacity.}$$

The quota fit and its service driver are described in the Q&A, not implemented in the released scheduler. A cache hit also leaves context to read; reducing new positions does not remove all attention work.

The storage placement changes a routing constraint. With block KV in a separate shared cluster, every prefill engine can hit any prefix. It need not wait for the compute machine that produced that prefix. This is the Q&A's reason for having no prefix-affinity routing.

Our lab holds the prefix and fetch costs fixed and changes only the eligible workers:

Listing 11.3: Choose from the local owner or both workers.
Book lab · labs/23_service.py · lines 19–25

```
    for request in range(4):
        candidates = range(2) if shared else [0]
        rank = min(candidates, key=lambda i: (prefill_free[i], i))
        start = max(1, prefill_free[rank])  # matching completes before routing
        end = start + (1024 - hit) // 128
        prefill_free[rank] = end
        prefilled.append((end + transfer, request, rank, start, end))
```

A local-cache service could instead route to an idle worker and recompute the prefix. That is another policy, not tested by the affinity case. Real shared storage pays for fetching and can itself queue. The experiment isolates the benefit of removing ownership affinity; it does not predict that shared storage always wins.

11.2.1 From work to a capacity estimate

Let λ be requests per second, L their fixed input length, O their fixed output length, and h the fraction of input tokens reused. Suppose v_P and v_D are sustained per-GPU rates for this workload. At a target utilisation u, a simple capacity estimate is

$$G_P = \left\lceil \frac{\lambda L(1-h)}{u v_P} \right\rceil, \qquad G_D = \left\lceil \frac{\lambda O}{u v_D} \right\rceil.$$

This linear teaching model suppresses context-dependent cost, batch shape and EP group granularity. Its units are GPU equivalents, not deployable group sizes. Lab 23 assigns $\lambda = 4$, $L = 1024$, $O = 64$, $v_P = 2048$, $v_D = 128$ and $u = 0.8$:

```
Capacity model (GPU equivalents, not deployment sizes):
reused input fraction 0%: P=3, D=3
reused input fraction 75%: P=1, D=3
```

The arithmetic moves prefill from three GPU equivalents to one. Output work stays at 256 tokens/s, so the decode estimate stays at three. That conclusion holds the arrival rate, output lengths and decode rate fixed. Faster prefill can still change when requests reach decode, as the opening experiment showed. It can also change achievable batching and latency. Production sizing cannot replace those effects with a constant v_D without checking the workload.

The meaning of h matters. One hundred requests might all hit ten tokens of a thousand-token prompt: the request hit rate is 100%, but only 1% of input tokens are reused. The Q&A sizes prefill from non-cached tokens and decode from running requests. It reports that shortening chatbot retention from seven days to about three costs 0.5–1% of KV-cache hit rate. It does not specify here whether that is a relative loss or a loss in percentage points, or give the corresponding uncached-token demand. Neither should be silently substituted into our formula [2].

Matching is on the routing path; fetching has a different dependency. The Q&A describes a separate process fetching block KV from 3FS into host shared memory, followed by an engine thread swapping it into device memory. Per-layer host-to-device copies can overlap forward computation once the needed state is ready. It describes using an RDMA network interface for these transfers through PCIe, avoiding GPU compute cores. This is a documented transfer design, not a path implemented by our queue simulator [2].

11.3 State lives in three places

Chapter 4 separated block KV from per-request state. At service scale these objects also have different homes and failure modes (Table 11.2).

Home	Contents and purpose	What reuse depends on
Device memory	In-flight block KV and live request state	Active engine ownership and sufficient capacity
Host memory	One copy of saved SWA state	A live host copy and a directory entry; remote copies can be fetched over the 400G VLAN
3FS storage	Persistent block KV for prefix reuse	A surviving coded object that has not been reclaimed

Table 11.2: Storage roles reported in the deployment Q&A. Saved SWA is never stored in 3FS; it is not an additional persistent replica of block KV.

The Entrypoint maintains an in-memory directory for the single host copy of SWA. A cross-machine hit pulls that state over the network. Host restart or eviction can therefore remove SWA while the block KV still exists. This is a normal trigger for chapter 9's encoder replay, distinct from decoder replay on every truncated prefill. The Q&A also says decode KV is written to storage for later reuse, but active decode does not use KV offloading [2].

11.3.1 The released handoff layout

A completion engine needs enough state to continue immediately. A future prefix hit needs only the stored region; chapter 9 explained why consumer SWA can be dropped from that cache entry. The released helper exposes this distinction through `TO_CE`. In the branch without extra prefill state, selection is a contiguous slice:

Listing 11.4: Choose a live-handoff size or a stored-prefix size.
DeepInfer · `deepinfer/worker/cache_engine.py` · lines 790–797

```
790 state_size = cls.total_bytes_inference if TO_CE else cls.state_size_to_disk
791 if state_size == 0:
792     return None
793 src = state_cache[state_id, :state_size]
794 if state_tensor is None:
795     return src
796 state_tensor.copy_(src)
797 return state_tensor
```

The live size, `total_bytes_inference`, includes the dropped region. The stored size, `state_size_to_disk`, names the prefix of that layout. Despite that identifier, the Q&A says this saved SWA goes to host memory, not to 3FS. The returned slice can alias the cache when no destination is supplied. A transport driver would have to preserve its source until the transfer finishes, using the ownership reasoning from chapters 4 and 6.

When extra prefill state is enabled, the rings need rearranging. Selection allocates the appropriate destination and passes the handoff flag through to the view builder:

Listing 11.5: Select ring views for the chosen destination.
DeepInfer · deepinfer/worker/cache_engine.py · lines 799–807

```
799 if state_tensor is None:
800     state_size = cls.total_bytes_inference if TO_CE else cls.state_size_to_disk
801     state_tensor = torch.empty(state_size, dtype=state_cache.dtype, device=
        ↪ state_cache.device)
802
803 cache_views, tensor_views = cls._carve_state_views(state_id, state_cache,
        ↪ state_tensor, TO_CE=TO_CE)
804 for tensor_view, cache_view in zip(tensor_views, cache_views, strict=True):
805     if tensor_view.numel() == 0:
806         continue
807     select_state_slice(tensor_view, cache_view, last_hit_position)
```

Inside the following `if TO_CE` branch, consumer SWA has already been rearranged. The remaining compression state is copied separately. The comment at line 812 says: "Consumer SWA has been rearranged by ring; copy compression state directly" (author's translation):

Listing 11.6: Include the remaining dropped state for a CE.
DeepInfer · deepinfer/worker/cache_engine.py · lines 813–816

```
813 dropped_compress_bytes = cls.dropped_bytes_inference - cls.
        ↪ swa_drop_total_bytes_inference_pad
814 state_tensor[dropped_tensor_offset : dropped_tensor_offset +
        ↪ dropped_compress_bytes].copy_(
815     state_cache[state_id, dropped_cache_offset : dropped_cache_offset +
        ↪ dropped_compress_bytes]
816 )
```

Loading a stored prefix into a prefill slot is a different operation. Its first check rejects `off` truncation when the model has dropped SWA layers:

Listing 11.7: Restore stored state, with a model-dependent guard.
DeepInfer · deepinfer/worker/cache_engine.py · lines 836–848

```
836 if cls.swa_drop_num_layer > 0:
837     assert envs.DEEPINFER_YOCO_PREFILL_TRUNCATION_MODE != "off"
838 if not envs.DEEPINFER_PREFILL_STORE_EXTRA_STATE:
839     if cls.state_size_to_disk == 0:
840         return
841     state_cache[state_id, : cls.state_size_to_disk] = state_tensor
842     return
843
844 cache_views, tensor_views = cls._carve_state_views(state_id, state_cache,
        ↪ state_tensor)
845 for tensor_view, cache_view in zip(tensor_views, cache_views, strict=True):
846     if tensor_view.numel() == 0:
847         continue
848     load_state_to_cache(tensor_view, cache_view, last_hit_position)
```

The helper either copies the stored slice or reconstructs its ring views. It does not fetch bytes from another host, select a destination engine, or rebuild the dropped consumer

state. In this release, calls to these two helpers outside `cache_engine.py` occur in `tests/worker/test_state_cache_ops.py`, not in a service driver [84].

That test also gives us a narrow, useful comparator. At lines 196–211 it selects state, changes the bytes, loads them back and selects again. The final comparison is:

Listing 11.8: The round-trip test compares the stored SWA payload.
DeepInfer · `tests/worker/test_state_cache_ops.py` · lines 213–215

```
213 reselected_swa = _stored_swa(reselected_state)
214 modified_swa = _stored_swa(modified_state)
215 assert torch.equal(reselected_swa, modified_swa)
```

`_stored_swa` excludes per-layer alignment padding before `torch.equal`. This checks a layout round trip for the tested configuration, not a network transfer or a full-model continuation. We inspected this CUDA test; the book's standard-library labs do not execute it.

11.3.2 Retention is not one number

A retention target, a capacity calculation and an eviction rule answer different questions. The Q&A gives all three, listed in Table 11.3. There TTL means time-to-live and LRU means least recently used.

Reported quantity	Meaning in the deployment description
More than 72 hours	System goal for retaining reusable KV
24 hours per server	Capacity-sizing recommendation alongside 20 SSDs of 30.73 TB each; server count scales with deployment
About 100 PiB SSD	Storage capacity of the described cluster
3 days / 7 days	TTL for agent / chatbot traffic
LRU and watermark	Per-storage-node GC workers reclaim least recently used KV to keep disk occupancy below a threshold

Table 11.3: Numbers and policies from the Q&A, not interchangeable guarantees.

The Q&A places expiry and reclamation in 3FS's GC, with no expiry logic in kvcache-store itself. It does not give the complete precedence rules between age and occupancy pressure. A server sized for a day's data does not by itself promise that every object will survive for that day, or that the whole cluster retains only a day [2].

Durability and read latency are separate again. The Q&A specifies 10+2 erasure coding, tolerating two simultaneous node failures, and says a read contacts 11 storage nodes and returns after any ten replies. Coding redundancy and read fan-out are different counts. The description does not explain why that read uses eleven nodes for twelve coded fragments; we do not infer a placement rule from those numbers. Nor does storage redundancy replicate the single host copy of SWA [2].

11.4 A hit has two lengths

Suppose a prompt has L tokens and block KV is available for a prefix of C tokens. The Q&A caps a candidate hit at $L - 134$, then queries an aligned block-KV hit L_1. Lab 24 makes the zero case explicit:

$$L_1 = 128 \left\lfloor \frac{\min(\max(0, L - 134), C)}{128} \right\rfloor,$$

$$H = \begin{cases} L_1, & \text{qualifying SWA exists,} \\ \max(0, L_1 - 128), & \text{otherwise.} \end{cases}$$

Here L_1 is the reusable block-KV length; H is the length skipped by computation. The lab assumes a contiguous available prefix. An actual lookup must discover that prefix and whether the matching SWA state is present.

```
python3 labs/24_cache_hits.py
```

```
Assigned full block-prefix availability; columns count tokens
L     block hit L1   compute hit: SWA present / missing
128   0                 0 / 0
261   0                 0 / 0
262   128             128 / 0
390   256             256 / 128
902   768             768 / 640
1024  768             768 / 640
Checks: 69666 cases; bounds, alignment, backoff, monotonicity
```

For $L = 902$, the cap is exactly 768. With SWA present, prefill starts at 768 and processes 134 new positions. Without it, prefill starts at 640. The interval $[640, 768)$ reconstructs encoder SWA; its block KV is already available and need not be rewritten. The remaining 134 positions are new. Calling both lengths simply "the hit" hides this distinction between reused bytes and avoided computation [2].

The entire translated policy fits in six lines:

Listing 11.9: Compute block reuse and the computation restart.
Book lab · `labs/24_cache_hits.py` · lines 14–19

```
14 def hit_lengths(length, cached, has_swa):
15     assert length >= 0 and cached >= 0
16     cap = max(0, length - RESERVE)
17     block_hit = min(cap, cached) // BLOCK * BLOCK
18     compute_hit = block_hit if has_swa else max(0, block_hit - BLOCK)
19     return block_hit, compute_hit
```

The checks enumerate aligned candidates independently and compare the result with the formula. They also verify nonnegative lengths, the cap, alignment, and a one-block backoff only when there is a positive hit and SWA is absent. Prompts shorter than $134 + 128 = 262$ cannot have a positive block hit under this policy. Monotonicity holds in the lab with cache extent and SWA availability fixed. It need not hold across changing directory entries in a live service.

The same $L = 902$ case exposes chapter 9's integration gap. A 140-row retained suffix starts at 762, six positions before a hit at 768. Without an additional validity restriction,

the released visibility formula can read those six consumer ring entries, but the stored-state restore does not fill them. Backing off after an SWA miss gives more rows; it does not resolve the SWA-present case. The numbers 128 (window), 134 (default ring and documented hit reserve) and 140 (suggested keep count) retain different meanings. The released helper's truncation guard narrows the allowed configurations, but does not disclose how the production driver reconciles the six entries. This experiment tests the Q&A's policy arithmetic, not that missing integration [84, 2].

11.5 Balancing, scaling and failure

The Q&A describes four load-balancing concerns: prefill, decode, vision encoding and expert parallelism. Chapter 7 already traced the released EPLB statistics, policy and weight movement. At the service boundary, the remaining question is which engines should receive work and how many should exist.

Prefill is divided by total context length into four tiers: below 16K, 16–64K, 64–256K and at least 256K. Each tier has a compute-quota cap. Engines pull requests according to remaining quota; within a DP group, new requests go to the rank with the least current quota load. The previous round's share of quota demand controls how many engines each tier receives. These policies use the hit-derived work estimate, rather than sending every request with a given prefix to its old owner [2].

Decode has two load measures. A rank with few running requests might already hold a large total KV length. The documented policy sends short requests towards low concurrency even if that rank has much KV, and long requests towards small total KV length. This is where the Q&A acknowledges DualPath's intra-group scheduling. Counting requests alone would miss the context-reading cost from chapters 2 and 3 [2].

Autoscaling follows the same division: prefill demand is non-cached tokens; decode demand is running requests. The Q&A adds rate limits, minimum and maximum sizes, spare quota and a rule forbidding scale-in above 0.8 quota utilisation. The capacity formula in lab 23 explains the direction of change, not those production thresholds or their control loop.

Scale-in and failure also have different contracts. On a planned shutdown a CE gets a one-minute grace period, then forwards unfinished requests and their KV in one transfer to other CEs. A crashed engine cannot promise to perform that handoff. The Q&A instead describes quota redundancy and request rerouting; restarts take minutes. A failed GPU invalidates the whole instance's EP communication domain. On restart, the platform forms an EP size from the healthy GPUs it can allocate. This describes recovery of service capacity, not evidence of uninterrupted, exactly identical output for an affected request [2].

11.6 Check a published curve

Once stages have separate capacities, we need to choose their operating points. A decode configuration may serve each user more slowly while producing more tokens per GPU overall. The useful comparison is a *Pareto frontier*: the points for which no other tested point is at least as good on both axes and strictly better on one.

The fixed release has three performance figures under `doc/images/`: `decode_pareto.png`, `prefill_pareto.png` and `wide_ep_decode_pareto.png`. It supplies a sweep script and plotting code, but no raw `results.jsonl`. The sweep creates that file when run. We can inspect the conversion and read an approximate point from the image; we cannot reanalyse absent raw measurements [85].

11.6.1 Build a frontier from known data

Lab 25 uses explicitly assigned request counts and step rates. It includes a poor operating point at 48 requests so that connecting all points differs from selecting a frontier:

```
python3 labs/25_frontier.py
```

```
Synthetic records: requests, steps/s, user tokens/s, tokens/s/GPU
 8 100 325.00 2600.00
16  90 292.50 4680.00
32  60 195.00 6240.00
48  35 113.75 5460.00
64  40 130.00 8320.00
Non-dominated request counts: [8, 16, 32, 64]
```

Both axes prefer larger values. The 48-request point gives 113.75 tokens/s/user and 5,460 tokens/s/GPU. The 32-request point improves both to 195 and 6,240. The lab removes the former from the frontier, as Figure 11.2 shows. Its assigned step rate also rises between 48 and 64 requests; these records are a deliberate counterexample, not a fitted hardware curve.

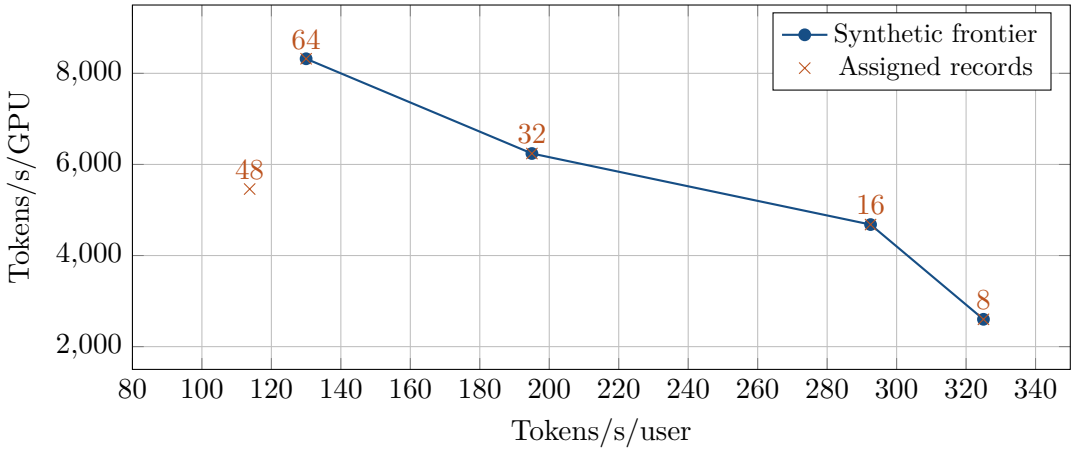


Figure 11.2: A frontier calculated by lab 25. Labels are requests per DP rank; CP is one. These are assigned rates, not the published benchmark points.

The released plotting script does something simpler: `load_points` retains the latest record at each scanned request count and checks a set of fixed configuration fields. Then line 150 plots all selected points in scan order. It does not calculate non-dominance or combine repeated runs statistically. A curve called "Pareto" still needs its points and assumptions examined [85].

11.6.2 The two meanings of acceptance

Chapter 2 established how to turn step rate into useful output. The decode plot makes that conversion explicit:

Listing 11.10: Both plotted rates use the supplied tokens-per-step factor.
DeepInfer · `examples/sweep/plot_pareto.py` · lines 66–68

```
66  xs = [p["decode_avg_sps"] * tau for p in points]
67  ys = [p["decode_avg_sps"] * p["decode_num_requests"] * tau / p["cp"] for p in points]
68  return xs, ys, [f"{p['decode_num_requests']:,}" for p in points]
```

Here $R =$ `decode_num_requests` is per DP rank, and SPS is the measured decode steps per second, `decode_avg_sps`. The horizontal rate is $x = \mathrm{SPS}\,\tau$; the vertical rate is $y = \mathrm{SPS}\,R\tau/\mathrm{CP}$. For the published decode figure, EP and DP are both 64, context length is 131,072, and `nextn=3` is fixed. Its subtitle supplies 75% acceptance and $\tau = 3.25$. The script calculates $\tau = 1 + \texttt{nextn} \times a$ at line 61 and rejects variable-length mode at line 60. The acceptance value is an input to the plot, not an acceptance measurement in these records [85].

There are two plausible meanings of a. If it means the mean number of accepted draft tokens divided by three, then $\tau = 1 + 3a$ is precisely the intended conversion, ignoring end-of-request clipping. But if it means a conditional acceptance probability of 0.75 at each successive draft position, chapter 8's accepted-prefix rule gives

$$\mathrm{E}[\text{accepted drafts}] = a + a^2 + a^3 = 0.75 + 0.5625 + 0.421875 = 1.734375, \qquad \tau = 2.734375.$$

The second and third positions count only when all preceding draft tokens were accepted. Equal conditional probabilities suffice for this product; the lab uses independent Bernoulli draws as a simple exact enumeration of that case. One main-model token is included per step, with no early EOS in this model.

Lab 25 uses rational arithmetic to enumerate all eight acceptance patterns. The conversion results and the separate approximate image reading are:

```
Mean accepted fraction 0.75: tau=3.250000
Conditional acceptance 0.75: tau=2.734375
Common axis shrink: 15.8654%
Published point, approximate: 246 / 3.25 = 75.69 steps/s
Implied throughput at R=64, CP=1: 15744 tokens/s/GPU
Checks: exact prefix expectation and common-scale frontier passed
```

The ratio $2.734375/3.25 = 175/208$ shrinks both axes by about 15.9%. It preserves their relative order and which points are dominated if the same factor applies everywhere. If acceptance changes with workload or configuration, point-specific factors can change the frontier itself. The subtitle's phrase "per-token acceptance" does not settle the definition, so this calculation identifies an assumption to check rather than proving that the figure is wrong.

11.6.3 Recover a step rate; specify the missing measurement

The published 64-request point is approximately 246 tokens/s/user, read from the image rather than a raw record. Under the figure's $\tau = 3.25$,

$$\mathrm{SPS} \simeq \frac{246}{3.25} = 75.69, \qquad y \simeq 75.69 \times 64 \times 3.25 = 15{,}744 \quad \text{tokens/s/GPU},$$

using CP=1. That agrees with the plotted point near 15,750. It is not the throughput of the entire 64-GPU instance. The image hash and fixed revision are recorded separately in the source manifest; the reading remains approximate.

Neither decode axis is a latency percentile. The reciprocal of user speed is an amortised time per output token; speculative steps can publish several tokens together. It does not reveal the gaps between bursts, queueing, or time to first token. The prefill figure uses mean measured step duration on its horizontal axis, which also omits a request's earlier waits. Chapter 2's request trace is the appropriate model for those quantities.

A measurement capable of changing the conclusion would record useful emitted tokens per request-step under the same model, prompt histories, sampling parameters and fixed

draft length. It should count early termination and rejected drafts, then estimate τ at each concurrency instead of supplying one acceptance number. Keep raw records and token timestamps so that steady-state rates and end-to-end gaps remain distinguishable. First this checks the figure's absolute scale; varying τ by point also tests its ordering.

The Wide-EP comparison needs a second control: repeat EP8 and EP64 on matched hardware and workload definitions, recording topology, model and dependency revisions, expert redundancy and the actual acceptance factor. Compare their frontiers at the same per-user target and per-GPU denominator, not merely at the same total request count. A different mapping from step rate to useful output could change the observed advantage. So could service queueing absent from the offline benchmark. This is a proposed measurement, not a result of our synthetic records.

11.7 Experiments to try next

1. In lab 23, use one decode worker instead of two. Predict which prefill saving survives in final completion time, then run your change. Identify the queue that absorbs the saved time.
2. Extend the local-cache case with a choice to recompute on the idle worker. Which hit lengths make affinity preferable under the assigned costs? State whether your objective is first completion, last completion or mean wait.
3. For a 902-token prompt, explain the different meanings of 768, 640 and 134 in lab 24. How many positions rebuild SWA when it is missing? Which block KV can still be reused? Why does this not resolve the 140-row keep setting?
4. A capacity estimate uses the fraction of requests with any cache hit as h. Construct two workloads with that same fraction but different uncached-token demand. Which measurement must replace it?
5. Change acceptance for just one synthetic operating point in lab 25. Can the set of non-dominated points change? Give the counts needed to measure that point's τ, including your treatment of completed requests.
6. Choose a service claim from Table 11.1. Locate the code, public project or Q&A passage that supports it, and name one observation needed to test its production behaviour.

11.8 One request, end to end

Follow request A with a 300-token prompt through the released offline path. Use a NORMAL engine, CP=1, ZOS enabled and fixed verification with `nextn=5`. Leave chunked prefill, YOCO truncation and structured output off. Assume enough free memory, no prefix hit or preemption, and an output limit that does not interrupt the steps below. Experts occupy the configured EP group; other requests keep decode calls running after A finishes. The lengths and acceptance outcome below are assigned to illustrate the inspected branches, not recorded engine output.

11.8.1 Admit the prompt and create the first output

`LLM.generate` admits A as a persistent `Sequence` in the waiting queue. When the scheduler selects its prompt, A receives three paged blocks, $\lceil 300/128 \rceil = 3$, and a state slot. The block

table locates compressed KV and index rows; the slot holds SWA and live compression state. No other request owns these allocations in this case (chapters 1, 4 and 5).

The worker sends the 300 prompt positions through embeddings and the model layers. Attention writes and reads the layer's cache views; MoE sends each position's expert work to the selected slots and returns the weighted results (chapters 3 and 7). Sampling uses the last prompt position's representation to choose A's first output. Prompt post-processing happens in the same worker call. The processed count is now 300, while token history has length 301. The output at position 300 still needs to become a model input.

11.8.2 Follow one six-position decode step

After further decode work, choose a step N whose inputs begin at position 320 and include five available drafts. During call N, the host processes $N-1$: A's committed count becomes 320 and its history length 321. Forward N is already queued. Its six queries occupy positions 320–325, with key length 326. For example, position 320 lies in A's third logical block at token-position offset 64; the block table supplies the physical block ID.

Assume verification accepts the first two drafts and rejects the third. The accepted prefix and its replacement contribute three output tokens. The six evaluated positions therefore advance the committed count by three, not six. The last output remains unprocessed:

Host post-processing milestone	Processed positions	History length
Prompt completes	300	301
Step $N-1$, during call N	320	321
Step N, during call $N+1$	323	324

Before that last update, scheduling $N+1$ sees 320 committed positions, six in flight and six new queries: an upper-bound key length of 332. On the device, the previous accepted count removes $5-2=3$, giving 329. The new queries actually cover positions 323–328. Their main input and drafts come from step N's device output via the request remap, even though the host does not yet know them (chapters 6 and 8). The three blocks still suffice. Discarded draft bytes may remain in storage; counts and visibility decide which rows can be read (chapter 4).

Only after queuing forward $N+1$ does the worker wait for `sample_end_evt(N)` and post-process those three outputs. The entrypoint clones the returned pooled records while their addresses remain valid. Counting the six queries as six useful outputs would double this step's progress; timing submission alone would miss the wait and publication boundary (chapters 1 and 2).

11.8.3 Finish, then change the boundary

Suppose a later step M produces EOS. Call $M+1$ has already queued A's extra forward before post-processing marks it finished. In call $M+2$, scheduling removes A; post-processing waits for the obsolete $M+1$ work and skips its output; `pre_sampler_sync` then releases A's remaining storage. Other requests keep these decode calls consecutive, as assumed in chapter 6's trace. At the eventual end of the whole generation loop, the final flush drains the remaining in-flight result. Request completion, output collection and storage release are distinct events.

Two changes connect this trace to the remaining chapters. Enabling compact prefill would introduce chapter 9's full-source and retained-query layouts; agreement with mask would still not prove agreement with this untruncated baseline. Moving A to a disaggregated service would introduce prefix matching, PE selection and a PE-to-CE state handoff. The

CE needs the live state extent, including the dropped region; the released layout helper does not implement that transport. These are changes to the case, not hidden steps in its offline execution.

Finally, choose the comparator before testing either change. Chapter 10's golden trajectory can hold A's tokens fixed for a numerical comparison; independently sampled outputs answer a different question. This single request now gives us both a route through the code and a way to investigate it: name the owner, locate the processed boundary, identify the completion condition, and state which observable a proposed change must preserve.

Appendix A

Compression and indexing

Chapter 3 followed a sliding window into attention; chapter 4 allocated blocks without explaining their compressed rows. Here we follow the other read path: completed compressed KV, a smaller index used to select rows, and the temporary state needed to produce the next row.

The question

If two token positions produce one compressed row, what happens when a prefill chunk ends after the first token? Can a later six-position verification step write all its inputs into a two-entry compression ring?

```
python3 labs/26_compression.py
```

```
C2 rows (before BF16/norm/RoPE): [('6', '8'), ('8', '3'), ('3', '5')]
Chunks [0,1), [1,4), [4,7): valid/reserved rows [(0, 1), (2, 2), (1, 1)]
Pending token positions: [6]
Six-position decode: ring 7 preserves every completed pair
Ring 6 corrupts first pair: ('42/5', '8')
Checks: 64 chunk partitions agree exactly; undersized ring fails
Default C2 state payload: 7 * 2 * 512 * 4 = 28672 bytes
```

Seven inputs produce three completed pairs and one pending input. Splitting the inputs into chunks of lengths 1, 3 and 3 does not change those pairs. A six-entry ring, however, lets the last new write overwrite the first pair's old input. Seven entries preserve it. Compression reduces the completed cache; it still needs live state between steps.

Scope and order. We first explain the gated mean and its boundaries, then follow `V4Attention` through the compressor and indexer to the final attention read. A second experiment separates scoring, candidate restriction and address conversion. Finally, we account for bytes and distinguish the objects shared across layers. The case is a sparse C2 layer that owns its main KV and index-K, without YOCO compaction or CP sharding; later sections identify where those other paths differ.

A.1 A pair is complete only at its boundary

There are two kinds of compression in this discussion. MLA compresses a *feature vector*, as chapter 3 explained. C2 combines *two token positions* into one row. It does not halve the number of channels in that row. C1 keeps one row per position. The released `Compressor`

accepts only ratios 1 and 2 and rejects the optional `compress_overlap` and `use_ape` settings [87].

For latent component d, the C2 operation over positions $2j, 2j+1$ is

$$z_{j,d} = \sum_{t=2j}^{2j+1} \frac{\exp(s_{t,d})}{\exp(s_{2j,d}) + \exp(s_{2j+1,d})} v_{t,d}.$$

The score s is a learned gate; v is the projected latent. Each channel has its own weights over the two positions. This is not attention over the whole context, nor an unweighted average. In `v4_modules/mla_attn.py`, `SoftmaxAPEGatedColumnParallelLora` constructs an A projection for values and a Gate projection for scores. Its name retains APE, although the runtime rejects that option. The single-position C1 softmax is one, so C1 needs no Gate [87].

The first lab pair has values $(0, 10)$ and $(8, 2)$. It substitutes positive integer weights $(1, 3)$ and $(3, 1)$ for the exponentials. The result is $(6, 8)$: the second token dominates channel zero, while the first dominates channel one. Fractions let us check this mathematical operation exactly. They do not reproduce floating-point exponentials or BF16 rounding.

The lab implements that mean with rational weights:

Listing A.1: A per-channel weighted mean before rounding.
Book lab · `labs/26_compression.py` · lines 18–20

```
18 def compress(rows):
19     return tuple(sum(F(w[d]) * v[d] for w, v in rows)
20                  / sum(w[d] for w, _ in rows) for d in range(2))
```

The source reference reduces over positions, then casts the result:

Listing A.2: A softmax over positions for each latent channel.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/compress.py` · lines 199–200

```
199 result = (latent_slice * score_slice.softmax(dim=0)).sum(dim=0).to(torch.bfloat16)
200 kv_compressed[compressed_start + blk_id - qstart // compress_ratio] = result
```

The production wrapper makes the C1/C2 distinction explicit:

Listing A.3: C1 bypasses temporal compression; C2 separates scores and values.
DeepInfer · `deepinfer/model_executor/models/v4_modules/compress.py` · lines 89–96

```
89 if self.kv_compress_ratio == 1:
90     assert projected_kv.dtype == torch.bfloat16 and state_cache is None
91     if input_metadata.mirror_cp_reverse_index is not None:
92         projected_kv = gather(projected_kv.contiguous(), input_metadata.
   ↪ mirror_cp_reverse_index)
93     return projected_kv
94
95 assert projected_kv.dtype == torch.float32 and state_cache is not None
96 score, latent = projected_kv.chunk(2, dim=-1)
```

C1 receives BF16 and no compression state. Its optional gather restores CP's input ordering. C2 receives FP32 score/latent pairs and requires state. The following branch calls the prefill or decode helper with the same ratio. These helpers return BF16 rows before the compressor's normalisation and RoPE.

A.1.1 Chunk boundaries and placeholder rows

Let s be the first new token position and e the exclusive end of a chunk. For ratio r, the number of newly completed rows is

$$n_c = \lfloor e/r \rfloor - \lfloor s/r \rfloor.$$

The prefill reference iterates exactly those compressed indices. If a pair began before s, it reads that input from the state ring; otherwise it reads from the current projection. It subsequently retains recent score/latent pairs in the ring for later chunks [88].

For $[0, 1)$, no pair is complete. For $[1, 4)$, pairs $(0, 1)$ and $(2, 3)$ finish: position zero must survive the first chunk. For $[4, 7)$, only $(4, 5)$ finishes. Position six remains pending. Lab 26 tries every choice of cuts between the seven inputs and compares the completed rows and pending suffix with processing the whole sequence.

Physical output shape is not the valid-row count. The prefill metadata reserves $\max(1, n_c)$ rows per request, even when $n_c = 0$. Its placeholder has position zero and store slot -1; the reference compression output is zero-filled there. Decode instead reserves one output row per query position, with zero output and no valid store slot for a position that does not complete a group. The C++ builder implements these conventions in `prepare_tensor_input_v4.cpp`:46–91. A placeholder is not an extra cached token [89].

A.2 Temporary state must survive speculative writes

A C2 cache row cannot replace the score and latent of an incomplete pair. The live `ca_state` stores both, in FP32, with shape

$$[\text{state slots},\ S,\ 2,\ \text{latent dimension}].$$

The axis of length two is score versus latent, not K versus V. `ca_cache` holds completed compressed KV in paged blocks. Their lifetimes and byte formats differ.

The decode reference writes every new position before computing completed rows. Its assertion states the required capacity:

Listing A.4: Write all inputs, then find positions that finish a group.
DeepInfer · `deepinfer/model_executor/layers/tilelang_ops/compress.py` · lines 223–238

```
223 compress_token = compress_ratio
224 state_cache_size = state_cache.shape[1]
225 assert state_cache.shape[1] >= compress_ratio + max_seqlen_q - 1
226
227 # store: each token writes to cache
228 for idx in range(total_q):
229     sidx = state_block_idx[idx].item()
230     cpos = (context_lens[idx].item() - 1) % state_cache_size
231     state_cache[sidx, cpos, 0, ...] = score[idx, ...]
232     state_cache[sidx, cpos, 1, ...] = latent[idx, ...]
233
234 # compute: check each token's position
235 kv_compressed = torch.zeros(total_q, dim, dtype=torch.bfloat16, device=score.device)
236 for idx in range(total_q):
237     abs_pos = context_lens[idx].item() - 1
238     if abs_pos % compress_ratio == compress_ratio - 1:
```

For a group of r positions and up to $q_{\max}$ new positions per state slot, retaining $r-1$ old inputs alongside all new inputs requires $S \geq r + q_{\max} - 1$. This is the same write-before-read hazard encountered with SWA, applied to a different ring.

In the lab, position zero is old; positions 1 through 6 are written in one step. With $S = 6$, position six takes position zero's slot before pair $(0, 1)$ is read. With $S = 7$, no required input is overwritten. The incorrect first component becomes $42/5$, rather than 6. No future position belongs in that pair, even though it has already been written to memory.

The cache engine derives its ring from the backtracking allowance, and separately decides whether an indexer needs its own state:

Listing A.5: Compression capacity and the index-state exception.
DeepInfer · deepinfer/worker/cache_engine.py · lines 123–133

```
123 def _needs_index_state(attn_module: AttrDict) -> bool:
124     if attn_module.cfg.get("index_topk", 0) <= 0:
125         return False
126     index_cfg = attn_module.cfg.get("attn_index")
127     return index_cfg is None or not index_cfg.get("use_kproj", False)
128
129
130 @lru_cache(8)
131 def get_inference_compress_block_size(compress_ratio: int) -> int:
132     """Compression ring plus rollback history; environment changes require
    ↪ clearing the cache."""
133     return compress_ratio + max(0, envs.DEEPINFER_MAX_BACKTRACKING_TOKENS - 1)
```

With the default `nextn=5`, the backtracking allowance is at least six. For C2 the default ring is $2 + (6 - 1) = 7$, not the SWA ring's 134. At dimension 512 its score/latent payload is $7 \times 2 \times 512 \times 4 = 28{,}672$ bytes per owning layer per state slot. That excludes other state regions and any layout padding. Rejecting a draft changes which positions are valid; it does not require clearing every byte, as chapter 4 established.

A.3 One compressed result feeds two caches

The main attention module connects ownership to computation:

Listing A.6: Give the compressor its completed cache and live state.
DeepInfer · deepinfer/model_executor/models/v4_modules/attn.py · lines 515–523

```
515 assert wait_projected_kv is not None
516 assert self.PackedWkv is not None
517 return self.PackedWkv.forward_store_kv_cache(
518     wait_projected_kv(),
519     kv_cache.ca_cache,
520     kv_cache.ca_state,
521     input_metadata,
522     return_pre_rope_kv=self.has_local_index,
523 )
```

The result forks after normalisation and before rotary position encoding. The main branch applies RoPE, casts to the main cache format and writes `ca_cache`. A local indexer also needs the normalised pre-RoPE latent:

Listing A.7: Retain the normalised latent while storing main KV.
DeepInfer · `deepinfer/model_executor/models/v4_modules/compress.py` · lines 158–167

```
    cast_args=CastArgs.from_cache_format(cache_format),
    out_args=KVCacheOutArgs(
        kv_cache,
        input_metadata.slot_mapping_store_ca[self.ratio_key],
        out=pre_rope_kv,
        out_stage="norm",
        layout=cache_format,
    ),
)
return pre_rope_kv[:, 0] if pre_rope_kv is not None else None
```

`out_stage="norm"` identifies the retained stage. The surrounding call supplies the compressed positions and RoPE parameters. For compressed row j, the C++ metadata uses position jr, the beginning of its group, while a query keeps its original token position. Equal row numbers therefore do not imply equal positional coordinates [89]. Figure A.1 shows where the two caches separate.

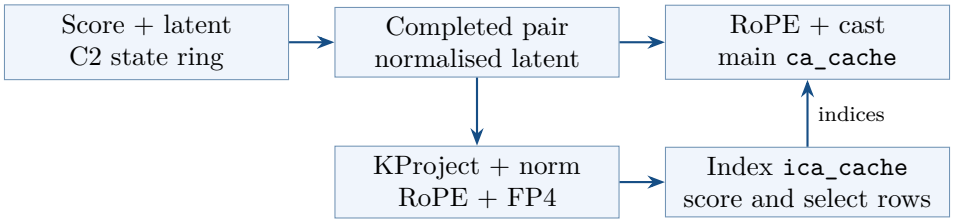


Figure A.1: The sparse layer's two caches. Index scores choose addresses in the main cache; they do not supply the attention values.

The index branch projects that latent to a smaller key. Its store method requires that no independent index compression state be present:

Listing A.8: Index-K is projected from the main compressed latent.
DeepInfer · `deepinfer/model_executor/models/v4_modules/index_attn.py` · lines 140–148

```
if not self.store_index_k:
    return
assert index_k_input is not None
assert state_cache is None
index_k = self.index_k_module.forward_pre_norm(index_k_input)
norm = self.index_k_module.norm
fused_norm_rope_hadamard_cast_store_kvcache(
    index_k,
    norm_args=NormArgs(norm.weight, norm.variance_epsilon),
```

The model configuration selects KProject with `attn_index.use_kproj=True`. The cache engine therefore leaves `ica_state=None` on this V4.1 path. The field remains in the generic interface for a separately compressed index path. It is not a second accumulator that every sparse layer allocates. `ica_cache` still exists: absence of live index state does not mean absence of completed index keys [90].

This is a model-driven requirement. A block allocator alone cannot decide that the index keys derive from the main compressed latent, that C1 needs no pending pair, or that a consumer layer reuses another layer's cache. Those decisions come from the model configuration and its runtime adapters.

A.4 Select rows, then translate their addresses

An indexer makes a cheaper scoring pass before main attention. The runtime describes its score for key k_j as

$$I_j = \sum_h w_h \max(0, q_h \cdot k_j).$$

These are learned index queries and head weights, not the main attention softmax probabilities. ReLU means $\max(0, x)$. DeepGEMM computes the logits; DeepSelect supplies the imported `topk`. Their kernel internals remain outside this walkthrough [90].

```
python3 labs/27_indexing.py
```

```
Scores: [2, 6, 3, 0, 300] | valid prefix: 4
Top-2 over valid rows: [1, 2]
Candidate block scores: [6, 3] | retained block: 0
Top-2 within retained block: [0, 1]
One valid row, three output slots: [0, -1, -1]
Prefill at gathered offset 10: [11, 12]
Decode logical rows: [1, 2, 4, -1] -> physical slots: [29, 30, 8, -1]
```

The lab uses two query heads, $(1, 0)$ and $(0, 1)$, with weights 1 and 2. Key $(0, 3)$ scores 6, while $(1, 1)$ scores 3. The fifth key would score 300, but it is outside the valid prefix. The chosen rows are 1 and 2, not 4. The calculations omit projection, RoPE and quantisation and use a stated index-based tie rule; they do not reproduce the device top-k implementation.

The teaching score uses ordinary integer arithmetic:

Listing A.9: Score one key against two query heads.
Book lab · `labs/27_indexing.py` · lines 16–18

```
16 def score(key):
17     return sum(w * max(0, sum(a * b for a, b in zip(q, key)))
18                for w, q in zip(WEIGHTS, Q))
```

A.4.1 Validity comes before ranking

A query at original position p can see $\lfloor (p+1)/r \rfloor$ completed compressed rows. The metadata supplies that count per query:

Listing A.10: Count completed compressed rows at each query position.
DeepInfer · csrc/engine/input_metadata/prepare_tensor_input_v4.cpp · lines 591–601

```
591 for (int r : args.sparse_compress_ratios) {
592     py::str name = std::to_string(r);
593     auto klen = get_or_create_tensor(klen_dict, has_out, name, {total_q}, torch::kInt32);
594     auto p_klen = klen.mutable_data_ptr<int32_t>();
595     i_q = 0;
596     for (auto p_seq : seq_metadata_list) {
597         for (int32_t j = p_seq->seqlen_k - p_seq->seqlen_q; j < p_seq->seqlen_k; ++j, ++i_q) {
598             p_klen[i_q] = (j + 1) / r;
599         }
600     }
601     TORCH_CHECK_EQ(i_q, total_q);
```

For C2, query zero sees no completed row; query one sees row zero; query two still sees only row zero. The most recent unpaired token remains available through the layer's SWA path. A large score in an unwritten or future slot must not enter the selection.

Decode passes an explicit valid range to top-k:

Listing A.11: Bound top-k and restore candidate-relative indices.
DeepInfer · deepinfer/model_executor/models/v4_modules/index_attn.py · lines 494–505

```
494 topk_index = topk(
495     p_sum,
496     self.topk_tokens,
497     sorted=False,
498     end=topk_lens,  # We must pass ‘end‘ to the topk kernel since OOB p_sum is not cleaned.
499     indices_type=torch.int32,
500     sorted_index=True,
501     idx_oob_fill_value=-1,
502     return_value=False,
503 )[1]
504 if candidate_blocks is not None:
505     restore_candidate_indices_inplace(topk_index, candidate_blocks, candidate_block_size)
```

`end` excludes invalid score storage, whose contents need not have been cleared. `idx_oob_fill_value=-1` fills unused output lanes. The request for `sorted_index=True` concerns index ordering, not decreasing score order. The lab returns a set of chosen rows in index order and checks its total score against all pairs of valid rows. It also checks empty and undersized ranges.

A.4.2 Candidate blocks are another restriction

Some layers first reduce index scores by candidate block, using a maximum, and select blocks for later layers to search. This grouping is distinct from the allocator's 128-token page. In the lab, two index keys form a candidate block: block scores are 6 and 3. Retaining only the first block changes token top-2 from $[1, 2]$ to $[0, 1]$. Thus top-k *within candidates* need not equal top-k over all valid keys, even with exact arithmetic.

The runtime's candidate consumer accepts block sizes 8 or 16 in compressed-row units and imposes further width constraints; the lab's block size two and single retained block are deliberately smaller teaching values. A source layer fills `block_candidate_idxs`; a consumer computes logits only within that set, then restores compact candidate offsets to logical row indices. Comparing this model with unrestricted attention would require a

model-quality check, not merely equality between two implementations of restricted top-k [90, 91].

Prefill limits the size of its intermediate score matrix with `compute_chunk_size`. It derives the row width from either the full compressed context or the candidate set, applies the memory budget, rounds to multiples of eight and caps the chunk at 8,192 queries. That is a temporary logit-storage budget, not the number of keys finally selected.

A.4.3 The same logical row has two physical representations

For prefill, `_load_prefill_kv` gathers keys into a concatenated sequence buffer. Top-k adds the request's `ks` offset, or restores candidate indices with that offset. With base 10, logical rows 1 and 2 become 11 and 12.

Decode keeps paged storage. With B original-token positions per allocator block, a ratio-r cache has $B_c = B/r$ compressed rows in that block. Logical row j maps to

$$\text{slot}(j) = \text{block table}[\lfloor j/B_c \rfloor]B_c + (j \bmod B_c).$$

The released indexer performs this conversion after candidate offsets have been restored:

Listing A.12: Convert selected logical rows to paged cache slots.
DeepInfer · `deepinfer/model_executor/models/v4_modules/index_attn.py` · lines 506–514

```
# topk outputs the logical index of the compressed token within the seq, which is
    ↪ converted in place into the kvcache
# physical slot through block_tables (-1 is kept)
convert_kv_index(
    topk_index.unsqueeze(1),
    compressed_block_size,
    topk_lens.unsqueeze(1),
    input_metadata.block_tables,
)
return topk_index
```

The lab's smaller page has $B = 8, r = 2, B_c = 4$, with block table $[7, 2]$. Rows 1, 2 and 4 map to slots 29, 30 and 8. Its last lane represents padding and remains -1. The CUDA conversion checks the lane's position against the valid count; callers must not put an arbitrary invalid index in a lane declared valid. The lab's sentinel guard is a teaching convenience, not a transcription of that kernel [92].

Finally, main attention reads the SWA and compressed branches together:

Listing A.13: Supply the selected main-KV rows alongside SWA.
DeepInfer · `deepinfer/model_executor/models/v4_modules/attn.py` · lines 640–647

```
if self.use_swa:
    attn_kv_args = dict(
        key_cache=kv_cache.swa_cache,
        indices_in_kvcache=kv_swa_index,
        key_cache_extra=kv_cache.ca_cache,
        indices_in_kvcache_extra=kv_ca_index,
        index_length_extra=dense_index_len,
    )
```

The following `decode_mla_mqa_sparse` call goes to FlashMLA. It receives `ca_cache`, not the smaller `ica_cache`, as its extra key/value storage. This completes the path from compression state to the actual attention read [93].

A.5 Count rows, payloads and owners separately

C2 halves the completed row count relative to C1. Low precision separately changes each row's byte count. The index format specifies 128 FP4 elements and one one-byte UE8M0 scale per 32 elements:

Listing A.14: The index cache stores packed values and scale factors.
DeepInfer · deepinfer/model_executor/layers/tilelang_ops/cache_common.py · lines 129–137

```
FP4_INDEX_KV_FORMAT = KVCacheFormat(
    quantized_dtype="e2m1",
    quantized_elements=128,
    elements_per_sf=32,
    bf16_elements=0,
    do_cast=False,
    sf_format="ue8m0",
    note="V4/V41, index kv.",
)
```

The same file describes main formats. Here E2M1 and E4M3 name floating-point encodings; UE8M0 is the unsigned exponent scale encoding. The relevant accounting is payload bits plus the stored scales:

Cache format	Data plus scales	Bytes per row
Main/SWA FP8	$512 + 512/32$	528
Main FP4	$512/2 + 512/16$	288
Index FP4	$128/2 + 128/32$	68

The main FP4 format uses one E4M3 scale per 16 values; its scale format differs from the index format. Main/SWA caches interleave data and scales within each row, whereas the index cache uses block-wide data and scale sections. The 68-byte figure is a logical per-row byte budget, not a claim that each physical index row is laid out as 64 immediately adjacent data bytes followed by four scales. Alignment and shared ownership affect total allocation [24, 90].

A 128-position C2 block has 64 completed-row slots. An owning main cache uses $64 \times 528 = 33{,}792$ payload bytes with FP8, or 18,432 with FP4; an owning index cache uses $64 \times 68 = 4{,}352$. The cache engine rounds regions to its alignment requirements and does not allocate another copy for a sharing consumer. The live 28,672-byte compression-state example is a separate per-request allocation. These equations explain chapter 4's rows without claiming a measured reduction in model error or end-to-end latency.

A.6 Sharing has several meanings

Chapter 9 separated YOCO's shared hidden state from segmented sharing. Here are the latter's distinct objects:

Map	Reused object	Work a consumer may still do
`layer_kv_map`	Main compressed KV	Project index-K or choose new indices
`layer_index_k_map`	Index keys	Score its own queries and choose indices
`layer_index_module_map`	Selected top-k indices	Main attention using its queries
`layer_candidate_source_map`	Candidate block set	Token top-k within those blocks

For a four-layer teaching example, main-KV sources might be $[0, 0, 0, 0]$, index-K sources $[0, 0, 2, 2]$, and index sources $[0, 1, 2, 3]$. Each finer segment lies inside a coarser one. An index-K source cannot project from a main KV belonging to a different segment. The topology checks this refinement:

Listing A.15: A finer sharing segment must stay inside its source segment.
DeepInfer · `deepinfer/model_executor/models/v4_modules/cross_layer.py` · lines 181–189

```
181 @staticmethod
182 def _validate_map_refines(fine_name: str, fine: list[int], coarse_name: str, coarse: list[int])
    ↪ -> None:
183     # Under the segmented assumption, coarse[fine[l]] == coarse[l] is equivalent to every segment
    ↪ of fine
184     # falling inside a single segment of coarse.
185     for layer_id, fine_src in enumerate(fine):
186         assert coarse[fine_src] == coarse[layer_id], (
187             f"shared {fine_name} map must refine {coarse_name} map: layer {layer_id} ->
    ↪ {fine_name} source {fine_src}, "
188             f"but {coarse_name} sources differ: {coarse[layer_id]} != {coarse[fine_src]}"
189         )
```

This is only an example of valid map nesting, not a replacement model configuration. Other checks require compatible layer settings, preceding source layers and a fresh KV source at the YOCO boundary. The topology is static; `CrossLayerRuntime` carries pre-RoPE KV, loaded index keys, indices and candidate data from source to consumer during a forward call. Under YOCO compaction, full-layout main/index KV may already have been written, so the compact query path skips those stores. Chapter 9 covered that boundary [91, 93].

A.7 Experiments to try next

The labs check rational compression, state lifetime and index/address arithmetic; they do not establish accelerator equivalence or acceptable generation quality. The released compression tests compare device outputs with a tensor reference and check state tensors. We inspected those assertions without running the tests.

1. Change the first pair's weights so each channel has equal weights. Which outputs become ordinary averages? Keep the chunk checks unchanged.
2. Explain why a zero-valid-row prefill chunk still reserves an output row. Which metadata prevents that placeholder from becoming a real cache entry?
3. For C2 with at most four new positions, derive a sufficient compression ring size and construct the write that would break a smaller ring.
4. Give an example where restricting candidates preserves token top-k, and one where it changes it. What would a model-quality evaluation need in addition?
5. Trace one selected compressed row through `ica_cache`, top-k, the block table and `ca_cache`. Which steps change values, and which only change addresses or the set of rows read?

The reading path distinguishes pending state, completed KV and selection keys. Use those distinctions to locate the next reader of a row and check its validity.

Appendix B

Repository and execution map

The chapters organise DeepInfer by mechanisms. This appendix organises the same implementation by processes, call boundaries and directories. Use it to choose a starting symbol, then return to the chapter that explains its behaviour. Line anchors refer to the pinned edition in Appendix C.

The question

On four devices, how can one configuration have four engine drivers and another have two drivers, while both have four device workers? Does adding expert parallelism create another pool of processes?

```
python3 labs/28_ranks.py
```

DP, CP and EP mean data, context and expert parallelism. The lab translates rank arithmetic; it starts no workers. DP4/CP1 gives four drivers, each using one device. DP2/CP2 gives two drivers, each with one spawned CP follower. EP4 groups those same four device workers in both configurations. For DP2/CP2 the computed memberships are:

```
DP groups: [[0, 2], [1, 3]]
CP groups: [[0, 1], [2, 3]]
EP groups: [[0, 1, 2, 3]]
```

The grouping arithmetic in the lab is small enough to inspect directly:

Listing B.1: Three groupings over one set of ranks.
Book lab · `labs/28_ranks.py` · lines 10–17

```
10 def layout(dp, cp, ep):
11     world = dp * cp
12     assert world % ep == 0
13     workers = [(e * cp + c, e, c, "driver" if c == 0 else "child")
14                for e in range(dp) for c in range(cp)]
15     dp_groups = [[e * cp + c for e in range(dp)] for c in range(cp)]
16     cp_groups = [[e * cp + c for c in range(cp)] for e in range(dp)]
17     ep_groups = [list(range(start, start + ep)) for start in range(0, world, ep)
    ↪ ]
```

Thus DP, CP and EP describe different groupings of the same workers. They are not three process pools to add together. The following map assumes one node, zero device offset and no extra Engram replicas. Production choices in the Q&A are a separate configuration, discussed in chapter 11.

B.1 Processes, devices and communication groups

The example launcher `examples/_launch.py`:26, `run_dp`, starts one process per engine rank assigned to the node (or uses the current process for one engine). Its `RANK`/`WORLD_SIZE` select the node's share of those ranks. Within each engine, `deepinfer/engine/llm_engine.py`:368, `_init_workers`, spawns CP ranks 1 through $C-1$. CP rank zero is a `Worker` in the engine process itself. The engine's Python controller is therefore not an additional device worker [94].

`WorkerBase` in `deepinfer/worker/base.py`:79 assigns

$$g = eC + c,$$

where e is engine rank, C is CP size and c is CP rank. The example factory's `setup_cuda_visible_devices` assigns each engine its contiguous slice of devices; within that slice a worker uses its local index. Figure B.1 shows the two example layouts with four local devices and no offset.

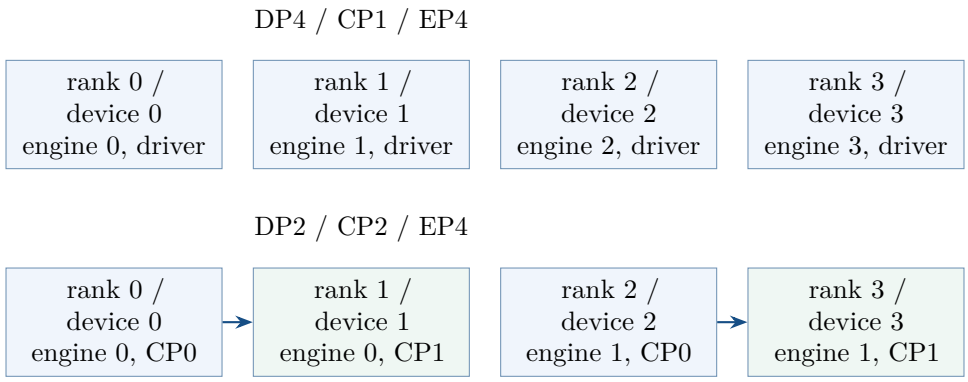


Figure B.1: Four device workers in either configuration. Lower-row arrows show driver-to-follower control within a CP group. EP includes all four ranks in each case; it does not add processes.

`ParallelProcessGroup.create_process_groups` builds DP groups across engines at a fixed CP position, CP groups within an engine, and contiguous EP groups:

Listing B.2: Group the existing ranks along different dimensions.
DeepInfer · `deepinfer/model_executor/parallel_utils/parallel_state.py` · lines 91–106

```
# DP group
dp_ranks = rearrange(ranks, "(replica dp cp) -> (replica cp) dp", dp=dp_size, cp=cp_size)
self.dp = _split_group(dp_ranks.tolist(), backend=backend, pg_options=options)
self.cpu_dp_group = _split_group(dp_ranks.tolist(), backend="gloo")

# CP group
cp_ranks = rearrange(ranks, "(dp cp) -> dp cp", cp=cp_size)
self.cp = _split_group(cp_ranks.tolist(), backend=backend, pg_options=options)
self.cpu_cp_group = _split_group(cp_ranks.tolist(), backend="gloo")

# EP groups: contiguous groups of ep_size ranks.
if ep_size == ranks.numel():
    self.ep = self.world
else:
    ep_ranks = rearrange(ranks, "(edp ep) -> edp ep", ep=ep_size)
    self.ep = _split_group(ep_ranks.tolist(), backend=backend, pg_options=options)
```

For DP2/CP2, this produces DP groups $[0, 2]$ and $[1, 3]$, CP groups $[0, 1]$ and $[2, 3]$, and the EP group $[0, 1, 2, 3]$. The membership formula is distinct from a workload split: CP workers cooperate on the same engine's requests, whereas DP engines admit different requests. EP determines where experts run.

Within a CP group, the query rows of each sequence are divided by a mirrored, or zigzag, rule. The reference expansion in `model_executor/layers/tilelang_ops/prepare_cp_mirror.py`, lines 135–146, cuts a sequence of q rows into $2C$ pieces of $\lceil q/2C \rceil$ rows; rank r takes pieces r and $2C - 1 - r$. With $C = 2$ and eight rows, rank 0 holds rows 0–1 and 6–7, while rank 1 holds rows 2–5. This is why the scheduler rounds prefill workloads to `cp_align`$= 2C$ (`csrc/engine/scheduler.hpp`:90; chapter 5). Under causal attention, later rows read more context, so pairing an early piece with a late one plausibly evens out the work. That is our reading of the README's "balanced CP", not a measured balance [94].

The general configuration is broader. `deepinfer/config.py`:150, `ParallelConfig`, allows Engram parallelism to enlarge the world; the world size is the maximum of that size and DP times CP. The lab deliberately excludes those replicas. CPU control groups and shared-memory queues also exist. Their transport implementation and all distributed failure paths are not covered here [94].

B.2 Follow one step into computation

Figure B.2 is a call-boundary map, not an elapsed-time chart. The scheduler returns work to the engine. It does not call the model runner. The worker separately launches main-model execution and sampling.

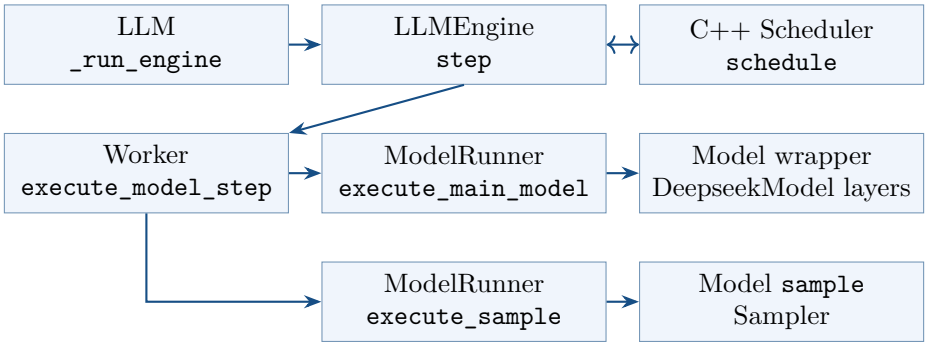


Figure B.2: Control and computation. The engine/scheduler boundary is a pybind call and return. The worker controls both launches; under ZOS, previous-step post-processing occurs between them.

The following anchors locate each hop. Paths in this table are relative to `deepinfer/`, except those starting with `csrc/`.

File and anchor	Symbol, boundary and reading
`entrypoints/llm.py`:90,168	`generate`, `_run_engine`; admission, step/flush loop and cloning. Chapter 1.
`engine/llm_engine.py`:860,885	`step` calls the scheduler and receives metadata. Chapters 1, 5, 6.
`csrc/engine/pybind_scheduler.cpp`:97	Binding of `Scheduler::schedule`; implementation in `scheduler.hpp`:667. Chapter 5.
`engine/llm_engine.py`:961,1017	`_execute_model_async_step` and `_run_workers`; route the call to the driver. Chapter 6.
`worker/base.py`:290	`run_workers`; broadcast the method to CP followers, then invoke the local worker. This appendix.
`worker/worker.py`:89,122,154	`execute_model_step`; launch forward, process earlier output where applicable, then launch sampling. Chapter 6.

The model runner first prepares input metadata. The eager path calls the model; the graph path replays captured work:

Listing B.3: The model call may be replaced by graph replay.
DeepInfer · `deepinfer/worker/model_runner.py` · lines 326–329

```
326 if run_cuda_graph:
327     model_output = self.cuda_graph_runner.replay(cuda_graph_used)
328 else:
329     model_output = self.model(kv_caches=kv_caches, input_metadata=input_metadata)
```

The registered architecture name is `DeepseekV41ForCausalLM`. It selects the `DeepseekVLForCausalLM` wrapper, whose forward prepares prompt embeddings when needed. It then delegates to the base wrapper's `DeepseekForCausalLM.forward`, which calls the core model [95].

File and anchor within `deepinfer/`	Symbol and reading
`worker/model_runner.py`:308,337	`execute_main_model`, `execute_sample`; input preparation and launch boundary. Chapters 3, 6, 8.
`model_executor/models/deepseek_vl.py`:226	VL `forward`; delegates to the base causal-LM wrapper. Navigation only for vision processing.
`model_executor/models/deepseek.py`:374,154	Causal-LM `forward`, then `DeepseekModel.forward`; layer loop. Chapters 3, 9; appendix A.
`model_executor/models/decoder_layer.py`:97	`forward`; attention and feed-forward sublayers. Chapters 3, 7, 9.
`model_executor/models/deepseek.py`:383	`sample`; main sampling and MTP work. Chapter 8.
`model_executor/layers/sampler.py`:114	`Sampler.forward`; constraints, verification and sampling inputs. Chapters 8, 10.

B.3 Follow results back to their owner

The reverse path is not simply `Sampler` returning a Python answer. Sampling completion permits post-processing; C++ updates sequence state and builds pooled output records; the Python entrypoint clones records it retains (Figure B.3).

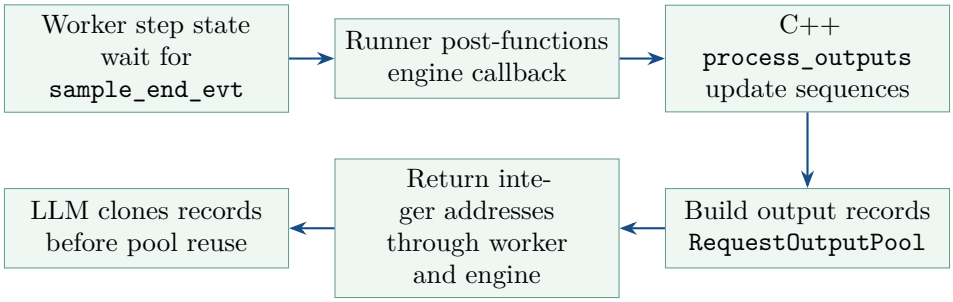


Figure B.3: Result ownership. Under ZOS, the output being processed usually belongs to the preceding decode step. A final flush processes the remaining in-flight result. Chapters 1 and 6 give the lifetime rules.

The engine installs its callbacks when constructing the driver:

Listing B.4: Connect worker completion to engine state changes.
DeepInfer · `deepinfer/engine/llm_engine.py` · lines 398–400

```
398 driver_kwargs = {
399     "engine_callbacks": EngineCallbacks(post_process_fn=self.post_process,
    ↪ pre_sampler_sync_fn=self.pre_sampler_sync),
400 }
```

Start at `worker/worker.py`:248, `_post_process_step`. It waits for the sampling event, runs the model runner's post-functions and invokes the engine callback on the driver. `engine/llm_engine.py`:984, `post_process`, enters `_process_model_outputs` at line 687. That method calls `scheduler.process_outputs` at line 727, then `build_request_outputs` at line 743. The binding is in `csrc/engine/pybind_output.cpp`:41; the builder and pool are in `csrc/engine/outputs.hpp`:94 and 45. The returned array holds integer addresses. `entrypoints/llm.py`:181 clones each retained record [5, 96].

These names identify a dependency path, not a guarantee that one `step()` returns the tokens it just launched. The worker may process step N while step $N + 1$ runs. To investigate missing final tokens, follow `flush_inflight_reqs`; to investigate stale addresses, follow the output pool's release, not Python reference counts.

B.4 Start from a README feature

The pinned README lists eight features. These tables connect each name to its controls and a source entry point. Batch invariance and deterministic sampling have separate rows because their conditions differ. Settings are not a complete launch recipe; follow the indicated chapter for defaults and limits [1, 97].

Paths are relative to `deepinfer/`, except `csrc/`, `vendor/` and `requirements.txt`. The example entry points' `ep` and `cp` arguments choose group sizes; `seq_seed` and `so_args` are `SamplingParams` fields. Names beginning with `DEEPINFER_` are environment variables.

README feature	Setting	Entry file and symbol	Read
Attention DP / balanced CP; EP + EPLB	`ep`, `cp`; `DEEPINFER_EXPERT_REARRANGEMENT_STEP_INTERVAL`	`model_executor/parallel_utils/parallel_state.py`: `create_process_groups`; `model_executor/eplb/`	7, B
DSpark	`DEEPINFER_ENABLE_MTP_VARLEN=1`; `DEEPINFER_USE_NEXTN_DECODE` limits draft length	`csrc/engine/mtp_varlen.hpp`: `MTPVarlen::schedule`	8
SWA bounded replay	`DEEPINFER_YOCO_PREFILL_TRUNCATION_MODE=compact`; `DEEPINFER_YOCO_PREFILL_KEEP_TOKENS`	`csrc/engine/input_metadata/prepare_tensor_input_v4.cpp`: `fill_yoco_truncation_metadata`. Decoder truncation; encoder hit recovery has a different release boundary.	9
Asynchronous scheduling	`DEEPINFER_ZERO_OVERHEAD_SCHEDULING`	`worker/worker.py`: `execute_model_step`	6

README feature	Setting	Entry file and symbol	Read
Batch-invariant inference	`DEEPINFER_BATCH_INVARIANT=1`; NVIDIA only	`worker/runtime_config.py`: `configure_worker_kernels`	10
Deterministic sampling	`DEEPINFER_BATCH_INVARIANT=1`, `DEEPINFER_ENABLE_MTP_VARLEN=0`, fixed `seq_seed`; NVIDIA only	`model_executor/layers/tilelang_ops/cuda/fused_sample.py`: position/role-keyed random draws. No released end-to-end test of this recipe.	10
Structured outputs	`so_args`	`model_executor/layers/sampler.py`: `_get_so_mask`; `structure_output/matcher.py`	10
Optimized kernels, model and sampler runtime	No single switch	`vendor/`; `requirements.txt`; `model_executor/layers/tilelang_ops/ops.py`: backend dispatch. Library internals are outside the book's detailed treatment.	3, A, B

B.5 Choose a directory before choosing a file

Coverage labels below describe the book, not the maturity of the software. *Deep* means a mechanism is traced in detail; *partial* means selected paths are analysed; *locate* gives responsibility and an entry point only; *outside* marks implementation details not analysed. A deep directory entry still does not mean every file in it was reviewed.

Runtime area	Responsibility and coverage
`deepinfer/entrypoints/`, `engine/`	Public LLM interface, request lifecycle and scheduling orchestration. Deep: chapters 1, 5, 6. The second path here and below is relative to `deepinfer/` unless stated otherwise.
`csrc/engine/`, `deepinfer/_C/`	Native scheduler, sequence/block state, metadata and output bindings; Python extension interface. Deep: chapters 1, 4–6, 8; appendix A.
`deepinfer/worker/`	Worker lifetime, model runner, cache engine and graphs. Deep for execution/state; partial for initialisation. Chapters 3, 4, 6, 9.
`model_executor/input_metadata/`	Python metadata views, ZOS preparation, YOCO and draft layouts. Partial: chapters 3, 6, 8, 9.
`model_executor/eplb/`	Statistics, replica/placement policy and weight movement. Deep: chapter 7.
`structure_output/`	Grammar compilation, matching and rollback. Partial: chapters 5, 10.

Model and kernel area within `deepinfer/`	Responsibility and coverage
`model_executor/models/v4_modules/`	Attention, compression, indexer, MoE, sharing and model adapters. Partial overall; deep paths in chapters 3, 7, 9 and appendix A.
`model_executor/models/mtp/`	DFlash proposals and confidence. Partial: chapter 8.
`model_executor/models/vision_encoder/`, `ve/`	Vision computation and multimodal processing workers. Locate; architecture and preprocessing outside.
`model_executor/layers/`	Sampling, linear/norm/rotary layers and attention backend interfaces. Partial: chapters 3, 8, 10; appendix A.
`model_executor/layers/tilelang_ops/`	Interfaces, references and device kernels. Selected formulas are analysed; general kernel tuning is outside.
`tilelang_ops/cuda/`, `tilelang_ops/asc/`	Relative to the preceding layers directory: GPU and Ascend implementations. CUDA has selected deep dives; Ascend internals are outside.

For the model modules not expanded in appendix A, start with `v4_modules/hc.py`: `HCLayerMixin` mixes the multiple residual streams around attention and feed-forward sublayers, while `HCHead` folds them back before the output head. `hc_blocks.py`, `hc_lazy.py` and `mhc_*` provide related mixing and deferred work. In `v4_modules/engram.py`, `EngramRuntime` fetches sharded n-gram embeddings; related metadata and parameter files describe their representation. These are navigation descriptions, not derivations or evaluations of either model mechanism [95].

Support area	Responsibility and coverage
`deepinfer/hugging_face/`	Configuration translation, checkpoint parameters, loading and tokenisation. Configuration is partial; loading internals outside. Start with `loading.py` and `weights.py`.
`deepinfer/model_executor/model_loader.py`	Registry lookup and model construction/loading. Locate; model structure also starts in `models/v4_modules/builder.py`.
`deepinfer/model_executor/parallel_utils/`, `deepinfer/distributed/`	Groups, collectives, buffers and shared-memory transport. Partial: chapters 6, 7 and this appendix; complete initialisation/recovery outside.
`deepinfer/platform.py`, `deepinfer/npu_ops/`, `deep_cluster/`	Platform selection, NPU adapters, device discovery and affinity. Locate; platform-specific algorithms outside.
`examples/`, `eval/`, `tests/`	Launch and benchmark entry points, evaluation and tests. Partial: chapters 2, 8–11; inspect test assertions rather than treating file names as guarantees.
`vendor/`, `scripts/`, `csrc/graph_timer/`	Independent library pins, build scripts and native timing. Dependency boundaries and timing are partial; library/kernel implementation surveys and build-system internals outside.
`doc/`, root README and requirements	Deployment account, usage and installation requirements. Chapter 11 separates described service logic from released code.

Platform dispatch has a useful search boundary: `model_executor/layers/tilelang_ops/ops.py` imports the backend selected by `PLATFORM.tilelang_ops_backend` and exports its `*_tl` factories. An interface module can therefore look platform-neutral while selecting CUDA or Ascend implementations. This is a route into those files, not evidence that their numerics or performance are identical. The independent DeepSeek libraries in `vendor/` include DeepGEMM, DeepEP, DeepSelect and FlashMLA; use their pinned submodule revisions when crossing that repository boundary. TileKernels is another independent DeepSeek library, installed through `tile-kernels==2.1.0` in `requirements.txt`, not a `vendor/` submodule [97].

B.6 Find the corresponding code in another edition

Begin with a symbol and responsibility, then use line anchors as a convenience. For example, from a DeepInfer checkout:

```
rg -n 'def execute_model_step|def _post_process_step' deepinfer/worker
rg -n 'class Compressor|class MQA' deepinfer/model_executor/models
rg -n '"schedule"|"process_outputs"' csrc/engine/pybind_scheduler.cpp
rg --files deepinfer/model_executor/parallel_utils
```

Follow callers as well as definitions: a helper's existence does not establish that the released engine drives it, as chapter 11's state-transfer example showed. Recheck defaults and the chosen model/platform branch. To compare a checkout with this book's evidence, run `make check` from the book with `DEEPINFER_SOURCE=/path/to/DeepInfer`. A mismatch detects a changed source edition; it does not identify which behavioural claims remain valid.

As a reading exercise, choose one object: an input position, a compressed row, a grammar matcher or a pooled output address. Locate its creator, its next reader and its release or reuse condition. The directory map supplies entry points; the ownership and evidence methods from the chapters supply the check.

Appendix C

Source edition and excerpts

The inspected DeepInfer files match upstream commit `774ce244b87cc0eb8d4b94ab9f5de2ab22049f01`. All DeepInfer file entries in `source-manifest.json` were compared by SHA-256 with that commit. This identifies the source used here; it does not pin a model checkpoint, submodule installation or a tested engine environment.

The experiments, saved outputs, excerpts, manifest and checking tool are published in the companion repository https://github.com/jzstark/inside-deepinfer-code under the Apache License 2.0. Paths such as `labs/`, `results/` and `excerpts/` below are relative to its root, and commands such as `make labs` and `make check` run there. Its tags identify the code for each published edition; use the tag that matches your copy of the book, since the default branch may contain later corrections.

Source paths in the chapters are relative to the DeepInfer repository. Printed excerpts are contiguous source lines saved under `excerpts/`, with original line numbers and common leading indentation removed. The manifest records paths, source ranges, removed indentation and hashes; code and comments are otherwise unchanged. These fragments are for reading, not standalone programs. DeepInfer's Apache 2.0 licence is included in `excerpts/LICENSE`; the references link to the fixed source edition. The book builds without an engine checkout. To verify the excerpts against a local snapshot, run `make check DEEPINFER_SOURCE=/path/to/DeepInfer`. The check verifies source hashes, line ranges, removed indentation and excerpt hashes without rewriting them.

The manifest also records a historical comparison with vLLM v0.6.3, commit `fd47e57f4b0d5f7920903490bce13bc9e49d8dba`. Its engine and sequence files support the terminology comparison; its scheduler supports chapter 5's queue and preemption comparison. Chapter 7 also compares DeepSeek's standalone EPLB at `d52c72d5b2f2fb4c41afbf8eb21366820239913d`, recording the policy and README hashes. These comparisons establish differences at fixed versions, not DeepInfer's fork point or direct derivation of its EPLB policy. `make check` covers the DeepInfer snapshot and its book excerpts; the separate comparison sources were checked manually.

Chapter 7's placement lab imports `labs/vendor/eplb_policy.py`, an unmodified copy of `deepinfer/model_executor/eplb/policy.py` that `make check` verifies like any excerpt. The same chapter quotes the gate's replica rule from two libraries outside the DeepInfer repository: DeepGEMM at the commit pinned by DeepInfer's `vendor/DeepGEMM` submodule, and the `tile-kernels` 2.1.0 package named in its `requirements.txt`. The manifest records those files' hashes under `dependencies`; `make check` does not fetch or verify them. Chapter 10 additionally records the FlashMLA submodule commit from DeepInfer's git tree and its repository URL from `.gitmodules`. That pin identifies the dependency; the inspected code is DeepInfer's caller, not the FlashMLA kernel implementation.

English renderings of Chinese comments are labelled *author's translation* in the surrounding prose, with a reference to the original line. A translation is editorial text, not an alteration of the quoted code. Source artifacts retain the original comments.

The book's English prose follows the project's writing guide [3], which draws on *The Economist*'s principles of clear, concrete and economical explanation.

The twenty-eight teaching labs have a separate record in `results/provenance.json`: script hashes, output hashes, Python version, platform and execution time. For chapter 2, it also records the input trace hash and the generated curve-data hash. The trace timestamps are assigned teaching data; the saved analysis is actual script output. Re-running `make labs` refreshes records for the labs it executes. If PyTorch is absent, lab 16's output and original run record are retained. Each run's `execution` field records its generating environment and time; the top-level `skipped` list describes skips in the current invocation. Chapter 8 adds exact rational enumeration and a translation of the verification budget search. Its provenance also records the sourced DSpark input table and the generated synthetic cost curves. The recorded output distinguishes DeepSeek's reported throughput estimates from calculations on assigned costs. Chapter 9 checks a rational window model and translated query-range and SWA pair-count formulas against direct enumeration. It does not execute the native FLOPs estimator or the model equivalence checks. Chapter 10 checks floating-point reduction order, a scalar metric counterexample, request-keyed teaching draws and exact PIT interval masses. Its inventory of released model tests is based on source inspection, not execution of those tests. Chapter 11 adds assigned service queues and capacity rates, a translation of the Q&A hit policy, and a frontier from synthetic records. The runner hashes its two generated curve CSVs. A separate `published_figures` manifest entry records the upstream PNG hash used for the approximate visual reading; it was checked against the fixed commit manually and is not covered by `make check`. The plotted CSVs contain no recovered benchmark measurements. Appendix A adds rational compression across chunk boundaries, a ring-lifetime counterexample, integer index scores and address conversion. Appendix B translates rank grouping for two four-worker layouts. These labs do not execute compression/indexing kernels or start distributed processes. These records attest to the teaching runs, not to execution of DeepInfer.

Terms and abbreviations

These entries locate explanations; they do not replace the conditions attached to each mechanism. Chapter numbers refer to the main text; A and B refer to the technical appendices.

Term	Meaning in this book	Where
Prefill / decode	Compute a prompt's states / continue generating from cached states. Verification can evaluate several positions per request in a decode step.	1, 8
KV cache	Stored key/value information reused by attention. MLA changes its representation; it need not contain two separate arrays.	1, 3
TTFT	Time to first token, including waiting and work before publication.	2
τ / SPS	Average useful output tokens per request per step / steps per second. Their product gives per-request token rate under the stated counting and measurement assumptions.	2, 8, 11
Pareto frontier	Non-dominated operating points: improving one objective requires worsening another among the compared points.	11
Roofline	A cost bound that compares arithmetic work with memory traffic; arithmetic intensity is operations per byte.	2
FLOP / MFU	Floating-point operation / model FLOPs utilisation. The chosen operation count and hardware denominator matter.	2
MLA	Multi-head Latent Attention: a shared compressed latent represents KV.	3
SWA	Sliding-window attention: a query sees a bounded recent interval.	3
RoPE	Rotary position embedding; a position-dependent transformation of attention queries and keys.	3
C1 / C2	One completed cache row per one / two token positions. Distinct from compression of a feature vector.	A
`ca_state` / `ca_cache`	Live score/latent compression state / completed compressed KV.	4, A
`ica_state` / `ica_cache`	Optional live index-compression state / completed index keys. The V4.1 KProject path needs no independent index state.	A
Top-k / top-p	Keep the highest-ranked k entries / a probability nucleus. Indexer top-k selects cache rows; sampling top-k selects possible tokens.	8, A
ZOS	Zero-overhead scheduling: overlap host work with the next forward while correcting information that is a step behind. The name does not guarantee that all host time is hidden.	1, 6
CUDA graph	Captured device work replayed using prepared buffers and a chosen size bucket.	6

Term	Meaning in this book	Where
HC	Hyper-connections: mix multiple residual streams around sublayers. Navigation only; the book does not derive this model mechanism.	B
Engram	The n-gram embedding mechanism; `EngramRuntime` fetches its sharded embeddings. A name, not an acronym expanded here. Navigation only.	B
MoE / EP	Mixture of experts / expert parallelism: route token work to expert weights distributed across ranks.	7
DP / CP	Data / context parallelism: engines handle different requests / workers cooperate within an engine. These groups can reuse the same ranks as EP.	7, B
EPLB	Expert parallelism load balancing: collect loads, choose replicas and placement, then move weights and update mappings.	7
MTP / nextn	Multi-token prediction / the configured draft count. A verification budget counts query positions, not just requests.	8
DFlash / DSpark	The draft-model path / the reported variable-length verification method. They are names; the implementation connects proposals, confidence and budgeting.	2, 8
YOCO / CED	You Only Cache Once / Causal Encoder-Decoder. Related model designs, not interchangeable names for every kind of cross-layer sharing.	9
Mask / compact	Two execution modes for truncated prefill: retain rows and mask work / physically remove rows. Their equivalence is distinct from agreement with full prefill.	9, 10
Batch invariance	The specified values remain identical when batch composition changes. State which values and comparator are tested.	10
Golden tokens	Prescribed token IDs that hold a test trajectory fixed while probabilities or other values are compared.	10
PIT / KS	Probability integral transform / Kolmogorov–Smirnov test. The sampler check uses randomised PIT values and tests their distribution.	10
PE / CE	Prefill engine / decode (completion) engine; they use the released `PROMPT` / `COMPLETION` roles. Separating roles does not supply production transfer and routing services.	5, 11
P/D; E/P/D	Prefill/decode; vision encoding/prefill/decode stage separation.	11
TTL / LRU	Time to live / least recently used. Retention time and capacity-driven eviction are separate mechanisms in the described service.	11
3FS	DeepSeek's separately released distributed file system. The deployment account places block KV there, not SWA state.	11

References

[1] DeepSeek-AI. *DeepInfer: release overview*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/README.md. Local file: `README.md`. Inspected 20 October 2026.

[2] DeepSeek-AI. *DeepSeek V4.1 Flash Inference Deployment Q&A*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/doc/inference_deployment_qa.md. Inspected 20 October 2026.

[3] Jzstark. *The Economist Writing Style: A Practical Reference*. https://github.com/jzstark/eco-writing-style/blob/main/eco-style.md. Read 20 October 2026. The project's chosen editorial reference.

[4] DeepSeek-AI. *Offline LLM entry point*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/entrypoints/llm.py. Symbols: `LLM.generate`, `LLM._add_request`, `LLM._run_engine`. Offline preemption default: line 68.

[5] DeepSeek-AI. *LLM engine*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/engine/llm_engine.py. YOCO prompt-logprob restriction: lines 528–536. Engine methods: `add_request`, `step`, `_execute_model_async_step` and `post_process`. Lifetime helper: `DeferredRelease`, lines 67–116. `finalize_seq`: lines 804–806; `pre_sampler_sync`: lines 830–846; metadata collection in `post_process`: line 1013.

[6] DeepSeek-AI. *Sequence state*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/sequence.hpp. Symbols: `SequenceData`, `SequenceStatus`.

[7] DeepSeek-AI. *Worker execution*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/worker.py. Symbols: `Worker.execute_model_step`, `Worker._post_process_step`, `Worker.flush_inflight_reqs`. Step order: lines 113–189; `_correct_seqlens_qk`: lines 191–245; event wait and graph-timer flush: lines 262–266. Local source hashes for these code references are recorded in `source-manifest.json`; the source links above pin the verified commit.

[8] DeepSeek-AI. *C++ scheduler*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/scheduler.hpp. Symbols: `Scheduler::add_sequence`, lines 135–142; `Scheduler::schedule`, lines 667–736; `Scheduler::_preempt`, lines 766–790.

[9] DeepSeek-AI. *Request output storage*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/outputs.hpp.

Symbols: `RequestOutputPool::allocate`, `RequestOutputPool::release`, lines 64–80.

[10] DeepSeek-AI. *Request output Python binding*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/pybind_output.cpp. `RequestOutput.clone` implementation: lines 16–18; declaration: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/_C/engine.pyi#L182.

[11] vLLM contributors. *Sequence definitions*, v0.6.3. https://github.com/vllm-project/vllm/blob/fd47e57f4b0d5f7920903490bce13bc9e49d8dba/vllm/sequence.py#L625. Symbol: `SequenceGroup`. A historical comparison, not an identified fork point.

[12] vLLM contributors. *LLM engine*, v0.6.3. https://github.com/vllm-project/vllm/blob/fd47e57f4b0d5f7920903490bce13bc9e49d8dba/vllm/engine/llm_engine.py#L1284. Symbol: `LLMEngine.step`; its docstring describes freeing sequence groups.

[13] vLLM contributors. *Scheduler*, v0.6.3. https://github.com/vllm-project/vllm/blob/fd47e57f4b0d5f7920903490bce13bc9e49d8dba/vllm/core/scheduler.py. Queues: lines 340–348; preemption selection: lines 1472–1499; recomputation: lines 1501–1510; swap-out: lines 1529–1543. Compared with DeepInfer's pinned scheduler, not an identified ancestor revision.

[14] DeepSeek-AI. *Main-model FLOPs estimator*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/flops_estimator.hpp. Symbols: `flops_coef`, `FlopsEstimator::calc_flops`, lines 13–45. Replay query ranges and cost terms: lines 237–320.

[15] DeepSeek-AI. *Offline benchmark*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/benchmark.py. Symbols: `measure_decode`, `run_decode_step`; endpoint storage, lines 380–381; measurement and profile windows, lines 499–519; token-accounting comments, lines 405, 407 and 539; topology and rank-0 reporting, lines 71–76 and 521–560; YOCO mask-mode option, lines 45–50.

[16] DeepSeek-AI. *Data-parallel process launcher*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/_launch.py. Symbols: `launch_dp`, `run_dp`.

[17] DeepSeek-AI. *Example engine configuration*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/_config.py. Symbol: `setup_cuda_visible_devices`, lines 55–64. Example chunked-prefill setting and caller overrides: lines 118–123.

[18] Samuel Williams, Andrew Waterman and David Patterson. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM, 52(4), 2009, pp. 65–76. https://doi.org/10.1145/1498765.1498785.

[19] Ashish Vaswani et al. *Attention Is All You Need*. 2017. Section 3.2.1: scaled dot-product attention. https://arxiv.org/abs/1706.03762.

[20] DeepSeek-AI. *DeepSeek model execution*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/deepseek.py. Symbol: `DeepseekModel.forward`, lines 154–248.

Wrapper sampling handoff: `DeepseekForCausalLM.sample`, lines 383–401. YOCO compatibility checks: lines 117–138.

[21] DeepSeek-AI. *Decoder layer*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/decoder_layer.py. V4 attention construction: line 85; attention call: lines 155–161. Feed-forward call: lines 169–173. Full/compact boundary normalisation: lines 134–152.

[22] DeepSeek-AI. *V4 attention runtime*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/attn.py. Class: `V4Attention`. Query and SWA projection: lines 386–464; decode attention: lines 628–670; forward path: lines 752–880. Full-source block KV: `_store_yoco_full_kv`, lines 672–721.

[23] DeepSeek-AI. *Cache engine and state layout*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/cache_engine.py. Format selection: lines 46–56; ring sizes: lines 88–110; stored/dropped classification: lines 113–116; SWA byte budgets: lines 261–280; cache view: lines 407–422; state-slot count: lines 201–202; cache fields: lines 206–219; `StateBlock`: lines 221–241 and 306; `select_state_slice_to_store`: lines 763–817; `load_state_from_store_to_prefill_cache`: lines 821–848.

[24] DeepSeek-AI. *KV-cache formats*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/cache_common.py. Symbols: `KVCacheFormat`, `V41_FORMAT`, lines 24–137, including main FP8/FP4 and index FP4 formats.

[25] DeepSeek-AI. *Attention metadata*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/input_metadata/input_metadata.py. V4 attention information: lines 343–385. CUDA decode address construction: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/cuda/prepare_decode_metadata.py#L138.

[26] DeepSeek-AI. *KV index expansion*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/get_kv_index.py. Decode SWA wrapper: lines 97–112. CUDA ring branch: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/cuda/get_kv_index.py#L59.

[27] DeepSeek-AI. *Attention backend*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/attention_backend.py. Symbol: `decode_mla_mqa_sparse`, lines 77–146.

[28] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. https://arxiv.org/html/2405.04434v5#S2.SS1.SSS2. Sections 2.1.2–2.1.3: joint KV compression and the separately cached position-encoding component.

[29] DeepSeek-AI. *MLA model projections*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/mla_attn.py. Low-rank projection container: lines

26–59; value and shared KV projections: lines 224–245. Runtime output projection: `_wv_and_o_proj` in reference [22], lines 191–225.

[30] DeepSeek-AI. *Environment defaults*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/environ.py. Draft limit and backtracking allowance: lines 129–132; extra state storage: lines 163–164; YOCO options: lines 188–194.

[31] DeepSeek-AI. *Verification query counts and decode dispatch*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/mtp_varlen.hpp#L150. Fixed-length query count: lines 150–158. The scheduler in reference [8] marks decode as non-prompt at lines 651–659 and transfers that flag at line 707. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/model_runner.py#L120. Input-metadata construction: lines 120–146; CUDA query positions: reference [25], kernel lines 112–144.

[32] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[33] DeepSeek-AI. *Scheduler memory ownership*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/scheduler.hpp. Joint checks and state cloning: lines 194–212; forked admission: lines 412–437 and 602–618; prompt allocation: lines 518–519; decode extension and preemption: lines 629–640; step key length: line 697; release: lines 738–745; `_append_slot`: lines 817–828; processed-count correction: lines 927 and 998–999; `fork_seq`: lines 1061–1122; block copies: lines 1288–1300.

[34] DeepSeek-AI. *Block and state-slot allocator*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/block_manager.hpp. Symbols: `Blocks`, lines 27–117; `BlockManager::can_allocate_nolock`, lines 128–136; `BlockManagerMutexAcquire::allocate_seq`, lines 207–214.

[35] DeepSeek-AI. *Scheduler and allocator Python bindings*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/pybind_scheduler.cpp. `unpin_blocks` bound to `BlockManager::free_blocks`: line 68.

[36] DeepSeek-AI. *Worker memory profiling*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/base.py. State reservation and block count: lines 237–246; cross-rank result: lines 264–267.

[37] DeepSeek-AI. *Scheduler admission, step selection and preemption*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/scheduler.hpp. Prefill cache watermark: line 80; completion-engine restriction: lines 102–104; step selection: lines 275–345; prefill counter reset: lines 675–680; prompt admission and chunking: lines 351–553; decode ordering: lines 624–659; metadata construction: lines 688–728; preemption: lines 766–790; stale-result comment and guard: lines 917–924; unfinished-chunk requeue: lines 1016–1019. State reset helpers: reference [6], lines 134–142, 223–227, 258–261 and 307–311.

[38] DeepSeek-AI. *V4 input preparation*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/input_metadata/prepare_tensor_input_v4.cpp. Prefill SWA write destinations: lines 98–109; YOCO visibility: lines 149–170; truncation metadata: lines 203–271; `update_yoco_prompt_v4`: lines 349–360.

[39] DeepSeek-AI. *Engine construction arguments*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/engine/arg_utils.py. Preemption, engine-type and chunking defaults: lines 40–42.

[40] DeepSeek-AI. *ZOS input preparation*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/zos.cpp. Fresh-row tokens: lines 99–113; `prepare_zos_inputs`: lines 142–196; `update_metadata_for_sampling_zos`: lines 279–305.

[41] DeepSeek-AI. *Worker step state and ZOS metadata*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/zos_status.py. `WorkerStepState`: lines 9–20; `MetadataZOS`: lines 23–40.

[42] DeepSeek-AI. *ZOS token gathering*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/gather_input_ids_zos.py. `gather_input_ids_zos` contract: lines 10–32. Caller: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/input_metadata/prepare_tensor_input_zos.py, `prepare_token_ids_zos`, lines 181–244.

[43] DeepSeek-AI. *ZOS key-length correction*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/compute_seqlens_k_zos.py. Kernel wrapper and formula: lines 10–33; reference implementation: lines 35–52.

[44] DeepSeek-AI. *Engine types and ZOS default*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/engine/engine_type.py. Default ZOS engine types: lines 16–17. Environment default applied in reference [39], lines 51–55.

[45] DeepSeek-AI. *CUDA graph runner*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/cuda_graph_runner.py. Graph data contract: lines 26–38; batch-size buckets: lines 41–51; selection and replay: lines 143–154. Runner construction: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/model_runner.py, lines 72–79; YOCO keep-length assertion: lines 107–113.

[46] DeepSeek-AI. *Host and device timers*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/timer.py. `CPUTimer.record`: lines 50–63; `CUDAGraphTimer.flush_out`: lines 196–212.

[47] DeepSeek-AI. *V4 MoE execution*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/mlp.py. Fused-path condition: lines 184–188; slot counts: lines 196–202; gate: lines 359–401; routed experts: lines 406–459; dispatch: lines 461–491; combine: lines 493–511; shared expert: lines 513–526; fused path: lines 528–563; `forward`: lines 565–616. Communication SM limit: https://github.com/deepseek-a

i/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/parallel_utils/userbuffer.py, lines 40–51.

[48] DeepSeek-AI. *Expert load balancer*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/eplb/__init__.py. Slot padding: lines 21–28; initial map: lines 31–39; state and window: lines 42–129; `step`: lines 131–165; `_rearrange_experts`: lines 222–288; per-layer views: lines 346–376. Step call site: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/deepseek.py, lines 402–404.

[49] DeepSeek-AI. *EPLB policy*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/eplb/policy.py. `_batch_greedy_indices`: lines 12–70; `rebalance_experts_global`: lines 73–140.

[50] DeepSeek-AI. *EPLB weight shuffle*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/eplb/shuffle.py. Point-to-point path: lines 65–147; intranode mirror path: lines 150–218.

[51] DeepSeek-AI. *Model loader*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/model_loader.py. Redundant-slot resolution: lines 83–107 and 122–123.

[52] DeepSeek-AI. *DeepGEMM SM100 mega gate*. https://github.com/deepseek-ai/DeepGEMM/blob/1fc9c66975ee322d628b3288534ca0e9ce12a64d/deep_gemm/include/deep_gemm/epilogue/sm100_mega_gate.cuh. Replica choice: lines 204–206. Commit pinned by DeepInfer's `vendor/DeepGEMM` submodule at the book's source edition.

[53] DeepSeek-AI. *TileKernels 2.1.0 top-k gate*. Package `tile-kernels==2.1.0`, required by DeepInfer's `requirements.txt`, line 17. File `tile_kernels/moe/moe_topk_gate_forward_cuda.py`: replica choice at lines 41 and 282–284. Package and file hashes are recorded in `source-manifest.json`.

[54] DeepSeek-AI. *Expert Parallelism Load Balancer*, standalone repository, commit `d52c72d`. https://github.com/deepseek-ai/EPLB/blob/d52c72d5b2f2fb4c41afbf8eb21366820239913d/eplb.py. Scalar packing: lines 27–40; replica allocation: lines 58–71; entry point and topology selection: lines 131–162. The README describes load prediction as outside the repository's scope: https://github.com/deepseek-ai/EPLB/blob/d52c72d5b2f2fb4c41afbf8eb21366820239913d/README.md.

[55] DeepSeek-AI. *DeepInfer dependency declarations*, at the book's fixed revision. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/.gitmodules; https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/requirements.txt. The gitlinks pin DeepGEMM and DeepEP; requirements pin TileKernels 2.1.0. Versions and inspected file hashes are recorded in `source-manifest.json`.

[56] Yaniv Leviathan, Matan Kalman and Yossi Matias. *Fast Inference from Transformers via Speculative Decoding*. ICML, PMLR 202, 19274–19286, 2023. Algorithm 1 and Appendix A.1. https://proceedings.mlr.press/v202/leviathan23a.html.

[57] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre and John Jumper. *Accelerating Large Language Model Decoding with Speculative Sampling*. 2023. Algorithm 2 and the distribution-preservation argument. https://arxiv.org/abs/2302.01318.

[58] DeepSeek-AI. *Speculative verification and accepted-prefix collection*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/sampler.py, `Sampler._verification`, beginning at line 411. `Sampler.forward`, lines 114–144, selects hidden rows and computes logits before verification or ordinary sampling. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/cuda/fused_sample.py, acceptance: lines 378–399; residual: lines 435–445; fallback: lines 506–518. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/cuda/sampler.py, `prepare_verification_metadata_tl`, `get_accept_result_tl` and `finalize_verification_tl`.

[59] DeepSeek-AI. *Sampling transforms and proposal vocabulary alignment*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/sampler_utils.py, `process_sampling`, lines 113–197. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/sampler.py, `remap_draft_topk`.

[60] DeepSeek-AI. *Draft probability penalty and its ideal-arithmetic bound*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/common.py. Docstring: lines 3–21; `DRAFT_PROB_PENALTY`: line 53.

[61] DeepSeek-AI. *Fixed-point sampling helpers*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/cuda/random_sampling.py. Symbols: `Config`, `float_to_fixed`, `accumulate`, `sample_offset`, `sample_select`.

[62] DeepSeek-AI. *Variable verification budget and runtime estimates*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/mtp_varlen.hpp. Estimators: lines 19–132; runtime update: lines 140–148; `schedule`: lines 150–267; confidence-round comment: lines 159–167. CPU prefix products: `csrc/engine/scheduler.hpp`, lines 1239–1250, at the same fixed revision (reference [37]).

[63] DeepSeek-AI. *Startup decode performance curve*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/model_runner.py, `profile_decode_perf`, lines 562–631. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/base.py, `warm_up_model`, lines 275–281. Feedback: `deepinfer/engine/llm_engine.py`, lines 995–1002, at the same revision (reference [5]).

[64] DeepSeek-AI. *DFlash proposal generation and confidence*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepin

fer/model_executor/models/mtp/dflash.py. Input embedding, draft forward and sampling: lines 230–345.

[65] DeepSeek-AI. *Stored MTP proposals*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/mtp_info_pool.py. `MTPInfoPool`: slot layout, reserved slot zero and `store`.

[66] DeepSeek-AI. *Device allocation of a fixed verification budget*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/cuda/schedule_mtp_varlen.py. `compute_seqlens_q_by_confidence_rates_tl`, lines 14–83. Call and D2H event: `deepinfer/worker/worker.py`, lines 195–234; sampling metadata update: lines 146–152 (reference [7]).

[67] DeepSeek-AI. *Verification confidence metric*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/sampler.py, `compute_confidence_error_ref`, lines 584–589. CUDA accumulation: `tilelang_ops/cuda/sampler.py`, lines 368–374; probability proxy: `tilelang_ops/cuda/fused_sample.py`, lines 471–479 and 520–522 (reference [58]). https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/engine/stats_logger.py, normalisation: lines 234–236; reported name: line 309.

[68] Yutao Sun et al. *You Only Cache Once: Decoder-Decoder Architectures for Language Models*. 2024, arXiv:2405.05254v1, sections 2.2–2.3. https://arxiv.org/html/2405.05254v1#S2.SS2.

[69] DeepSeek-AI. *DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression*. 2026, arXiv:2609.19969v1, sections 2.2, 3.2.2 and 4.2.1. https://arxiv.org/html/2609.19969v1#S2.SS2.

[70] DeepSeek-AI. *Hugging Face configuration adapter*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/hugging_face/config.py. SWA shared-hidden default: line 47; segmented maps: lines 66–85; `_n_prefill_layer`: lines 158–161; window mapping: line 284.

[71] DeepSeek-AI. *YOCO runtime predicates and compaction*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/yoco.py. `is_yoco_consumer_layer`: lines 19–21; `compact_yoco_boundary`: lines 93–123.

[72] DeepSeek-AI. *Cross-layer sharing topology and runtime*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/cross_layer.py. Boundary validation: lines 114–116; `CrossLayerRuntime`: lines 277–325.

[73] DeepSeek-AI. *YOCO input metadata views*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/input_metadata/yoco.py. `get_mask_view`: lines 84–92; `get_consumer_view`: lines 94–128 for the single-context-rank path, with redistribution below.

[74] DeepSeek-AI. *FLOPs estimator reference checks*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/tests/engine

/test_flops_estimator.py. Fixture: lines 26–31; per-query reference: lines 34–145; mode, chunk and Engram cases: lines 148–178. Inspected, not run for this book.

[75] DeepSeek-AI. *YOCO mask/compact equivalence checks*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/yoco/README.md. Driver: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/yoco/yoco_equiv.py. Baseline selection: line 536; baseline validation: lines 793–797. Full-vocabulary guard: line 566; temperature/top-p: lines 674–675. Inspected, not run for this book.

[76] DeepSeek-AI. *Batch-invariance configuration and library boundaries*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/runtime_config.py, lines 54–69. Attention predicates and calls: deepinfer/model_executor/models/v4_modules/attn.py, lines 87–92, 666 and 1002 (reference [22]). https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/parallel_utils/userbuffer.py, lines 72–80. FlashMLA imports and sparse-decode flag forwarding: deepinfer/model_executor/layers/attention_backend.py, lines 2 and 141; direct fused-operation import: deepinfer/model_executor/models/v4_modules/attn.py, line 14. The gitlink at vendor/FlashMLA pins 2e5429fc5653bab6e081f09477126f731882a6a9; .gitmodules identifies the DeepSeek repository. All caller evidence and the gitlink are from the same DeepInfer revision.

[77] DeepSeek-AI. *Released numerical model checks*. Inspected, not run for this book. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/tests/models/test_batch_invariant.py, result fields at lines 41–47 and golden replay at lines 96–105. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/tests/models/test_ve_batch_invariant.py, embedding comparison at lines 113–127. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/tests/models/test_kl.py, golden generation at lines 80–82; comparison at lines 177–209. Prompt sampling parameters: https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/prepare_sample.cpp, lines 186–188; prompt-logprob calculation: deepinfer/model_executor/layers/sampler_utils.py, lines 52–78. Temperature, top-p/top-k and log-softmax reference: deepinfer/model_executor/layers/tilelang_ops/fused_sample.py, lines 137–150 and 209–211 (reference [79]). https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/testing.py, cal_diff, lines 4–12. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/tests/models/test_offline_inference.py, platform configuration and stored offline_decode_ppl.json comparison.

[78] DeepSeek-AI. *Golden overrides and verification metadata*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/sampler.py, reference metadata at lines 13–25 and override at lines 227–250. CUDA implementation: deepinfer/model_executor/layers/tilelang_ops/cuda/sampler.py, lines 252–276, at the same revision (reference [58]).

[79] DeepSeek-AI. *Sampling keys and evaluation defaults*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/

model_executor/layers/tilelang_ops/fused_sample.py, MTP offset at line 15 and namespace selection at lines 93 and 208. CUDA keys: tilelang_ops/cuda/fused_sample.py, lines 227, 392, 450 and 481 (relative to deepinfer/model_executor/layers/; reference [58]). Packed positions: tilelang_ops/sampler.py, lines 56–66, and tilelang_ops/common.py, line 52; expansion caller: sampler_utils.py, lines 162–163 (reference [59]). DFlash proposal positions: deepinfer/model_executor/models/mtp/dflash.py, lines 290–294 (reference [64]). https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/eval/run_eval.py, per_sample_seed and _setup_env, including lines 121–129.

[80] DeepSeek-AI. *Randomised PIT and sampler checks*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/tests/models/test_sampler.py, settings at lines 39–45, statistical helpers at lines 50–94 (trajectory rejection filter at lines 85–88), and failure conditions at lines 242–250. Inspected, not run for this book. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/sum_smaller_probs.py, inclusive CDF definition and reference at lines 10–44.

[81] DeepSeek-AI. *Structured-output matcher state and sampling masks*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/structure_output/matcher.py, rollback capacity at line 20; token acceptance, masking and rollback at lines 134–235; failed futures at lines 297–338. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/sampler.py, matcher updates at lines 339–386 and masking at lines 433–437. Engine enqueue failure: deepinfer/engine/llm_engine.py, lines 621–622; scheduler failure/preemption: csrc/engine/scheduler.hpp, lines 769–788 and 973–980 (references [5, 37]). Trigger/bound interface: deepinfer/sampling_params.py, lines 39–43. XGrammar version: requirements.txt, line 21, all at the same revision.

[82] DeepSeek-AI. *Structured-output examples and format assertions*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/_structured_output.py, settings at lines 37–41; termination and format assertions at lines 42–63. Invoked by tests/models/test_offline_inference.py with check_structured_output=True. Inspected, not run for this book.

[83] DeepSeek-AI. *Engine roles and the offline measurement boundary*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/engine/arg_utils.py, NORMAL default at line 41. Prompt-only assertion: deepinfer/engine/llm_engine.py, lines 947–949, at the same revision. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/README.md, “Stage decoupling and metrics”. Benchmark construction: examples/benchmark.py, lines 173–188; example factory: examples/_config.py, lines 112–125, at the same revision.

[84] DeepSeek-AI. *State selection, restoration and layout tests*. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/cache_engine.py, select_state_slice_to_store, lines 763–818; load_state_from_store_to_prefill_cache, lines 821–848. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/tests/worker/test_state_cache_ops.py, padding removal at lines 76–84; round-trip comparison at lines 188–215. Inspected, not executed for this book.

[85] DeepSeek-AI. *Performance sweeps, plotting code and published decode figure.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/examples/sweep/plot_pareto.py, record selection at lines 37–55; conversion and axes at lines 58–75; subtitle at lines 91–100; point connection at line 150. Sweep driver: `examples/sweep/run_sweep.sh`, at the same revision. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/doc/images/decode_pareto.png. Chapter 11's 246 tokens/s/user is an approximate visual reading at 64 requests per DP rank, not a recovered benchmark record.

[86] DeepSeek-AI. *3FS* and *DeepSeek Recipe.* https://github.com/deepseek-ai/3FS; https://github.com/deepseek-ai/deepseek-recipe. Public project boundaries checked on 22 October 2026. The DeepInfer deployment Q&A does not identify the deployed revisions; this book does not inspect their internal service implementations.

[87] DeepSeek-AI. *Runtime temporal compression.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/compress.py, `KProject` and `Compressor`, especially lines 74–167. Projection: `deepinfer/model_executor/layers/mla_attn.py`, `SoftmaxAPEGatedColumnParallelLora`, at the same revision.

[88] DeepSeek-AI. *Compression interfaces, references and checks.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/compress.py, `compress_and_update_state_prefill`, `compress_and_update_state_decode`, reference formulas at lines 178–248, and device tests in the same file. Tests inspected, not run. Live state allocation: `deepinfer/worker/cache_engine.py`, lines 123–133, 441–465, at the same revision.

[89] DeepSeek-AI. *Compressed positions, store slots and visible lengths.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/input_metadata/prepare_tensor_input_v4.cpp, lines 46–91 and 591–601.

[90] DeepSeek-AI. *Index attention and candidate selection.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/index_attn.py, `MQA`, `store_index_kv`, `compute_chunk_size`, `_topk_prefill`, `_topk_decode` and candidate selection. Configuration: `deepinfer/hugging_face/config.py`, lines 111–134, at the same revision. Index scoring and top-k implementations in independent libraries are identified at the import/call boundary, not audited here.

[91] DeepSeek-AI. *Segmented sharing maps and runtime data.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/cross_layer.py, `SharedLayerTopology`, refinement checks at lines 181–189, and `CrossLayerRuntime`.

[92] DeepSeek-AI. *Paged index conversion.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/layers/tilelang_ops/flash_mla_utils.py, `convert_kv_index`; CUDA implementation in `deepinfer/model_executor/layers/tilelang_ops/cuda/flash_mla_utils.py`, lines 31–37, at the same revision. The kernel and tensor reference use

the output lane's position relative to the valid count; they do not use a sentinel-value guard.

[93] DeepSeek-AI. *Main/index KV storage and sparse attention read.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/v4_modules/attn.py, main/index storage at lines 515–587; decode cache arguments and attention call at lines 640–669; full-layout YOCO storage at lines 672–705.

[94] DeepSeek-AI. *Engine drivers, workers and rank groups.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/worker/base.py, `WorkerBase` rank setup and `run_workers`. https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/parallel_utils/parallel_state.py, `ParallelProcessGroup.create_process_groups`. At the same revision: `examples/_launch.py`, `run_dp`; `examples/_config.py`, `setup_cuda_visible_devices`; `deepinfer/config.py`, `ParallelConfig`; `deepinfer/engine/llm_engine.py`, `_init_workers`; `deepinfer/model_executor/layers/tilelang_ops/prepare_cp_mirror.py`, lines 135–146; `csrc/engine/scheduler.hpp`, line 90.

[95] DeepSeek-AI. *Model registration, wrappers and navigation entries.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/deepinfer/model_executor/models/__init__.py, `_MODELS`; `deepinfer/model_executor/models/deepseek_vl.py`, `DeepseekVLForCausalLM.forward`, at the same revision. Core model: `deepinfer/model_executor/models/deepseek.py`. Navigation only: `deepinfer/model_executor/models/v4_modules/hc.py`, `HCLayerMixin`, `HCHead`; `v4_modules/engram.py`, `EngramRuntime`; `v4_modules/builder.py` (the latter two paths relative to the models directory). These entries do not evaluate the model mechanisms or weight-loading implementation.

[96] DeepSeek-AI. *Native scheduler and output bindings.* https://github.com/deepseek-ai/DeepInfer/blob/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01/csrc/engine/pybind_scheduler.cpp, `schedule` and `process_outputs` bindings. At the same revision: `csrc/engine/pybind_output.cpp`, `build_request_outputs`; `csrc/engine/outputs.hpp`, `RequestOutputPool` and output construction.

[97] DeepSeek-AI. *Repository layout and platform dispatch.* https://github.com/deepseek-ai/DeepInfer/tree/774ce244b87cc0eb8d4b94ab9f5de2ab22049f01. Platform dispatch: `deepinfer/model_executor/layers/tilelang_ops/ops.py`; platform selection: `deepinfer/platform.py`; dependency repositories: `.gitmodules`. Directory entries in Appendix B are navigation, not a claim to have inspected all files in each directory.